

2025-1-17

8676 快速开发手册

PVT软件部

掌锐电子软件部

目录

一、bootloader	6
1.1 code path	6
1.2 lk2 log开关	6
1.3 lk to kernel简单时序图	8
二、yocto kernel	9
2.1 config & dts	9
2.2 增加ko driver流程	9
2.3 增加kernel的命令行启动参数	11
2.3.1 增加 pageowner的命令行启动参数	11
2.3.2 增加 cma的命令行启动参数修改cma内存大小	11
三、android kernel	11
3.1 config&dts	11
3.2 增加ko driver流程	12
3.3 增加kernel的命令行启动参数	13
3.3.1 增加 pageowner的命令行启动参数	13
四、yocto	14
4.1 增加一个应用流程(包括自启动以及相关属性配置)	14
4.2 YOCTO SDK 应用	17
4.3 ivi-shell 配置	21
五、android	24
六、谦川SDK	24
6.1 物理中断统计(irq_count)	24
6.2 各个pCPU上瞬时正在运行的vCPU的信息 (pcpu_attr)	26
6.3 各个vCPU在一段时间内的负载 (top)	26
6.4 vcpu 累计运行时间 (vcpu_exec_accum)	28
6.5 vmexit原因统计(vcpu_exit_reason)	28
七、系统	30
7.1 bring up流程	30
7.1.1 开机流程	30
7.1.2 安全启动流程	31
7.2 目录结构介绍	33
7.2.1 代码结构	33
7.3 编译、下载、烧录	36
7.3.1 编译环境配置	36
7.3.2 代码下载	37

7.3.3 编译	38
7.3.4 烧录	39
7.4 系统虚拟化框架	43
7.5 内存	44
7.5.1 OS size控制	44
7.5.2 内存layout	44
7.5.3 查询内存信息	48
7.5.4 新增reserved 样例	48
7.6 partition	49
7.6.1 partition layout	49
7.6.2 如何新增一个partition	53
7.6.3 partition debug	58
7.7 OS间通信	60
7.7.1 SOS & UOS 间通信	61
7.7.2 UOS & UOS间通信	79
7.7.3 MCU & CPU间通信	80
7.8 外设通信协议	82
7.8.1 SPI	82
7.8.2 I2C	90
7.8.3 UART	91
7.8.4 GPIO	95
7.9 保活机制	97
7.9.1 进程保活	97
7.9.2 系统保活	97
7.9.3 控制虚拟机状态	100
7.10. misc debug	101
7.10.1 log【抓取，分析】	101
7.10.2 adb【连接，切换】	104
7.10.3 sos uos console 切换	105
7.10.4 android yocto 通过console 数据交互	106
7.10.5 remount挂载方法	106
7.11 cpu配置	107
7.11.1 虚拟机CPU配置	107
7.11.2 Nebula CPU调度策略配置	107
7.12 中断配置	108
7.13 STR	111

7.13.1 STR机制概要说明	111
7.13.2 STR框架	112
7.13.3 控制guest os suspend/resume方式	116
7.13.4 调试方法	117
7.14 OS间内存共享	121
7.14.1 Vdmabuf	121
7.15 OS内进程间内存共享	122
7.15.1 通过Binder机制	122
7.16 debug入口办法	127
7.16.1 常用debug方式	127
7.16.2 串口log入手确认启动时序	128
7.16.3 从问题种类选择合适debug工具	130
7.16.4 基础器件debug 方式	131
7.17 dmabuf	135
7.17.1 接口介绍	135
八、模块	136
8.1 Display	136
8.1.1 Display 总览	136
8.1.2 Configuration / Customization Guideline	142
8.1.3 Debug Tips	149
8.1.4 Troubleshooting	153
8.2 Camera	154
8.2.1 camera虚拟化概念介绍	154
8.2.2 camera虚拟化框架介绍	155
8.2.3 camera主要code位置	155
8.2.4 配置/客制化指南	156
8.2.5 Debug手段	159
8.3 thermal	162
8.3.1 Architecture	162
8.3.2 Thermal sensor	163
8.3.3 Linux thermal framework	163
8.3.4 配置文件	166
8.3.5 thermal debug	172
8.4 virtio_input	173
8.4.1 虚拟化框架介绍	173
8.4.2 Input数据传输	174

8.4.3 input快速开发	175
8.4.4 input debug	177
8.5 Audio	177
8.5.1 Audio AFE HW	177
8.5.2 Audio HAL	183
8.5.3 Audio Virtualization	189
8.6 secure boot	199
8.6.1 sign flow	199
8.6.2 verify flow	200
8.6.3 key配置	200
8.6.4 常用的配置	202
8.7 optee	205
8.7.1 optee简介	205
8.7.2 optee应用开发	206
8.8 OTA	208
8.8.1 OTA简介	208
8.8.2 如何编译Hypervisor(L+L+A) OTA升级包	210
8.8.3 如何进行Hypervisor OTA升级	213

一、bootloader

1.1 code path

ATF: mt8676/vendor/mediatek/proprietary/trustzone/tf-a-2.8/

LK1: yocto/src/bsp/lk2/

LK2: yocto/src/bsp/lk2/

LK1和LK2 共享一套代码，只是编译选项不一样。

1.2 lk2 log开关

```
yocto/src/bsp/lk2/top/include/lk/trace.h
```

```
12  /* trace routines */
13  #define TRA
CE_ENTRY printf("%s: entry\n", __func__)
14  #define TRACE_EXIT printf("%s: exit\n", __func__)
15  #define TRACE_ENTRY_OBJ printf("%s: entry obj %p\n", __func__, this)
16  #define TRACE_EXIT_OBJ printf("%s: exit obj %p\n", __func__, this)
17  #define TRACE printf("%s:%d\n", __func__, __LINE__)
18  #define TRACEF(str, x...) do { printf("%s:%d: " str, __func__, __LINE__, ## x); } while (0)
19
20  /* trace routines that work if LOCAL_TRACE is set */
21  #define LTRACE_ENTRY do { if (LOCAL_TRACE) { TRACE_ENTRY; } } while (0)
22  #define LTRACE_EXIT do { if (LOCAL_TRACE) { TRACE_EXIT; } } while (0)
23  #define LTRACE do { if (LOCAL_TRACE) { TRACE; } } while (0)
24  #define LTRACEF(x...) do { if (LOCAL_TRACE) { TRACEF(x); } } while (0)
25  #define LTRACEF_LEVEL(level, x...) do { if (LOCAL_TRACE >= (level)) { TRACEF(x); } } while (0)
```

可以看到是根据level和LOCAL_TRACE的值确定是否打印log的。这个值都是放到每个模块自己去设定是否打印的。比如

lk2 init模块

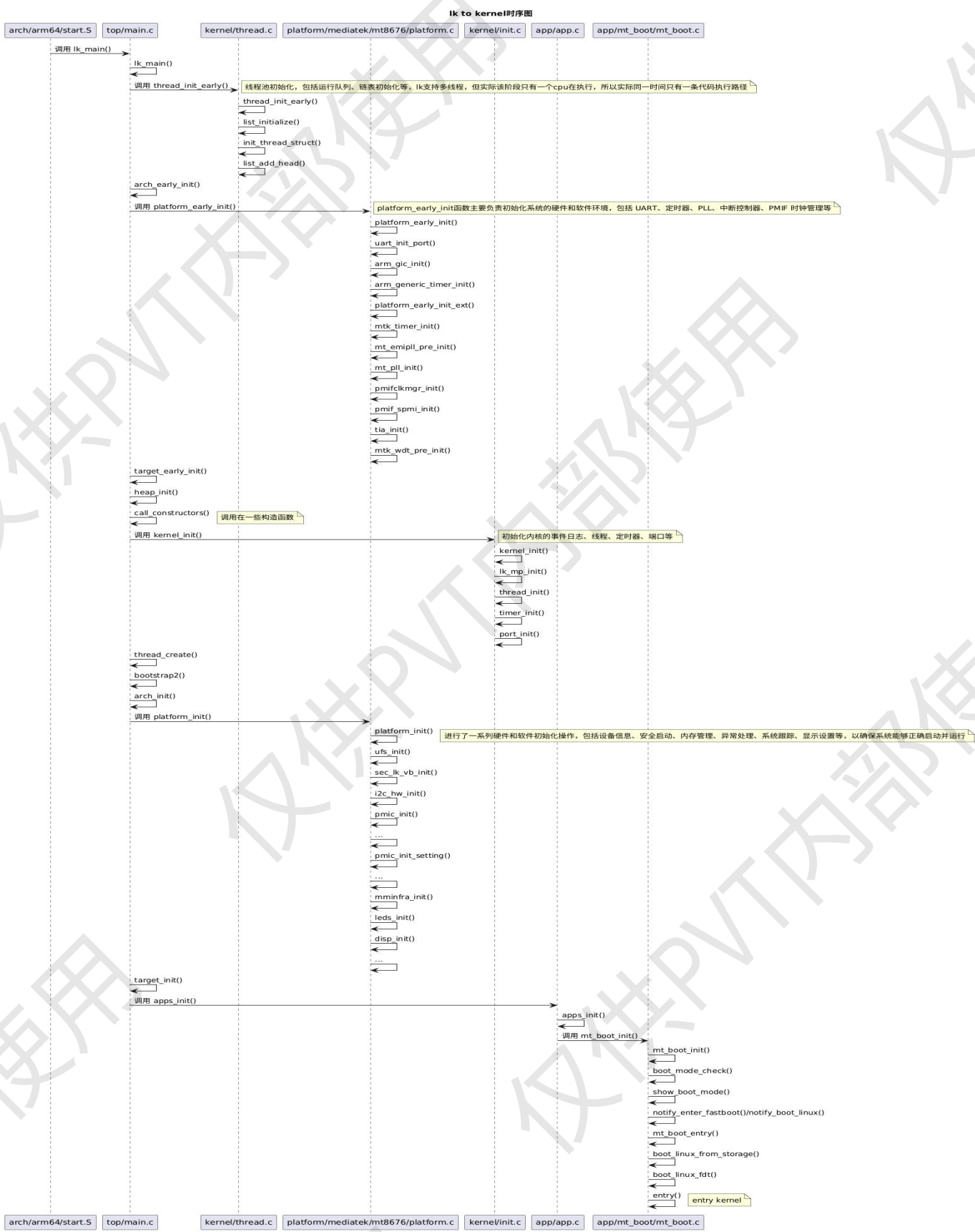
```
yocto/src/bsp/lk2/top/init.c
```

```
#define LOCAL_TRACE 0
```

将上面的 宏设置为1即可打开init模块的log

```
#define LOCAL_TRACE 1
```

1.3 lk to kernel简单时序图



二、yocto kernel

2.1 config & dts

多系统项目config & dts路径:

yocto/src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/configs

yocto/src/kernel/linux/v6.1_mt8676/co_device_module/kernel/configs/

多系统平台dts路径:

yocto/src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/mt6897.dts

yocto/meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/auto8676p1_64_hyp.dts

多系统dws路径:

yocto/src/devtools/dct/dws/mt6897/auto8676p1_64_hyp.dws

导出yocto板端的设备树文件/sys/firmware/fdt可以看到实际所有的dts信息（因为有的是lk阶段动态往dtb文件写的）

通过命令dtc -f -I dtb -O dts -o ydts fdt反汇编

config overlay:

gki_defconfig as base, merge下面Kernel configs

mgk_64_k61_defconfig, userdebug.config, nebula_hypervisor.config

(nebula_hypervisor.config层级最高, 上面merge的configs 是按照merge层级从低到高排列的)

sos 的 config可以从下面文件看到加的是否生效:

yocto/build-sos/tmp/work/auto8676p1_64_hyp-poky-linux/linux-mtk-extension/6.1/build/common/include/generated/autoconf.h

tbox的config 可以从下面文件看到加的是否生效:

yocto/build-tbox/tmp/work/auto8676p1_64_uos_tbox-poky-linux/linux-mtk-extension-uos/6.1/build/common/include/generated/autoconf.h

2.2 增加ko driver流程

以添加himax_touch触摸驱动为例, 描述如何添加驱动

(1) 在config文件中添加配置项, 例如: arch/arm64/configs/mgk_64_k66_defconfig中添加

```
CONFIG_TOUCHSCREEN_HIMAX_CHIPSET=m
CONFIG_TOUCHSCREEN_HIMAX_COMMON=y
CONFIG_TOUCHSCREEN_HIMAX_DEBUG=y
CONFIG_TOUCHSCREEN_HIMAX_INCELL=y
CONFIG_TOUCHSCREEN_HIMAX_IC_HX83193=y
CONFIG_TOUCHSCREEN_HIMAX_SELF_TEST=y
```

(2) 在设备树中添加对应的设备树配置，例如：

arch/arm64/boot/dts/mediatek/cust_mt8676_display_interface.dtsi中添加

```
hxchipset_ts@48 {
    status = "okay";
    compatible = "himax,hxcommon";
    reg = <0x48>;
    interrupt-parent = <&pio>;
    interrupts = <183 IRQ_TYPE_LEVEL_LOW>;
    himax,TSIX = <&pio 183 0x0>;
    himax,panel-coords = <0 2560 0 1440>;
    himax,display-coords = <0 2560 0 1440>;
};
```

(3) 在kernel drivers对应目录中添加设备驱动源码和编译Kconfig和Makefile：

```
config TOUCHSCREEN_HIMAX_CHIPSET
tristate "Himax touchpanel CHIPSET"
depends on I2C
help
Say Y here if you have a Himax CHIPSET touchscreen.
HIMAX controllers are multi touch controllers which can
report 10 touches at a time.
If unsure, say N.
source "$(KCONFIG_EXT_PREFIX)drivers/input/touchscreen/hxchipset/Kconfig"
```

Makefile中添加：

```
obj-$(CONFIG_TOUCHSCREEN_HIMAX_CHIPSET) += hxchipset/
```

(4) 如果是编译为ko文件，需要在 kernel/kleaf/mgk_64_k61.bzl中添加对应的ko路径

```
"drivers/input/touchscreen/hxchipset/himax_mmi.ko",
```

(5) 如果是编译为ko文件，需要在recipe对应路径yocto/meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/ko_order_table/ 目录下对应的子目录的ko_order_table.csv文件中添加对应的ko声明，以便生成的ko文件能被包含到对应的镜像中

```
himax_mmi.ko,../kernel_device_modules-
6.1/drivers/input/touchscreen/hxchipset/himax_mmi.ko,vendor,Y,N,user
```

```
/userdebug/eng
```

2.3 增加kernel的命令行启动参数

yocto/src/bsp/lk2/app/blxboot/blxboot.c 的函数update_dtb增加kcmdline_append(""); 添加启动参数

2.3.1 增加 pageowner的命令行启动参数

```
static void update_dtb(void)
{
....
    kcmdline_append("page_owner=on panic_on_taint=20");
....
}
```

2.3.2 增加 cma的命令行启动参数修改cma内存大小

```
static void update_dtb(void)
{
....
    kcmdline_append("cma=256M");
....
}
```

三、android kernel

3.1 config&dtb

多系统config路径:

android/kernel/kernel_device_modules-6.1/arch/arm64/configs/mgk_64_k61_defconfig

android/kernel/kernel_device_modules-6.1/kernel/configs/

多系统项目dtb路径:

android/kernel/kernel_device_modules-6.1/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts

多系统dws路径:

android/vendor/mediatek/proprietary/tools/dct/dws/mt6897/auto8676p1_64_bsp_vm.dws

config overlay:

gki_defconfig as base , merge下面Kernel configs

mgk_64_k61_defconfig auto.config , wifionly.config , userdebug.config, nebula_vm.config

(nebula_vm.config层级最高, 上面merge的configs 是按照merge层级从低到高排列的)

编译后可以从下面文件看config有没有生效

android/out/target/product/auto8676p1_64_bsp_vm/obj/KERNEL_OBJ/kernel-6.1/include/generated/autoconf.h

导出Android板端的设备树文件/sys/firmware/fdt可以看到实际所有的dts信息 (因为有的是lk阶段动态往dtb文件写的)

通过命令dtc -f -l dtb -O dts -o ydts fdt反汇编

3.2 增加ko driver流程

android/kernel/kernel_device_modules-6.1/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts

android/kernel/kernel_device_modules-6.1/arch/arm64/boot/dts/mediatek/mt6897.dts

多系统android dws路径,以8676为例:

以添加serdes-dp驱动为例, 描述如何添加驱动

(1) 在config文件中添加配置项, 例如: android/kernel/kernel_device_modules-6.1/arch/arm64/configs/mgk_64_k61_defconfig中添加

```
CONFIG_DRM_SERDES_DP=m
```

(2) 在设备树中添加对应的设备树配置, 例如: android/kernel/kernel_device_modules-6.1/arch/arm64/boot/dts/mediatek/mt6897.dts中添加

```
serdes_dp: serdes_dp {  
    compatible = "maxim,serdes-dp";  
    status = "okay";  
};
```

(3) 在kernel drivers对应目录中添加设备驱动源码和编译Kconfig和Makefile:

android/kernel/kernel_device_modules-6.1/drivers/gpu/drm/bridge

config DRM_SERDES_DP

tristate "DRM Serdes DP Support for Platform SoCs"

depends on OF

select SERDES_DP

help

Enable support for the Serdes-based DisplayPort (DP) interface

on Mediatek SoCs. This driver provides the necessary functionality

to initialize and manage the Serdes hardware that handles the high-speed data lanes used for DisplayPort connections on these platforms. Enabling this option will allow the DRM subsystem to communicate with DisplayPort devices via the Serdes interface.

android/kernel/kernel_device_modules-6.1/drivers/gpu/drm/bridge/Makefile 中添加:

```
obj-$(CONFIG_DRM_SERDES_DP) += serdes-dp.o
```

(4) 如果是编译为ko文件, 需要在 android/kernel/kernel_device_modules-6.1/kernel/kleaf/mgk_64_k61.bzl 中添加对应的ko路径

```
mgk_64_k61_device_modules.append("drivers/gpu/drm/bridge/serdes-dp.ko"
```

(5) 如果是编译为ko文件, 需要在recipe对应路径

android/device/mediateksample/auto8676p1_64_bsp_vm/ko_order_table.csv 中添加对应的ko声明, 以便生成的ko文件能被包含到对应的镜像中

```
serdes-dp.ko,../kernel_device_modules-6.1/drivers/gpu/drm/bridge/serdes-  
dp.ko,ramdisk,Y,Y,user/userdebug/eng
```

3.3 增加kernel的命令行启动参数

yocto/src/bsp/lk2/platform/mediatek/common/aee/debug_cmd_init.c 的函数kernel_mm_cmd_init增加
kcmdline_append(""); 添加启动参数

3.3.1 增加 pageowner的命令行启动参数

```
static void kernel_mm_cmd_init(void *fdt, int chosen_offset)
{
    #if LK_DEBUGLEVEL > 0
    ....
    kcmdline_append("page_owner=on panic_on_taint=20");
    #endif
    ....
}
```

四、yocto

4.1 增加一个应用流程(包括自启动以及相关属性配置)

(1) 添加recipes-*/[模块]/[模块].bb

如meta/meta-mediatek-mt8676/recipes-support/mcu_messenger/mcu_messenger.bb

```
#
DESCRIPTION = "pvt mcu messenger"
LICENSE = "MediaTekProprietary"
#
DEPENDS = "platform-libs platform-libs-common"
LICENSE = "BSD-3-Clause & MediaTekProprietary"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/BSD-3-
Clause;md5=550794465ba0ec5312d6919e203a55f9 \
file://${PROPRIETARY_LICENSE_PATH}/MediaTekProprietary;md5=c5d17c6905715d0948a3d6087602d12d
"
PN='mcu_messenger'
# fog systemd
inherit fog systemd
#
SRC_URI= "fog://src/apps/spm-base/mcu_messenger;name=mcu_messenger;prebuilt=none"
SRCREV_mcu_messenger= "${AUTOREV}"
S = "${WORKDIR}/git"
SYSTEMD_PACKAGES = "${PN}"
#
SYSTEMD_SERVICE_${PN} = "mcu_messenger.service"
FILES:${PN}:append = " \
${bindir} \
${systemd_unitdir}/system/mcu_messenger.service \
"
# do_compile
do_compile() {
oe_runmake \
CFLAGS="-lpthread ${CFLAGS}" \
LDFLAGS="-lstdc++ -lpthread -lm ${LDFLAGS}" \
```

```

BOOTDEV_TYPE=${BOOTDEV_TYPE}
}
# do_install
do_install(){
install -d ${D}/usr/bin/
install -m 0777 ${S}/mcu_messenger_bin ${D}/usr/bin/
if ${@bb.utils.contains('DISTRO_FEATURES','systemd','true','false',d)}; then
install -d ${D}${systemd_unitdir}/system/
install -m 0644 ${B}/mcu_messenger.service ${D}${systemd_unitdir}/system
#/etc/systemd/system/basic.target.wants
install -d ${D}/${sysconfdir}/systemd/system/basic.target.wants/
ln -s /lib/systemd/system/mcu_messenger.service ${D}/${sysconfdir}/systemd/system/basic.target.wants
/mcu_messenger.service
fi
}

```

编译命令：bitbake -s [模块] && bitbake [模块]

(2) 添加源码和service文件

```

yocto/src/apps/spm-base/mcu_messenger$ tree
|— main.cpp
|— Makefile
|— mcu_messenger.service
#mcu_messenger.service
[Unit]
Description=MCU Messenger Daemon #描述
After=android-tools-adbd.service #设置service依赖，启动顺序

[Service]
ExecStartPre=/bin/sleep 15
ExecStart=/usr/bin/mcu_messenger_bin
Restart=always
#StandardOutput=kmsg+console
Type=simple
StandardOutput=tty
StandardError=tty

[Install]
WantedBy=basic.target

```

(2) service配置介绍

[Unit]区块通常是配置文件的第一个区块，用来定义 Unit 的元数据，以及配置与其他 Unit 的关系。它的主要字段如下。

- Description: 简短描述
- Documentation: 文档地址
- Requires: 当前 Unit 依赖的其他 Unit，如果它们没有运行，当前 Unit 会启动失败
- Wants: 与当前 Unit 配合的其他 Unit，如果它们没有运行，当前 Unit 不会启动失败
- BindTo: 与Requires类似，它指定的 Unit 如果退出，会导致当前 Unit 停止运行
- Before: 如果该字段指定的 Unit 也要启动，那么必须在当前 Unit 之后启动
- After: 如果该字段指定的 Unit 也要启动，那么必须在当前 Unit 之前启动
- Conflicts: 这里指定的 Unit 不能与当前 Unit 同时运行
- Condition...: 当前 Unit 运行必须满足的条件，否则不会运行
- Assert...: 当前 Unit 运行必须满足的条件，否则会报启动失败

[Install]通常是配置文件的最后一个区块，用来定义如何启动，以及是否开机启动。它的主要字段如下。

- WantedBy: 它的值是一个或多个 Target，当前 Unit 激活时（enable）符号链接会放入 /etc/systemd/system 目录下面以 Target 名 + .wants 后缀 构成的子目录中
- RequiredBy: 它的值是一个或多个 Target，当前 Unit 激活时，符号链接会放入 /etc/systemd/system 目录下面以 Target 名 + .required 后缀构成的子目录中
- Alias: 当前 Unit 可用于启动的别名
- Also: 当前 Unit 激活（enable）时，会被同时激活的其他 Unit

[Service]区块用来 Service 的配置，只有 Service 类型的 Unit 才有这个区块。它的主要字段如下。

- Type: 定义启动时的进程行为。它有以下几种值。
- Type=simple: 默认值，执行ExecStart指定的命令，启动主进程
- Type=forking: 以 fork 方式从父进程创建子进程，创建后父进程会立即退出
- Type=oneshot: 一次性进程，Systemd 会等当前服务退出，再继续往下执行
- Type=dbus: 当前服务通过D-Bus启动
- Type=notify: 当前服务启动完毕，会通知Systemd，再继续往下执行
- Type=idle: 若有其他任务执行完毕，当前服务才会运行
- ExecStart: 启动当前服务的命令
- ExecStartPre: 启动当前服务之前执行的命令
- ExecStartPost: 启动当前服务之后执行的命令
- ExecReload: 重启当前服务时执行的命令
- ExecStop: 停止当前服务时执行的命令
- ExecStopPost: 停止当前服务之后执行的命令
- RestartSec: 自动重启当前服务间隔的秒数

- **Restart:** 定义何种情况 **Systemd** 会自动重启当前服务，可能的值包括**always**（总是重启）、**on-success**、**on-failure**、**on-abnormal**、**on-abort**、**on-watchdog**
- **TimeoutSec:** 定义 **Systemd** 停止当前服务之前等待的秒数
- **Environment:** 指定环境变量

(3) Unit 管理

立即启动一个服务

```
$ systemctl start qtdemo.service
```

立即停止一个服务

```
$ systemctl stop qtdemo.service
```

重启一个服务

```
$ systemctl restart qtdemo.service
```

杀死一个服务的所有子进程

```
$ systemctl kill qtdemo.service
```

查看一个服务的状态

```
$ systemctl status qtdemo.service
```

服务开机自启动

```
$ systemctl enable qtdemo.service
```

服务开机不自启动

```
$ systemctl disable qtdemo.service
```

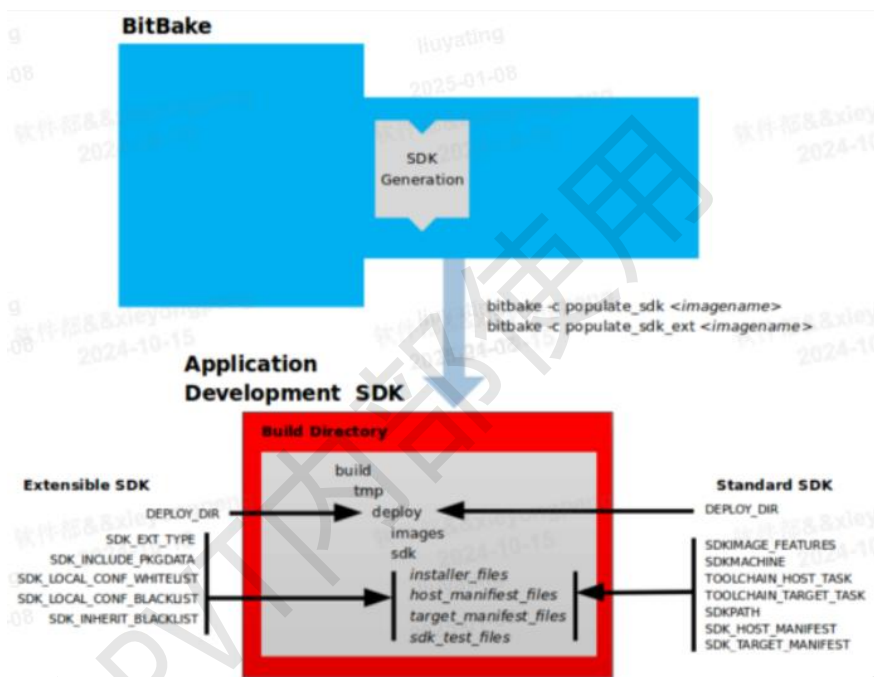
显示服务依赖关系

```
$systemctl list-dependencies [服务名] [--reverse]
```

4.2 YOCTO SDK 应用

(1) 介绍

在Yocto工程添加软件，整个编译过程还是较麻烦的。如果只编译测试工具，可以通过**yocto sdk**达到快速编译的目的，并将编译好工具再加入映像中即可。 以下为**SDK**生成输出工作流。



构建输出的是一组特定形式文件，其中包括SDK自解压安装程序(*.sh)、主机、目标清单文件以及用于SDK测试的文件。SDK详细介绍可参阅 <https://www.yoctoproject.org/docs/2.7/sdk-manual/sdk-manual.html>。

(2) SDK构建及安装

a. 执行命令编译构建SDK

针对8676平台的

```
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-hyp/conf/templates/auto8676p1_64_hyp
source meta/poky/oe-init-build-env
bitbake -c populate_sdk mtk-core-image-auto8676
```

b. 查看编译输出

编译输出位于 build/tmp/deploy/sdk 目录：

```
xieyongpeng@pvt-bianyi1010:~/hy8678/yocto_8/build/tmp/deploy/sdk$ ls
poky-glibc-x86_64-mtk-core-image-auto8678-hyp-aarch64-auto8678p1_64_hyp-toolchain-5.0.host.manifest
poky-glibc-x86_64-mtk-core-image-auto8678-hyp-aarch64-auto8678p1_64_hyp-toolchain-5.0.host.spdx.tar.zst
poky-glibc-x86_64-mtk-core-image-auto8678-hyp-aarch64-auto8678p1_64_hyp-toolchain-5.0.sh
poky-glibc-x86_64-mtk-core-image-auto8678-hyp-aarch64-auto8678p1_64_hyp-toolchain-5.0.target.manifest
poky-glibc-x86_64-mtk-core-image-auto8678-hyp-aarch64-auto8678p1_64_hyp-toolchain-5.0.target.spdx.tar.zst
poky-glibc-x86_64-mtk-core-image-auto8678-hyp-aarch64-auto8678p1_64_hyp-toolchain-5.0.testdata.json
```

其中 *.sh 文件是SDK自解压程序，执行该文件将在指定目录下安装SDK。

c. 安装sdk

在任意目录下创建mysdk目录并记录其绝对路径，然后执行build/tmp/deploy/sdk目录下sdk安装脚本并指定安装目录为[自定义]/mysdk，结果如下：


```

xieyongpeng@pvt-bianyi010:~/hy8678/yocto_8$
xieyongpeng@pvt-bianyi010:~/hy8678/yocto_8$ mkdir mysdk
xieyongpeng@pvt-bianyi010:~/hy8678/yocto_8$ cd mysdk/
xieyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk$ pwd
/aosp/pvt2/hy8678/yocto_8/mysdk
xieyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk$ cd ../build/tmp/deploy/sdk/
xieyongpeng@pvt-bianyi010:~/hy8678/yocto_8/build/tmp/deploy/sdk$ ./poky-glibc-x86_64-mtk-core-image-auto8678-hyp-aarch64-auto8678p1_64_hyp-toolchain-5.0.sh
Poky (Yocto Project Reference Distro) SDK installer version 5.0
=====
Enter target directory for SDK (default: /opt/poky/5.0): /aosp/pvt2/hy8678/yocto_8/mysdk
You are about to install the SDK to "/aosp/pvt2/hy8678/yocto_8/mysdk". Proceed [Y/n]? Y
Extracting SDK.....
.....done
Setting it up...done
SDK has been successfully set up and is ready to be used.
Each time you wish to use the SDK in a new shell session, you need to source the environment setup script e.g.
$ . /aosp/pvt2/hy8678/yocto_8/mysdk/environment-setup-aarch64-poky-linux
$ . /aosp/pvt2/hy8678/yocto_8/mysdk/environment-setup-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi
xieyongpeng@pvt-bianyi010:~/hy8678/yocto_8/build/tmp/deploy/sdk$

```

进入/aosp/pvt2/hy8678/yocto_8/mysdk目录查看，结果如下：

```

xieyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk$ ls -la
总用量 56
drwxr-xr-x 3 xieyongpeng xieyongpeng 4096 10月 15 18:12 .
drwxrwxr-x 12 xieyongpeng xieyongpeng 4096 10月 15 18:10 ..
-rw-r--r-- 1 xieyongpeng xieyongpeng 3952 10月 15 18:11 environment-setup-aarch64-poky-linux
-rw-r--r-- 1 xieyongpeng xieyongpeng 4497 10月 15 18:11 environment-setup-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi
-rw-r--r-- 1 xieyongpeng xieyongpeng 11785 10月 15 18:11 site-config-aarch64-poky-linux
-rw-r--r-- 1 xieyongpeng xieyongpeng 11785 10月 15 18:11 site-config-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi
drwxr-xr-x 4 xieyongpeng xieyongpeng 4096 10月 15 18:11 sysroots
-rw-r--r-- 1 xieyongpeng xieyongpeng 119 10月 15 18:11 version-aarch64-poky-linux
-rw-r--r-- 1 xieyongpeng xieyongpeng 119 10月 15 18:11 version-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi

```

SDK是由一个交叉开发工具链、一组库、头文件以及一个SDK环境设置脚本组成，如上所示sysroots就是最终打包到映像中的文件系统。

d. 配置QT编译环境

当前sdk的QT环境缺失一些配置info，需要修改。

i、mysdk/sysroots/x86_64-pokysdk-linux/usr/bin/qt.conf 定义的路径要跟mysdk路径对应
改前

```

[Paths]
Prefix = /usr
Headers = /usr/include
Libraries = /usr/lib64
ArchData = /usr/lib64
Data = /usr/share
Binaries = /usr/bin
LibraryExecutables = /usr/lib64/libexec
Plugins = /usr/lib64/plugins
Qml2Imports = /usr/lib64/qml
Translations = /usr/share/translations
Documentation = /usr/share/doc
settings = /etc
Examples = /usr/share/examples
Tests = /usr/share/tests
HostBinaries = /usr/bin
HostData = /usr/lib64
HostLibraries = /usr/local/oecore-x86_64/sysroots/aarch64-poky-linux/usr/lib64
HostSpec = linux-oe-g++
TargetSpec = linux-oe-g++
ExternalHostBinaries = /usr/local/oecore-x86_64/sysroots/x86_64-oesdk-linux/usr/bin
Sysroot = /usr/local/oecore-x86_64/sysroots/aarch64-poky-linux
[EffectivePaths]
HostBinaries = /usr/local/oecore-x86_64/sysroots/x86_64-oesdk-linux/usr/bin
HostLibraries = /usr/local/oecore-x86_64/sysroots/x86_64-oesdk-linux/usr/lib
HostData = /usr/local/oecore-x86_64/sysroots/aarch64-poky-linux/usr/lib64
HostPrefix = /usr/local/oecore-x86_64/sysroots/x86_64-oesdk-linux

```

改后

```
[Paths]
Prefix = /usr
Headers = /usr/include
Libraries = /usr/lib64
ArchData = /usr/lib64
Data = /usr/share
Binaries = /usr/bin
LibraryExecutables = /usr/lib64/libexec
Plugins = /usr/lib64/plugins
Qml2Imports = /usr/lib64/qml
Translations = /usr/share/translations
Documentation = /usr/share/doc
Settings = /etc
Examples = /usr/share/examples
Tests = /usr/share/tests
HostBinaries = /usr/bin
HostData = /usr/lib64
HostLibraries = /aosp/pvt2/hy8678/yocto_8/mysdk/sysroots/aarch64-poky-linux/usr/lib64
HostSpec = linux-oe-g++
TargetSpec = linux-oe-g++
ExternalHostBinaries = /aosp/pvt2/hy8678/yocto_8/mysdk/sysroots/x86_64-pokysdk-linux/usr/bin
Sysroot = /aosp/pvt2/hy8678/yocto_8/mysdk/sysroots/aarch64-poky-linux
[EffectivePaths]
HostBinaries = /aosp/pvt2/hy8678/yocto_8/mysdk/sysroots/x86_64-pokysdk-linux/usr/bin
HostLibraries = /aosp/pvt2/hy8678/yocto_8/mysdk/sysroots/x86_64-pokysdk-linux/usr/lib
HostData = /aosp/pvt2/hy8678/yocto_8/mysdk/sysroots/aarch64-poky-linux/usr/lib64
HostPrefix = /aosp/pvt2/hy8678/yocto_8/mysdk/sysroots/x86_64-pokysdk-linux
```

ii、sdk缺mkspecs

复制一个qtdemo的程序下的

yocto\build\tmp\work\aarch64-poky-linux\homeui\1.0\recipe-sysroot\usr\lib64\mkspecs到sdk:

yocto\mysdk\sysroots\aarch64-poky-linux\lib64\mkspecs

(3) SDK使用

进入mysdk目录下，执行 . environment-setup-aarch64-poky-linux命令（注意前面有个 . 号且.号后面有空格）

，结果如下。 注意：每次使用SDK都需要执行一次该脚本（或新打开终端）。

执行 \$CC -v 命令查看编译器详情:

```
xieyongpeng@pvt-bianyi1010:~/hy8678/yocto_8/mysdk$ CC -v
Using built-in specs.
COLLECT_GCC=aarch64-poky-linux-gcc
COLLECT_LTO_WRAPPER=/aosp/pvt2/hy8678/yocto_8/mysdk/sysroots/x86_64-pokysdk-linux/usr/libexec/aarch64-poky-linux/gcc/aarch64-poky-linux/13.2.0/lto-wrap
per
Target: aarch64-poky-linux
Configured with: ../../../../../../work-shared/gcc-13.2.0-r0/gcc-13.2.0/configure --build=x86_64-linux --host=x86_64-pokysdk-linux --target=aarch64-pok
y-linux --prefix=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-pokysdk-linux/usr --exec_prefix=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x
86_64-pokysdk-linux/usr --bindir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-pokysdk-linux/usr/bin/aarch64-poky-linux --sbindir=/usr/local/oe
-sdk-hardcoded-buildpath/sysroots/x86_64-pokysdk-linux/usr/bin/aarch64-poky-linux --libexecdir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-po
kysdk-linux/usr/libexec/aarch64-poky-linux --datadir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-pokysdk-linux/usr/share --sysconfdir=/usr/lo
cal/oe-sdk-hardcoded-buildpath/sysroots/x86_64-pokysdk-linux/etc --sharedstatedir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-pokysdk-linux/c
om --localstatedir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-pokysdk-linux/var --libdir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_
64-pokysdk-linux/usr/lib/aarch64-poky-linux --includedir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-pokysdk-linux/usr/include --oldincludedi
r=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-pokysdk-linux/usr/include --infodir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-pokys
dk-linux/usr/share/info --mandir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-pokysdk-linux/usr/share/man --disable-silent-rules --disable-dep
endency-tracking --with-libtool-sysroot=/aosp/pvt2/hy8678/yocto/build/tmp/work/x86_64-native-sdk-pokysdk-linux/gcc-cross-canadian-aarch64/13.2.0/recipe-
sysroot --with-gnu-ld --enable-shared --enable-languages=c,c++ --enable-threads=posix --enable-multilib --enable-default-pie --enable-c99 --enable-long
-long --enable-symvers=gnu --enable-libstdc++-pch --program-prefix=aarch64-poky-linux --without-local-prefix --disable-install-libiberty --disable-lib
ssp --enable-libitm --enable-lto --disable-bootstrap --with-system-zlib --with-linker-hash-style=gnu --enable-linker-build-id --with-ppl=no --with-clo
g=no --enable-checking=release --enable-headers=c_global --without-isl --with-gxx-include-dir=/not/exist/usr/include/c++/13.2.0 --with-build-time-tool
s=/aosp/pvt2/hy8678/yocto/build/tmp/work/x86_64-native-sdk-pokysdk-linux/gcc-cross-canadian-aarch64/13.2.0/recipe-sysroot-native/usr/aarch64-poky-linux/
bin --with-sysroot=/not/exist --with-build-sysroot=/aosp/pvt2/hy8678/yocto/build/tmp/work/x86_64-native-sdk-pokysdk-linux/gcc-cross-canadian-aarch64/13.
2.0/recipe-sysroot --with-plugin=ld --enable-poison-system-directories --disable-static --enable-nls --with-glibc-version=2.28 --enable-initfini-arr
ay --enable-_cxa_atexit
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 13.2.0 (GCC)
```


执行 qmake -v 命令查看qt编译器详情:

```
xiyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk$ qmake -v
QMake version 3.1
Using Qt version 5.15.13 in /aosp/pvt2/hy8678/yocto_8/mysdk/sysroots/aarch64-poky-linux/usr/lib64
xiyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk$
```

编译Makefile

```
xiyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk/demo/drm$ ls
main.cpp Makefile
xiyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk/demo/drm$ make
aarch64-poky-linux-g++ -fstack-protector-strong -O2 -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/aosp/pvt2/hy8678/yocto_8/mysdk/sysroots/aarch64-poky-linux ./main.cpp -O1 -ldl -pthread -I /aosp/pvt2/hy8678/yocto_7/mysdk/sysroots/aarch64-poky-linux/usr/include/drm -I ./ -ldrm -o drm_demo_bin
xiyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk/demo/drm$ ls
drm_demo_bin main.cpp Makefile
xiyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk/demo/drm$ file drm_demo_bin
drm_demo_bin: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-aarch64.so.1, BuildID[sha1]=5c518b61322b4106e01f9716c564337f79f0a3a0, for GNU/Linux 5.15.0, with debug info, not stripped
```

编译qt pro

```
xiyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk/demo/meter-kde$ ls
com main.cpp meter.pro meter.pro.user presentation presentation.qrc protocol.cpp protocol.h qml qml.qrc qtdemo2.service qtdemo.service
xiyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk/demo/meter-kde$ qmake meter.pro
Info: creating stash file /aosp/pvt2/hy8678/yocto_8/mysdk/demo/meter-kde/.qmake.stash
xiyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk/demo/meter-kde$ make
aarch64-poky-linux-g++ -fstack-protector-strong -O2 -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/aosp/pvt2/hy8678/yocto_8/mysdk/sysroots/aarch64-poky-linux -O2 -std=gnu++11 -Wall -Wextra -D_REENTRANT -fPIC -DOT_DEPRECATED_WARNINGS -DOT_NO_DEBUG -DOT_WIDGETS_LIB -DOT_QUICK_LIB -DOT_GUI_LIB -DOT_QMLMODELS_LIB -DOT_QML_LIB -DOT_NETWORK_LIB -DOT_CORE_LIB -I -I ./ -I ../sysroots/aarch64-poky-linux/usr/include/QtWidgets -I ../sysroots/aarch64-poky-linux/usr/include/QtQuick -I ../sysroots/aarch64-poky-linux/usr/include/QtGui -I ../sysroots/aarch64-poky-linux/usr/include/QtQmlModels -I ../sysroots/aarch64-poky-linux/usr/include/QtQml -I ../sysroots/aarch64-poky-linux/usr/include/QtNetwork -I ../sysroots/aarch64-poky-linux/usr/include/QtCore -I -I ../sysroots/aarch64-poky-linux/usr/lib64/mkspecs/linux-oe-g++ -o readthread.o com/readthread.cpp
xiyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk/demo/meter-kde$ ls
com meter.pro moc_protocol.o presentation.qrc qrc_contacts.cpp qrc_navigation.cpp qrc_telldates.cpp writethread.o
handler.o meter.pro.user moc_readthread.cpp protocol.cpp qrc_contacts.o qrc_navigation.o qrc_telldates.o
main.cpp moc_handler.cpp moc_readthread.o protocol.h qrc_menu.cpp qrc_navigation.o qrc_telldates.o
main.o moc_handler.o moc_writethread.cpp protocol.o qrc_menu.o qrc_navigation.o qtdemo2.service
Makefile moc_predefs.h moc_writethread.o qml qrc_music.cpp qrc_settings.cpp readthread.o
meter moc_protocol.cpp presentation qml.qrc qrc_music.o qrc_settings.o socket.o
xiyongpeng@pvt-bianyi010:~/hy8678/yocto_8/mysdk/demo/meter-kde$ file meter
meter: ELF 64-bit LSB shared object, ARM aarch64, version 1 (GNU/Linux), dynamically linked, interpreter /lib/ld-linux-aarch64.so.1, BuildID[sha1]=61213cfe1836df87b2bac24276002bf17f382ae, for GNU/Linux 5.15.0, with debug info, not stripped
```

4.3 ivi-shell 配置

(1) 介绍

通过配置文件ivi_shell_cfg.pb.txt的方式, 方便配置UI应用的显示信息: 显示大小、尺寸、位置和层级。

目录结构:

yocto/meta/meta-mediatek-mt8676/recipes-graphics/wayland/

- ├── pvtsoft-layer-surfaces
 - ├── add_layer_surface.service //服务
 - ├── ivi_cfg.pb.cpp //protobuf.3.21.5.0生成
 - ├── ivi_cfg.pb.h //protobuf.3.21.5.0
 - ├── ivi_cfg.proto //protobuf数据结构
 - ├── ivi_shell_cfg.pb.txt //配合文件, 如上
 - ├── Makefile
 - ├── pvtsoft-layer-surfaces.cpp //源码
 - └── pvtsoft-layer-surfaces.bb //yocto编译脚本

单编: bitbake -c cleansstate pvtsoft-layer-surfaces && bitbake pvtsoft-layer-surfaces

采用protobuf数据结构管理图层配置, 服务解析/vendor/etc/hyper_android/ivi_shell_cfg.pb.txt

```

display_info {                                #第一个显示屏
    connector_name: "DP-1"                    #必要参数，用于匹配connector_name
    screen_width: 1920,                       #非必要，从weston获取屏参
    screen_height: 720,                      #非必要，从weston获取屏参
    layer_count: 6,                          #非必要，从下面layers个数获取
    layers {                                  #第一个layer
        layer_id: 90500,                     #必要参数，用于创建layer
        opacity: 1.0,                       #必要参数，设置layer透明度
        sourceX: 0,                         #可选，截取layer显示区域x，不填为0
        sourceY: 0,                         #可选，截取layer显示区域y，不填为0
        source_width: 1920,                 #可选，截取layer显示区域w，不填为screen_width
        source_height: 720,                #可选，截取layer显示区域h，不填为 screen_height
        destX: 0,                           #可选，设置layer显示起始位置x，不填为0
        destY: 0,                           #可选，设置layer显示起始位置y，不填为0
        dest_width: 1920,                  #可选，设置layer显示起始位置w，不填为screen_width
        dest_height: 720,                  #可选，设置layer显示起始位置h，不填为 screen_height
        visibility: true,                  #必要参数，设置layer是否显示
        surface_count: 1,                  #非必要，从下面surfaces个数获取
        surfaces {                          #layer: 90500下的第一个surface
            surface_id: 90500500,          #必要参数，用于识别surface，大于0时，用于识别ivi_surface client
            surface_name: "hmi",           #可选，当surface_id为0时，用于识别xdg_surface client
            opacity: 1.0,                  #必要参数，设置surface透明度
            sourceX: 0,                   #可选，截取surface显示区域x，不填为0
            sourceY: 0,                   #可选，截取surface显示区域y，不填为0
            source_width: 1920,           #可选，截取surface显示区域w，不填为client width
            source_height: 720,           #可选，截取surface显示区域h，不填为 client height
            destX: 0,                     #可选，设置surface显示起始位置x，不填为0
            destY: 0,                     #可选，设置surface显示起始位置y，不填为0
            dest_width: 1920,             #可选，设置surface显示起始位置w，不填为client width
            dest_height: 720,             #可选，设置surface显示起始位置h，不填为 client height
            visibility: true,             #必要参数，设置surface是否显示
        }
    }
    surfaces {                                #layer: 90500下的第二个surface，层级是往下越高，即显示在上层
        surface_id: 90500501,
    }
}

```

```

.....
}
}
layers {                #DP-1下的第二个layer，层级是往下越高，即显示在上层
    layer_id: 90700,
    .....
    surfaces {
        surface_id: 90700700,
        .....
    }
}
}
display_info {          #第二个显示屏
    connector_name: "DSI-1"
    .....
}

```

ivi_shell_cfg.pb.txt配置说明：

- 1、动态分配layer， 一个screen上添加几组layer，就分配几组layer，且layer组往后定义的层级越高。
- 2、同一layer下可以绑定多个surface，surface组往后定义的层级越高。但只能定义ivi_surface。需要通过surface_id来设置层级。
- 3、id可以定义十进制和十六进制（0x）
- 4、client可以是ivi_surface和xdg_surface。通过surface_id是否为0区分。
- 5、同一surface可以和多个layer绑定，实现多显。

FAQ:

- 1、client显示超过屏大小是否会闯屏？

答：在ivi模式下不会，设定layer和surface大小大于screen大小，不会显示到第二屏上。只会显示绑定的screen上。

也通过ivi可以设定layer和surface大小，严格规定显示范围。

- 2、ivi-shell是否给出旋转接口

答：否，在ilm_control.h里面没找到到接口，建议在client实现。

五、android

较为开源，暂无详细说明

六、谦川SDK

板卡上目录结构：

```
```bash
sh-3.2# ls -Fl /sys/kernel/debug/nebula_trace/
total 0
-r--r--r-- 1 root root 0 Apr 28 17:42 available_trace # 所有debug trace列表
drwxr-xr-x 2 root root 0 Apr 28 17:42 comcat_link/ # GRT internal
drwxr-xr-x 2 root root 0 Apr 28 17:42 cpu-ipi-delay/ # GRT internal
-r--r--r-- 1 root root 0 Apr 28 17:42 irq-count # 中断数量
drwxr-xr-x 2 root root 0 Apr 28 17:42 irq-delay/ # GRT internal
drwxr-xr-x 2 root root 0 Apr 28 17:42 ktrace/ # Nebula Ktrace, 详情见后续章节
-r--r--r-- 1 root root 0 Apr 28 17:42 pcpu-status # PCPU当前运行状态
drwxr-xr-x 2 root root 0 Apr 28 17:42 top/ # Nebula内部top
-r--r--r-- 1 root root 0 Apr 28 17:42 vcpu-exit-reason # per-VM/per-vcpu退出原因统计
-r--r--r-- 1 root root 0 Apr 28 17:42 vcpu-run-time-accumulate # per-VM总运行时间
drwxr-xr-x 2 root root 0 Apr 28 17:42 vcpu-sched-delay/ # per-VM/per-vcpu调度延迟统计
...
```
```

6.1 物理中断统计(irq_count)

如下图，可以和 Linux /proc/interrupts 对照

```
```bash
sh-3.2# cat irq_count
[Nebula] GIC V3 irq count:
irq-80 : 1
irq-289 : 1104
irq-302 : 10
```
```

irq-343 : 828
irq-352 : 432
irq-361 : 11766
irq-362 : 9388
irq-363 : 11082
irq-374 : 8
irq-375 : 7
irq-377 : 11
irq-380 : 6
irq-381 : 2
irq-428 : 1
irq-449 : 15
irq-468 : 1
irq-479 : 1
irq-482 : 482096
irq-483 : 481341
irq-515 : 241080
irq-631 : 1013
irq-636 : 34402
irq-638 : 72280
irq-640 : 6369
irq-716 : 1
irq-717 : 5
irq-722 : 122
irq-742 : 3
irq-744 : 3
irq-821 : 9
irq-822 : 10
irq-823 : 14
irq-825 : 1
irq-845 : 418143
irq-932 : 3
irq-1023 : 6893
...

6.2 各个pCPU上瞬时正在运行的vCPU的信息 (pcpu_attr)

```
```bash
sh-5.1# cat pcpu_attr
pcpu0 is running on hypervisor
pcpu1 is running on hypervisor
pcpu2, vm0, vcpu2, vcpu_thread_priority:17
pcpu3, vm1, vcpu3, vcpu_thread_priority:16
pcpu4 is running on hypervisor
pcpu5 is running on hypervisor
pcpu6 is running on hypervisor
pcpu7 is running on hypervisor
```
```

6.3 各个vCPU在一段时间内的负载 (top)

各 VM 对应的 vcpu 线程名:

sos:vcpu-N

uos:vcpu-N

```
```bash
sh-5.1# cd top/
sh-5.1# echo 1 > enable
sh-5.1# cat top
PID TID TIME% STATE NAME
2349 2705 12.93 block sos:vcpu-0
2349 2813 11.28 block sos:vcpu-1
2349 2823 8.54 block sos:vcpu-2
2349 2833 6.36 block sos:vcpu-3
2985 3746 6.15 block uos:vcpu-0
2985 4020 4.11 block uos:vcpu-1
2985 4030 3.22 block uos:vcpu-2
2985 4041 2.64 block uos:vcpu-3
2349 2873 1.57 block sos:vcpu-7
2985 4051 1.54 block uos:vcpu-4
2349 3649 1.44 block sos:vhm-sos-thread
2985 3654 0.98 block uos:vhm-uos-thread
2349 2863 0.92 block sos:vcpu-6
```
```


| | | |
|------|------|--|
| 2985 | 4061 | 0.82 block uos:vcpu-5 |
| 2349 | 2843 | 0.62 block sos:vcpu-4 |
| 2349 | 2853 | 0.57 block sos:vcpu-5 |
| 2985 | 3189 | 0.44 block uos:trapbell-handler-0 |
| 2985 | 3194 | 0.42 block uos:trapbell-handler-1 |
| 2985 | 4071 | 0.38 block uos:vcpu-6 |
| 2985 | 4081 | 0.20 block uos:vcpu-7 |
| 2349 | 2715 | 0. 6 run sos:nbl_loading_show |
| 2349 | 3724 | 0. 0 block sos:fastcall-srv |
| 1716 | 1808 | 0. 0 block devhost:misc:initial-thread |
| 1044 | 1047 | 0. 0 block bin/devmgr:bin/devmgr |
| 1044 | 1100 | 0. 0 block bin/devmgr:loader-service |
| 1304 | 1411 | 0. 0 block devhost:sys:initial-thread |
| 1439 | 1536 | 0. 0 block devhost:test:initial-thread |
| 1557 | 1688 | 0. 0 block devhost:root:initial-thread |
| 1716 | 1967 | 0. 0 block devhost:misc:debug-reader |
| 1113 | 1134 | 0. 0 block fshost:initial-thread |
| 1113 | 1189 | 0. 0 block fshost:root-dispatcher |
| 1113 | 1213 | 0. 0 block fshost:loader-service |
| 1233 | 1291 | 0. 0 block crashlogger:initial-thread |
| 1233 | 1573 | 0. 0 block crashlogger:self-dump-thread |
| 1309 | 1385 | 0. 0 block netsvc:initial-thread |
| 2720 | 2771 | 0. 0 block sh:console:initial-thread |
| 1440 | 1617 | 0. 0 block appmgr:initial-thread |
| 2056 | 2076 | 0. 0 block sysmgr:initial-thread |
| 2349 | 2368 | 0. 0 block sos:initial-thread |
| 2349 | 2559 | 0. 0 block sos:io-handler-0 |
| 2349 | 2564 | 0. 0 block sos:io-handler-1 |
| 2349 | 2569 | 0. 0 block sos:trapbell-handler-0 |
| 2349 | 2574 | 0. 0 block sos:trapbell-handler-1 |
| 2349 | 2682 | 0. 0 block sos:pthread_t:0x4a5a8a493000 |
| 2349 | 2699 | 0. 0 block sos:thrd_t:0xc67f198bb000/TLS=0x650 |
| 2349 | 2906 | 0. 0 block sos:thrd_t:0xd400e8387000/TLS=0x650 |
| 2985 | 3004 | 0. 0 block uos:initial-thread |
| 2985 | 3179 | 0. 0 block uos:io-handler-0 |
| 2985 | 3184 | 0. 0 block uos:io-handler-1 |
| 2985 | 3628 | 0. 0 block uos:thrd_t:0x62e42c8c3000/TLS=0x660 |
| ... | | |

6.4 vcpu 累计运行时间 (vcpu_exec_accum)

vCPU的实际运行时间的累加值，单位秒，精确到ms。

```
```bash
sh-5.1# cat vcpu_exec_accum
[Nebula] VM execution time:
time elapsed since system boot: 7123.642
PCPU0:
VM0: 838.782
VM1: 466.687
PCPU1:
VM0: 679.307
VM1: 310.521
PCPU2:
VM0: 559.256
VM1: 223.804
PCPU3:
VM0: 453.999
VM1: 169.751
PCPU4:
VM0: 50.562
VM1: 103.563
PCPU5:
VM0: 54.33
VM1: 72.762
PCPU6:
VM0: 63.133
VM1: 42.438
PCPU7:
VM0: 108.358
VM1: 33.726
```
```

6.5 vmexit原因统计(vcpu_exit_reason)

该值为系统上电后的累计值。

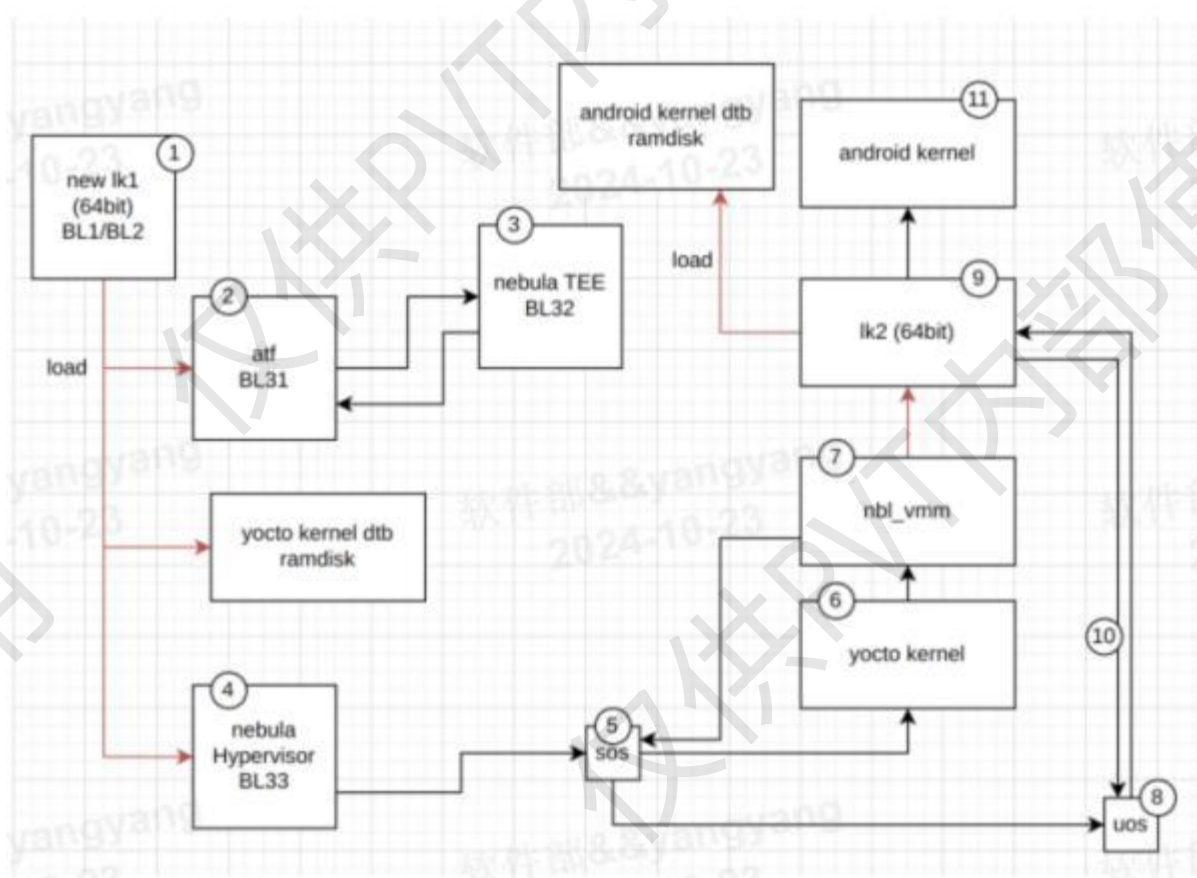

```
```bash
sh-5.1# cat vcpu_exit_reason
[Nebula] VCPU exit reason:
VCPU0:
VM0:
physical-interrupt: 4067174
wfi_wfe_instruction: 9753815
smc_instruction: 1431739
hvc_instruction: 0
system_instruction: 14631183
instruction_abort: 0
data_abort: 219786
unknown: 0
failure: 0
VM1:
physical-interrupt: 1425605
wfi_wfe_instruction: 16490743
smc_instruction: 38869
hvc_instruction: 0
system_instruction: 2227845
instruction_abort: 0
data_abort: 1077634
unknown: 0
failure: 0
VCPU1:
VM0:
physical-interrupt: 2592685
wfi_wfe_instruction: 9365716
smc_instruction: 1536844
hvc_instruction: 0
system_instruction: 9264545
instruction_abort: 0
data_abort: 33467
unknown: 0
failure: 0
VM1:
physical-interrupt: 931260
wfi_wfe_instruction: 15116129
smc_instruction: 8885
```

hvc\_instruction: 0  
system\_instruction: 1956201  
instruction\_abort: 0  
data\_abort: 1119534  
unknown: 0  
failure: 0  
...

## 七、系统

### 7.1 bring up流程

#### 7.1.1 开机流程



1.New LK1加载atf, yocto kernel/dtb/ramdisk, Nebula TEE/Hypervisor。

2.ATF初始化完成，跳TEE。

3.Nebula TEE初始化完成，回ATF。ATF跳Nebula hypervisor。

4.Nebula hypervisor初始化完成，启动sos进程。

5.sos进程启动sos虚拟机，vcpu0跳yocto kernel。

6.yocto kernel开机，进入userspace，启动nbl\_vmm。

7.nbl\_vmm启动，加载LK2镜像到android内存中。发送启动虚拟机命令给sos，启动uos进程。

8.uos进程启动，与sos建立mailbox连线。uos虚拟机启动，vcpu0跳到LK2。

9.LK2加载android kernel/dtb/ramdisk，调用smc接口让uos进程patch device tree

10.uos patch android device tree，加入mailbox咨询

11.lk2启动android kernel

### 7.1.1.1 启动各个阶段log特征

#### 1. preloader/lk阶段

可以在各个os下面 cat /proc/pl\_lk， pl\_lk这个里面保存的是preloader跟lk的log

#### 2. nbl阶段

从“welcome to Nebula”开始，一直到yocto Kernel log起来

#### 3. Yocto Kernel

Yocto Kernel的开始log “Booting Linux on physical CPU”

可以从yocto系统的/data/debuglogger/mobilelog/APLog\_XX/boot\_\_normal/kernel\_log\_XX里面看到

#### 4. nbl\_vmm

nbl\_vmm service的 log可以搜“nbl\_vm\_srv” 这个target

#### 5. Android Kernel

Android Kernel的开始log “Booting Linux on physical CPU”

可以从android系统的/data/debuglogger/mobilelog/APLog\_XX/boot\_\_normal/kernel\_log\_XX里面看到

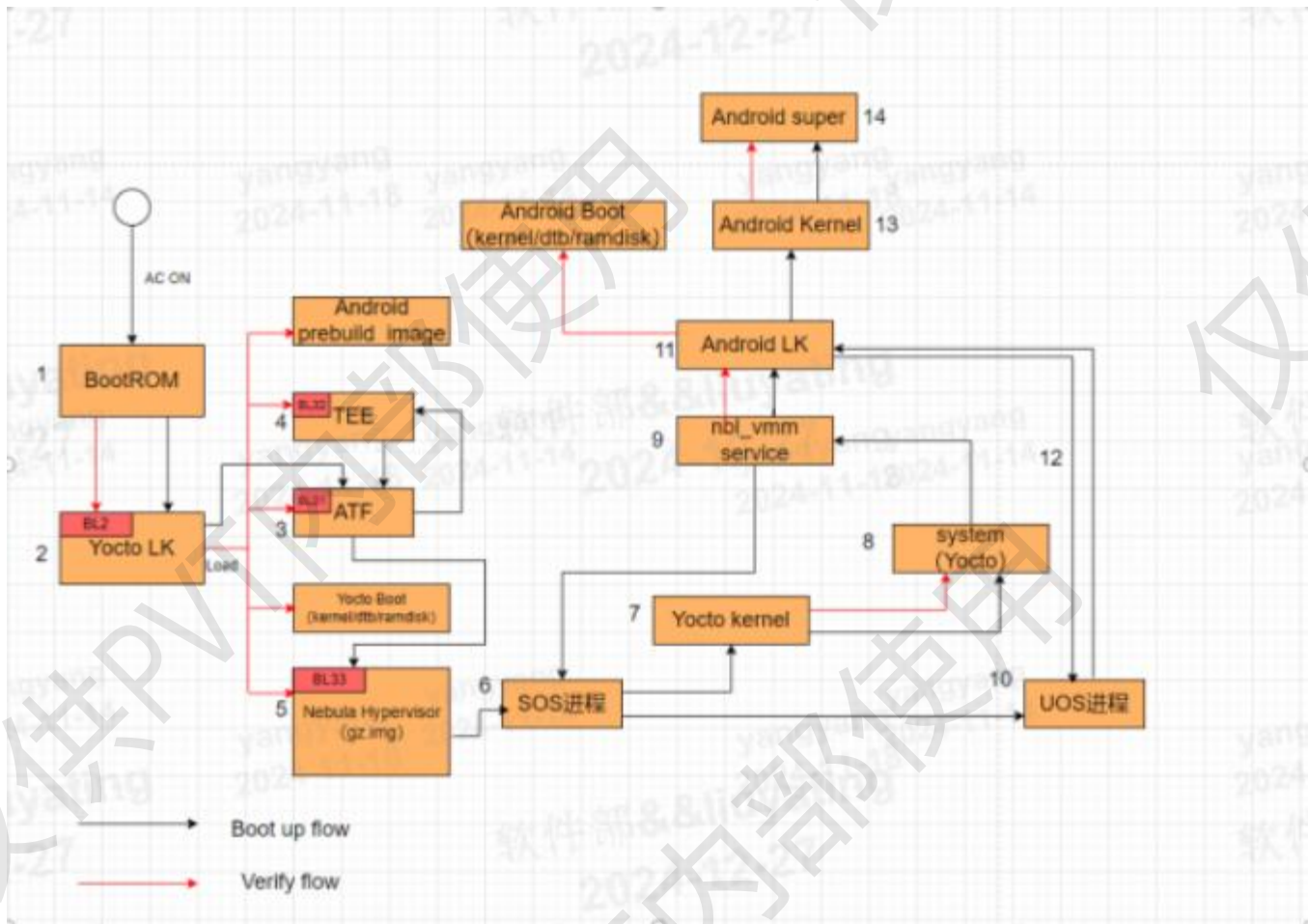
#### 6. Android init的log

Android init的log从“/init as init process” 这个开始

#### 7. Android systemserver的log

Android systemserver的log 从“InitBeforeStartServices”这个开始

### 7.1.2 安全启动流程



1. 上电启动，运行BootRom代码， 校验Yocto LK镜像(bl2.img)，校验通过加载Yocto LK镜像到sram
2. Yocto LK 校验MTK Android Format镜像跟Yocto Format镜像，校验通过后加载到dram，主要有tee， atf， Yocto boot， Nebula Hypervisor镜像
3. 跳转到ATF， ATF初始化，跳转TEE
4. TEE环境初始化完成，跳回ATF
5. ATF跳转到Nebula Hypervisor， 初始化完成后启动SOS进程
6. SOS进程启动SOS虚拟机， vcpu0跳Yocto Kernel
7. Yocto Kernel开机， 校验system镜像，
8. 进入userspace， 启动nbl\_vmm service
9. Nbl\_vmm运行， 校验LK2镜像， 校验成功后加载LK2到Android内存中， 发送启动虚拟机命令给SOS， 启动UOS进程
10. UOS进程启动， 与SOS建立mailbox连接。UOS虚拟机启动， vcpu0跳到LK2
11. Lk2校验Android Boot镜像， 校验成功后加载Android Kernel/dtb/ramdisk， 调用smc接口让UOS进程 patch device tree
12. Uos patch Android device tree， 加入mailbox资讯
13. LK2启动Android Kernel， Kernel运行校验super镜像
14. Android 启动

## 7.2 目录结构介绍

### 7.2.1 代码结构

yocto的代码结构如下图所示



meta目录包含所有yocto元数据。

patch目录包含MediaTek对OSS打的补丁。

prebuilt目录包含预编译好的文件，例如可执行文件，动态库，镜像和工具链等。

src目录包含所有的源代码文件

⋮

#### 7.2.1.1 meta layer

8676 yocto meta layers，主要由Yocto core layer，OSS layer，MediaTek feature layer，MediaTek platform layer 4部分组成，下表详细说明了每个layer包含的内容

Layer	Explanation
meta-mediatek-classes-overlay	覆盖Yocto bbclass，自定义bbclass
meta	包含OpenEmbedded的核心元数据，不包含发行版本，仅提供对模拟器的支持
meta-poky	包含poky发行版本的配置和元数据

meta-yocto-bsp	包含Yocto项目参考硬件的BSP
meta-fileSystems	包含文件系统相关的元数据，例如fuse，owfs，ntfs-3g等
meta-python	包含Python相关的元数据
meta-multimedia	包含多媒体相关的元数据
meta-networking	包含网络相关的元数据
meta-oe	其他共享的OE元数据
meta-clang	包含clang/llvm相关的元数据
meta-qt5	包含qt5相关的元数据
meta-mediatek	MediaTek基础Layer，包含MediaTek写的bbclass和与平台无关的软件包
meta-mediatek-gpl	MediaTek gpl Layer，包含u-boot等软件包
meta-mediatek-gplv2	MediaTek gplv2 layer，包含grep等软件包的GPLv2版本
meta-mediatek-gstreamer	MediaTek Gstreamer layer
meta-mediatek-ml-np	包含MediaTek NeuroPilot相关的软件包
meta-mediatek-mt8676	MT8676 BSP layer，包含MT8676 BSP配置文件和元数据

### 7.2.1.2 src layer

src 主要模块说明如下表所示

layer	Explanation
src/bsp/lk src/bsp/dramk_8676	bootloader源代码
src/kernel/linux/v6.1_mt8676/co_common	Linux Kernel 6.1源代码
src/kernel/linux/v6.1_mt8676/co_device_module	Mediatek Linux kernel 6.1设备驱动
src/kernel/modules	out-of-tree设备驱动

src/apps/spm-base src/apps/atom-base	应用程序源代码
src/connectivity	Wi-Fi 相关的源代码
src/ml/neuropilot	NeuroPilot相关的源代码
src/telephony src/telephonyware	车机tbox（telephony）data/call/sms FW相关的源代码
src/tinysys	tinysys相关的源代码

### 7.2.1.3 常用的文件路径

yocto系统常用的文件路径及应用如下表所示

路径	说明
meta/meta-mediatek-mt8676/conf/machine/auto8676p1_64.conf	项目配置文件
meta/meta-mediatek-mt8676/recipes-bsp/ptgen/files/auto8676p1_64/partition_table_emmc_ab.csv	分区表
meta/meta-mediatek-mt8676/recipes-kernel/linux/ko_order_table/auto8676p1_64/ko_order_table.csv	ko table，决定ko 加载顺序
src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/auto8676p1_64.dts	kernel dts 文件
src/kernel/linux/v6._mt8676/co_device_module/arch/arm64/configs/auto8676p1_64_defconfig	kernel defconfig 文件
src/devtools/dct/dws/mt6897/auto8676p1_64.dws	dws文件，主要用于配置GPIO
meta-mediatek-mt8676/recipes-core/base-files/base-files/auto8676p1_64/fstab	fstab 文件，控制分区挂载
meta-mediatek-mt8676/recipes-auto/images/mtk-core-image-auto8676.bb	MT8676 镜像 bb，用于生成根文件系统

### 7.2.1.4 android常用的目录介绍

路径	说明
----	----



android/vendor/mediatek/proprietary/trustzone/tf-a-2.8/	atf代码
android/vendor/mediatek/proprietary/trustzone/optee/3.18.0/	optee代码
android/kernel-6.1/	linux kernel 6.1源码
android/kernel/kernel_device_modules-6.1/	MediaTek linux kernel 6.1 设备驱动

## 7.3 编译、下载、烧录

### 7.3.1 编译环境配置

基于Ubuntu-20.04

#### 7.3.1.1 安卓编译环境配置

```
sudo apt-get update
```

```
sudo apt-get upgrade
```

```
sudo apt-get install git gnupg flex bison gperf build-essential zip curl g++-multilib gnulib tofrodos python-markdown
libxml2-utils xsltproc openjdk-8-jdk samba cifs-utils vim git-gui libxml-simple-perl git git-gui gitk gnupg flex bison build-
essential unzip curl libxml-simple-perl libtool libmpc-dev libmpfr-dev libgmp-dev flashplugin-installer tcl tk expect
exuberant-ctags cscope ruby ruby-dev xinetd tftpd gcc g++ binutils patch bzip2 flex make gettext pkg-config unzip
zlib1g-dev lib32z1 libc6-dev subversion libncurses5-dev gawk sharutils curl libxml-parser-perl python-yaml ocaml-nox
ocaml ocaml-findlib libssl-dev u-boot-tools device-tree-compiler libfdt-dev texlive minicom libswitch-perl -y
```

先安装上边的编译环境然后继续

```
sudo apt update
```

安装python2.7

```
sudo apt install python2-minimal
```

```
sudo update-alternatives --install /usr/bin/python python /usr/bin/python2.7 1
```

```
sudo update-alternatives --install /usr/bin/python python /usr/bin/python3.8 2
```

```
sudo update-alternatives --config python
```

选择 Python 2.7

```
pvt@pvt-BianYiJi009:/home/pvt1/zhifei/android$ sudo update-alternatives --config python
有 2 个候选项可用于替换 python (提供 /usr/bin/python)。

 选择 路径 优先级 状态

0 /usr/bin/python3.8 2 自动模式
* 1 /usr/bin/python2.7 1 手动模式
2 /usr/bin/python3.8 2 手动模式

要维持当前值[*]请按<回车键>，或者键入选择的编号:
```

安装编译环境

```
sudo apt-get install libx11-dev:i386 libreadline6-dev:i386 libgl1-mesa-dev g++-multilib git flex bison gperf build-essential libncurses5-dev:i386 tofrodos python-markdown libxml2-utils xsltproc zlib1g-dev:i386 dpkg-dev libSDL1.2-dev git-core gnupg flex bison gperf build-essential zip curl zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386 x11proto-core-dev libx11-dev libgl1-mesa-dev libxml2-utils xsltproc unzip m4 lib32z-dev ccache libswitch-perl libxml2-utils libssl-dev libncurses5 gawk wget git-core diffstat unzip texinfo gcc-multilib build-essential chrpath socat cpio python3 python3-pip python3-pexpect xz-utils debianutils iputils-ping python3-git python3-jinja2 libegl1-mesa libsdl1.2-dev pylint3 xterm rpm2cpio zstd qtchooser cppcheck gcovr -y
```

7.3.1.2 yocto编译环境配置

```
sudo apt-get install gawk wget git-core diffstat unzip texinfo gcc-multilib build-essential chrpath socat cpio python3 python3-pip python3-pexpect xz-utils debianutils iputils-ping python3-git python3-jinja2 libegl1-mesa libsdl1.2-dev pylint3 xterm rpm2cpio
sudo apt install qtchooser
```

7.3.2 代码下载

共4个部分

内容	形式	必要性	Note
yocto源码	repo仓	必要	
android源码	repo仓	必要	
downloads	压缩包	建议提前放置，不然编译服务器需可访问外网	Yocto编译时需要在线访问外网拉取的一些依赖库，可在编译前放于yocto根目录
Hypervisor SDK	随yocto的源码释放，客户如果没有谦川【hypervisor厂商】	必要	Yocto路径： /prebuilt/hypervisor/grt

	SDK授权，需要额外 向谦川申请，否则不 会释放		
--	--------------------------------	--	--

### 7.3.3 编译

#### 7.3.3.1 userdebug版本

userdebug版本

Yocto(SOS)

```
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-hyp/conf/templates/auto8676p1_64_hyp
source meta/poky/oe-init-build-env build-sos
bitbake mtk-core-image-auto8676 2>&1 | tee build.log
```

Yocto(UOS Tbox)

```
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-
hyp/conf/templates/auto8676p1_64_uos_tbox
source meta/poky/oe-init-build-env build-tbox
bitbake mtk-core-image-auto8676-uos 2>&1 | tee build.log
```

注：yocto和tbox编译生成的文件夹分别是根目录下的build-sos和build-tbox

Android

```
python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py --run
full_auto8676p1_64_bsp_vm-userdebug 2>&1 | tee build.log
```

#### 7.3.3.2 user版本

user版本

如需编译user版本，保证Yocto也是编译的User版本

Yocto(SOS)

```
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-hyp/conf/templates/auto8676p1_64_hyp
export TARGET_BUILD_VARIANT=user
source meta/poky/oe-init-build-env build-sos
bitbake mtk-core-image-auto8676 2>&1 | tee build.log
```

Yocto(UOS Tbox)

```
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-
hyp/conf/templates/auto8676p1_64_uos_tbox
export TARGET_BUILD_VARIANT=user
source meta/poky/oe-init-build-env build-tbox
bitbake mtk-core-image-auto8676-uos 2>&1 | tee build.log
```

注：yocto和tbox编译生成的文件夹分别是根目录下的build-sos和build-tbox

Android:

```
python vendor/mediatek/proprietary/scripts/releasetools/splt_build_helper.py --run
full_auto8676p1_64_bsp_vm-user --target -j24 2>&1 |
tee build.log
```

### 7.3.4 烧录

分别将Android和Tbox编译生成的部分img拷贝到yocto路径下面统一烧录，具体img名称如图所示：

```
cp ${ALPS}/boot.img ${SOS}/boot.img
cp ${ALPS}/boot-debug.img ${SOS}/boot-debug.img
cp ${ALPS}/dtbo.img ${SOS}/dtbo.img
cp ${ALPS}/userdata.img ${SOS}/userdata.img
cp ${ALPS}/vendor_boot.img ${SOS}/vendor_boot.img
cp ${ALPS}/init_boot.img ${SOS}/init_boot.img
cp ${ALPS}/super.img ${SOS}/super.img
cp ${ALPS}/vbmeta.img ${SOS}/vbmeta.img
cp ${ALPS}/vbmeta_system.img ${SOS}/vbmeta_system.img
cp ${ALPS}/vbmeta_vendor.img ${SOS}/vbmeta_vendor.img
cp ${ALPS}/tee.img ${SOS}/tee.img

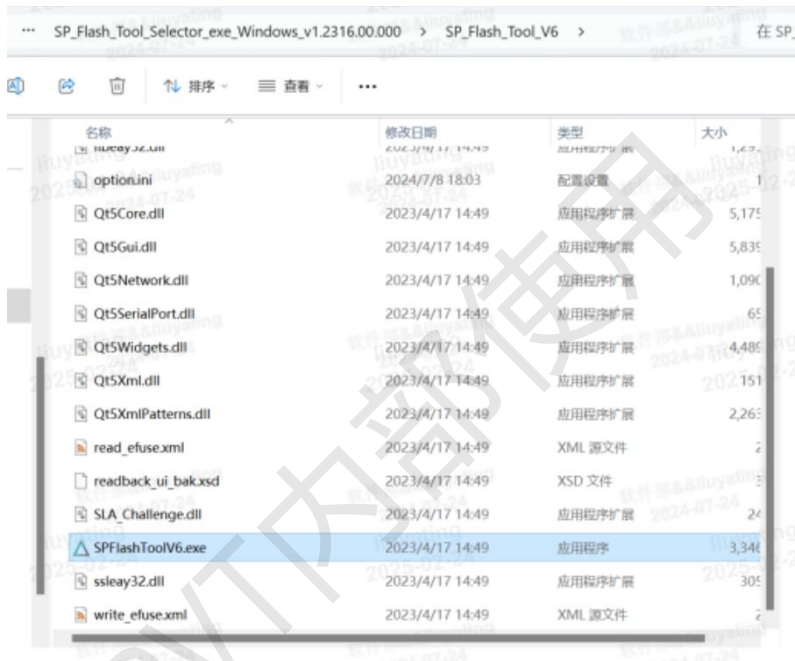
cp ${TBOX}/bl2-tbox.img ${SOS}/bl2-tbox.img
cp ${TBOX}/boot_uos_tbox.img ${SOS}/boot_uos_tbox.img
cp ${TBOX}/system_uos_tbox.img ${SOS}/system_uos_tbox.img
cp ${TBOX}/userdata_uos_tbox.ext4 ${SOS}/userdata_uos_tbox.ext4
```

Yocto load 路径：build-sos/tmp/deploy/images/auto8676p1\_64\_hyp

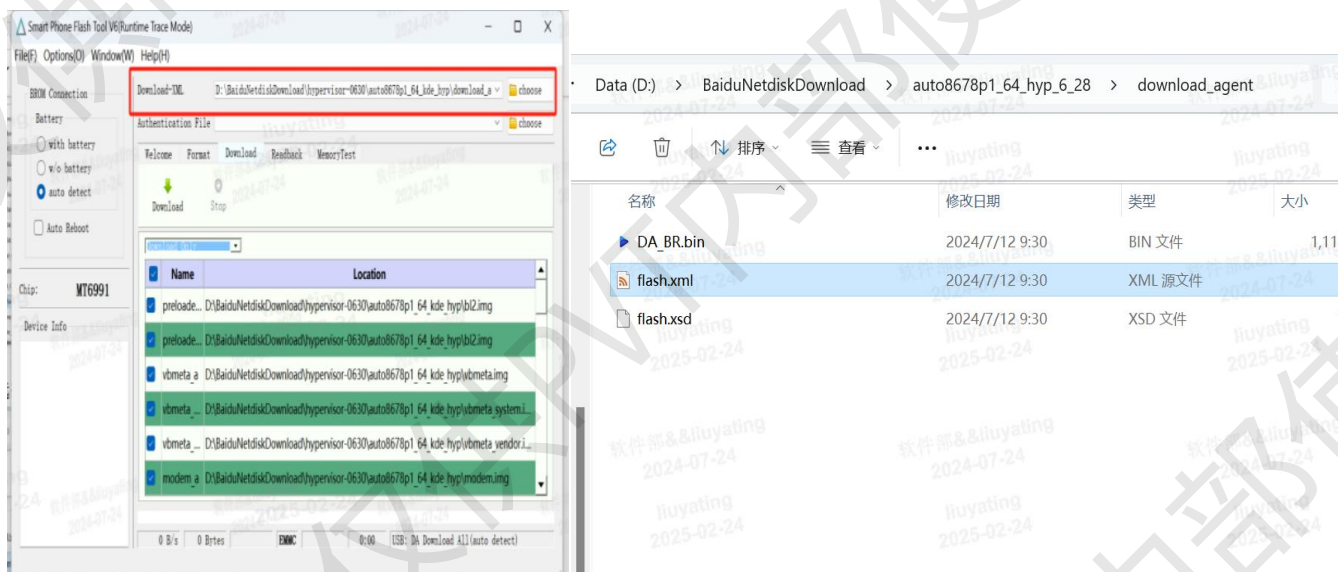
Android load 路径：out/target/product/auto8676p1\_64\_bsp\_vm/merged

Tbox load 路径：build-tbox/tmp/deploy/images/auto8676p1\_64\_uos\_tbox

1、打开SP\_flash\_V6烧录工具

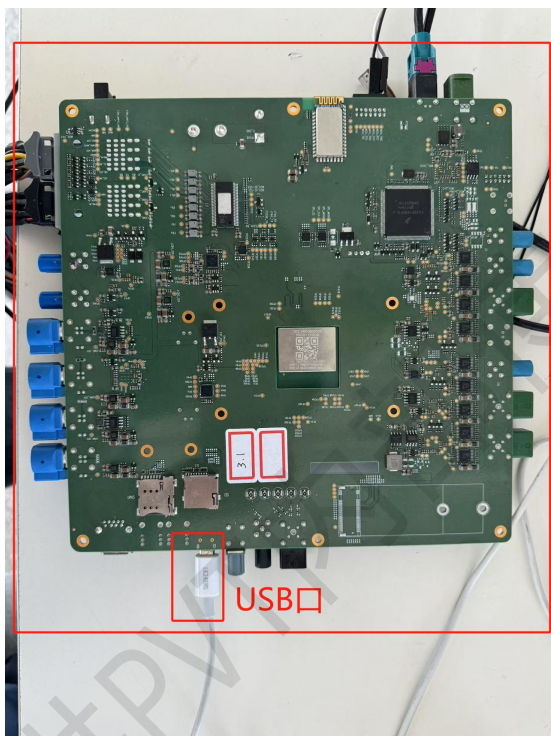


## 2、选择XML文件



## 3、烧录

烧录时，USB线一端连接evb底板的USB口,一端连接PC端USB口



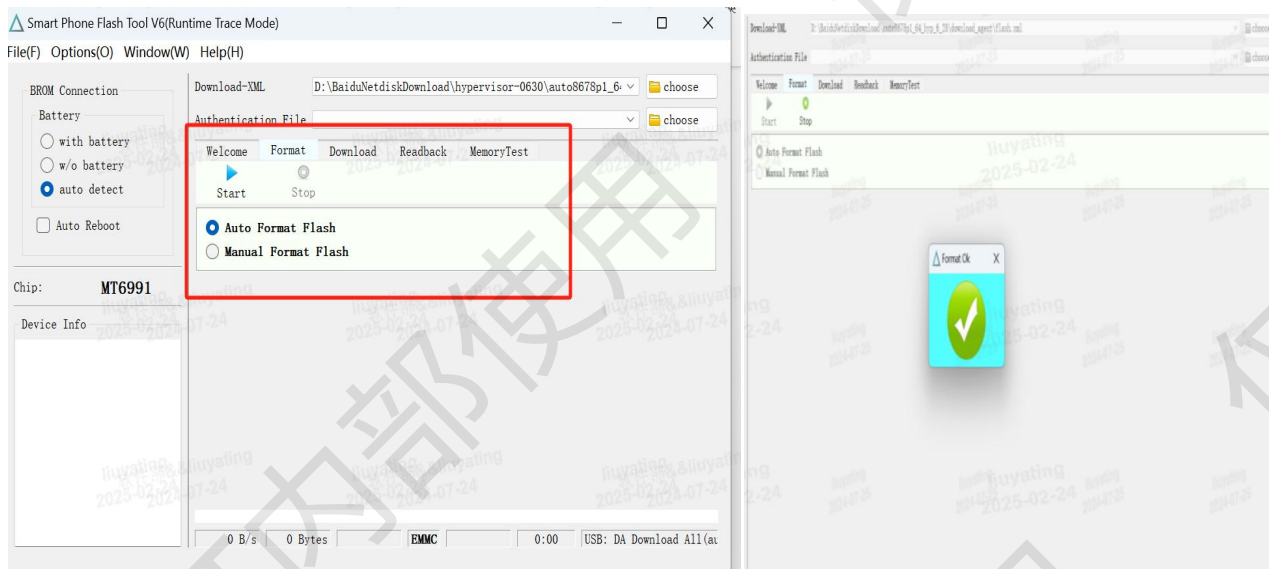
注：烧录新的load,需要对板子进行短接和格式化

①短接方式：使用镊子等工具将COLO和GMD短接

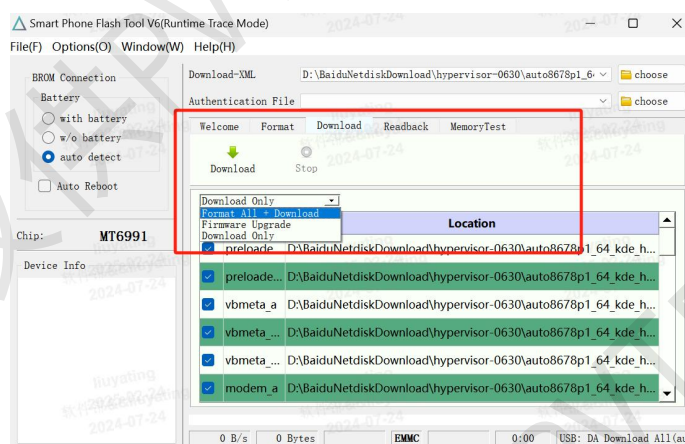


②格式化：选择Format,点击Start,再给板子上电，等底部开始跑进度条的时候再松开短接脚，格式化完成后，成功后会跳出弹窗提示



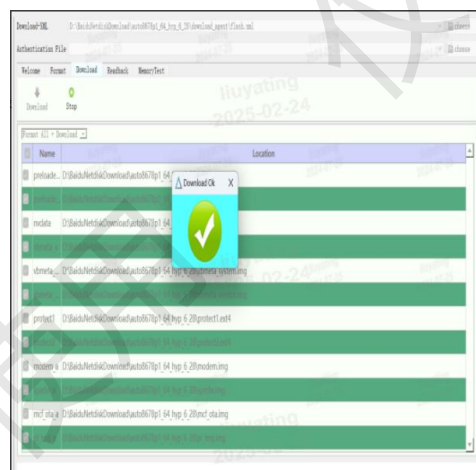


### ③烧录：选择Download,选择Format All+Download模式



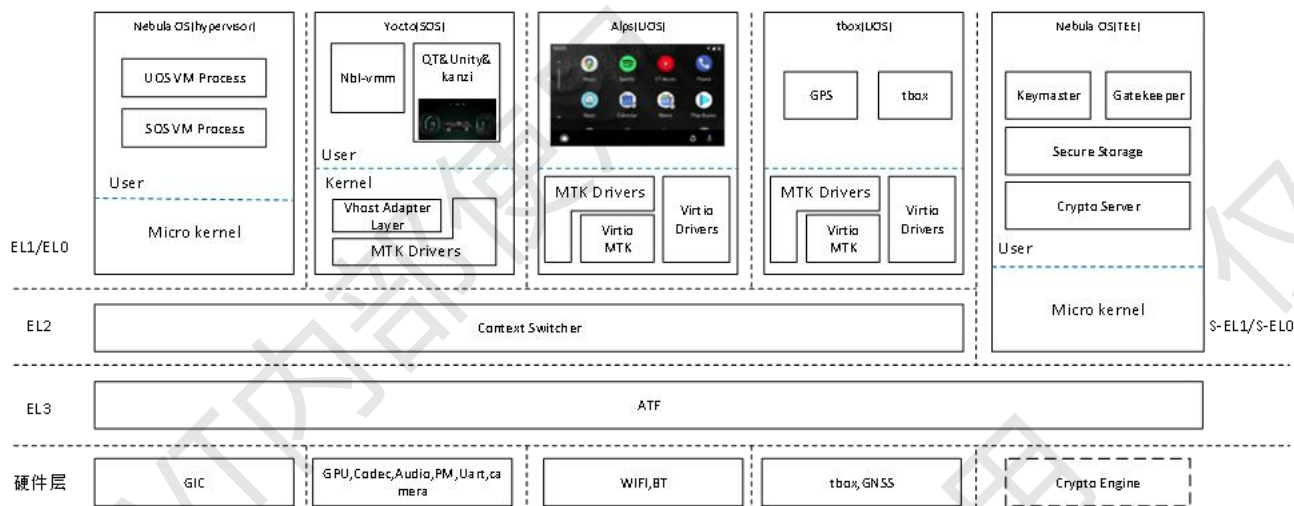
### ④点击Download后，再给板子上电

⑤烧录成功后，有弹窗提示成功，烧录完成，刷机完成后，设备手动断电，再重新上电一次，设备就可以正常开机





## 7.4 系统虚拟化框架



名词说明:

硬件:

硬件按照安全级别由高到低的顺序:TEE>Hypervisor>SOS>UOS, 划分为4个级别。

ATF:

负责Normal World和Secure World间的切换。

Context Switcher :

负责Hypervisor、SOS、UOS间的切换。

Hypervisor:

虚拟机监视器, 负责执行底层的虚拟化操作, 例如映射内存到SOS、UOS, 虚拟CPU调度等, 还有一些设备的共享, 例如GIC。

SOS:

Service Operating System, 特殊的VM, 负责将指定的设备共享给UOS使用, 共享设备只能在SOS或Hypervisor中实现。

UOS:

User Operating System, 可以使用SOS或Hypervisor共享的设备, 还可以以pass-through的方式使用专门分配给它的设备。

TEE:

可信执行环境, 用于执行安全敏感的操作, 例如密钥管理、加解密、指纹识别等。

Nebula OS:

谦川基于Google Fuchsia开发的微内核架构的操作系统, 目前应用于Hypervisor和TEE中。

VM Process:

负责执行VM的创建、配置、启动、销毁等操作, 在VM运行过程中转发UOS的IO请求到SOS进行处理。

nbl-vmm:

负责将SOS管理的设备虚拟化成virtio设备共享给UOS使用。虚拟的设备分为两类：一类是virtio backend仅使用用户空间接口的虚拟设备，例如virtio-gpu, virtio-video；一类是virtio vhost backend需要一部分在内核实现的虚拟设备，例如virtio-iommu, virtio-net。

Vhost Adapter Layer:

Virtio backend在内核中的适配层，用于将内核中实际的设备驱动接口转换为virtio通讯协议支持的接口。

Virtio MTK:

基于virtio标准扩展的用于MTK专有设备的驱动组件及接口，例如virtio-mbox, virtio-cmdq。

Keymaster:

AndroidKeystore在TEE端的backend。

## 7.5 内存

### 7.5.1 OS size控制

8676 pvt\_lla总体内存信息划分，hyper 128M，SOS 6G， android 9G ,tbox 1G。

修改uos内存size 方式

在文件 yocto/meta/meta-mediatek-mt8676-hyp/conf/common/memory-size.inc中修改和配置SOS, TBox和UOS Android的DRAM大小。SOS的内存大小由变量SOS\_DRAM\_SIZE决定，默认是6GB。UOS-TBox的内存大小由变量UOS\_TBOX\_DRAM\_SIZE决定，默认值是1GB。UOS-Android使用剩余的所有内存，不受某个变量的控制。需要注意的是，一些模块会在bootloader申请保留内存，这些内存是从SOS\_DRAM\_SIZE分配的。为了保证SOS正常运行，SOS\_DRAM\_SIZE不建议小于2GB。memory关键参数如下：

```
DRAM BASE, is fixed value, don't modify it
DRAM_BASE = "0x40000000"
SOS Yocto memory size
SOS_DRAM_SIZE = "0x180000000"
UOS Tbox memory size
for 2os, no UOS TBox, so value is 0x0
UOS_TBOX_DRAM_SIZE = "0x0"
for 3os
UOS_TBOX_DRAM_SIZE:auto8676p1_64_hyp_sos = "0x40000000"
```

### 7.5.2 内存layout

mt8676\_pvt\_lla/yocto/src/bsp/lk2/platform/mediatek/mt8678/include/platform/memory\_layout.h

nebula会把内存地址全部映射给SOS，所以防止sos踩到其他os内存地址，即需要在SOS端需要对其进行reserved。

#### SOS端内存视图

gz\_nebula 为hypevisor 内存段

mblock-R[28]	0x7c000000	0x1230000	18624	adsp_image_reserved
mblock-R[42]	0xfa000000	0xe30000	14528	adsp_shared_reserved
mblock-R[6]	0x4d000000	0x10000	64	aee_debug_kinfo
mblock-R[36]	0x8c000000	0x1b60000	28032	ap_md_c_smem
mblock-R[35]	0x8e000000	0x130000	1216	ap_md_nc_smem
mblock-R[30]	0x8ed60000	0x2a0000	2688	apu_apusys-rv_aee-coredum
mblock-R[29]	0x8fa00000	0x600000	6144	apu_apusys-rv_secure
mblock-R[31]	0x8f900000	0x100000	1024	apu_sapu_apurvzgctrlshm
mblock-R[11]	0x48b00000	0x200000	2048	atf-log-reserved
mblock-R[17]	0x48080000	0x200000	2048	BL31-reserved
mblock-R[32]	0xbfff0000	0x10000	64	ccci_tag_mem
mblock-R[46]	0x89e00000	0xd00000	13312	consys_emi_reserved
mblock-R[41]	0x9f17f000	0xc00000	12288	debug_met_res-ram
mblock-R[8]	0x1afff0000	0x2000	8	dramc-rk0
mblock-R[9]	0x43fff0000	0x2000	8	dramc-rk1
mblock-R[15]	0x7d600000	0x800000	8192	emi_isu_buf
mblock-R[12]	0xfd40d000	0x2bf2000	45000	framebuffer
mblock-R[51]	0x1b4c80000	0x2000000	32768	grt_dmabuf_heap
mblock-R[3]	0x1c0000000	0x27ff00000	10484736	gz-guest-rk0
mblock-R[23]	0x53400000	0x8000000	131072	gz_nebula
mblock-R[1]	0x47c00000	0x400000	4096	LK_DT
mblock-R[0]	0x40000000	0x7c00000	126976	LK_KERNEL
mblock-R[2]	0x5b400000	0x4000000	65536	LK_RAMDISK
mblock-R[7]	0x48180000	0x80000	512	log_store_new
mblock-R[27]	0x7d500000	0x100000	1024	MCUPM-reserved

mblock-R[34]	0xd0000000	0x25a0000	38528	md_mem_usage
mblock-R[37]	0xd3000000	0x140000	1280	md_mem_usage
mblock-R[38]	0xd4000000	0x63c0000	102144	md_mem_usage
mblock-R[39]	0xda400000	0x390000	3648	md_mem_usage
mblock-R[40]	0xed000000	0x1000000	16384	md_mem_usage
mblock-R[16]	0x1bff0000	0x100000	1024	me_cmdq_reserved
mblock-R[45]	0x1bfd00000	0x200000	2048	me_GPUUEB_SHARED
mblock-R[33]	0x7ba00000	0x600000	6144	me_GPUmputab_PMA
mblock-R[24]	0xbf000000	0x800000	8192	me_vcp_reserved
mblock-R[5]	0x48170000	0x10000	64	minirdump
mblock-R[52]	0x1b4c7f000	0x1000	4	nbl_sched
mblock-R[49]	0x1b6d00000	0x9000000	147456	nbl_trace
mblock-R[50]	0x1b6c80000	0x80000	512	nebulalog_reserved
mblock-R[14]	0xfcc00000	0x280000	2560	platform-debug_dfdmcu_dum
mblock-R[13]	0xfd000000	0x300000	3072	platform-debug_dfdsoc_dum
mblock-R[43]	0xfd3c0000	0x40000	256	platform-debug_slbc_shared
mblock-R[10]	0x7de00000	0x2200000	34816	platform_mtksmmu_protpgd
mblock-R[4]	0x48090000	0xe0000	896	pstore
mblock-R[26]	0x8f000000	0x898000	8800	reserve-memory-scp_share
mblock-R[53]	0x1b0c7f000	0x4000000	65536	rproc_reserved
mblock-R[55]	0x1a7ff0000	0x4000000	65536	rproc_reserved_sostuous
mblock-R[25]	0xbe700000	0x900000	9216	SCP-reserved
mblock-R[48]	0x8bf00000	0x20000	128	scp_emi_reserved
mblock-R[47]	0x8ab00000	0x1400000	20480	shared-dma-pool_wifi-res
mblock-R[54]	0x1abff0000	0x4000000	65536	sos_tboxuos_rproc_reser
mblock-R[21]	0x50800000	0x100000	1024	spmfw
mblock-R[22]	0x9fd7f000	0x280000	2560	SSPM-reserved
mblock-R[44]	0x9f06f000	0x110000	1088	sspm_ap-shared

mblock-R[18]	0x48a00000	0x100000	1024	system_bl31_dtb
mblock-R[20]	0x71000000	0x3200000	51200	tee-reserved
mblock-R[19]	0xfc600000	0x600000	6144	tee-secmem

#### Tbox

起始地址	size	name
0x1fca00000	0x1400000	shared-dma-pool_wifi-reserve-memory_dma
0x1fbe00000	0xc00000	consys_emi_reserved
0x1fde00000	0x200000	me_GPUEB_SHARED
0x1fec00000	0x1400000	shared-dma-pool_wifi-reserve-memory_dma
0x1fe000000	0xc00000	consys_emi_reserved
0x1e8180000	0x80000	log_store_new
0x1e8080000	0x10000	aee_debug_kinfo
0x1e8170000	0x10000	minirdump
0x1e8090000	0xe0000	pstore
0x89e00000	0xd00000	consys
0x1b4c7f000	0x1000	nbl_sched
0x1b4c80000	0x2000000	grt_dmabuf_heap
0x1b6d00000	0x9000000	nbl_trace

#### Android

起始地址	size	name
0x23de00000	0x200000	me_GPUEB_SHARED
0x23ec00000	0x1400000	shared-dma-pool_wifi-reserve-memory_dma
0x23e000000	0xc00000	consys_emi_reserved
0x228180000	0x80000	log_store_new
0x228080000	0x10000	aee_debug_kinfo
0x228170000	0x10000	minirdump

0x228090000	0xe0000	pstore
0x48b00000	0x200000	atf_log_reserved
0x8ab00000	0x1400000	wifi_vm
0x89e00000	0xd00000	consys
0x1b4c7f000	0x1000	nbl_sched
0x1b4c80000	0x2000000	grt_dmabuf_heap
0x1b6d00000	0x9000000	nbl_trace

### 7.5.3 查询内存信息

1, cat /proc/meminfo

auto8676p1\_64\_bsp\_vm:/ # cat /proc/meminfo

```
MemTotal: 9164208 kB
MemFree: 5148124 kB
MemAvailable: 6119552 kB //free+cached
Buffers: 5416 kB
Cached: 1162592 kB
```

2, Dmabuf

adb shell cat /proc/dma\_heap/all\_heaps > all\_dma\_heaps.txt

此文件为MTK平台独有，在MTK平台

debug时推荐优先查看

adb shell /system/bin/dmabuf\_dump > dmabuf\_dump.txt

此文件会按进程分开列出每个进程的dma

buf持有情况

adb shell dmabuf\_dump -a > dmabuf\_dump\_a.txt

该文件为Android默认节点，其列表形式比较容易查

看Fd ref/ Map ref count持有情况

adb shell dmabuf\_dump -b > dmabuf\_dump\_b.txt

该文件为Android默认节点，主要时按per buffer进行

统计汇总

### 7.5.4 新增reserved 样例

1.kmalloc 不能分配那么大。

2.alloc\_pages MAX\_ORDER为11，也就是最大分配2的11次方的页。

reserverd memory在最开始时候分配好，这块内存不会再分配给kernel用。主要更改devicetree。

```

reserved-memory {
#address-cells = <2>;
#size-cells = <2>;
ranges;
test_reserved: test@1c000000 {
compatible = "test,test-memory";
reg = <0 0x1d7ff0000 0 0x4000000>; //起始内存，内存大小
no-map; //禁止操作系统做逻辑地址映射,开始我没加这句话，ioremap出错了。
};
};

```

验证代码：

```

p_malloc = (char *)ioremap(0x1c000000,0x4000000); //做映射
char *p_tmp = p_malloc;
if(p_malloc == NULL) {
printf("virtual add is error \n");
}else {
for(i = 0; i < 1024; i++) {
*p_tmp++ = i;
}
p_tmp = p_malloc;
for(i = 0; i < 1024; i++) {
printf("%x \n",*p_tmp++);
}
}

```

## 7.6 partition

### 7.6.1 partition layout

#### 1.分区基本介绍



8676的分区表是一个csv文件，每一列的说明如下图所示

表头	说明
Partition_Name	分区名字
Type	分区类型，EXT4 或 Raw data
Size_KB	分区大小，单位是 KB
Region(emmc)	分区所在的 eMMC 物理分区，EMMC_BOOT1、EMMC_BOOT2 或 EMMC_USER
Region(ufs)	分区所在的 UFS 物理分区，UFS_LU0、UFS_LU1 (BOOT) 或 UFS_LU2 (USER)
Reserved	保留分区，Y 或 N
Download	是否烧录，Y 或 N。如果填 Y，Download_File 不能为空。
Download_File	烧录的文件名
OTA_Update	OTA 是否升级
EmptyBoot_Needed	未使用
Operation_Type	BOOTLOADERS: preloader 分区 AUTO: 普通分区 PROTECTED: 固件更新的备份分区 NEEDRESIZE: 用户数据分区 RESERVED: 与 Reserved 字段含义相同，保留分区
Default_Exist	Y 或 N，Y 表示分区存在

双系统分区表如下图所示， android跟yocto的分区都在下面文件中

```
xref: /mt8676_pvt_demo/yocto/meta/meta-mediatek-mt8676-hyp/recipes-bsp/ptgen/files/auto8676p1_64_hyp/partition_table_emmc_ab_hyp.csv
```

```

1 Partition_Name,Type,Size_KB,Region,,Reserved,Download,Download_File,OTA_Update,EmptyBoot_Needed,Operation_Type,Default_Exis
2 ,,,emmc,ufs,,,,,,,,,
3 preloader_a,Raw data,1024,EMMC_BOOT1,UFS_LU0,N,Y,b12.img,Y,N,BOOTLOADERS,Y
4 preloader_b,Raw data,1024,EMMC_BOOT2,UFS_LU1,N,Y,b12.img,Y,N,BOOTLOADERS,Y
5 pgpdt,Raw data,32,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
6 misc,Raw data,512,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
7 para,Raw data,512,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
8 expdb,Raw data,131072,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
9 frp,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
10 nvcfg,EXT4,32768,EMMC_USER,UFS_LU2,N,N,NONE,N,N,PROTECTED,Y
11 nvdata,EXT4,81920,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
12 vbmeta_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,vbmeta.img,Y,N,AUTO,Y
13 vbmeta_system_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,vbmeta_system.img,Y,N,AUTO,Y
14 vbmeta_vendor_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,vbmeta_vendor.img,Y,N,AUTO,Y
15 vbmeta_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
16 vbmeta_system_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
17 vbmeta_vendor_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
18 metadata,Raw data,75744,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
19 protect1,EXT4,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,PROTECTED,Y
20 protect2,EXT4,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,PROTECTED,Y
21 seccfg,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
22 otp,Raw data,44032,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
23 pstore,Raw data,136192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
24 modem_a,Raw data,204800,EMMC_USER,UFS_LU2,N,Y,modem.img,Y,N,AUTO,Y
25 spmfw_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,spmfw.img,Y,N,AUTO,Y
26 mcf_ota_a,EXT4,32768,EMMC_USER,UFS_LU2,N,Y,mcf_ota.img,Y,N,AUTO,Y
27 pi_img_a,Raw data,2048,EMMC_USER,UFS_LU2,N,Y,pi_img.img,Y,N,AUTO,Y
28 dpm_a,Raw data,4096,EMMC_USER,UFS_LU2,N,Y,dpm.img,Y,N,AUTO,Y
29 scp_a,Raw data,12288,EMMC_USER,UFS_LU2,N,Y,scp.img,Y,N,AUTO,Y
30 ccu_a,Raw data,4096,EMMC_USER,UFS_LU2,N,Y,ccu.img,Y,N,AUTO,Y
31 vcp_a,Raw data,6144,EMMC_USER,UFS_LU2,N,Y,vcp.img,Y,N,AUTO,Y
32 sspm_a,Raw data,2048,EMMC_USER,UFS_LU2,N,Y,sspm.img,Y,N,AUTO,Y
33 mcupm_a,Raw data,2048,EMMC_USER,UFS_LU2,N,Y,mcupm.img,Y,N,AUTO,Y
34 gpub_b,Raw data,2048,EMMC_USER,UFS_LU2,N,Y,gpub.img,Y,N,AUTO,Y
35 apusys_a,Raw data,4096,EMMC_USER,UFS_LU2,N,Y,apusys.img,Y,N,AUTO,Y
36 gz_a,Raw data,32768,EMMC_USER,UFS_LU2,N,Y,gz.img,Y,N,AUTO,Y
37 boot_a,Raw data,65536,EMMC_USER,UFS_LU2,N,Y,boot.img,Y,N,AUTO,Y
38 vendor_boot_a,Raw data,65536,EMMC_USER,UFS_LU2,N,Y,vendor_boot.img,Y,N,AUTO,Y
39 init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot.img,Y,N,AUTO,Y
40 dtbo_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,dtbo.img,Y,N,AUTO,Y
41 tee_a,Raw data,5120,EMMC_USER,UFS_LU2,N,Y,tee.img,Y,N,AUTO,Y
42 connsys_bt_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,connsys_bt.img,Y,N,AUTO,Y
43 connsys_wifi_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,connsys_wifi.img,Y,N,AUTO,Y
44 connsys_gnss_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,connsys_gnss.img,Y,N,AUTO,Y
45 logo_a,Raw data,16384,EMMC_USER,UFS_LU2,N,Y,logo.img,Y,N,AUTO,Y
46 audio_dsp_a,Raw data,16384,EMMC_USER,UFS_LU2,N,Y,audio_dsp.img,Y,N,AUTO,Y
47 sec1,Raw data,2048,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
48 proinfo,Raw data,3072,EMMC_USER,UFS_LU2,N,N,NONE,N,N,PROTECTED,Y
49 nvram,Raw data,65536,EMMC_USER,UFS_LU2,N,N,NONE,N,N,BINREGION,Y
50 boot_para,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
51 dram_para,Raw data,26624,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
52 modem_b,Raw data,204800,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
53 spmfw_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
54 mcf_ota_b,EXT4,32768,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
55 pi_img_b,Raw data,2048,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
56 dpm_b,Raw data,4096,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
57 scp_b,Raw data,12288,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
58 ccu_b,Raw data,4096,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
59 vcp_b,Raw data,6144,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
60 sspm_b,Raw data,2048,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
61 mcupm_b,Raw data,2048,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
62 gpub_b,Raw data,2048,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
63 apusys_b,Raw data,4096,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
64 gz_b,Raw data,32768,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
65 boot_b,Raw data,65536,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
66 vendor_boot_b,Raw data,65536,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
67 init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
68 connsys_bt_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
69 connsys_wifi_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
70 connsys_gnss_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
71 dtbo_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
72 logo_b,Raw data,16384,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
73 tee_b,Raw data,11264,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
74 audio_dsp_b,Raw data,16384,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
75 super,Raw data,9437184,EMMC_USER,UFS_LU2,N,Y,super.img,Y,N,AUTO,Y
76 system,EXT4,5242880,EMMC_USER,UFS_LU2,N,Y,system.ext4,N,N,AUTO,Y
77 yocto-userdata,EXT4,3145728,EMMC_USER,UFS_LU2,N,Y,userdata.ext4,N,N,NEEDRESIZE,Y
78 b12-an,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,b12-an.img,N,N,AUTO,Y
79 yocto-boot_a,Raw data,131072,EMMC_USER,UFS_LU2,N,Y,yocto-boot.img,Y,N,AUTO,Y
80 yocto-boot_b,Raw data,131072,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
81 yocto-expdb,Raw data,131072,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
82 userdata EXT4 3145728 EMMC_USER UFS_LU2,N,Y userdata.img N,N,NEEDRESIZE,Y

```



```
83 sgpt,Raw data,32,EMMC_USER,UFS_LU2,Y,N,NONE,N,N,RESERVED,Y
84
```

LLA分区表如下图所示， android跟yocto的分区都在下面文件中

yocto/meta/meta-mediatek-mt8676-hyp/recipes-

bsp/ptgen/files/auto8676p1\_64\_hyp/partition\_table\_emmc\_ab\_hyp.c

```
4 preloader_b,Raw data,1024,EMMC_USER,UFS_LU1,N,Y,bl2_img,Y,N,BOOTLOADERS,Y
5 pgpt,Raw data,32,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
6 misc,Raw data,512,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
7 para,Raw data,512,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
8 expdb,Raw data,131072,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
9 frp,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
10 nvctg,EXT4,32768,EMMC_USER,UFS_LU2,N,N,NONE,N,N,PROTECTED,Y
11 nvdata,EXT4,81920,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
12 vbmeta_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,vbmeta_img,Y,N,AUTO,Y
13 vbmeta_system_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,vbmeta_system_img,Y,N,AUTO,Y
14 vbmeta_vendor_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,vbmeta_vendor_img,Y,N,AUTO,Y
15 vbmeta_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
16 vbmeta_system_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
17 vbmeta_vendor_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
18 metadata,Raw data,75744,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
19 protect1,EXT4,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,PROTECTED,Y
20 protect2,EXT4,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,PROTECTED,Y
21 seccfg,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
22 otp,Raw data,44032,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
23 pstore,Raw data,136192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
24 modem_a,Raw data,204800,EMMC_USER,UFS_LU2,N,Y,modem_img,Y,N,AUTO,Y
25 spmfw_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,spmfw_img,Y,N,AUTO,Y
26 mcf_ota_a,EXT4,32768,EMMC_USER,UFS_LU2,N,Y,mcf_ota_img,Y,N,AUTO,Y
27 pi_img_a,Raw data,2048,EMMC_USER,UFS_LU2,N,Y,pi_img_img,Y,N,AUTO,Y
28 dpm_a,Raw data,4096,EMMC_USER,UFS_LU2,N,Y,dpm_img,Y,N,AUTO,Y
29 scp_a,Raw data,12288,EMMC_USER,UFS_LU2,N,Y,scp_img,Y,N,AUTO,Y
30 ccu_a,Raw data,4096,EMMC_USER,UFS_LU2,N,Y,ccu_img,Y,N,AUTO,Y
31 vcp_a,Raw data,6144,EMMC_USER,UFS_LU2,N,Y,vcp_img,Y,N,AUTO,Y
32 sspm_a,Raw data,2048,EMMC_USER,UFS_LU2,N,Y,sspm_img,Y,N,AUTO,Y
33 mcupm_a,Raw data,2048,EMMC_USER,UFS_LU2,N,Y,mcupm_img,Y,N,AUTO,Y
34 gpueb_a,Raw data,2048,EMMC_USER,UFS_LU2,N,Y,gpueb_img,Y,N,AUTO,Y
35 apusys_a,Raw data,4096,EMMC_USER,UFS_LU2,N,Y,apusys_img,Y,N,AUTO,Y
36 gz_a,Raw data,32768,EMMC_USER,UFS_LU2,N,Y,gz_img,Y,N,AUTO,Y
37 boot_a,Raw data,65536,EMMC_USER,UFS_LU2,N,Y,boot_img,Y,N,AUTO,Y
38 vendor_boot_a,Raw data,65536,EMMC_USER,UFS_LU2,N,Y,vendor_boot_img,Y,N,AUTO,Y
39 init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot_img,Y,N,AUTO,Y
40 dtbo_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,dtbo_img,Y,N,AUTO,Y
41 tee_a,Raw data,5120,EMMC_USER,UFS_LU2,N,Y,tee_img,Y,N,AUTO,Y
42 connsys_bt_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,connsys_bt_img,Y,N,AUTO,Y
43 connsys_wifi_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,connsys_wifi_img,Y,N,AUTO,Y
44 connsys_gnss_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,connsys_gnss_img,Y,N,AUTO,Y
45 logo_a,Raw data,16384,EMMC_USER,UFS_LU2,N,Y,logo_img,Y,N,AUTO,Y
46 audio_dsp_a,Raw data,16384,EMMC_USER,UFS_LU2,N,Y,audio_dsp_img,Y,N,AUTO,Y
47 secl,Raw data,2048,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
48 proInfo,Raw data,3072,EMMC_USER,UFS_LU2,N,N,NONE,N,N,PROTECTED,Y
49 nvram,Raw data,65536,EMMC_USER,UFS_LU2,N,N,NONE,N,N,BINREGION,Y
50 boot_para,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
51 dram_para,Raw data,26624,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
52 modem_b,Raw data,204800,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
53 spmfw_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
54 mcf_ota_b,EXT4,32768,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
55 pi_img_b,Raw data,2048,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
56 dpm_b,Raw data,4096,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
57 ccu_b,Raw data,4096,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
58 vcp_b,Raw data,6144,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
59 sspm_b,Raw data,2048,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
60 mcupm_b,Raw data,2048,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
61 gpueb_b,Raw data,2048,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
62 apusys_b,Raw data,4096,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
63 gz_b,Raw data,32768,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
64 boot_b,Raw data,65536,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
65 vendor_boot_b,Raw data,65536,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
66 init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
67 connsys_bt_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
68 connsys_wifi_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
69 connsys_gnss_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
70 dtbo_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
71 logo_b,Raw data,16384,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
72 tee_b,Raw data,11264,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
73 audio_dsp_b,Raw data,16384,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
74 super,Raw data,9437184,EMMC_USER,UFS_LU2,N,Y,super_img,Y,N,AUTO,Y
75 system,EXT4,5242880,EMMC_USER,UFS_LU2,N,Y,system_ext4,N,N,AUTO,Y
76 yocto-userdata,EXT4,3145728,EMMC_USER,UFS_LU2,N,Y,userdata_ext4,N,N,NEEDRESIZE,Y
77 bl2-an,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,bl2-an_img,N,N,AUTO,Y
78 yocto-boot_a,Raw data,131072,EMMC_USER,UFS_LU2,N,Y,yocto-boot_img,Y,N,AUTO,Y
79 yocto-boot_b,Raw data,131072,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
80 yocto-expdb,Raw data,131072,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
81 bl2-tbox_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,bl2-tbox_img,Y,N,AUTO,Y
82 bl2-tbox_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
83 boot_uos_tbox_a,Raw data,65536,EMMC_USER,UFS_LU2,N,Y,boot_uos_tbox_img,Y,N,AUTO,Y
84 boot_uos_tbox_b,Raw data,65536,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
85 system_uos_tbox,EXT4,2097152,EMMC_USER,UFS_LU2,N,Y,system_uos_tbox_ext4,N,N,AUTO,Y
86 userdata_uos_tbox,EXT4,4194304,EMMC_USER,UFS_LU2,N,Y,userdata_uos_tbox_ext4,N,N,NEEDRESIZE,Y
87 expdb_uos_tbox,Raw data,131072,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
88 userdata,EXT4,3145728,EMMC_USER,UFS_LU2,N,Y,userdata_img,N,N,NEEDRESIZE,Y
```

```
0 sgpt,Raw data,32,EMMC_USER,UFS_LU2,Y,N,NONE,N,N,RESERVED,Y
```

```
work/8676-lla/yocto/meta/meta-mediatek-mt8676-hyp/recipes-bsp/ptgen/files/auto8676p1_64_hyp/partition_table_emmc_ab_hyp.csv [FORM
```

## 2.yocto启动阶段各个镜像存放分区

image	启动阶段	partition	说明
bl2.img	BL2阶段	preloader_a	这个是bootloader（传统的preloader跟lk2阶段合成一个了）
tee.img (包括 ATF镜像 tee os镜像)	BL31阶段 BL32阶段	tee_a	跳转ATF，跟初始化TEE OS环境
gz.img (Nebula hypervisor镜像)	BL33	gz_a	运行hypervisor os
yocto-boot.img (包括 yocto kernel镜像 yocto dtb镜像 yocto ramdisk镜像)	yocto kernel阶段	yocto-boot_a	yocto kernel的启动包括kernel初始化，initramfs初始化
system.ext4	yocto 根文件系统	system	挂载根文件系统，运行systemd服务
bl2-an.img	android BL33 lk	bl2-an	加载android的boot镜像,lk2_an

## 7.6.2 如何新增一个partition

### 7.6.2.1 新增一个partition给SOS

1. 新增分区在yocto/meta/meta-mediatek-mt8676-hyp/recipes-bsp/ptgen/files/auto8676p1\_64\_hyp/partition\_table\_emmc\_ab\_hyp.csv 这个文件中添加， yocto跟Android的分区都在这个文件中添加

新增分区的格式说明请参考7.6.1中的分区表格式每一列的说明，如果需要配置成A/B分区， partition name后面需要加上\_a/\_b

note: 新增分区需要放在sgpt这一行上面，不然编译会报错

2. 怎么挂载这个分区

如果新增的是空的分区没有烧写镜像， ext4格式需要下面脚本里面加给分区Format

xref: /mt8676\_pvt\_11a/yocto/meta/meta-mediatek-mt8676/recipes-kernel/initramfs-platform/files/format

Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download

```
1 #!/bin/sh
2
3 format_enabled() {
4 return 0
5 }
6
7 # $1 device path
8 format_to_ext4() {
9 device_path=$1
10 if [! -b "$device_path"]; then
11 msg "format_ext4: device $device_path not exist, ignore"
12 return
13 fi
14
15 if blkid "$device_path" | grep -q 'TYPE="ext4"'; then
16 msg "format_ext4: skip ext4 partition $device_path"
17 return
18 fi
19
20 msg "format_ext4: format ${device_path} to ext4"
21 mkfs.ext4 "${device_path}"
22 }
23
24 format_run() {
25 format_to_ext4 "/dev/disk/by-partlabel/nvdata"
26 format_to_ext4 "/dev/disk/by-partlabel/protect1"
27 format_to_ext4 "/dev/disk/by-partlabel/protect2"
28 }
29
```

在fstab里面配置挂载：

xref: /mt8676\_pvt\_11a/yocto/meta/meta-mediatek-mt8676-hyp/recipes-core/base-files/base-files/auto8676pl\_64\_hyp/fstab (revision 436b6513)

Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download Search current directory

1	/dev/root	/	auto	defaults	0	0	
2	proc	/proc	proc	defaults	0	0	
3	devpts	/dev/pts	devpts	mode=0620,gid=5	0	0	
4	tmpfs	/run	tmpfs	mode=0755,nosuid,strictatime	0	0	
5	tmpfs	/var	tmpfs	defaults	0	0	
6	tmpfs	/var/volatile	tmpfs	defaults	0	0	
7	/dev/disk/by-partlabel/misc	/misc	tmpfs	defaults	0	0	
8	/dev/disk/by-partlabel/yocto-userdata	/data	ext4	data=ordered,inlinecrypt,noatime,inode_readahead_blks=4096,nodelalloc,noauto_da_alloc	0	2	
9	/dev/disk/by-partlabel/mcf_ota_a	/mnt/vendor/ndota	ext4	ro,data=ordered,noatime,inode_readahead_blks=4096,nodelalloc,noauto_da_alloc	0	0	
10	/dev/disk/by-partlabel/nvdata	/mnt/vendor/nvdata	ext4	data=ordered,noatime,inode_readahead_blks=4096,nodelalloc,noauto_da_alloc	0	0	
11	/dev/disk/by-partlabel/protect1	/mnt/vendor/protect_f	ext4	data=ordered,noatime,inode_readahead_blks=4096,nodelalloc,noauto_da_alloc	0	0	
12	/dev/disk/by-partlabel/protect2	/mnt/vendor/protect_s	ext4	data=ordered,noatime,inode_readahead_blks=4096,nodelalloc,noauto_da_alloc	0	0	
13							

3.启动阶段获取slot信息

启动阶段下面脚本会解析slot信息



xref: /mt8676\_pvt\_1la/yocto/meta/poky/meta/recipes-core/initrdscripts/initramfs-framework/init (revision fb91a493)

Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download  Search ☐ current directory

```
85 if [-d $EFI_DIR]; then
86 mount -t efivarfs none /sys/firmware/efi/efivars
87 fi
88
89 # populate bootparam environment
90 for p in `cat /proc/cmdline`; do
91 if [-n "$quoted"]; then
92 value="$value $p"
93 if [`echo $p | sed -e 's/\\"$/"' != "$p"]; then
94 eval "bootparam_${quoted}=\${value}"
95 unset quoted
96 fi
97 continue
98 fi
99
100 opt=`echo $p | cut -d '=' -f1`
101 opt=`echo $opt | sed -e 'y/_/-/'`
102 if [`echo $p | cut -d '=' -f1` = "$p"]; then
103 eval "bootparam_${opt}=true"
104 else
105 value=`echo $p | cut -d '=' -f2`
106 if [`echo $value | sed -e 's/\\"$/"' != "$value"]; then
107 quoted=$(opt)
108 continue
109 fi
110 eval "bootparam_${opt}=\"\${value}\""
111 fi
112 done
```

## 7.6.2.2 新增一个partition给uos(android)

### 1、在分区表新增2G分区

```
diff --git a/recipes-bsp/ptgen/files/auto8676p1_64_hyp/partition_table_emmc_ab_hyp.csv b/recipes-bsp/ptgen/files/auto8676p1_64_hyp/partition_table_emmc_ab_hyp.csv
index a1bbe1..4724945 100644
--- a/recipes-bsp/ptgen/files/auto8676p1_64_hyp/partition_table_emmc_ab_hyp.csv
+++ b/recipes-bsp/ptgen/files/auto8676p1_64_hyp/partition_table_emmc_ab_hyp.csv
@@ -49,8 +49,7 @@ protoinfo,Raw data,3072,EMMC USER, UFS_LU2,N,N,NONE,N,N,PROTECTED,Y
 nvrw,Raw data,65536,EMMC USER, UFS_LU2,N,N,NONE,N,N,EMBEDDED,Y
 boot_para,Raw data,1024,EMMC USER, UFS_LU2,N,N,NONE,N,N,AUTO,Y
 dram_para,Raw data,26624,EMMC USER, UFS_LU2,N,N,NONE,N,N,AUTO,Y
+android_log,EXT4, 2097152,EMMC USER, UFS_LU2,N,N,NONE,N,N,AUTO,Y
 modem_b,Raw data,204800,EMMC USER, UFS_LU2,N,N,NONE,Y,N,AUTO,Y
 spmfw_b,Raw data,1024,EMMC USER, UFS_LU2,N,N,NONE,Y,N,AUTO,Y
 mcf_ota_b,EXT4,32768,EMMC USER, UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

yocto代码目录: meta/meta-mediatek-mt8676-hyp/recipes-

bsp/ptgen/files/auto8676p1\_64\_hyp/partition\_table\_emmc\_ab\_hyp.csv

```
diff --git a/vendor/file_contexts b/vendor/file_contexts
index 7b9a018..5ba2a7f 100644
--- a/vendor/file_contexts
+++ b/vendor/file_contexts
@@ -635,6 +635,7 @@
/dev/block/platform/mtk-b(misc|ufs) \b\.[0-9]+\.\b(msdc|ufs0) \b/by-name/secro
/dev/block/platform/mtk-b(misc|ufs) \b\.[0-9]+\.\b(msdc|ufs0) \b/by-name/system
/dev/block/platform/mtk-b(misc|ufs) \b\.[0-9]+\.\b(msdc|ufs0) \b/by-name/userdata
+/dev/block/platform/mtk-b(misc|ufs) \b\.[0-9]+\.\b(msdc|ufs0) \b/by-name/android_log
/dev/block/platform/mtk-b(misc|ufs) \b\.[0-9]+\.\b(msdc|ufs0) \b/by-name/cache
/dev/block/platform/mtk-b(misc|ufs) \b\.[0-9]+\.\b(msdc|ufs0) \b/by-name/recovery
/dev/block/platform/mtk-b(misc|ufs) \b\.[0-9]+\.\b(msdc|ufs0) \b/by-name/protect1
@@ -727,6 +728,7 @@
/dev/block/by-name/system([_a-z])?
/dev/block/by-name/tee([12])[_a-z]
/dev/block/by-name/userdata
+/dev/block/by-name/android_log
/dev/block/by-name/vmmeta([_a-z] | system[_a-z])?([_a-z])?
/dev/block/by-name/vcp([_a-z])?
/dev/block/by-name/vendor([_a-z])?
```

### 2、在android端配置file contexts

Android代码目录: device/mediatek/sepolicy/base/vendor/file\_contexts

userdata\_block\_device这个selinux权限是使用data的,也可以根据需要自己创建

### 3、在android端配置fstab挂载

```
diff --git a/mt6897/fstab.in b/mt6897/fstab.in
index 8ba6f37..494e30e 100644
--- a/mt6897/fstab.in
+++ b/mt6897/fstab.in
@@ -168,6 +168,7 @@ DEVPATH(protect2) /mnt/vendor/protect_s ext4 FS_FLAG_COMMIT FSMGR_FLAG_F
DEVPATH(nvdata) /mnt/vendor/nvdata ext4 FS_FLAG_COMMIT FSMGR_FLAG_FMT
#endif
DEVPATH(nvcfg) /mnt/vendor/nvcfg ext4 FS_FLAG_COMMIT FSMGR_FLAG_FMT
+DEVPATH(android_log) /mnt/vendor/android_log ext4 FS_FLAG_COMMIT FSMGR_FLAG_FMT

#ifdef _PERSIST_PARTITION_SUPPORT
DEVPATH(persist) /mnt/vendor/persist ext4 FS_FLAG_COMMIT FSMGR_FLAG_FMT
diff --git a/mt6897/fstab.in.emmc b/mt6897/fstab.in.emmc
index b73a239..d3ec0e9 100644
--- a/mt6897/fstab.in.emmc
+++ b/mt6897/fstab.in.emmc
@@ -168,6 +168,7 @@ DEVPATH(protect2) /mnt/vendor/protect_s ext4 FS_FLAG_COMMIT FSMGR_FLAG_F
DEVPATH(nvdata) /mnt/vendor/nvdata ext4 FS_FLAG_COMMIT FSMGR_FLAG_FMT
#endif
DEVPATH(nvcfg) /mnt/vendor/nvcfg ext4 FS_FLAG_COMMIT FSMGR_FLAG_FMT
+DEVPATH(android_log) /mnt/vendor/android_log ext4 FS_FLAG_COMMIT FSMGR_FLAG_FMT

#ifdef _PERSIST_PARTITION_SUPPORT
DEVPATH(persist) /mnt/vendor/persist ext4 FS_FLAG_COMMIT FSMGR_FLAG_FMT
```

Android代码目录：vendor/mediatek/proprietary/hardware/fstab/mt6897/fstab.in和  
vendor/mediatek/proprietary/hardware/fstab/mt6897/fstab.in.emmc

### 7.6.2.3 Sos目录作为一个partition共享给Android

#### 1、在pb文件配置backends

```
diff --git a/auto8676p1_64_hyp/vm_srv_cfg.pb.txt b/auto8676p1_64_hyp/vm_srv_cfg.pb.txt
index e7a3a42..970f15e 100644
--- a/auto8676p1_64_hyp/vm_srv_cfg.pb.txt
+++ b/auto8676p1_64_hyp/vm_srv_cfg.pb.txt
@@ -48,6 +48,30 @@ vmrecords {
 args: "--xattrmap=map:security.selinux:trusted.virtiofs.:"
 args: "--modcaps=sys_admin"
}
+
+ backends {
+ exec: "virtiofsd"
+ args: "--socket-path=/tmp/virtiofs_socket2"
+ args: "--shared-dir=/mnt/vendor/sos_log"
+ args: "--tag=sos_log"
+ args: "--sandbox=none"
+ args: "--security-label"
+ args: "--selabel=u:object_r:sos_log:s0"
+ args: "--xattrmap=map:security.selinux:trusted.virtiofs.:"
+ args: "--modcaps=sys_admin"
+ }
+ }
```

yocto代码目录：prebuilt/hypervisor/grt\_mt8676/auto8676p1\_64\_hyp/vm\_srv\_cfg.pb.txt

参数修改说明：

- socket-path 后面的参数是递增关系，保证和其他的不一样，/tmp/virtiofs\_socket0，/tmp/virtiofs\_socket1，...等
- shared-dir 后面的参数是yocto这边需要共享的实际目录，如/mnt/vendor/sos\_log
- tag 这个参数是给uos唯一识别的tag号，用于uos系统边mount使用
- 其他参数不变

#### 2、在用于android的lua文件配置相应值



```
diff --git a/auto8676p1_64_hyp/uos_alps_pv8676.lua b/auto8676p1_64_hyp/uos_alps_pv8676.lua
index b4e0d25..321c5ed 100644
--- a/auto8676p1_64_hyp/uos_alps_pv8676.lua
+++ b/auto8676p1_64_hyp/uos_alps_pv8676.lua
@@ -117,6 +117,8 @@ void setFs(std::string type, std::string path)
--]]
uos_config:setFs("media", "/tmp/virtiofs_socket0")
uos_config:setFs("nvdata", "/tmp/virtiofs_socket1")
+uos_config:setFs("sos_log", "/tmp/virtiofs_socket2")
```

yocto代码目录: `prebuilt/hypervisor/grt_mt8676/auto8676p1_64_hyp/uos_alps_pv8676.lua`

参数修改说明:

setFs前面参数为backends里的tag, 后面参数为backends里的socket-path

### 3、在uos系统android边mount

```
diff --git a/init.project.rc b/init.project.rc
index 0eba22b..9b0c490 100644
--- a/init.project.rc
+++ b/init.project.rc
@@ -50,6 +50,14 @@ on post-fs-data
chmod 0771 /mnt/vendor/nvdata
chown root system /mnt/vendor/nvdata
restorecon --recursive --cross-fileSYSTEMS /mnt/vendor/nvdata
+
+ mkdir /mnt/vendor/sos_log
+ mount virtiofs sos_log /mnt/vendor/sos_log
+ restorecon --recursive --cross-fileSYSTEMS /mnt/vendor/sos_log
+
```

android代码目录: `device/mediateksample/auto8676p1_64_bsp_vm/init.project.rc`

先在android边创建一个空目录, 再用mount挂载到这个目录

## 7.6.2.4 Sos目录作为一个partition共享给tbox

### 1、在pb文件配置给tbox的backends

```
diff --git a/auto8676p1_64_hyp/vm_srv_cfg.pb.txt b/auto8676p1_64_hyp/vm_srv_cfg.pb.txt
index e7a3a42..9fdcb2c 100644
--- a/auto8676p1_64_hyp/vm_srv_cfg.pb.txt
+++ b/auto8676p1_64_hyp/vm_srv_cfg.pb.txt
@@ -67,4 +67,22 @@ vmrecords {
 args: "--sandbox=none"
 args: "--modcaps=sys_admin"
}
+
+ backends {
+ exec: "virtiofsd"
+ args: "--socket-path=/tmp/virtiofs_tbox_socket1"
+ args: "--shared-dir=/mnt/vendor/sos_log"
+ args: "--tag=sos_log"
+ args: "--sandbox=none"
+ args: "--modcaps=sys_admin"
+ }
+
```

yocto代码目录: `prebuilt/hypervisor/grt_mt8676/auto8676p1_64_hyp/vm_srv_cfg.pb.txt`

参数修改说明:

--socket-path 后面的参数是递增关系, 保证和其他的不一样, /tmp/virtiofs\_tbox\_socket0,

/tmp/virtiofs\_tbox\_socket1, ...等, 注意virtiofs\_tbox\_socket值不同于给android的

--shared-dir 后面的参数是yocto这边需要共享的实际目录, 如/mnt/vendor/sos\_log

--tag 这个参数是给uos唯一识别的tag号, 用于uos系统边mount使用

--其他参数不变

2、在用于tbox的lua文件配置相应值

yocto代码目录：prebuilt/hypervisor/grt\_mt8676/auto8676p1\_64\_uos\_tbox/uos\_tbox\_pv8676.lua

setFs前面参数为backends里的tag，后面参数为backends里的socket-path

3、在uos系统tbox边 mount

```
diff --git a/recipes-core/base-files/base-files/auto8676p1_64_uos_tbox/fstab b/recipes-core/base-files/base-files/auto8676p1_64_uos_tbox/fstab
index ed21f07..665d2ad 100644
--- a/recipes-core/base-files/base-files/auto8676p1_64_uos_tbox/fstab
+++ b/recipes-core/base-files/base-files/auto8676p1_64_uos_tbox/fstab
@@ -7,6 @@ tmpfs /var tmpfs defaults 0 0
/dev/disk/by-partlabel/protect1 /mnt/vendor/protect_f ext4 data=ordered,noatime,inode_readahead_blks=4096,nodealloc,noauto_da_alloc 0 0
/dev/disk/by-partlabel/protect2 /mnt/vendor/protect_s ext4 data=ordered,noatime,inode_readahead_blks=4096,nodealloc,noauto_da_alloc 0 0
tbox_nvdata /mnt/vendor/nvdata virtiofs defaults 0 0
+sos_log /mnt/vendor/sos_log virtiofs defaults 0 0
```

yocto代码目录：meta/meta-mediatek-mt8676-hyp/recipes-core/base-files/base-

files/auto8676p1\_64\_uos\_tbox/fstab

## 7.6.3 partition debug

(1).分区的start addr跟len信息

可以在生成的MT6897\_Android\_scatter.xml文件中看到start address跟len，如下图所示

```
<is_upgradable>false</is_upgradable>
<empty_boot_needed>false</empty_boot_needed>
<combo_partsize_check>false</combo_partsize_check>
<reserve>0x00</reserve>
</partition_index>
<partition_index name="SYS79">
 <partition_name>userdata</partition_name>
 <file_name>userdata.img</file_name>
 <is_download>true</is_download>
 <type>EXT4_IMG</type>
 <linear_start_addr>0x4c130000</linear_start_addr>
 <physical_start_addr>0x4c130000</physical_start_addr>
 <partition_size>0xc000000</partition_size>
 <region>UFS_LU2</region>
 <storage>HW_STORAGE_UFS</storage>
 <boundary_check>true</boundary_check>
 <is_reserved>false</is_reserved>
 <operation_type>NEEDRESIZE</operation_type>
 <is_upgradable>false</is_upgradable>
 <empty_boot_needed>false</empty_boot_needed>
 <combo_partsize_check>false</combo_partsize_check>
 <reserve>0x00</reserve>
</partition_index>
<partition_index name="SYS80">
 <partition_name>demo_a</partition_name>
 <file_name>demo.img</file_name>
 <is_download>true</is_download>
 <type>NORMAL_ROM</type>
 <linear_start_addr>0x58130000</linear_start_addr>
 <physical_start_addr>0x58130000</physical_start_addr>
 <partition_size>0x200000</partition_size>
 <region>UFS_LU2</region>
 <storage>HW_STORAGE_UFS</storage>
 <boundary_check>true</boundary_check>
 <is_reserved>false</is_reserved>
 <operation_type>UPDATE</operation_type>
 <is_upgradable>false</is_upgradable>
 <empty_boot_needed>false</empty_boot_needed>
 <combo_partsize_check>false</combo_partsize_check>
 <reserve>0x00</reserve>
</partition_index>
<partition_index name="SYS81">
```

lk中有关分区的信息log如下

```

mbr partition table dump:
 0: status 0x0, type 0xee, start 0x1, len 0x1dcd7ff
 1: status 0x0, type 0x0, start 0x0, len 0x0
 2: status 0x0, type 0x0, start 0x0, len 0x0
 3: status 0x0, type 0x0, start 0x0, len 0x0
found GPT
name:misc uuid_str is 00000000-39c2-4488-9bb0-00cb43c9ccd4
name:para uuid_str is 00000001-39c2-4488-9bb0-00cb43c9ccd4
name:expdb uuid_str is 00000002-39c2-4488-9bb0-00cb43c9ccd4
name:frp uuid_str is 00000003-39c2-4488-9bb0-00cb43c9ccd4
name:nvcfg uuid_str is 00000004-39c2-4488-9bb0-00cb43c9ccd4
name:nvdata uuid_str is 00000005-39c2-4488-9bb0-00cb43c9ccd4
name:vbmeta_a uuid_str is 00000006-39c2-4488-9bb0-00cb43c9ccd4
name:vbmeta_system_a uuid_str is 00000007-39c2-4488-9bb0-00cb43c9ccd4
name:vbmeta_vendor_a uuid_str is 00000008-39c2-4488-9bb0-00cb43c9ccd4
name:vbmeta_b uuid_str is 00000009-39c2-4488-9bb0-00cb43c9ccd4
name:vbmeta_system_b uuid_str is 0000000a-39c2-4488-9bb0-00cb43c9ccd4
name:vbmeta_vendor_b uuid_str is 0000000b-39c2-4488-9bb0-00cb43c9ccd4
name:metadata uuid_str is 0000000c-39c2-4488-9bb0-00cb43c9ccd4
name:protect1 uuid_str is 0000000d-39c2-4488-9bb0-00cb43c9ccd4
name:protect2 uuid_str is 0000000e-39c2-4488-9bb0-00cb43c9ccd4
name:seccfg uuid_str is 0000000f-39c2-4488-9bb0-00cb43c9ccd4
name:otp uuid_str is 00000010-39c2-4488-9bb0-00cb43c9ccd4
name:pstore uuid_str is 00000011-39c2-4488-9bb0-00cb43c9ccd4
name:modem_a uuid_str is 00000012-39c2-4488-9bb0-00cb43c9ccd4
name:spmfw_a uuid_str is 00000013-39c2-4488-9bb0-00cb43c9ccd4
name:mcf_ota_a uuid_str is 00000014-39c2-4488-9bb0-00cb43c9ccd4
name:pi_img_a uuid_str is 00000015-39c2-4488-9bb0-00cb43c9ccd4
name:dpm_a uuid_str is 00000016-39c2-4488-9bb0-00cb43c9ccd4
name:scp_a uuid_str is 00000017-39c2-4488-9bb0-00cb43c9ccd4
name:ccu_a uuid_str is 00000018-39c2-4488-9bb0-00cb43c9ccd4
name:vcp_a uuid_str is 00000019-39c2-4488-9bb0-00cb43c9ccd4
name:sspm_a uuid_str is 0000001a-39c2-4488-9bb0-00cb43c9ccd4
name:mcupm_a uuid_str is 0000001b-39c2-4488-9bb0-00cb43c9ccd4
name:gpueb_a uuid_str is 0000001c-39c2-4488-9bb0-00cb43c9ccd4
name:apusys_a uuid_str is 0000001d-39c2-4488-9bb0-00cb43c9ccd4
name:gz_a uuid_str is 0000001e-39c2-4488-9bb0-00cb43c9ccd4
name:boot_a uuid_str is 0000001f-39c2-4488-9bb0-00cb43c9ccd4
name:vendor_boot_a uuid_str is 00000020-39c2-4488-9bb0-00cb43c9ccd4
name:init_boot_a uuid_str is 00000021-39c2-4488-9bb0-00cb43c9ccd4
name:dtbo_a uuid_str is 00000022-39c2-4488-9bb0-00cb43c9ccd4
name:tee_a uuid_str is 00000023-39c2-4488-9bb0-00cb43c9ccd4
name:connsys_bt_a uuid_str is 00000024-39c2-4488-9bb0-00cb43c9ccd4
name:connsys_wifi_a uuid_str is 00000025-39c2-4488-9bb0-00cb43c9ccd4
name:connsys_gnss_a uuid_str is 00000026-39c2-4488-9bb0-00cb43c9ccd4
name:logo_a uuid_str is 00000027-39c2-4488-9bb0-00cb43c9ccd4
name:audio_dsp_a uuid_str is 00000028-39c2-4488-9bb0-00cb43c9ccd4
name:secl uuid_str is 00000029-39c2-4488-9bb0-00cb43c9ccd4
name:proinfo uuid_str is 0000002a-39c2-4488-9bb0-00cb43c9ccd4
name:nvram uuid_str is 0000002b-39c2-4488-9bb0-00cb43c9ccd4
name:boot_para uuid_str is 0000002c-39c2-4488-9bb0-00cb43c9ccd4
name:dram_para uuid_str is 0000002d-39c2-4488-9bb0-00cb43c9ccd4
name:modem_b uuid_str is 0000002e-39c2-4488-9bb0-00cb43c9ccd4
name:spmfw_b uuid_str is 0000002f-39c2-4488-9bb0-00cb43c9ccd4

```

## (2).dump ufs上分区的原始数据

使用flashtool 可以回读ufs分区的数据， 分区的start address 跟length可以从上面的xml中找到， 最后点击 read back就可以把分区信息写到后面的路径文件中





# 7.7.1 SOS & UOS 间通信

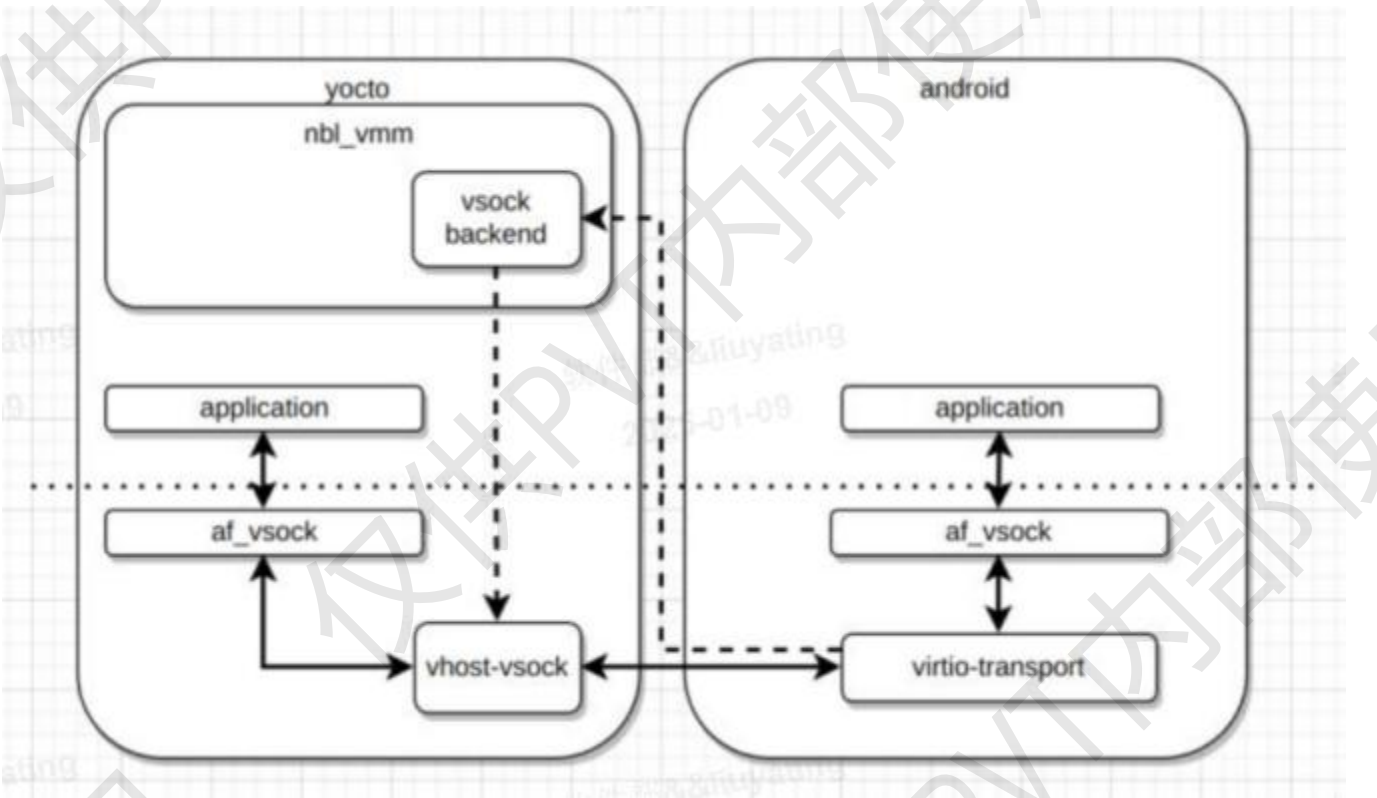
## 7.7.1.1 vsock 使用方法

### 1、介绍

virtio-socket（简称vsock）是一种虚拟套接字，用于在不同虚拟机或虚拟机和宿主机之间提供高性能、可靠的通信。现阶段用在GPS的虚拟化、camera虚拟化、modem虚拟化和android display虚拟化上。

传输速率如下表

方向	A->L	A->T	L->A	L->T	T->A	T->L
吞吐量(Gbps)	3.98	2.54	1.66	2.17	8.33	1.82



应用程序，使用<cid,port>作为套接字地址层，支持套接字APIAF\_VSOCK地址族，实现vsock核心传输，在客户机和主机之间传输数据。传输层是最需要通信

的，其他三个层只需要在内核中实现标准接口。顾名思义，传输用于在客户机和主机之间传输数据，就像网卡在本地和远程套接字之间传输数据一样。根据数据的流向有两种传输方式。

G2H: guest->host传输，它们运行在客户机中，客户机vsock网络协议使用它与主机通信；

H2G: host→guest传输，它们运行在主机中，主机vsock网络协议使用它与客户机通信。

指定类型操作：

SOCK\_STREAM: Provides sequenced, reliable, two-way, connection-based byte streams. An out-of-band data transmission mechanism may be supported.

SOCK\_DGRAM: Supports datagrams (connectionless, unreliable messages of a fixed maximum length).

## 2、接口介绍

- (1) int socket (int af, int type, int protocol) : 创建一个vsock socket
- (2) int bind (int sock, struct sockaddr \*addr, socklen\_t addrlen) : 将socket绑定到本地地址
- (3) int connect (int sock, struct sockaddr \*serv\_addr, socklen\_t addrlen) : 将socket连接到远程地址
- (4) int listen(int sock,int backlog): 监听进来的连接
- (5) int accept (int sock, struct sockaddr \*addr, socklen\_t \*addrlen): 接收来自对端的连接, 并获取其文件

描述符

- (6) ssize\_t send(int fd, const void\* buf, size\_t n, int flags): 向对端发送信息
- (7) ssize\_t recv(int fd, void\* buf, size\_t n, int flags): 接收来自对端的信息
- (8) int close(int fd): 关闭socket

## 3、示例

UOS&SOS 都可以作为server或者client

注: CID值:

sos 2

android 3

tbox 4

- (1) server



```

// Create a vsock socket with stream type and default protocol
int s = socket(AF_VSOCK, SOCK_STREAM, 0); //
// Define a response string to send to the client
const char *response = "This is a custom string from yocto server.";
// Initialize a sockaddr_vm structure with the given port and any CID
sockaddr_vm addr;
memset(&addr, 0, sizeof(struct sockaddr_vm));
addr.svm_family = AF_VSOCK;
addr.svm_port = port; //
addr.svm_cid = VMADDR_CID_ANY;
// Bind the socket to the local address
bind(s, (sockaddr *)&addr, sizeof(struct sockaddr_vm));
// Listen for incoming connections
listen(s, 0); //
// Initialize a sockaddr_vm structure for the peer address
sockaddr_vm peer_addr;
socklen_t peer_addr_size = sizeof(struct sockaddr_vm);
// Accept a connection from the peer and get its file descriptor
int peer_fd = accept(s, (sockaddr *)&peer_addr, &peer_addr_size); //
// Define a buffer to receive the message from the client
char buf[64];
// Receive the message from the client
ssize_t received_len = recv(peer_fd, &buf, 64, 0); //
// Log the received message
GTEST_LOG_ (INFO) << "Server received: " << buf;
// Send the response string to the client
send(peer_fd, response, strlen(response), 0); //
// Log the sent response
GTEST_LOG_ (INFO) << "Server sent: " << response;
// Close the peer socket and the server socket
close(peer_fd);
close(s);

```

(2) client

```

int s = socket(AF_VSOCK, SOCK_STREAM, 0); //
sockaddr_vm addr;
memset(&addr, 0, sizeof(struct sockaddr_vm));
addr.svm_family = AF_VSOCK;
addr.svm_port = port; //
addr.svm_cid = cid;
connect(s, (sockaddr*)&addr, sizeof(struct sockaddr_vm)); //server
const char *message = "Hello, world!";
size_t message_len = strlen(message);
send(s, message, message_len, 0); //
char buf[64];
ssize_t received_len = recv(s, &buf, 64, 0); //
close(s);

```

## 7.7.1.2 vnet

### 7.7.1.2.1 名词解释

**Virtio:** 一种跨虚拟机的通信机制。

**Virtio-net协议:** 一种基于virtio的协议，可以虚拟出一个以太网卡，让虚拟机间可透过tcp/ip通信。

**Virtio-vsock协议:** 一种基于virtio的协议，可以用于跨虚拟机通信。

**Vhost:** 可将virtio后端对virtio-queue的控制权转让给内核的机制。让内核模块与前端驱动直接通信。

**Socket接口:** 一种可用于网路通信的接口。

**Tap:** layer 2虚拟网路设备，可以虚拟出一个以太网卡，并提供可以注入/接收以太网讯框（ethernet frame）的接口，供其他模块使用。

**SNAT:** 一种网络地址转换（NAT）技术，用于修改数据包的源IP地址。SNAT通常用于将内部网络的私有IP地址转换为公共IP地址，以便在互联网中进行通信

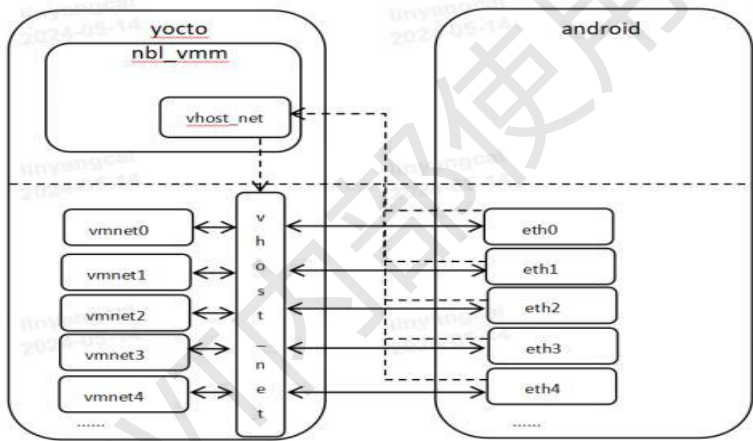
### 7.7.1.2.2 OS间虚拟化通路搭建

#### 1、概述

虚拟网路设备目的在于提供yocto/android虚拟机沟通的机制，有两种类型vnet与vsock。这两种机制都能让yocto/android虚拟机通信，差别在vnet会经过 tcp/ip网路堆叠，而vsock不会。因此，vsock更高效（因为没有protocol overhead），而vnet更通用（因为走tcp/ip，相容于网路应用，也可用来访问外网）。目前nebula hypervisor的设计是modem放在yocto端，因此android端是没有网路可用的。因此会使用vnet在android端建立虚拟以太网卡，然后在yocto端在把收到的讯框转发给modem，间接让android虚拟机也可以透过modem上网。

Vsock主要用于内网通信，yocto/android虚拟机上如果有些系统服务想要共享一些讯息，则可以使用vsock来达成。

2、架构流程概述



在虚拟网路应用场景，为了效能考量，直接使用了vhost机制，让yocto与android内核可以直接通信。

3、虚拟通路的配置

目前通路的配置可以修改lua文件进行配置，可以根据需要增删通路数量及ip等配置

路径：yocto/prebuilt/hypervisor/grt\_mt8676/uos\_alps\_pv8676\_mix.lua

详细配置如下：

```
-- vmnet
--[
void setVmnet(std::string id, std::string ip, std::string netmask)
--]
uos_config:setVmnet("vmnet0", "192.168.1.1", "255.255.255.0");
uos_config:setVmnet("vmnet-vccmni0", "172.17.0.1", "255.255.255.0")
uos_config:setVmnet("vmnet-vccmni1", "172.17.1.1", "255.255.255.0")
uos_config:setVmnet("vmnet-vccmni2", "172.17.2.1", "255.255.255.0")
uos_config:setVmnet("vmnet-vccmni3", "172.17.3.1", "255.255.255.0")

```

其中vmnet0这一路是用于以太网虚拟化，vmnet-vccmni0到vmnet-vccmni3是用于modem网络虚拟化（默认配置），也可根据需要自行配置 vmnet0—eth0能互相ping通ip则说明通路OK

7.7.1.2.3 公网通路搭建

实际的物理网卡接入以后会生成对应的网卡，安卓端需要配置路由网关和DNS确保网络数据的转发能走虚拟通路送到yocto端

路径：

alps/vendor/mediatek/proprietary/packages/overlay/vendor/ConnectivityOverlay/res\_auto/values/config.xml

```

<resources xmlns:xliff="urn:oasis:names:tc:xliff:document:1.2">
<string-array translatable="false" name="config_ethernet_interfaces">
<!-- for adb, no internet -->
<item>eth0;11,13,14;ip=192.168.1.2/24 gateway=192.168.1.1</item>
<!-- for data share, internet, Override Transport Cellular transport -->
<item>eth1;11,12,13,14,15,28;ip=172.17.0.2/24 gateway=172.17.0.1 dns=8.8.8.8;0</item>
<!-- for xx -->
<item>eth2;11,12,13,14,15,28;ip=172.17.1.2/24 gateway=172.17.1.1</item>
<!-- for xx -->
<item>eth3;11,12,13,14,15,28;ip=172.17.2.2/24 gateway=172.17.2.1</item>
<!-- for xx -->
<item>eth4;11,12,13,14,15,28;ip=172.17.3.2/24 gateway=172.17.3.1</item>
<!-- for xx -->
<item>eth5;11,12,13,14;ip=172.20.1.44/24 gateway=172.20.1.1</item>
</string-array>
</resources>

```

yocto端需要搭建SNAT把内网IP转换成公网IP，以便在互联网中进行通信。

路径：src/support/mtknetworkmanager/src/Connection.cpp

```

const char* NAT_CMD[MAX_PDN_NUMBER] = {
"iptables -t nat -%s POSTROUTING -s 172.17.0.0/24 -o %s -j MASQUERADE",
"iptables -t nat -%s POSTROUTING -s 172.17.1.0/24 -o %s -j MASQUERADE",
"iptables -t nat -%s POSTROUTING -s 172.17.2.0/24 -o %s -j MASQUERADE",
"iptables -t nat -%s POSTROUTING -s 172.17.3.0/24 -o %s -j MASQUERADE",
};

```

里面有关于网络连接的相关配置，感兴趣可以看看

至此，安卓端就能通过虚拟化共享yocto端的网络了

### 7.7.1.2.4 vhost-net接口

vhost-net在kernel中是miscdevice的形态存在的，在linux驱动中把无法归类的五花八门的设备定义为混杂设备（用miscdevice结构体表述）。miscdevice共享一个主设备号MISC\_MAJOR（即10），但是次设备号不同。所有的miscdevice设备形成一个链表，对设备访问时内核根据次设备号查找对应的miscdevice设备，然后调用file\_operations结构中注册的文件操作接口进行操作。

对应vhost-net，其file\_operations为：

```
static const struct file_operations vhost_net_fops = {
 .owner = THIS_MODULE,
 .release = vhost_net_release,
 .read_iter = vhost_net_chr_read_iter,
 .write_iter = vhost_net_chr_write_iter,
 .poll = vhost_net_chr_poll,
 .unlocked_ioctl = vhost_net_ioctl,
 .compat_ioctl = compat_ptr_ioctl,
 .open = vhost_net_open,
 .llseek = noop_llseek,
};
```

在nbl\_vmm中会open(“/dev/vhost-net”, O\_RDWR), 此时会调用vhost\_net\_open初始化vhost\_net, 使用ioctl命令可以对vhost-net设备进行配置

常用的配置参数如下:

```
VHOST_NET_SET_BACKEND //设置后端
VHOST_GET_FEATURES //获取设备支持的特性, 并将其复制到用户空间。
VHOST_SET_FEATURES //从用户空间获取特性
VHOST_GET_BACKEND_FEATURES //获取后端支持的特性
VHOST_SET_BACKEND_FEATURES //设置后端特性。
VHOST_RESET_OWNER //重置设备所有者
VHOST_SET_OWNER //设置设备所有者
```

### 7.7.1.2.5 Debug

#### 1、yocto端iptables命令

```
iptables -t nat -A POSTROUTING -s 172.17.0.0/24 -o ccmni1 -j MASQUERADE //手动配置SNAT
```

```
iptables -t nat -L -v -n //NAT 表中的规则, 并提供详细信息。
```

#### 2、yocto端modem命令

```
/usr/bin/mlclient_test --slot_id=0 --gtest_filter=MLClientTest.ML_Sim_GetCardStatus //确认是否识别到
```

卡 present or absent, 正常为present

```
/usr/bin/mlclient_test --slot_id=0 --gtest_filter=MLClientTest.ML_GetNetState //确认是否驻网成
```

功 attach or detach, 正常为attach

```
/usr/bin/mlclient_test --gtest_filter=MLClientTest.ML_DataCallStart_IotPublic //拨号上网
```

```
/usr/bin/mlclient_test --slot_id=0 --gtest_filter=MLClientTest.ML_GetCSQ //获取信号质量
```

#### 3、安卓端查看网卡配置

```
cmd overlay lookup --verbose com.android.connectivity.resources
com.android.connectivity.resources:array/config_ethernet_interfaces
```

#### 4、网络配置相关信息

安卓端网络服务信息dump

# 获取 netd 服务信息

```
adb -e shell dumpsys netd > netd.log
```

# 获取 network\_stack 服务信息

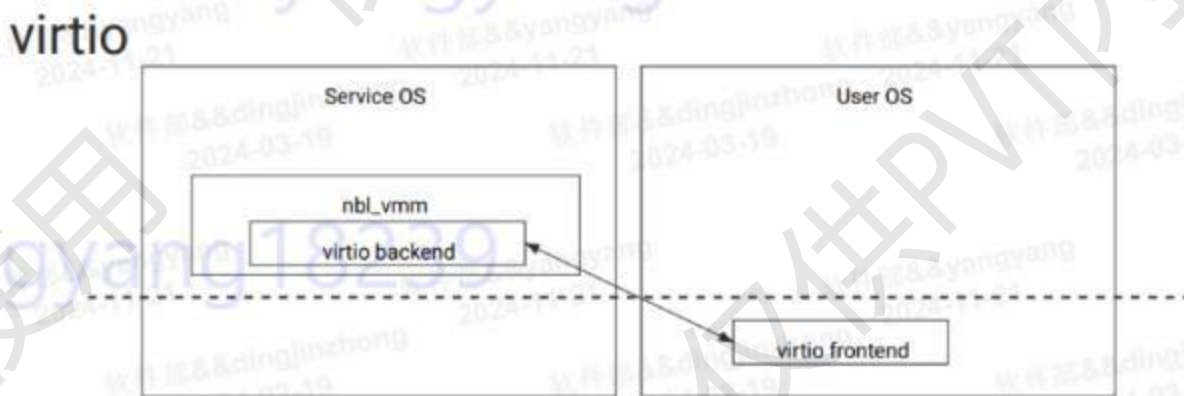
```
adb -e shell dumpsys network_stack > network_stack.log
获取 connectivity 服务信息
adb -e shell dumpsys connectivity > connectivity.log
获取 tethering 服务信息
adb -e shell dumpsys tethering > tethering.log
获取 ethernet 服务信息
adb -e shell dumpsys ethernet > ethernet.log
获取 dnsresolver 服务信息
adb -e shell dumpsys dnsresolver > dnsresolver.log
获取系统属性
adb -e shell getprop > prop.log
获取 ARP 表
adb -e shell cat /proc/net/arp > arp.log
获取 IP 规则
adb -e shell ip rule > rule.log
获取所有路由表
adb -e shell ip route show table all > route.log
```

### 7.7.1.3 vmctl

此driver提供给 Android 的内核，

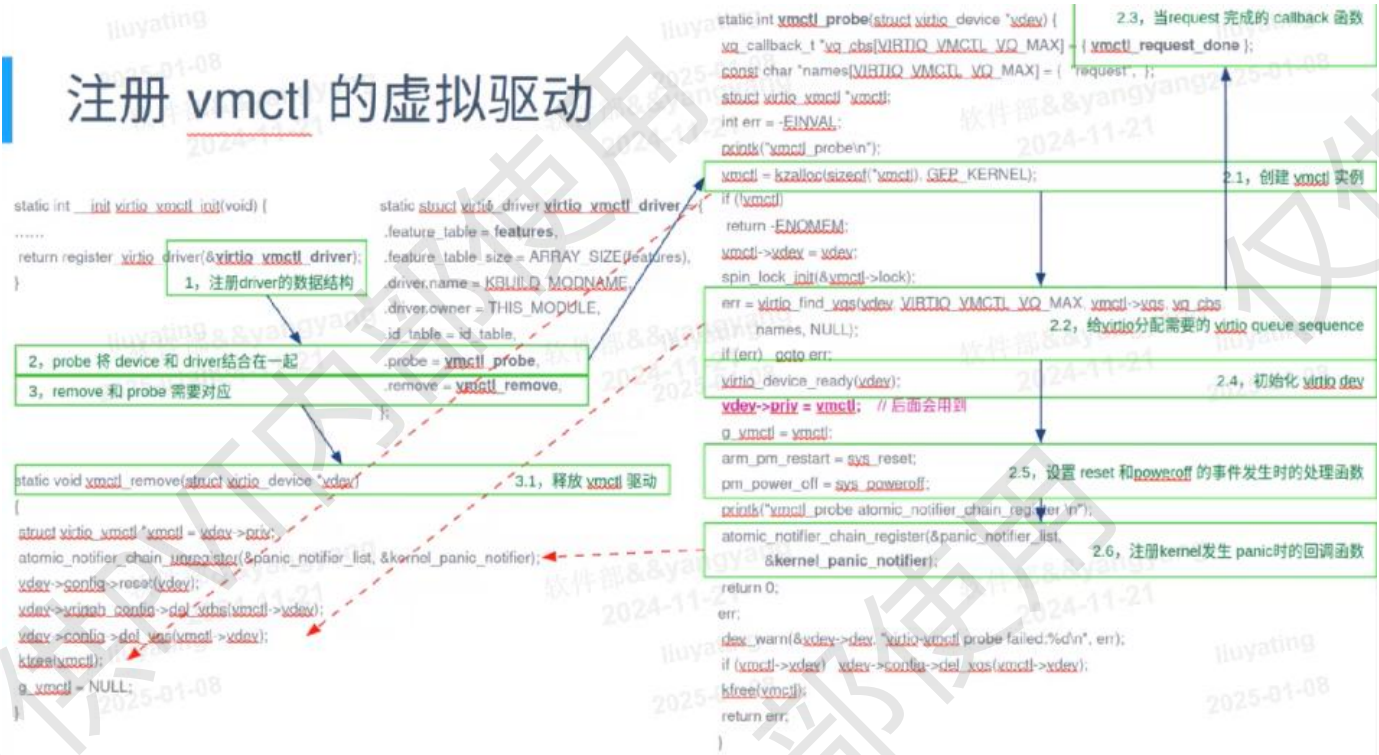
- 用于监听Android kernel中的 power-off / restart / panic 事件，然后通知 nbl\_vm\_srv 做出相应的处理。
- 另外提供 “vmctl\_kick\_wdt” 函数给其他的driver，可以唤醒 wdt (watchdog timer)，这个wdt是 yocto那管理的，如果有一段时间没有kick wdt，那么hypervisor 会启动 Android。

采用virtio实现vmctl通信，用下面这种方式实现

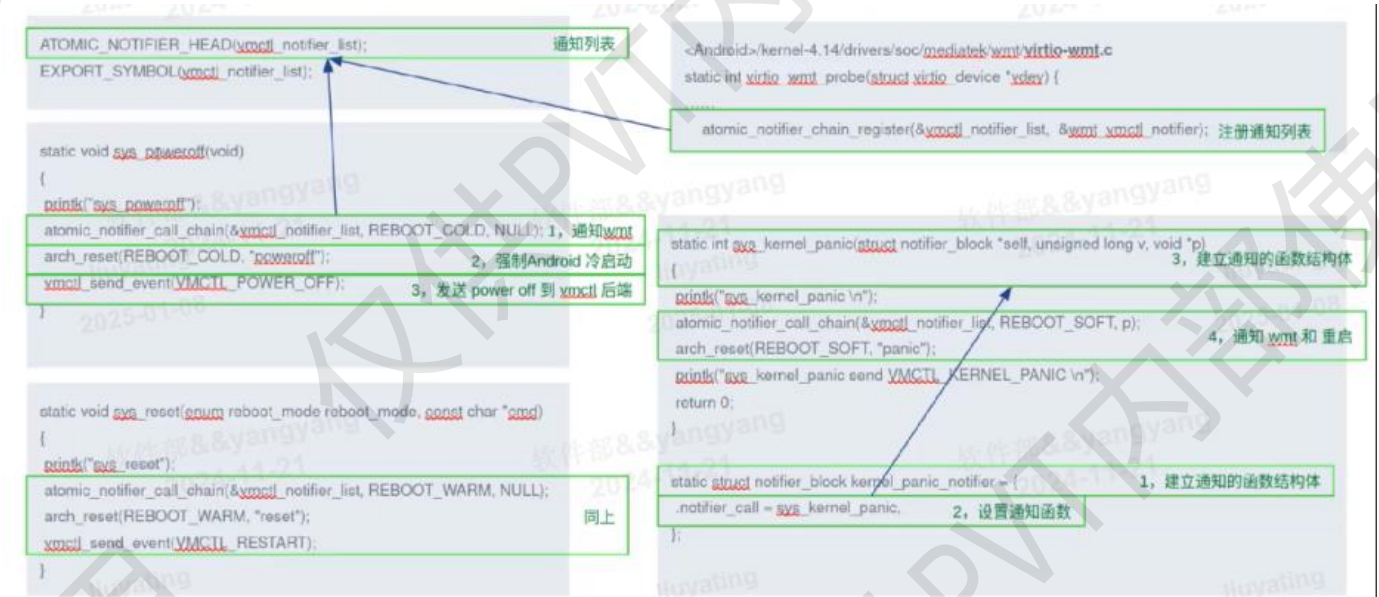




vmctl虚拟驱动注册流程:



当kernel 发生power off / reset / panic 时



后端处理:

xref: /mt8676\_pvt\_demo/grt\_sdk/grt/thyp-sdk/vmm/nbl\_vm\_srv/vm\_controller.cc (revision c7d059fc)

Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download  Search

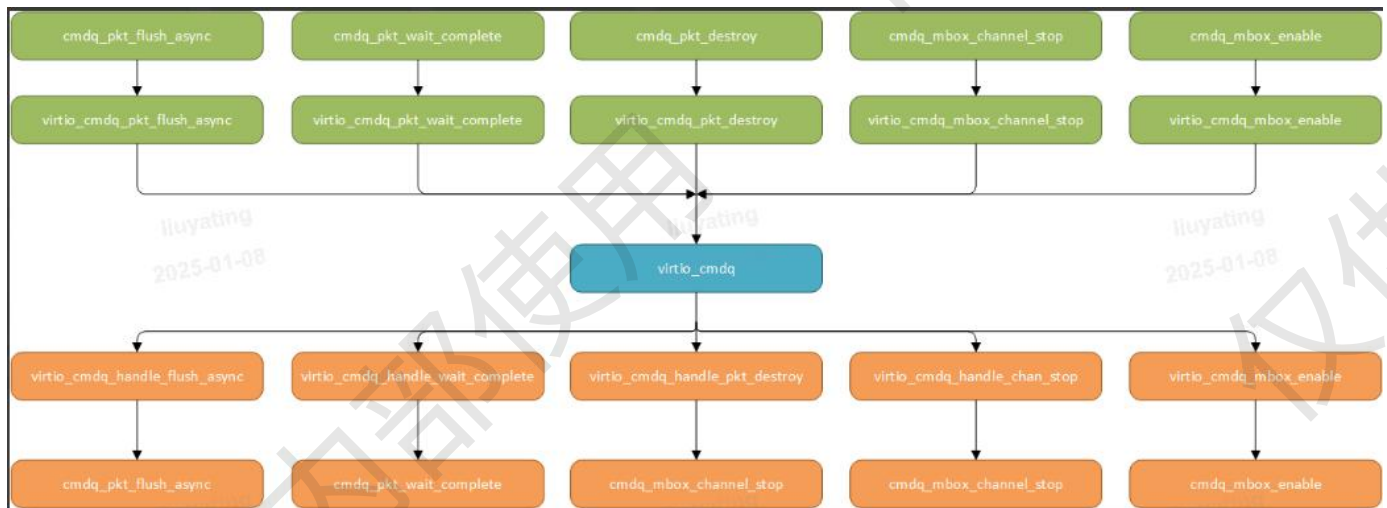
```
236 }
237
238 void VmController::StartVmStateMonitor() {
239 monitor_thread_ = std::thread([this] {
240 while (!monitor_thread_stop_) {
241 VmStateEvent event;
242 auto success = event_reader->Read(&event);
243 if (!success)
244 return;
245
246 auto state = event.state();
247 switch (state) {
248 case VmState::POWER_OFF:
249 LOG(INFO) << "received VmState: POWER_OFF";
250 async::PostTask(async_, [this] { StopVm(); });
251 break;
252 case VmState::RESTART:
253 LOG(INFO) << "received VmState: RESTART";
254 async::PostTask(async_, [this] { RestartVm(); });
255 break;
256 case VmState::KERNEL_PANIC:
257 LOG(INFO) << "received VmState: KERNEL_PANIC, try to reboot VM";
258 async::PostTask(async_, [this] {
259 DumpVmState(1);
260 RestartVm();
261 });
262 break;
263 case VmState::KICK_WDT: {
264 std::unique_lock<std::mutex> lk(wdt_timer_mutex_);
265 LOG(INFO) << "received VmState: KICK_WDT";
266 wdt_cv_.notify_all();
267 wdt_kick_start_ = true;
268 break;
269 }
270 default:
```

#### 7.7.1.4 cmdq

##### 1、介绍

CMDQ: 即command queue, 由于Display的driver中涉及大量的寄存器读写操作, 如果全部用CPU读写, 会占用大量的CPU资源。所以MTK使用一个硬件来负责此 项工作, cmdq就是辅佐这个硬件工作的软件架构。

Display把需要读、写、和轮询硬件寄存器的一连串命令, 打包成命令队列交给cmdq, cmdq会单独去启一个 线程做这些事情。cmdq的实现依赖gce和IPI中断。



display直接向DMA硬件设备发送命令，完成对物理内存的读写操作。

cmdq\_handle\_flush\_async完成硬件寄存器命令的提交，cmdq\_handle\_pkt\_destroy销毁chan，代表本次写操作结束。

## 2、流程

dsiplanes

0drm\_atomic\_helper\_commit\_planes

1mtk\_drm\_crtc\_atomic\_flush

2mtk\_crtc\_gce\_flush

3cmdq\_pkt\_flush\_threaded

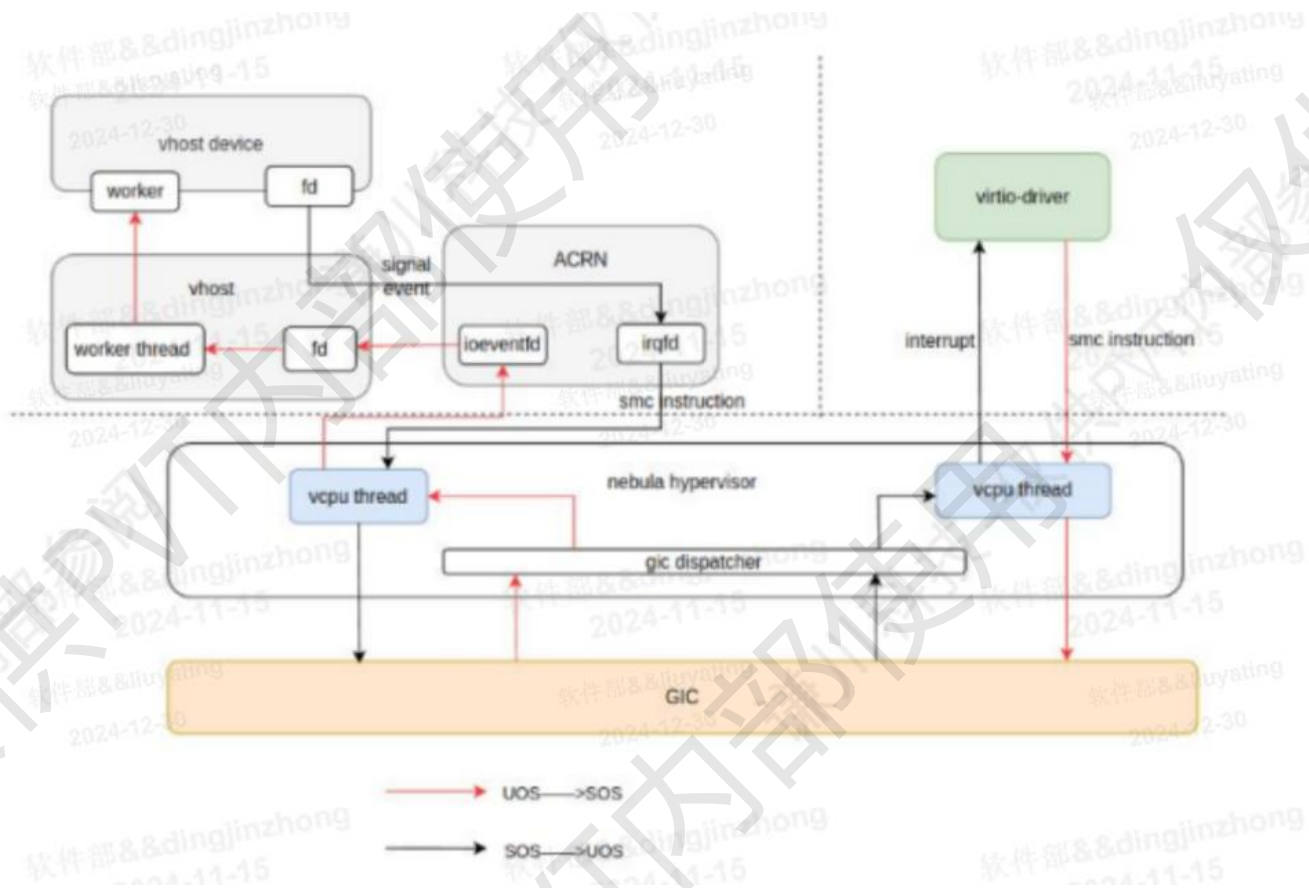
4cmdq\_pkt\_flush\_async

5virtio\_cmdq\_pkt\_flush\_async

0virtio\_cmdq\_handle\_flush\_async

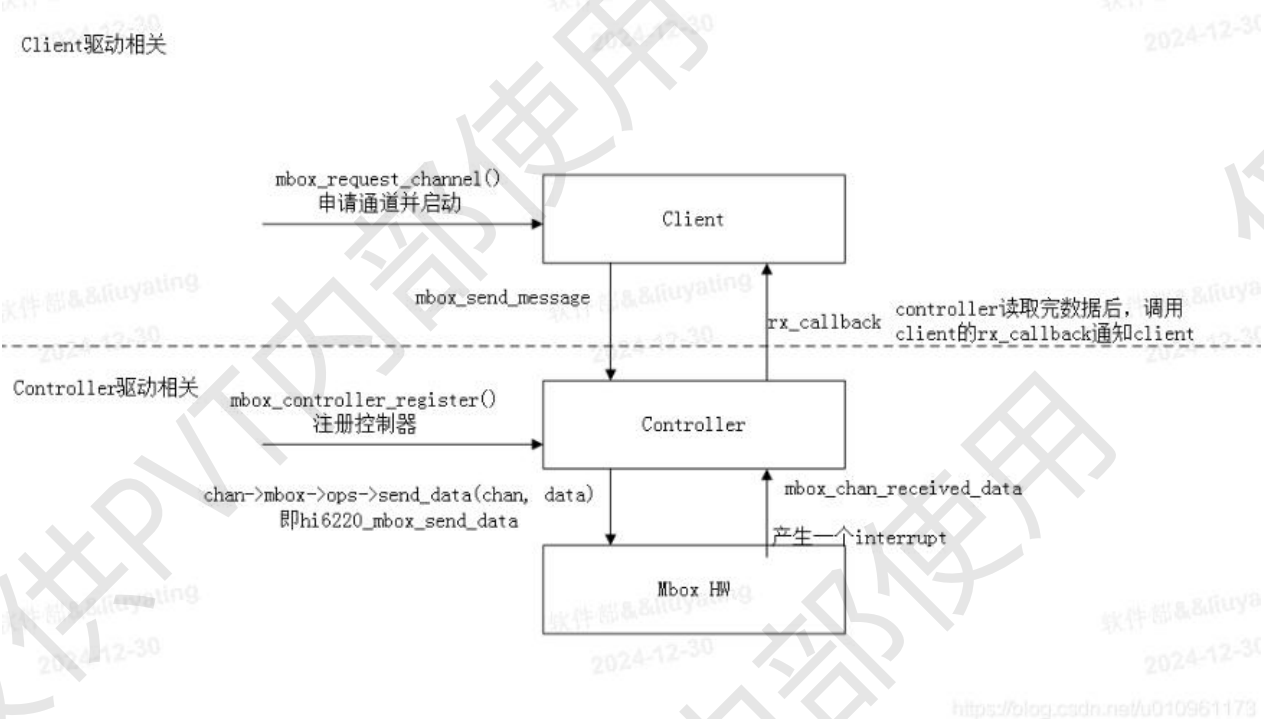
1cmdq module cmdq\_pkt\_flush\_async //api

### 7.7.1.5 hw-irq



- 基于原生virtio协议vhost机制进行的优化;
- 将物理中断与ioeventfd和irqfd绑定, 即收到中断后触发ioeventfd,或者写入irqfd后, 会产生物理中断;
- 使用hwirq的模块: vsoc、virtio-apu,virtio-gpu

### 7.7.1.6 mailbox



#### 1、mailbox controller api

文件: kernel/include/linux/mailbox\_controller.h

##### (1) 注册、注销控制器

```
int mbox_controller_register(struct mbox_controller *mbox); /* can sleep */-----probe中调用
void mbox_controller_unregister(struct mbox_controller *mbox); /* can sleep */-----probe中调用
```

##### (2) (对外接口) 将底层收到的数据回调给上层应用

```
void mbox_chan_received_data(struct mbox_chan *chan, void *data); /* atomic */
```

##### (3) 通知上层当前数据已经发送完成

```
void mbox_chan_txdone(struct mbox_chan *chan, int r); /* atomic */
```

#### 2、mailbox client api

文件: kernel/include/linux/mailbox\_client.h

##### (1) 发送数据前，申请通道

```
struct mbox_chan *mbox_request_channel_byname(struct mbox_client *cl, const char *name);
struct mbox_chan *mbox_request_channel(struct mbox_client *cl, int index);
```

##### (2) 数据发送

```
int mbox_send_message(struct mbox_chan *chan, void *mssg);
void mbox_client_txdone(struct mbox_chan *chan, int r); /* atomic */
```

##### (3) 数据记录

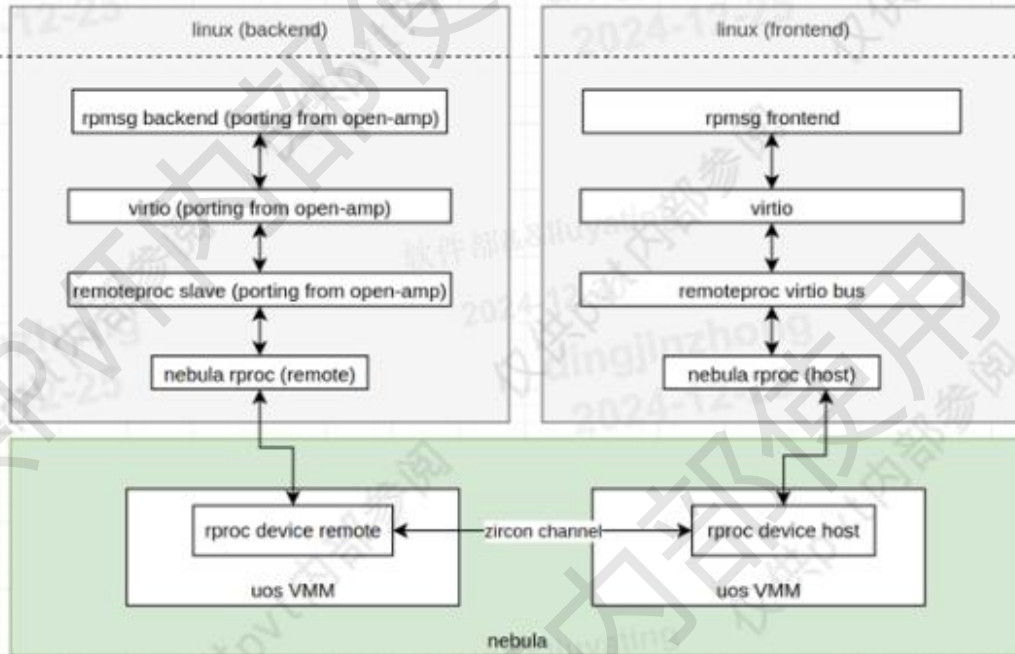
```
bool mbox_client_peek_data(struct mbox_chan *chan); /* atomic */
```



#### (4) 释放通道

```
void mbox_free_channel(struct mbox_chan *chan); /* may sleep */
```

### 7.7.1.7 rproc



#### Server端:

- 结构体sample\_device\_descriptor 为rproc virtio设备描述符:

```
struct sample_device_descriptor {
 struct fw_rsc_vdev vdev;
 struct fw_rsc_vdev_vring vrings[NUM_VRINGS];
} __packed;

static const struct sample_device_descriptor sample_vdev_desc = {
 .vdev = {
 .type = RSC_VDEV, //设备类型
 .id = VIRTIO_ID_SAMPLE, //虚拟设备id, 是唯一的, 定义在virtio_ids.h文件中
 .dfeatures = 0, //指定固件支持的virtio设备功能
 .config_len = 0, //此vdev的virtio配置空间的大小
 .num_of_vrings = NUM_VRINGS, //vring个数
 },
 .vrings = { //定义vring的发送和接收参数
 [RING_TX] = {
 .da = FW_RSC_U32_ADDR_ANY, //设备地址
 }
 }
};
```



```

 .align = VRING_ALIGN, //排列方式
 .num = VRING_SIZE, //vring大小
 },
 [RING_RX] = {
 .da = FW_RSC_U32_ADDR_ANY,
 .align = VRING_ALIGN,
 .num = VRING_SIZE,
 },
},
};

```

- nebula\_rproc\_vdev\_ops为rproc virtio设备的文件操作，是系统调用和驱动程序之间的桥梁。用户程序对设备文件进行的操作函数，下面有创建、销毁和重置

```

static struct nebula_rproc_vdev_ops sample_vdev_ops = {
 .on_create = sample_device_create,
 .on_destroy = sample_device_destroy,
 .on_reset = sample_device_reset,
};

```

## 1、int nebula\_rproc\_register\_device(const void \*rsc, size\_t rsc\_size, struct nebula\_rproc\_vdev\_ops \*vdev\_ops)

函数功能：注册一个rproc设备，并将其与指定的资源表和虚拟设备操作相关联。通过调用此函数，可以初始化和配置远程处理设备，以便进行跨处理器通信或其他相关操作。

- const void \*rsc: 虚拟化设备描述符，为结构体，如上面的sample\_vdev\_desc
- size\_t rsc\_size: 为虚拟化设备描述符结构体大小
- struct nebula\_rproc\_vdev\_ops \*vdev\_ops: rproc virtio设备的文件操作结构体的指针，如上面sample\_vdev\_ops

## 2、int virtio\_create\_virtqueues(struct virtio\_device \*vdev, unsigned int flags, unsigned int nvqs, const char \*names[], vq\_callback callbacks[], void \*callback\_args[])

函数功能：该函数用于为指定的 Virtio 设备创建多个 Virtqueue，并为每个队列设置名称、回调函数及其参数。

Virtio 驱动程序使用这个函数来初始化和配置设备所需的队列，以便进行数据交换。

- struct virtio\_device \*vdev: 指向 Virtio 设备结构体的指针。
- unsigned int flags: 创建 Virtqueue 时使用的标志。
- unsigned int nvqs: 要创建的 Virtqueue 数量。
- const char \*names[]: 每个 Virtqueue 的名称数组。
- vq\_callback callbacks[]: 每个 Virtqueue 的回调函数数组。

- `void *callback_args[]`: 每个回调函数的参数数组。

### 3、`void virtqueue_disable_cb(struct virtqueue *vq)`

函数功能：该函数禁用指定 `Virtqueue` 的回调通知。`Virtio` 驱动程序使用这个函数来停止接收设备发出的中断或通知，从而避免在不需要时处理不必要的回调。

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。

### 4、`static inline void virtqueue_set_shmem_io(struct virtqueue *vq, void *io)`

函数功能：该函数将共享内存 I/O 操作设置到指定的 `Virtqueue` 中。`Virtio` 驱动程序使用这个函数来配置共享内存操作，以便在主机和设备之间进行高效的数据交换。

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。
- `void *io`: 指向共享内存 I/O 操作结构体的指针。

### 5、`void *virtqueue_get_available_buffer(struct virtqueue *vq, uint16_t *avail_idx, uint32_t *len)`

函数功能：该函数从指定的 `Virtqueue` 中获取一个可用的缓冲区，并将其索引和长度存储在提供的参数中。`Virtio` 驱动程序使用这个函数来从设备获取数据。

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。
- `uint16_t *avail_idx`: 用于存储可用缓冲区在 `Virtqueue` 中的索引。
- `uint32_t *len`: 用于存储可用缓冲区的数据长度。

### 6、`int virtqueue_add_consumed_buffer(struct virtqueue *vq, uint16_t head_idx, uint32_t len)`

函数功能：该函数将已消耗的缓冲区添加到指定的 `Virtqueue` 中。`Virtio` 驱动程序使用这个函数来通知设备某个缓冲区已经被使用完毕，并且可以被重新利用

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。
- `uint16_t head_idx`: 已消耗缓冲区在 `Virtqueue` 中的索引。
- `uint32_t len`: 已消耗缓冲区的数据长度。

### 7、`void virtqueue_kick(struct virtqueue *vq)`

函数功能：该函数用于通知 `Virtio` 设备，`Virtqueue` 中有新的缓冲区可供处理。`Virtio` 驱动程序使用这个函数来触发设备处理已添加到队列中的缓冲区。

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。

代码实例路径：`yocto:src/kernel/modules/mt8676/virt/grt/nebula_rproc/nebula_virtio_sample.c`

**client端():**

- 结构体 `virtio_driver`，是一个用于描述 `rproc Virtio` 驱动程序的结构体。它定义了驱动程序如何与 `Virtio` 设备进行交互，包括设备匹配、初始化、移除等操作，具体实例如下：

```
static struct virtio_driver perftest_driver = {
 .feature_table_size = 0,
 .driver.name = KBUILD_MODNAME,
 .driver.owner = THIS_MODULE,
 .id_table = id_table,
 .probe = perftest_probe,
 .remove = perftest_remove,
};
```

### 1、int `register_virtio_driver(struct virtio_driver *driver)`

函数功能：该函数用于注册一个 `Virtio` 驱动程序，使其能够与内核中的 `Virtio` 子系统进行交互。通过调用此函数，可以将驱动程序添加到内核中，以便处理相应的 `Virtio` 设备。

- `struct virtio_driver *driver`: 指向 `Virtio` 驱动程序结构体的指针。如上面 `perftest_driver`。

### 2、int `virtio_find_vqs(struct virtio_device *vdev, unsigned nvqs, struct virtqueue *vqs[], vq_callback_t *callbacks[], const char * const names[], struct irq_affinity *desc)`

函数功能：该函数用于查找并初始化指定数量的 `Virtqueue`，并为每个队列设置名称、回调函数及其中断亲和性。`Virtio` 驱动程序使用这个函数来配置设备所需的队列，以便进行数据交换。

- `struct virtio_device *vdev`: 指向 `Virtio` 设备结构体的指针。
- `unsigned nvqs`: 要查找和初始化的 `Virtqueue` 数量。
- `struct virtqueue *vqs[]`: 用于存储找到的 `Virtqueue` 的数组。
- `vq_callback_t *callbacks[]`: 每个 `Virtqueue` 的回调函数数组。
- `const char * const names[]`: 每个 `Virtqueue` 的名称数组。
- `struct irq_affinity *desc`: 中断亲和性描述符（可选）。

### 3、unsigned int `virtqueue_get_vring_size(struct virtqueue *vq)`

函数功能：该函数用于获取指定 `Virtqueue` 的 `vring` 大小。`Virtio` 驱动程序使用这个函数来查询队列中可用的描述符数量，以便进行数据交换和管理。

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。

### 4、int `virtqueue_add_inbuf(struct virtqueue *vq, struct scatterlist *sg, unsigned int num, void *data, gfp_t gfp)`

函数功能：该函数用于向指定的 `Virtqueue` 添加一个输入缓冲区。`Virtio` 驱动程序使用这个函数来将数据缓冲区添加到队列中，以便设备可以将数据写入这些缓冲区。

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。

- `struct scatterlist *sg`: 指向散列表 (scatter-gather list) 的指针, 描述输入缓冲区。
- `unsigned int num`: 散列表中的条目数量。
- `void *data`: 与该缓冲区关联的数据指针, 通常用于在回调中识别该缓冲区。
- `gfp_t gfp`: 分配内存时使用的标志。

#### 5、`void virtqueue_disable_cb(struct virtqueue *vq)`

函数功能: 该函数禁用指定 `Virtqueue` 的回调通知。`Virtio` 驱动程序使用这个函数来停止接收设备发出的中断或通知, 从而避免在不需要时处理不必要的回调。

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。

#### 6、`bool virtqueue_kick_prepare(struct virtqueue *vq)`

函数功能: 该函数用于检查指定的 `Virtqueue` 是否需要通知 `Virtio` 设备。`Virtio` 驱动程序使用这个函数来决定是否应该发送通知, 以便设备处理已添加到队列中的缓冲区。这在某些情况下可以避免不必要的中断, 提高性能。

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。

#### 7、`void virtio_device_ready(struct virtio_device *dev)`

函数功能: 该函数用于通知 `Virtio` 子系统, 指定的 `Virtio` 设备已经完成初始化并且可以开始正常工作。`Virtio` 驱动程序在完成设备配置和初始化后调用此函数, 以便设备可以开始处理队列中的数据。

- `struct virtio_device *dev`: 指向 `Virtio` 设备结构体的指针。

#### 8、`bool virtqueue_notify(struct virtqueue *vq)`

函数功能: 该函数用于通知 `Virtio` 设备, `Virtqueue` 中有新的缓冲区可供处理。`Virtio` 驱动程序使用这个函数来触发设备处理已添加到队列中的缓冲区。

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。

#### 9、`void *virtqueue_get_buf(struct virtqueue *vq, unsigned int *len)`

函数功能: 该函数用于从指定的 `Virtqueue` 中获取一个已处理完成的缓冲区, 并返回该缓冲区的数据指针和长度。`Virtio` 驱动程序使用这个函数来检索设备已经处理完毕的数据缓冲区, 以便进行进一步处理或释放资源。

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。
- `unsigned int *len`: 指向存储缓冲区长度的指针。

#### 10、`void virtqueue_kick(struct virtqueue *vq)`

函数功能: 该函数用于通知 `Virtio` 设备, `Virtqueue` 中有新的缓冲区可供处理。`Virtio` 驱动程序使用这个函数来触发设备处理已添加到队列中的缓冲区。

- `struct virtqueue *vq`: 指向 `Virtqueue` 结构体的指针。

代码实例路径: android:vendor/mediatek/kernel\_modules/virt/nebula\_rproc/nebula\_virtio\_perftest.c

## 7.7.2 UOS & UOS间通信

### 7.7.2.1 vsockx

vsockx 是谦川基于 vsock 开发的 UOS 之间 socket 通信的方式,基于rproc的vsock使用实例如下:

对于 UOS 间 vsock,即 vsockx

tbox CID:4

android CID: 2 //VMADDR\_CID\_HOST

server 端通讯案例(C++):

```
#define AF_VSOCKX AF_DECnet
int main()
{
 int s = socket(AF_VSOCKX, SOCK_STREAM, 0);
 const struct sockaddr_vm addr = {
 .svm_family = AF_VSOCK,
 .svm_prot = 9999,
 .svm_cid = VMADDR_CID_HOST,
 }
 bind(s,(const sockaddr *)&addr, sizeof(struct sockaddr_vm));
 listen(s, 0);
 struct sockaddr_vm peer_addr;
 socklen_t peer_addr_size = sizeof(struct sockaddr_vm);
 int peer_fd = accept(s,(sockaddr *)&peer_addr,&peer_ddd_size);

 char buf[54];
 ssize_t msg_len;
 while ((msg_len = recv(peer_fd ,&buf, 64, 0)) > 0){
 printf("Received %zd bytes: %.*s\n", msg_len, msg_len, buf);
 }
 return 0;
}
```

client 端通讯案例(C++):

```
#define AF_VSOCKX AF_DECnet
int main()
```

```

{
int s = socket(AF_VSOCKX, SOCK_STREAM, 0);

struct sockaddr_vm ddr;
memset(&addr, 0, sizeof(struct sockaddr_vm));
addr.svm_family = AF_VSOCK;
addr.svm_prot = 9999;
addr.svm_cid = VMADDR_CID_HOST;

int rc = connect(s, &addr, sizeof(struct sockaddr_vm));
if(rc < 0){

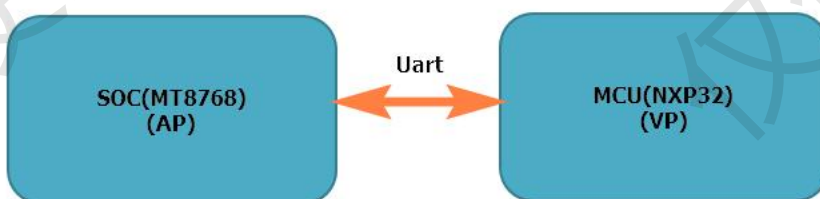
printf("rc = %d %s\n", rc, strerror(errno));
return rv;
}
send(s "Hello,world!", 13, 0);
close(s);
return 0;
}

```

## 7.7.3 MCU & CPU间通信

### 7.7.3.1 单系统uart

- 1、MT8676和MCU通过UART进行通信协议，使用波特率 115200 bps ， 8bit数据 ， 1个停止位， 无奇偶校验位。
- 2、分需要回应ACK和不需要回应NACK的数据。对于需要回应的数据ACK，在100ms内做回应，超过100ms，重新发送，3次无响应，抛去此帧。
- 3、传输框图如下：





### 7.7.3.2 单系统spi

SOC作为主机，MCU作为从机，波特率暂定12MHz

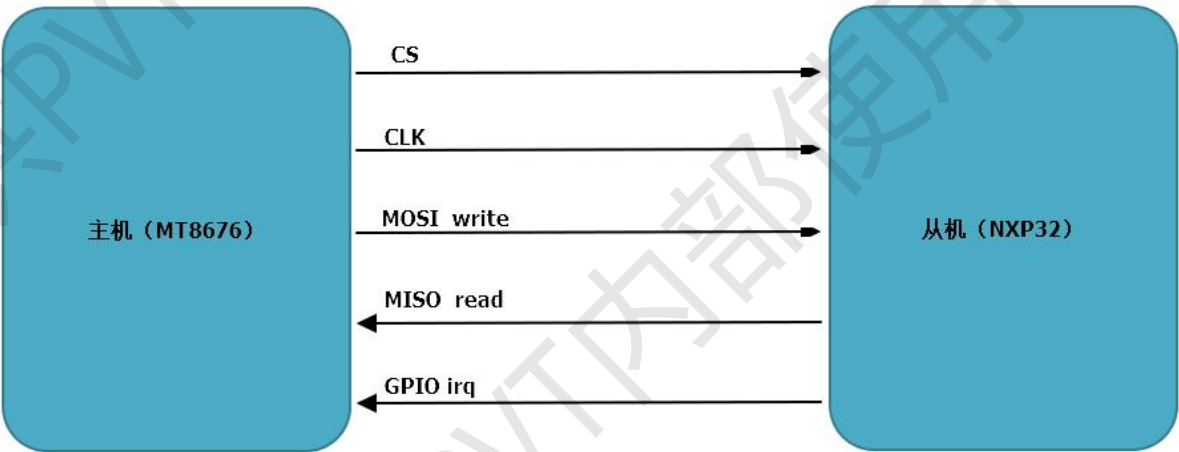
接口write是主机发送给从机数组，从机无法将回传的值写入，写入主机读不到。

接口read是主机发起的，从机在spi clk传输过程中将值给主机读取，主机无法写入值。

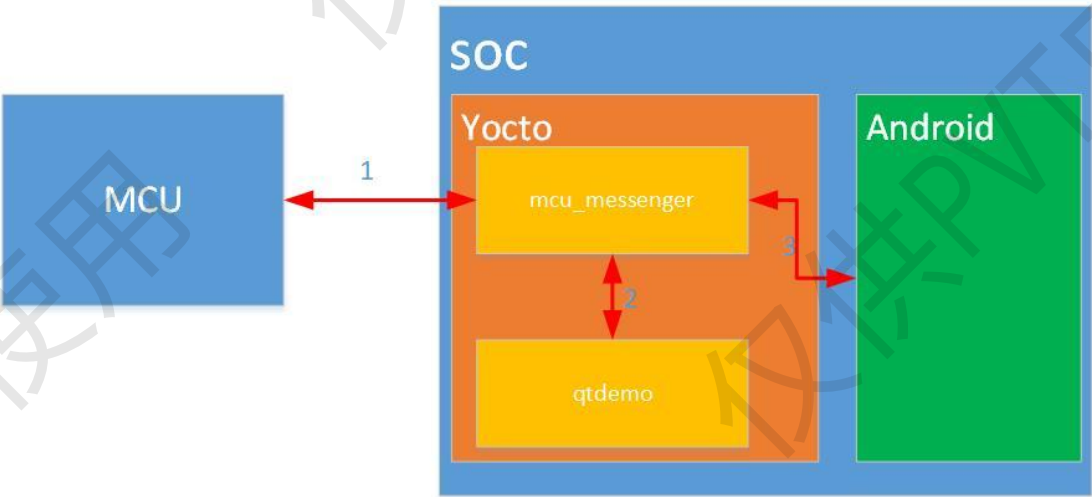
主机写由于只能发送主机命令，并由主机发起，所以write长度是不定长的，根据主机数据量来定，只需要限制最大值即可，比如最大500bytes。

Read函数是由主机发起的，不知道从机有多少数据量，所以这个是定长的，比如固定为 200、300 、 500等。

当从机有中断脚送给主机机制后，主机可以省去周期读，主机检测高，表示从机有消息给主机，主机就去read。



### 7.7.3.2 双系统



1: uart      2: socket      3: vsocket

1、mcu通过uart与Yocto的mcu\_messenger进行通讯，实现数据的透传。

2、mcu\_messenger根据协议帧的type进行分发：mcu\_2\_yocto、mcu\_2\_android、yocto\_2\_mcu、android\_2\_mcu、yocto\_2\_android、android\_2\_yocto。

## 7.8 外设通信协议

### 7.8.1 SPI

#### 7.8.1.1 名词解释

CPHA: CLock Phase时钟相位

CPOL: Clock Polarity时钟极性

CS: Chip Select芯片选择引脚

DMA: Direct Memory Access直接内存访问

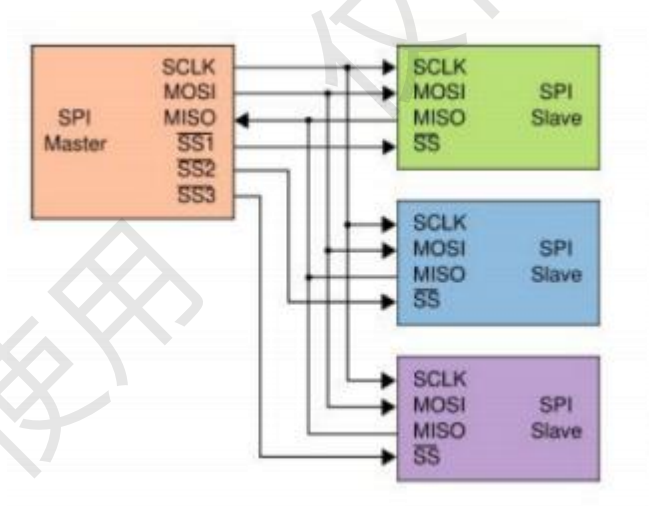
FIFO: First In First Out先进先出

MISO: Master In Slave Out,SPI Master输入数据和SPI Slave输出数据

MOSI: Master Out Slave In,SPI Master输出数据和SPI Slave输入数据

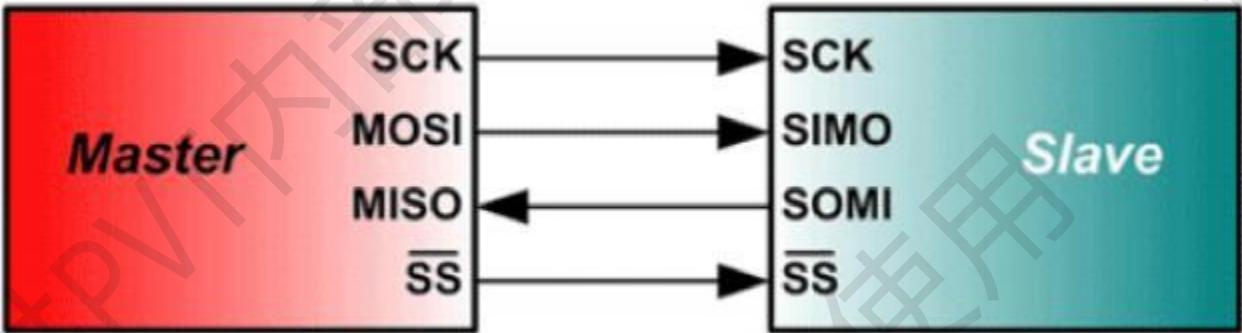
#### 7.8.1.2 SPI介绍

1、SPI总线结构 SPI(Serial Peripheral Interface)串行外设接口，是一种高速的，全双工，同步的通信总线。采用主从模式（Master Slave）架构，支持多个slave，一般仅支持单Master。



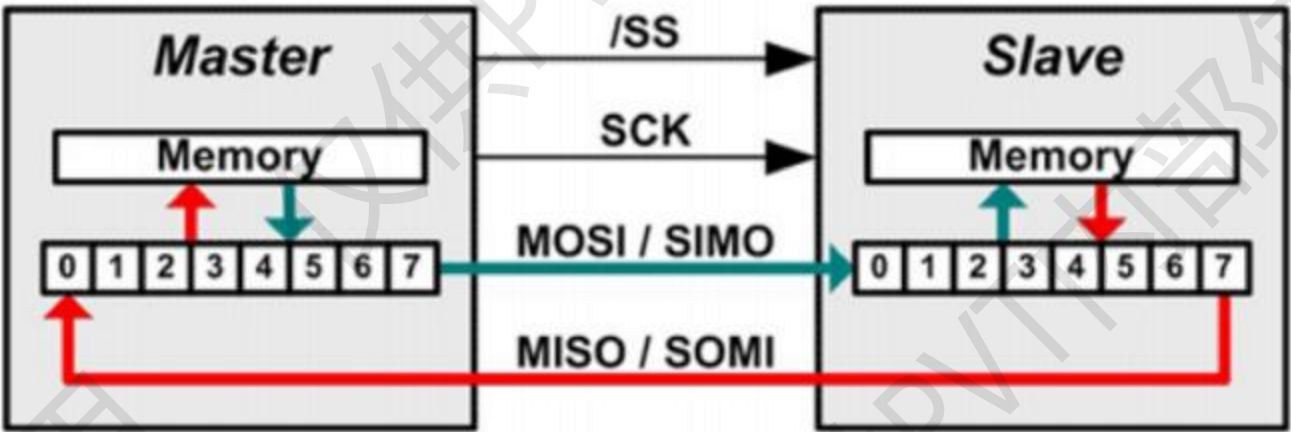
SPI接口共有4根信号线，分别是：设备选择线(SS)、时钟线(SCK)、串行输出数据线(MOSI)、串行输入数据线(MISO)。①、CS/SS，Slave Select/Chip

Select，这个是片选信号线，用于选择需要进行通信的从设备。 I2C 主机是通过发送从机设备地址来选择需要进行通信的从机设备的，SPI 主机不需要发送从机设备，直接将相应的从机设备片选信号拉低即可。 ②、SCK, Serial Clock, 串行时钟, 和 I2C 的 SCL 一样，为 SPI 通信提供时钟。 ③、MOSI /SDO, Master In Slave Out/Serial Data Output, 简称主出从入信号线，这根数据线 只能用于主机向从机发送数据，也就是主机输出，从机输入。 ④、 MISO/SDI, Master In Slave Out/Serial Data Input, 简称主入从出信号线，这根数据线只 能用户从机向主机发送数据，也就是主机输入，从机输出。

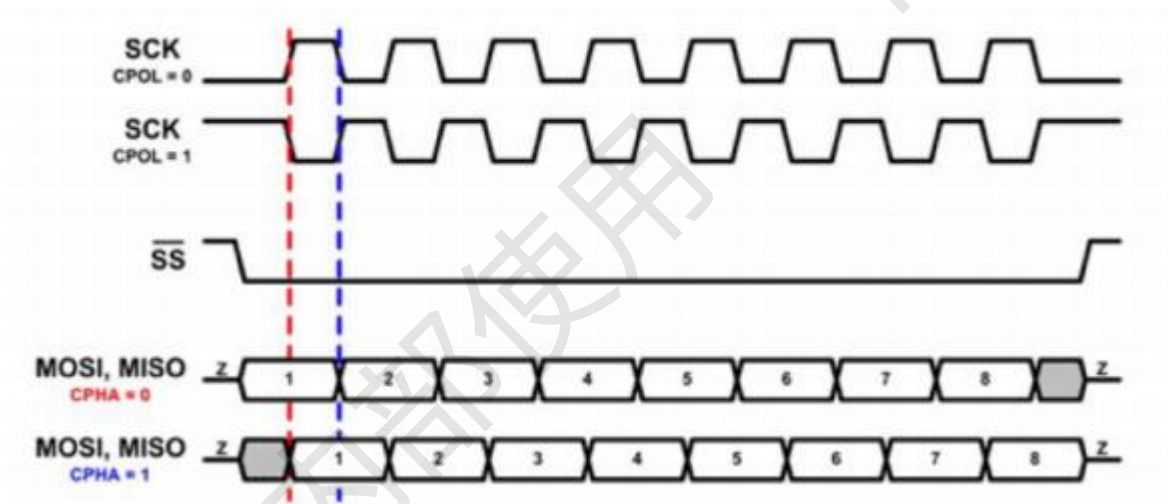


2、数据传输过程

主节点通过MOSI线输出数据，从节点在SIMO处从主节点读取数据。同时，也在通过SOMI输出MSB （最高位），主节点会在 MISO处读取从节点的数据， 整个过程将一直持续，直至交换完所有数据。



3、总线时序

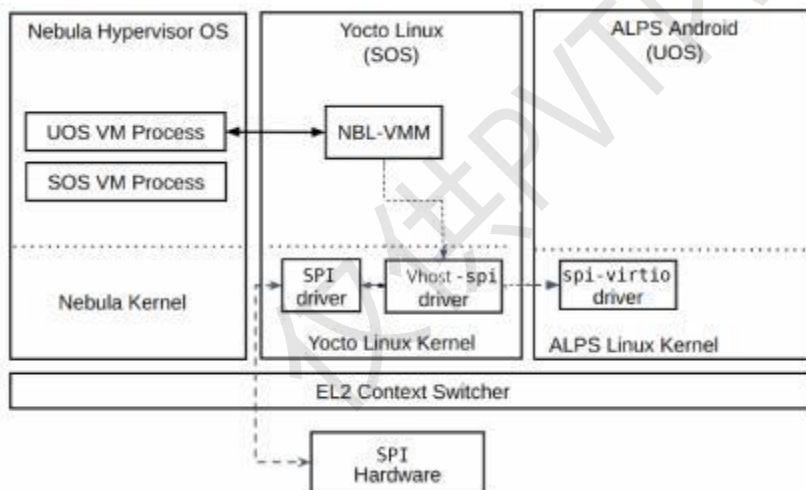


CPOL极性：决定时钟空闲时为高电平还是低电平 CPOL=0：CLK空闲时是低电平，CLK有效时是高电平

CPOL=1：CLK空闲时是高电平，CLK有效时是低电平 CPHA

相位：决定何时进行数据采样(数据位读取) CPHA=0：第一个边沿采样 CPHA=1：第二个边沿采样 根据CPOL和CPHA的不同组合，SPT被分为4种模式

### 7.8.1.3 SPI虚拟化框架



NBL-VMM:将spi虚拟化features写入pci配置空间中，便于UOS probe。 vhost-spi :spi back-end driver,调用spi device和建立与front end的通讯通路。

spi-virtio :spi front-end driver,, 给Android 上层调用。

### 7.8.1.4 SPI配置

#### 1、虚拟化配置

```

//
//spi
&spi1 {
pinctrl-names = "default";
pinctrl-0 = <&ak7709_codec_spi>;
/*pinctrl-1 = <&ak7709_codec_pdn>;*/
status = "okay";
ak7709: ak7709@0 {
status = "okay";
compatible = "akm,ak7709";
#sound-dai-cells = <0>;
reg = <0>;
spi-max-frequency = <1000000>;
/*ak7709, pdn-gpio = <&pio 147 0>;*/
};
};
//pingctrlSPI pinmux
&pio {
ak7709_codec_spi: spimode-default {
pins-cmd-dat {
pinmux = <PINMUX_GPIO53__FUNC_SPI1_MO>,
<PINMUX_GPIO52 FUNC_SPI1_MI>,
_ _ _ _
<PINMUX_GPIO51 FUNC_SPI1_CSB>,
_ _ _ _
<PINMUX_GPIO50 FUNC_SPI1_CLK>;
_ _ _ _
};
};
//
vhost_spi: vhost-spi {
compatible = "grt,vhost-spi";
};
//
spi1_virtio: spi1-virtio {
compatible = "mediatek,virtio-spi";
id = <1>;
status = "ok";
};

```

## 2、PAD\_SEL

在有些IC上， 一组SPI会有不同的GPSIO可以选择，根据GPIO表格信息确定SPI使用哪组GPIO。此时需要在dts中添加mediatek,pad-select=<x>;

```
spi0: spi0@11010000 {
compatible = "mediatek,mt6985-spi";
mediatek,pad-select = <0>; //x=0 1 2A B C
mediatek,tickdly = <2>, <0>;
reg = <0 0x11010000 0 0x100>;
interrupts = <GIC_SPI 242 IRQ_TYPE_LEVEL_HIGH 0>;
clocks = <&topckgen_clk CLK_TOP_MAINPLL_D6_D2>,
<&topckgen_clk CLK_TOP_SPI0_SEL>,
<&pericfg_ao_clk CLK_PERAOP_SPI0_BCLK>;
clock-names = "parent-clk", "sel-clk", "spi-clk";
clock-source-type = "mainpll";
};
```

## 3、频率

SPI控制器支持偶数倍分频，支持的最大频率SPI控制器为52MHz，SPI\_SCK 小于等于52 MHz。

可以通过两种方式设置SPI\_CLK;

(1)、在设备树中，配置属性spi-max-frequency，例如，spi-max-frequency = <1000000>;设置默认SPI\_CLK=1MHz。

(2)、在设备驱动中，配置结构spi\_transfer的字段spedd\_hz,例如xfer→speed\_hz = 1000000;设置SPI\_CLK=1MHz。

## 4、SPI模式

可以通过两种方式设置SPI模式

(1) 在设备数中设置SPI模式



```

&spi4{
pinctrl-names = "default","sleep";
pinctrl-0 = <&spi4_pins_default>;
pinctrl-1 = <&spi4_pins_sleep>;
cs-gpios = <&pio 7 GPIO_ACTIVE_HIGH>;
ak4619: ak4619@0 {
compatible = "akm,ak4619";
reg = <0>;
spi-max-frequency = <4000000>;
spi-cpha;
spi-cpol;
ak4619,pdn-gpio = <&pio 223 0>;
ak4619,mute-gpio = <&pio 130 0>;
status = "okay";
};
};

```

(2) 在设备驱动程序中设置SPI模式

```

int xxx_probe(struct spi_device *spi)
{
...;
spi->mode = SPI_MODE_0; //SPI_MODE_0;SPI_MODE_1;SPI_MODE_2;SPI_MODE_3;
...;
}

```

## 5、FIFO和DMA模式

SPI控制器支持DMA模式和FIFO模式传输数据。控制器通过传输数据长度自动选择。FIFO模式的长度小于或等于32字节，DMA模式的长度大于32字节。

## 6、支持多个设备

如果要在SPI总线上添加多个设备，可以使用GPIO作为支持SPI\_CS。例如，在SPI总线0上添加两个SPI device，使用GPIO136和GPIO30作为SPI\_CS1和 SPI\_CS2，如下所示

```

&spi4{
pinctrl-names = "default","sleep";
pinctrl-0 = <&spi4_pins_default>;
pinctrl-1 = <&spi4_pins_sleep>;
cs-gpios = <&pio 136 GPIO_ACTIVE_HIGH><&pio 30 GPIO_ACTIVE_HIGH>;
ak4619: ak4619@0 {
compatible = "akm,ak4619";
reg = <0>;
spi-max-frequency = <4000000>;
spi-cpha;
spi-cpol;
ak4619,pdn-gpio = <&pio 223 0>;
ak4619,mute-gpio = <&pio 130 0>;
status = "okay";
};
};

```

### 7.8.1.5 SPI 虚拟化配置方法

Yocto配置

```
CONFIG_DEVICE_MODULES_VHOST_SPI=m
```

Android配置

```
CONFIG_VIRTIO_SPI=m
```

```

//
//spi
&spi1 {
pinctrl-names = "default";
pinctrl-0 = <&ak7709_codec_spi>;
/*pinctrl-1 = <&ak7709_codec_pdn>;*/
status = "okay";
ak7709: ak7709@0 {
status = "okay";
compatible = "akm,ak7709";
#sound-dai-cells = <0>;
reg = <0>;
spi-max-frequency = <1000000>;
/*ak7709,pdn-gpio = <&pio 147 0>;*/
};

```

```

};
//pingctrlSPI pinmux
&pio {
ak7709_codec_spi: spimode-default {
pins-cmd-dat {
pinmux = <PINMUX_GPIO53__FUNC_SPI1_MO>,
<PINMUX_GPIO52_FUNC_SPI1_MI>,
- - - -
<PINMUX_GPIO51_FUNC_SPI1_CSB>,
- - - -
<PINMUX_GPIO50_FUNC_SPI1_CLK>;
- - - -
};
};
//
vhost_spi: vhost-spi {
compatible = "grt,vhost-spi";
};
//
spi1_virtio: spi1-virtio {
compatible = "mediatek,virtio-spi";
id = <1>;
status = "ok";
};

```

### 7.8.1.6 SPI passthrough 配置方法

理论当yocto spi adapter正常情况下，以及Android spi adapter驱动已经加载的情况下，只需要把spi的中断号配置给Android。以spi1为例

1、yocto disable

```

&spi1 {
status = "disabled";
}

```

2、android enable

```

&spi1 {
status = "ok";
}

```

3、spi1中断号配置给Android

```

yocto/prebuilt/hypervisor/grt_mt8676/uos_alps_pv8676_xx.lua
+uos_config:setVgiclrq(275)

```

```
src > kernel > linux > v6.1_mt8676 > co_device_module > arch > arm64 > boot > dts > mediatek > mt6897.dts
31 / {
4826 soc {
7404 };
7405
7406 spi1: spi@11011000 {
7407 compatible = "mediatek,mt6985-spi";
7408 mediatek,pad-select = <0>;
7409 mediatek,tickdly = <3>, <0>;
7410 reg = <0 0x11011000 0 0x100>;
7411 interrupts = <GIC_SPI 243 IRQ_TYPE_LEVEL_HIGH 0>;
7412 clocks = <&topckgen_clk CLK_TOP_MAINPLL_D6_D2>,
7413 <&topckgen_clk CLK_TOP_SPI1_SEL>,
7414 <&pericfg_ao_clk CLK_PERAOP_SPI1_BCLK>;
7415 clock-names = "parent-clk", "sel-clk", "spi-clk";
7416 clock-source-type = "mainpll";
7417 };
```

spi中断号=设备数interrupts243+32 = 275

## 7.8.2 I2C

76总共有0~13 14路i2c，速率最高可配3.4Mb/s，默认所有i2c 控制器的中断都配置到sos。

### 7.8.2.1 I2C 虚拟化配置方法

1, front-driver

dts 中id对应真实i2c bus id。

```
i2c0_virtio: i2c0-virtio {
 compatible = "mediatek,virtio-i2c";
 id = <0>;
 status = "okay";
};
```

driver code path

vendor/mediatek/kernel\_modules/virt/i2c\_virtio

2, backend driver

kernel/modules/mt8678/virt/grt/i2c\_vhost:

### 7.8.2.2 I2C passthrough 配置方法

理论当yocto i2c adapter正常情况下，只需要把i2c 的中断号配置给Android。

以i2c10直通Android为例

1.1 Android

1, i2c-mt65xx ko

diff --git a/ko\_order\_table.csv b/ko\_order\_table.csv

+i2c-mt65xx.ko,./../kernel\_device\_modules-6.1/drivers/i2c/busses/i2c-mt65xx.ko,vendor,Y,N,user/userdebug/eng

1.2 yocto

1, i2c10 gpio配置

```
diff --git a/dws/mt6897/auto8676p1_64_hyp.dws b/dws/mt6897/auto8676p1_64_hyp.dws
```

```
+ <gpio157>
+ <eint_mode>>false</eint_mode>
+ <def_mode>1</def_mode>
+ <inpull_en>>true</inpull_en>
+ <inpull_selhigh>>true</inpull_selhigh>
+ <def_dir>IN</def_dir>
+ <out_high>>false</out_high>
+ <smt>>true</smt>
+ <ies>>true</ies>
+ </gpio157>
+ <gpio158>
+ <eint_mode>>false</eint_mode>
+ <def_mode>1</def_mode>
+ <inpull_en>>true</inpull_en>
+ <inpull_selhigh>>true</inpull_selhigh>
+ <def_dir>IN</def_dir>
+ <out_high>>false</out_high>
+ <smt>>true</smt>
+ <ies>>true</ies>
+ </gpio158>
```

1.3 thyp-sdk

i2c10中断号配置给Android, lua文件配置需要 根据dts中配置的中断号加偏移, +32

i2c10 dts 中断配置如下:

```
interrupts = <GIC_SPI 196 IRQ_TYPE_LEVEL_HIGH 0>;
```

uos\_alps\_pv8676.lua 则需要 $196+32 = 228$

```
+uos_config:setVgiclrq(228)
```

编译到版本

```
cp uos_alps_pv8676.lua yocto/prebuilt/hypervisor/grt_mt8676/uos_alps_pv8676_mix.lua
```

## 7.8.3 UART

76总共有0~3 4路uart, 速率最高可配3Mb/s, 默认所有uart 控制器的中断都配置到sos

### 7.8.3.1 UART 虚拟化配置方法

采用virtio-console方式

1, front driver

kernel/drivers/char/virtio\_console.c

2, backend driver

kernel/modules/mt8676/virt/grt/vhost\_uart/uart.c

3, 测试前端生成的uart 虚拟节点

find . -name vport\*

获取uart数据

cat /dev/vport\*

uart发送数据

echo 123 > /dev/vport\*

### 7.8.3.2 UART passthrough 配置方法

理论当yocto 和android 端uart驱动加载正常情况下，只需要把uart 的 中断号配置给Android，uart便直通到Android。

1, Android

uart dts配置

```
--- a/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts
+++ b/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts
+&pio {
+ uart2_pins_default: uart2-pins-default {
+ pins-rx {
+ pinmux = <PINMUX_GPIO117__FUNC_URXD2>;
+ input-enable;
+ bias-pull-up;
+ };
+ pins-tx {
+ pinmux = <PINMUX_GPIO121__FUNC_UTXD2>;
+ };
+ pins-rts {
+ pinmux = <PINMUX_GPIO162__FUNC_URTS2>;
+ output-enable;
+ };
+ pins-cts {
```



```

+ pinmux = <PINMUX_GPIO161__FUNC_UCTS2>;
+ input-enable;
+ };
+ };
+};
+
&uart2 {
- status = "disabled";
+ pinctrl-names = "default";
+ pinctrl-0 = <&uart2_pins_default>;
+ status = "okay";
};
&uarthub {

- status = "disabled";
+ status = "okay";
};

```

uart驱动ko加入到ko\_order\_table.csv

```

--- a/ko_order_table.csv
+++ b/ko_order_table.csv
+mtk-uart-apdma.ko,./kernel_device_modules-6.1/drivers/dma/mediatek/mtk-uart-
apdma.ko,ramdisk,Y,Y,user/userdebug/eng
+uarthub_drv.ko,./kernel_device_modules-
6.1/drivers/misc/mediatek/uarthub/uarthub_drv.ko,ramdisk,Y,Y,user/userdebug/eng
+8250_mtk.ko,./kernel_device_modules-
6.1/drivers/tty/serial/8250/8250_mtk.ko,ramdisk,Y,Y,user/userdebug/eng

```

2, yocto

```

disable uart2
--- a/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts
+++ b/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts
@@ -393,7 +393,7 @@
&uart2 {
pinctrl-names = "default";
pinctrl-0 = <&uart2_pins_default>;
- status = "okay";
+ status = "disabled";

```

```
};
```

uart2 gpio配置成串口 mode

```
--- a/dws/mt6897/auto8676p1_64_hyp.dws
+++ b/dws/mt6897/auto8676p1_64_hyp.dws
@@ -1,5 +1,5 @@
<?xml version="1.0" encoding="UTF-8"?>
-<!--dct_version="2.5" buid_sn="161102" dws_modification_time="11.12.2024.15:42:26"-->
+<!--dct_version="2.5" buid_sn="161102" dws_modification_time="11.13.2024.14:22:06"-->
<dct_cfg>
<general chip="MT6897">: : :
<proj>Ponsot_Phone</proj>
@@ -1265,10 +1265,10 @@
</gpio116>
<gpio117>
<eint_mode>>false</eint_mode>
-
+
<def_mode>0</def_mode>
+
<def_mode>3</def_mode>
<inpull_en>true</inpull_en>
-
+
<inpull_selhigh>>false</inpull_selhigh>
-
+
<def_dir>OUT</def_dir>

+
+
<inpull_selhigh>true</inpull_selhigh>
+
+
<def_dir>IN</def_dir>
<out_high>>false</out_high>
<smt>true</smt>
<ies>true</ies>
@@ -1306,10 +1306,10 @@
</gpio120>
<gpio121>
<eint_mode>>false</eint_mode>
-
+
<def_mode>0</def_mode>
-
+
<inpull_en>true</inpull_en>
+
+
<def_mode>3</def_mode>
+
+
<inpull_en>>false</inpull_en>
<inpull_selhigh>>false</inpull_selhigh>
```

```
- <def_dir>OUT</def_dir>
+ <def_dir>IN</def_dir>
<out_high>>false</out_high>
<smt>>true</smt>
<ies>>true</ies>
```

3, thyp-sdk

uart2控制器中断号 284配置给Android, DMA中断255, 256也要配置给Android。

```
uos_alps_pv8676.lua
+uos_config:setVgiclrq(284)
+uos_config:setVgiclrq(255)
+uos_config:setVgiclrq(256)
```

## 7.8.4 GPIO

此处介绍下GPIO概况, 包括76有多少路GPIO, 当前公板配置状态

### 7.8.4.1 名词解释

DATAIN: pin读到的值, 仅在IES = 1时有效

DATAOUT: GPIO mode时设定output High/Low

DIR: GPIO mode时设置Pin input/output

DRV: Pin的普通driving设定

EH: I2C类Pin专有的driving设定

IES: input enable

MODE: Pinmux设定, 填入mode number既可

PD: 部分pin打开pull down时, PU需为0

PU: 部分pin打开pull up时, PD需为0

PUPD: 部分内部上下拉阻值可调的Pin的pull up/down设定

R0/R1: 上下拉电阻启用 (部分Pin可用)

RSEL: 部分Pin调整上下拉阻值

SMT: 波形过滤功能

### 7.8.4.2 架构介绍

General-Purpose Input/Output (GPIO), 意为通用输入/输出。在嵌入式系统中, GPIO是一种用于与外部设备进行数字信号交互的接口。它可以通过设置为输入或输出模式来读取或控制外部设备的状态。

MediaTek SoC 提供 PIN Controller 硬件单元以实现:

引脚功能配置。例如该 I/o pin 是一个普通的 GPIO 还有一些特殊功能引脚(例如 EMMC 上的 CMD 信号)。

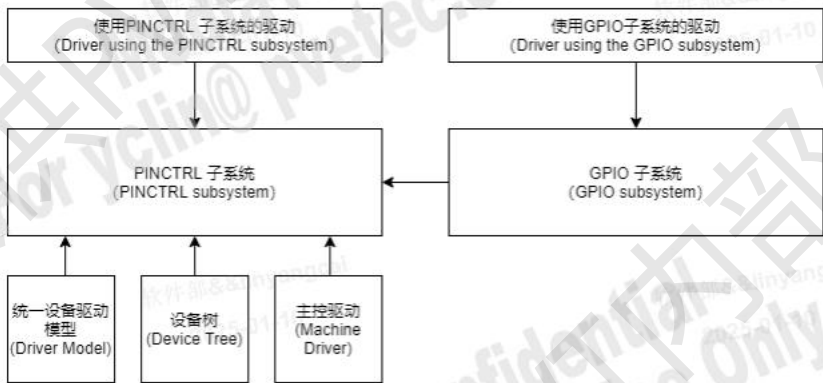
引脚特性配置。例如 pull-up/down 电阻的设定, drive-strength 的设定。

提供 GPIO Controller 硬件单元以实现:

配置 GPIO的方向。

如果是输出, 可以配置为 high level 或者 low level.

如果是输入, 可以获取 GPIO 引脚上的电平状态。



### 7.8.4.3 GPIO使用

安卓和yocto中gpio的配置节点都在

路径: /proc/mtk\_gpio/soc.pinctrl

如图:

```
console:/ # cat /proc/mtk_gpio/soc.pinctrl
pins base: 262
PIN: (MODE)(DIR)(DOUT)(DIN)(DRIVE)(SMT)(IES)(PULL_EN)(PULL_SEL)(R1 R0)
000: 0001001111
001: 0111001110
002: 0000001110
003: 0111001111
004: 0001001111
```

可直接操作该节点手动配置gpio

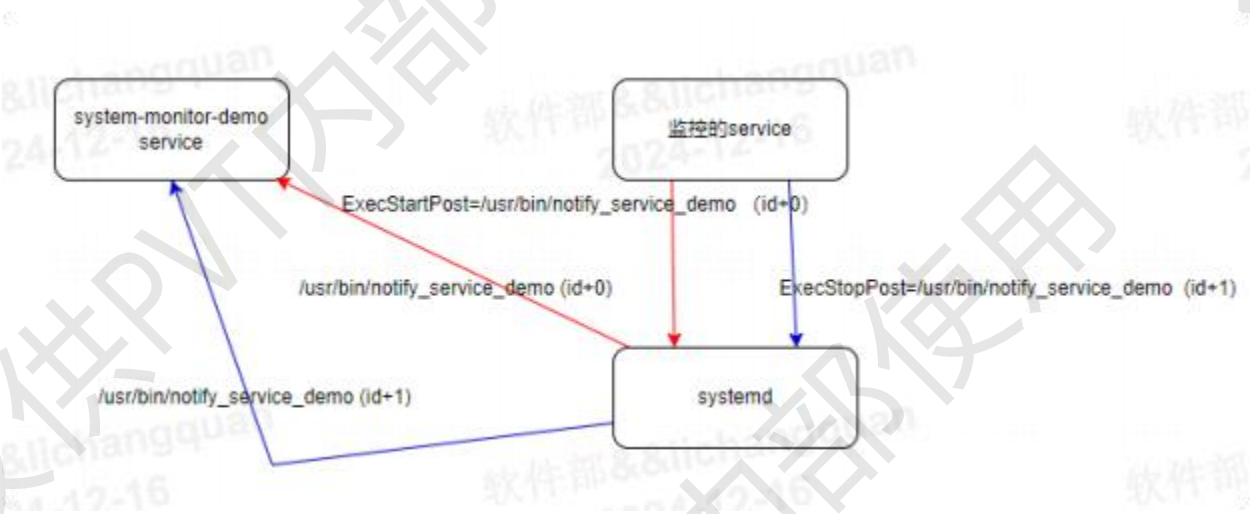
手动拉高某个gpio口 (135)

echo out 135 1 > /proc/mtk\_gpio/soc.pinctrl

## 7.9 保活机制

### 7.9.1 进程保活

#### 7.9.1.1 设计框图



1, notify\_service\_demo

notify\_service\_demo此应用notify system-monitor-demo，实时信号机制，参数是服务id+服务状态（0：start，1：stop）

2, system-monitor-demo

功能是 监测服务程序 启动和关闭情况，做下记录。

### 7.9.2 系统保活

介绍下有哪些虚拟化机，如何获取虚拟机状态，如何控制虚拟机，如何保活虚拟机

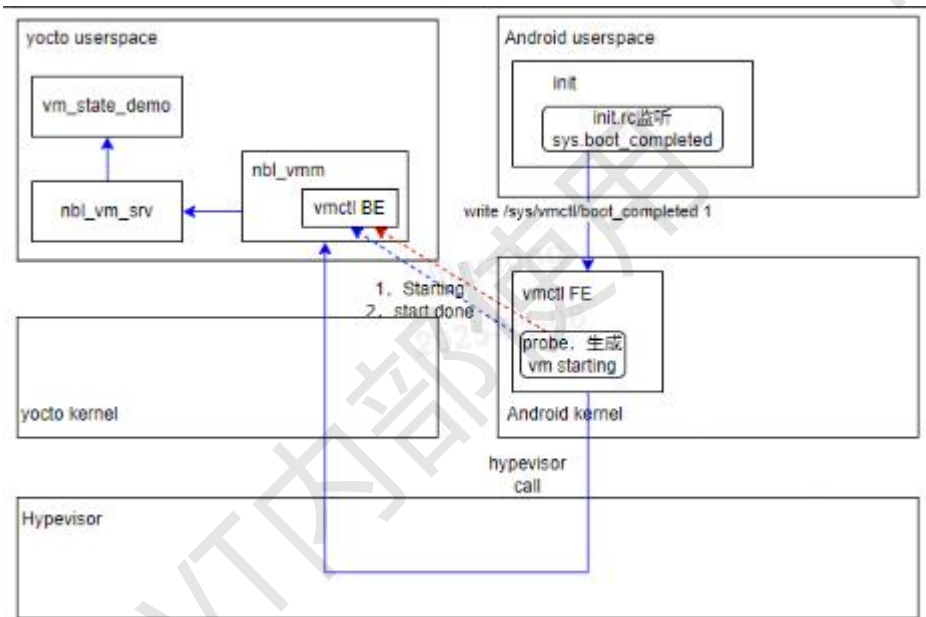
#### 7.9.2.1 保活机制介绍

虚拟机保活机制主要是监听到虚拟机异常时，重新拉起虚拟机。

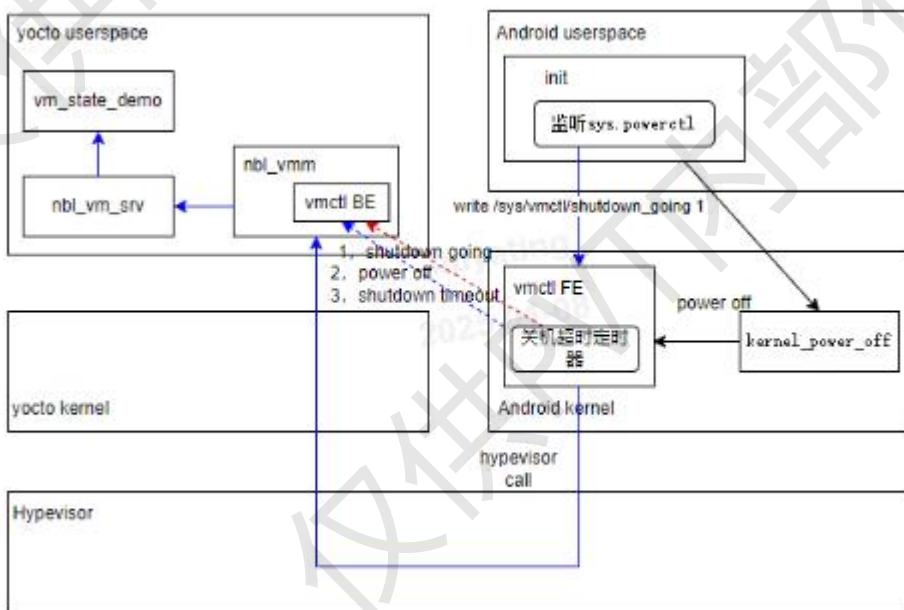
VM状态通过vmctl 传递到nbl\_vmm。

新增VM状态流转：

1, 开机状态



## 2, 关机状态



## 7.9.2.2 获取虚拟机状态

虚拟机状态是由VM OS vmctl 传递状态到nbl\_vmm。

以8676 双系统为例

1, Android vmctl

[vendor/mediatek/kernel\\_modules/virt/vmctl/vmctl.c](#)



```

static int sys_kernel_panic(struct notifier_block *self, unsigned long v,
 void *p)
{
 // struct pt_regs saved_regs;
 dump_stack();
 pr_info("sys_kernel_panic\n");
 atomic_notifier_call_chain(&vmctl_notifier_list, REBOOT_SOFT, p);
 // crash_setup_regs(&saved_regs, NULL);
 // mrdump_common_die(AEE_REBOOT_MODE_KERNEL_PANIC,
 // "Kernel Panic", &saved_regs);
 mtk_wdt_set_sw_rst_status();
 pr_info("sys_kernel_panic send VMCTL_KERNEL_PANIC\n");
 vmctl_send_event(VMCTL_KERNEL_PANIC, false);
 return 0;
}

static struct notifier_block kernel_panic_notifier = {
 .notifier_call = sys_kernel_panic,
};

```

==>

```

static int vmctl_send_event(uint8_t type, bool async)
{
 unsigned long flags;
 bool notify = false;
 int err;
 struct virtio_vmctl_vbuffer *vbuffer = build_request(type, async);
 struct virtqueue *vq;

 BUG_ON(g_vmctl == NULL || vbuffer == NULL || g_vmctl->lock == NULL
 || g_vmctl->vbuf_lock == NULL);
 vq = g_vmctl->vqs[VIRTIO_VMCTL_VQ_REQ];

 spin_lock_irqsave(g_vmctl->lock, flags);
 err = submit_request(vq, vbuffer);
 BUG_ON(err != 0);

 if (virtqueue_kick_prepare(vq))
 notify = true;
 spin_unlock_irqrestore(g_vmctl->lock, flags);

 if (notify)
 virtqueue_notify(vq);

 if (!async) {
 while (!vbuffer->completed)
 cpu_relax();

 err = vbuffer->res.ret;
 kfree(vbuffer);
 }

 return err;
}

```

2, nbl\_vmm vmctl.cc

获取到Android 端状态。

```
zx_status_t VirtioVmctl::HandleCommand(VirtioQueue* queue,
 uint16_t head,
 uint32_t* used) {
 virtio_desc_t desc;
 auto status = queue->ReadDesc(head, &desc);
 CHECK(status == ZX_OK);

 auto req = (struct vmctl_request*)desc.addr;
 switch (req->type) {
 case VMCTL_POWER_OFF:
 LOG(INFO) << "Host vmm recieve VMCTL_POWER_OFF";
 reporter->Report(nbl_vmm::VmState::POWER_OFF);
 break;
 case VMCTL_RESTART:
 LOG(INFO) << "Host vmm recieve VMCTL_RESTART";
 reporter->Report(nbl_vmm::VmState::RESTART);
 break;
 case VMCTL_KERNEL_PANIC:
 LOG(INFO) << "Host vmm recieve VMCTL_KERNEL_PANIC";
 reporter->Report(nbl_vmm::VmState::KERNEL_PANIC);
 break;
 case VMCTL_KICK_WDT:
 LOG(INFO) << "Host vmm recieve VMCTL_KICK_WDT";
 reporter->Report(nbl_vmm::VmState::KICK_WDT);
 break;
 case VMCTL_SUSPEND:
 LOG(INFO) << "Host vmm recieve VMCTL_SUSPEND";
 reporter->Report(nbl_vmm::VmState::VM_SUSPEND);
 break;
 case VMCTL_RESUME:
 LOG(INFO) << "VHost vmm recieve VMCTL_RESUME";
 reporter->Report(nbl_vmm::VmState::VM_RESUME);
 break;
 default:
 CHECK(false) << "unknown vmctl req: " << (uint32_t)req->type;
 } // end switch req->type
}
```

## 7.9.3 控制虚拟机状态

### 7.9.3.1 休眠唤醒android方法(L+A)

Android 单独休眠唤醒

echo powerkey > /sys/guest\_os/android/pm\_state //android 休眠

echo 2 > /sys/guest\_os/android/resume //android 唤醒

cat /sys/guest\_os/android/pm\_state //查看android 状态

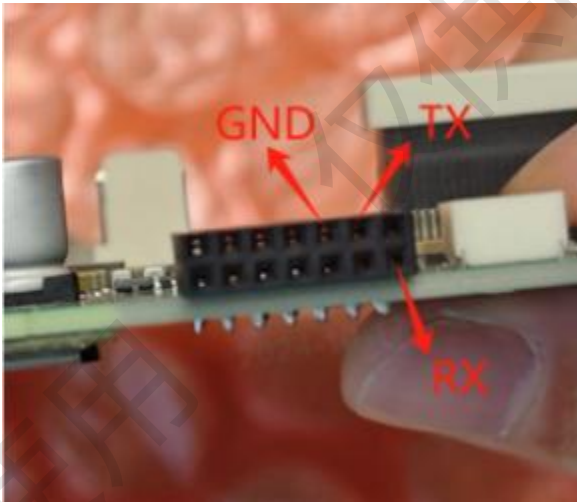
### 7.9.3.2 stop & start 方法 (L+A)

```
sh-3.2# nbl_vm_ctl -h
Usage: nbl_vm_ctl [options]
shell: attach to virtual machine console
start: start virtual machine
stop: stop virtual machine
restart: restart virtual machine
dump: dump vcpus state
vminfo: dump vmid ,state information list
option 参数是 - vmid (1,2,3...), shell某个VM指令如下:
nbl_vm_ctl shell --vmid 1 是tbox,
nbl_vm_ctl shell --vmid 0是android
```

## 7.10. misc debug

### 7.10.1 log 【抓取，分析】

#### 7.10.1.1 串口log抓取



串口log中包含yocto、android以及nebula log (SDK)

1.其中android log带有nbl\_vm\_srv为标志的log

如

```
[20241211_10:13:57438][37.119358] nbl_vm_srv[2681]: [13.479003][T600335]
modprobe: [touch_boost][name:touch_boost&]
touch_boost_init, cpu_num:3
[20241211_10:13:57438][37.128079] nbl_vm_srv[2681]: [13.494917][T600335]
modprobe: [touch_boost][name:touch_boost&]
cpu_policy_init, policy[0]: first:0, min:1900000, max:1900000
```

## 2.nebula log带有nbl\_vmm或者nbl\_vm\_srv

如

```
[414.533316] nbl_vm_srv[4681]: nbl_vmm E 02-27 17:32:51 4681 4681
guest_config.h:88] setVmid: 0
[414.533351] nbl_vm_srv[4681]: nbl_vmm E 02-27 17:32:51 4681 4681
guest_config.h:94] setCpus:
[414.533370] nbl_vm_srv[4681]: nbl_vmm E 02-27 17:32:51 4681 4681
guest_config.h:100] setBindcpus: 2
[20241211_10:16:41022][18.669227] nbl_vm_srv[2688]: nbl_vm_srv I 11-30 14:06:04 2688
2688 vm_controller.cc:183] nbl vm
controller start vm finish vmid:0
[20241211_10:16:41023][18.680110] nbl_vm_srv[2688]: nbl_vm_srv I 11-30 14:06:04 2688
2688 vm_controller.cc:421] VM Controller
parse UOS json vmid:1
```

## 3. 不带任何标识的就是yocto log

```
[20241211_10:16:27462][5.101556][T1601003] modprobe:
[name:tcpc_class&]tcpc_class_init (2.0.25_MTK)
[20241211_10:16:27462][5.102498][T1601003] modprobe: [name:tcpc_class&]Init Richtek
RegMap 1.2.1_G
[20241211_10:16:27462][5.103469][T1601003] modprobe: [name:tcpc_class&]size:0x10
tag:0x41000802 mode:0x0 type:0x2
```

注：由于安卓端log由vm\_console统一管理，所以只有当安卓端vm\_console起来之后才有log输出，此前的log暂时没有

## 4、Log动态开关

关闭yocto 侧kernel log:

```
echo 0 > /proc/sys/kernel/printk
```

关闭android 侧 kernel log:

```
echo 0 > /proc/sys/kernel/printk
```

打开串口log

```
echo 1 >/proc/mtprintk
```

### 7.10.1.2 MTK log 抓取

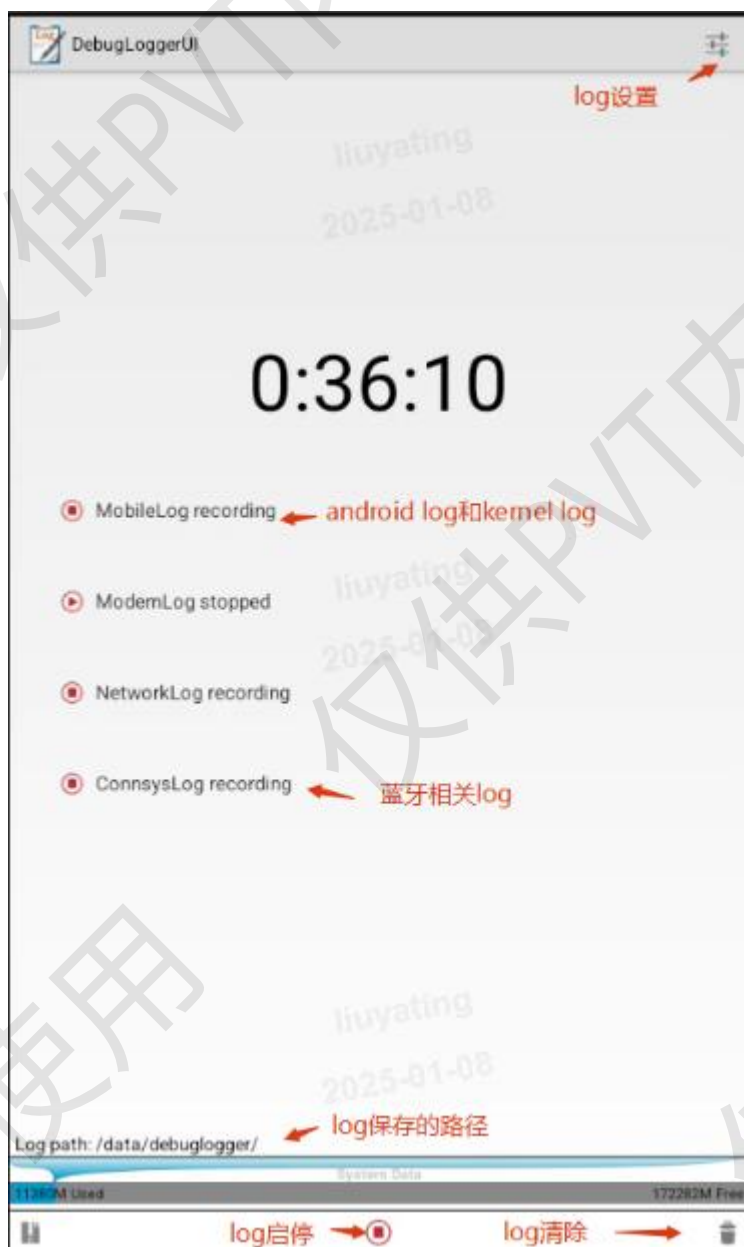
1, Android端

Log保存默认路径是/data/debuglogger/

(1) adb 命令显示DebugLoggerUI主界面

```
adb shell am start -n com.debug.loggerui/com.debug.loggerui.MainActivity
```

UI界面控制如下:



2、yocto端

(1) 重启后生效启停命令

deep\_start/deep\_stop 启/停Mobilelog，且启停状态重启后延续：

```
adb shell mobile_log_d --control deep_stop
```

```
adb shell mobile_log_d --control deep_start
```

(2) 只影响此次开机启停命令

```
adb shell mobile_log_d --control start
```

```
adb shell mobile_log_d --control stop
```

### 7.10.1.3 MTK AEE log抓取

aee log会默认开启

db 存储路径

/data/aee\_exp

/data/vendor/aee\_exp

## 7.10.2 adb【连接，切换】

1.将usb切到安卓(双系统)

```
adb -d root
```

```
adb -d shell sysenv-test write USB_SWITCH A
```

```
#adb -d shell sysenv-test read USB_SWITCH //查看是否成功
```

```
adb -d reboot //重启
```

2.将usb重新切回yocto(双系统)

串口中查看yocto adb中/data/network\_adb，存在则删除

下载附件的usb\_android\_to\_yocto.rar，双击bat就行

再重启

3.USB切到yocto后，安卓如何使用adb(双系统)

```
adb forward tcp:7777 tcp:6666
```

```
adb connect 127.0.0.1:7777
```

```
adb -e shell //android shell
```

4.USB切到Android后，Yocto如何使用adb(双系统)

在Android adb shell执行：

```
mkfifo /data/local/tmp/fifo
```

```
nc -l -p 6666 0</data/local/tmp/fifo | nc 192.168.1.1 5555 1>/data/local/tmp/fifo &
```

```
adb forward tcp:7777 tcp:6666
```

```
adb connect 127.0.0.1:7777
```

```
adb -e shell //yocto shell
```

5. 三系统network\_adb



虽然USB目前是直通给其中一侧，但也能通过network\_adb的方式连接到其他两个系统的adb，以下列出每个系统进入adb shell的指令，如USB默认在SOS，所以用第一行指令。

USB	SOS	UOS Android	UOS Tbox
SOS	adb -d shell	adb -d forward tcp:7666 tcp:6666 adb connect 127.0.0.1:7666 adb -s 127.0.0.1:7666 shell	adb -d forward tcp:7667 tcp:6667 adb connect 127.0.0.1:7667 adb -s 127.0.0.1:7667 shell
UOS Android	adb -d forward tcp:7665 tcp:6665 adb connect 127.0.0.1:7665 adb -s 127.0.0.1:7665 shell	adb -d shell	adb -d forward tcp:7667 tcp:6667 adb connect 127.0.0.1:7667 adb -s 127.0.0.1:7667 shell
UOS Tbox	adb -d forward tcp:7665 tcp:6665 adb connect 127.0.0.1:7665 adb -s 127.0.0.1:7665 shell	adb -d forward tcp:7666 tcp:6666 adb connect 127.0.0.1:7666 adb -s 127.0.0.1:7666 shell	adb -d shell

6. 连接usb，利用USB直连方式也可以切换

注：需要切换到哪个系统的 USB，就运行对应的命令

安卓系统执行下方命令给需要 root 权限

命令执行后，重启才能生效

sysenv_test write USB_SWITCH T	//Tbox
sysenv_test write USB_SWITCH Y	//Yocto
sysenv_test write USB_SWITCH A	//android
sysenv_test read USB_SWITCH	//查看是否生效

### 7.10.3 sos uos console 切换

仅支持sos为基准进入uos console

nbl\_vm\_ctl shell --vmid 0                   \\进入安卓console

nbl\_vm\_ctl shell --vmid 1                   \\进入tbox

uos回到sos (使用快捷键ctrl+D)

注：此种方式，无法使用 adb 投屏等功能

## 7.10.4 android yocto 通过console 数据交互

可以通过virtiofs进行yocto和Android的数据交互

安卓端执行

```
su
mkdir /mnt/sdcard/yocto_data
mount -t virtiofs media /mnt/sdcard/yocto_data/
```

目前yocto端可共享的文件夹有/data/share/media和/mnt/vendor/nvdata

安卓端是默认挂载这两个文件夹的

代码路径: device/mediateksample/auto8676p1\_64\_bsp\_vm/init.project.rc

```
#virtio-fs sample
mkdir /data/vendor/share/
mkdir /data/vendor/share/media/
mount virtiofs media /data/vendor/share/media
restorecon --recursive --cross-file systems /data/vendor/share/media
mkdir /mnt/vendor/nvdata
mount virtiofs nvdata /mnt/vendor/nvdata
chmod 0771 /mnt/vendor/nvdata
chown root system /mnt/vendor/nvdata
restorecon --recursive --cross-file systems /mnt/vendor/nvdata
```

## 7.10.5 remount挂载方法

三系统remount挂载方法如下:

1、在烧录的时需要替换如下几个文件:

Preloader\_a 选择 bl2\_sboot\_dis.img

Preloader\_b 选择 bl2\_sboot\_dis.img

Bl-an 选择 bl2-an\_sboot\_dis.img

Boot\_a 选择 boot-debug.img

vendor\_boot\_a 选择 vendor\_boot-debug.img

2、android端: adb -d shell remount

3、yocto端: adb -s 127.0.0.1:7665 shell "mount -o remount,rw /"

## 7.11 cpu配置

### 7.11.1 虚拟机CPU配置

对于MT8676，它有8个物理核。其中有4个小核，分别是Core0~Core3。有3个中核，分别是Core4~Core6。有1个大核，是Core7。

对于每一个物理CPU，在hypervisor中，都有一个虚拟的vCPU与之对应。

物理CPU核既可以让一个VM独享，也可以让几个VM共享。如果CPU核为某一个VM 独享，需要在除该VM之外的所有其他VM的配置文件中，将该CPU设置成 offline。

如果要多个VM共享CPU，则要在相应的VM配置文件中，让该CPU处于onLine。

• MT8676设置SOS CPU offline的方法如下：

在编译时修改：

在文件yocto/prebuilt/hypervisor/grt\_mt8676/nbl\_vm\_pre.sh中，添加  
echo 0 > /sys/devices/system/cpu/cpux/online //cpux → cpu0, cpu1, cpu2, ... cpu7

对于SOS 来说，Core0用于接收中断，不能关闭。该方法是编译时生效。

在系统运行时修改：

通过SOS adb shell进入目录/vendor/etc/hyper\_android，修改nbl\_vm\_pre.sh文件，保存重启。

• MT8676设置UOS(Android) CPU offline的方法如下：

在编译时修改：

在文件alps/devices/mediatek/mt6897/init.mt6897.rc中，添加  
write /sys/devices/system/cpu/cpux/online 0 //cpu x → cpu0, cpu1, cpu2, ... cpu7

对于UOS(Android)， Core2用于接收中断，不能设置成offline。

在运行时修改：

通过Android adb shell进入目录/vendor/etc/init/hw，修改init.mt8676.rc文件，保存重启。

### 7.11.2 Nebula CPU调度策略配置

#### 7.11.2.1 Nebula对共享核调度策略

对于VM独享的CPU，nebula不参与VM内部进程调度。由VM的Linux调度程序来调度。

对于VM共享的CPU，nebula参与VM间的进程调度。调度策略为：

1. 将共享的CPU运行时间，以period为单位进行划分。让每个VM中的进程都有机会在period时间内运行，不会让一个VM独占一个period时间。

2. 在period时间单位的基础上，规定每个VM在period的运行时间budget。即每个VM在period内最多只能运行的时间为budget，时间到了之后，就必须让出CPU，让其他的VM进行运行。
3. 对VM内部的进程，nebula将其分4类，分别为IRQ, Spin\_Lock, RT\_Thread和Normal\_Thread。这四种进程的默认等级优先级为IRQ > Spin\_Lock = RT\_Thread > Normal\_Thread。
4. 对于相同优先级的进程，同时要在共享核上运行，则nebula会通过调度，让其交替运行。这四类进程的时间片分别是：IRQ(100us), Spin\_Lock (200us), RT\_Thread(200us), Normal Thread(budget)。例如，如果SOS和UOS上同时有两个RT Thread进程要在相同的CPU上运行，则RT\_Thread (SOS)和RT\_Thread(UOS)交替200us运行。

### 7.11.2.2 Nebula共享核调度策略参数配置

- 设置共享核period的方法：

在文件yocto/src/hypervisor/grt/thy-sdk/products/mt8676-mix/guest-configs/sos\_mt8676.json中，“period”字段对应的值

为period时间，单位us。8个核，所以有8个值。

- 设置共享核budget的方法：

SOS设置在文件yocto/src/hypervisor/grt/thy-sdk/products/mt8676-mix/guest-configs/sos\_mt8676.json中，“budgets”字段对应的值为budgets时间，单位us。8个核，所有有8个值。

UOS设置在文件yocto/src/hypervisor/grt/thy-sdk/products/mt8676-mix/guest-configs/uos\_alps\_pv8676.lua中，budgets数组

中的值为budgets时间，单位us。8个核，所有有8个值。

- 设置RT\_Thread, Spin\_Lock和IRQ的优先级和时间片。

在文件yocto/src/hypervisor/grt/thy-sdk/products/mt8676-mix/guest-configs/sos\_mt8676.json中。“sched-priority”对应的字段为相对于Normal Thread的优先级，分别对应RT\_Thread, Spin\_Lock和IRQ进程优先级。

“sched-timeslice”对应的字段为RT\_Thread, Spin\_Lock和IRQ的时间片，单位us。

## 7.12 中断配置

以8676 3os为例

涉及中断修改的配置文件如下：

- 1，分配给sos的中断配置文件：

yocto/prebuilt/hypervisor/grt\_mt8676/auto8676p1\_64\_hyp\_sos/sos\_mt8676.json

- 2，rproc os间通信用的中断配置文件

yocto/prebuilt/hypervisor/grt\_mt8676/auto8676p1\_64\_hyp\_sos/rproc.pb.txt

- 3，android 用的中断配置文件

yocto/prebuilt/hypervisor/grt\_mt8676/auto8676p1\_64\_hyp\_sos/uos\_alps\_pv8676.lua

4, tbox 用的中断配置文件

yocto/prebuilt/hypervisor/grt\_mt8676/auto8676p1\_64\_uos\_tbox/uos\_tbox\_pv8676.lua

由于这些配置文件都会配置中断，若超过两个文件同时配置一个中断号，必然会导致冲突或是覆盖。

加载先后顺序

rproc.pb.txt->sos\_mt8676.json->uos\_alps\_pv8676.lua, uos\_tbox\_pv8676.lua

uos\_alps\_pv8676.lua, uos\_tbox\_pv8676.lua 会覆盖sos\_mt8676.json配置；

rproc.pb.txt, uos\_alps\_pv8676.lua, uos\_tbox\_pv8676.lua 三者之间会形成冲突关系。

案例一：uart2直通Android

uart2涉及中断有3个，255，256，284；在没配置rproc.pb.txt，nebula 默认给rproc占用了255，256中断号。

当Android端使用uart2时，会造成整个系统timeout之后重启。

log如下：

```
[16:48:36.709]收←◆[85.709945] nbl_vm_srv[2975]: console:/ # echo 12341 > /dev/ttyS2
[16:48:55.671]收←◆(5)[00111.185] 02344.03815> #####
[16:48:55.703]收←◆ handle_smc_instruction 506 irq:321 timeout.
(5)[00111.185] 02344.03815> irq pending: vector:321 not recorded
(5)[00111.185] 02344.03815> ##### dump_gic_states 950 irq:321
(0)[00111.188] 01751.01981> ##### dump_gic_states_task 898 irq:321
(0)[00111.188] 01751.01981> GICD_IROUTER<321>:0x0
(0)[00111.188] 01751.01981> ##### irq 321 bind to vcpu: 0, pcpu: 0
(5)[00111.188] 02344.03815> ##### handle_smc_instruction 506 irq:322 timeout.
(5)[00111.188] 02344.03815> irq pending: vector:322 not recorded
(5)[00111.188] 02344.03815> ##### dump_gic_states 950 irq:322
(0)[00111.192] 01751.01981> ##### dump_gic_states_task 898 irq:322
(0)[00111.192] 01751.01981> GICD_IROUTER<322>:0x0
(0)[00111.192] 01751.01981> ##### irq 322 bind to vcpu: 0, pcpu: 0
(5)[00111.192] 02344.03815> ##### handle_smc_instruction 506 irq:339 timeout.
(5)[00111.192] 02344.03815> irq pending: vector:339 not recorded
(5)[00111.192] 02344.03815> ##### dump_gic_states 950 irq:339
(0)[00111.200] 01751.01981> ##### dump_gic_states_task 898 irq:339
(0)[00111.200] 01751.01981> GICD_IROUTER<339>:0x0
(0)[00111.200] 01751.01981> ##### irq 339 bind to vcpu: 0, pcpu: 0
(5)[00111.200] 02344.03815> ##### handle_smc_instruction 506 irq:339 timeout.
(5)[00111.200] 02344.03815> irq pending: vector:339 not recorded
(5)[00111.200] 02344.03815> ##### dump_gic_states 950 irq:339
(0)[00111.202] 01751.01981> ##### dump_gic_states_task 898 irq:339
(0)[00111.202] 01751.01981> GICD_IROUTER<339>:0x0
```

(0)[00111.202] 01751.01981> ##### irq 339 bind to vcpu: 0, pcpu: 0  
[16:49:02.647]收 ←◆(6)[00118.161] 02344.03825> ##### handle\_smc\_instruction 506 irq:487  
timeout.  
(6)[00118.161] 02344.03825> irq pending: vector:487 not recorded  
(6)[00118.161] 02344.03825> ##### dump\_gic\_states 950 irq:487  
(0)[00118.166] 01751.01981> ##### dump\_gic\_states\_task 898 irq:487  
(0)[00118.166] 01751.01981> GICD\_IROUTER<487>:0x0  
(0)[00118.166] 01751.01981> ##### irq 487 bind to vcpu: 0, pcpu: 0  
[16:49:02.856]收 ←◆(6)[00118.369] 02344.03825> #####  
[16:49:02.972]收 ←◆ handle\_smc\_instruction 506 irq:487 timeout.  
(6)[00118.369] 02344.03825> irq pending: vector:487 not recorded  
(6)[00118.369] 02344.03825> ##### dump\_gic\_states 950 irq:487  
(0)[00118.372] 01751.01981> ##### dump\_gic\_states\_task 898 irq:487  
(0)[00118.372] 01751.01981> GICD\_IROUTER<487>:0x0  
(0)[00118.372] 01751.01981> ##### irq 487 bind to vcpu: 0, pcpu: 0  
[16:49:03.060]收 ←◆(6)[00118.573] 02344.03825> ##### handle\_smc\_instruction 506 irq:487  
timeout.  
(6)[00118.573] 02344.03825> irq pending: vector:487 not recorded  
(6)[00118.573] 02344.03825> ##### dump\_gic\_states 950 irq:487  
(0)[00118.574] 01751.01981> ##### dump\_gic\_states\_task 898 irq:487  
(0)[00118.574] 01751.01981> GICD\_IROUTER<487>:0x0  
(0)[00118.574] 01751.01981> ##### irq 487 bind to vcpu: 0, pcpu: 0  
[16:49:03.264]收 ←◆(6)[00118.777] 02344.03825> ##### handle\_smc\_instruction 506 irq:487  
timeout.  
(6)[00118.777] 02344.03825> irq pending: vector:487 not recorded  
(6)[00118.777] 02344.03825> ##### dump\_gic\_states 950 irq:487  
(0)[00118.780] 01751.01981> ##### dump\_gic\_states\_task 898 irq:487  
(0)[00118.780] 01751.01981> GICD\_IROUTER<487>:0x0  
(0)[00118.780] 01751.01981> ##### irq 487 bind to vcpu: 0, pcpu: 0  
[16:49:03.468]收 ←◆(6)[00118.981] 02344.03825> ##### handle\_smc\_instruction 506 irq:487  
timeout.  
(6)[00118.981] 02344.03825> irq pending: vector:487 not recorded  
(6)[00118.981] 02344.03825> ##### dump\_gic\_states 950 irq:487  
(0)[00118.982] 01751.01981> ##### dump\_gic\_states\_task 898 irq:487  
(0)[00118.982] 01751.01981> GICD\_IROUTER<487>:0x0  
(0)[00118.982] 01751.01981> ##### irq 487 bind to vcpu: 0, pcpu: 0  
[16:49:03.680]收 ←◆(6)[00119.193] 02344.03825> ##### handle\_smc\_instruction 506 irq:487  
timeout.



会有hypevisor irq处理timeout log。

### 7.13.1 STR机制概要说明

在 MT8676 平台上, suspend 即 suspend to RAM, 此状态使所有的设备进入低功耗状态, 仅保留 RAM 自刷新。

## Suspend Power State:

- Tasks “freezed”
- 外部设备/内部部分模块关电或者进入低功耗模式
- System PLL/clock close
- ARM off
- DRAM 自刷新
- PMIC 进入低功耗模式
- VCORE off
- SPM run
- 等待硬件唤醒事件

在 MT8676 Hypervisor ( L+L+A ) 系统上, Host OS SoS 与 Guest OS Tbox 系统都是 Yocto , Guest OS IVI 系统是Android。系统级 Suspend 由 Host OS 发起, 触发 Guest OS 运行 Suspend Flow , Guest OS 都完成 Suspend Flow 之后 , Host OS 可以进入 Suspend , Host OS Suspend 完成之后整个系统才能进入省电的深度休眠模式。

### 7.13.2 STR框架

以EINT触发系统进出休眠为例

### 7.13.2.1 suspend flow

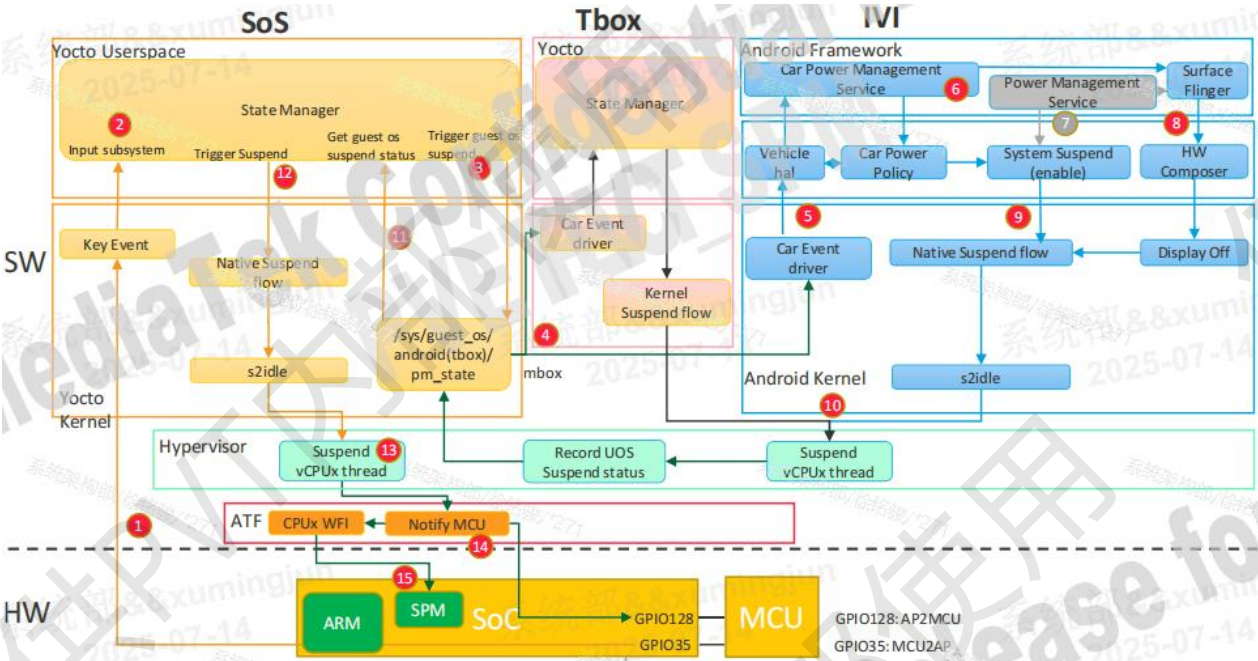


图 1-1. Hypervisor Suspend 流程

流程说明：

1. 平台进出休眠主要由 MCU 发起， MCU 通过 GPIO35 (MCU2AP) 的状态控制 SoC 进出休眠， GPIO35 被拉低后， 会触发中断， 该中断会被 Hypervisor 分发给 SoS 系统处理 SoS Kernel 收到中断后， 将 Key Event 上报。
2. 在 SoS userspace 中， MTK 增加了一段控制逻辑 SleepManager (state manager)， 用于管理 Guest OS 的电源状态， 这层逻辑主要是 soc 通过 input sub system 获取 key event 来触发的。
3. 当检测到休眠事件后， SoS userspace 写 powerkey event 到 /sys/guest\_os/android/pm\_state， 向两个 Guest OS (Android/Yocto) 发起休眠请求。
4. SoS 的 kernel driver 通过 Mailbox 方式将 suspend 请求通知到两个 Guest OS 的 car event driver。
5. 在 car event driver 中， 上报虚拟 Power Key (Keyevent=87)， 触发 Guest OS 端完整的休眠流程。
6. Guest OS IVI 由于 Vehicle HAL 没有现成的接口对接 Car EVENT Driver， 所以这里增加 Input subsystem 支持， 等待 Kernel 端送来的虚拟 Power Key， VHAL 收到后会通知 Car Power Management Service 启动休眠流程。
7. Power Management Service 依然监听 Power Key Event (Keyevent=116)， 这里没有执行 PMS Flow。
8. 触发关闭显示相关硬件。
9. CPMS 在没有唤醒锁检查的情况下触发 Suspend Flow， 触发 Android Kernel Suspend。 Kernel Suspend Flow 与原生系统中的流程相同。 根据执行顺序， 它依次调用设备注册的 Prepare/Suspend/Suspend\_late 回调函数， 在休眠阶段为每个 Device driver 完成必要的准备工作。

10. 各device 休眠后，送S2idle SMC指令到ATF关闭CPU Core，指令会被Hypevisor拦截，并挂起对应的vCPU线程。Android的休眠完成，Hypevisor会记录 Android状态，将running改为suspend，方便Host os通过结点查询。
11. 触发Android端休眠后，会阻塞并等待，直到从节点/sys/guest\_os/android (tbbox)/pm\_state 获取到两个 Guest OS 都为 Suspend 的状态。
12. 待 Guest OS 都 Suspend 后，Host OS 端上报 key code 87 触发关闭显示相关，然后再检查 Wakelock 状态，如无锁，则触发 Host OS Kernel Suspend Flow (echo mem > /sys/power/state)
13. 执行Yocto kernel suspend，完成Device suspend后，送S2idle SMC指令到ATF关闭CPU core，指令会被Hypevisor 拦截，并挂起对应的vCPU线程，然后 虚拟机做好backup，Hypevisor将S2idle指令再发送给 ATF，将物理CPU真正关闭。
14. 在 ATF 中，CPU0 执行 WFI 指令前，先将 GPIO128 (AP2MCU)拉低，通知 MCU ，SoC 端已经完成 Host OS 和 Guest OS 的 Suspend Flow。
15. 待Arm相关的硬件被关闭后，SPM（system power manager，用于电源管理的MCU）将接管整个系统的资源，控制DRAM进入自刷新模式，关闭26M和Vcore，通知PMIC进入Low power Mode。至此，整个休眠流程完成，系统处于低功耗状态，等待唤醒事件发生。

### 7.13.2.2 resume flow

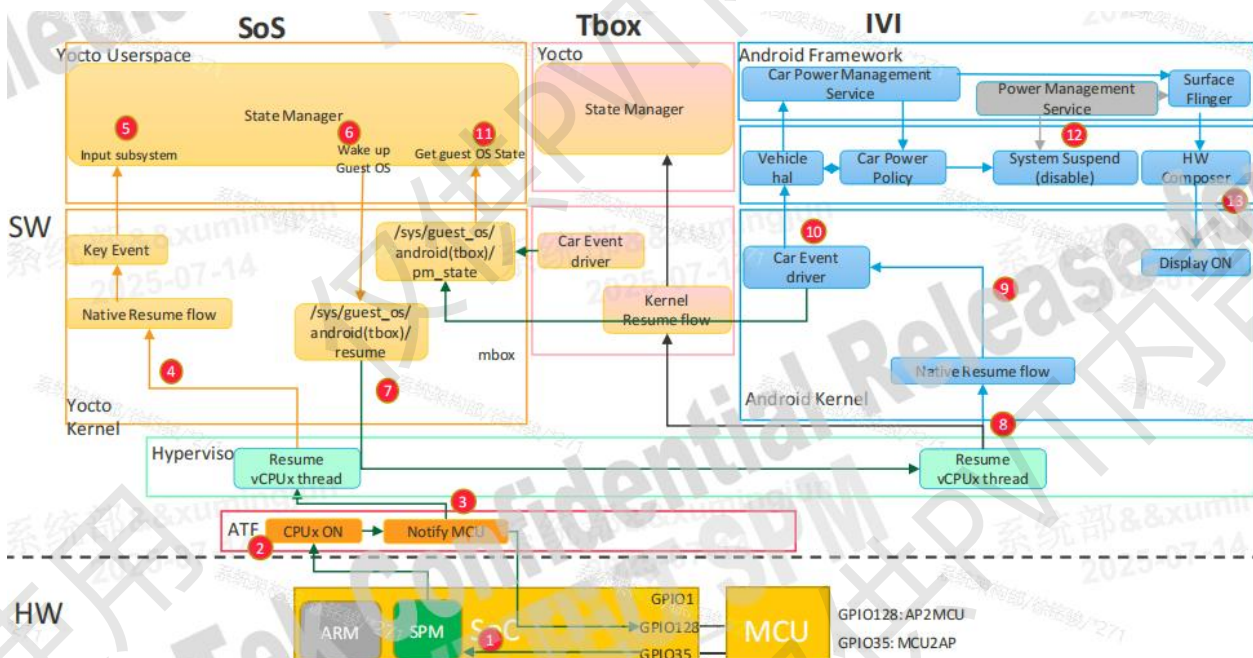


图 1-2. Hypervisor Resume 流程

流程说明：

1. MCU 收到唤醒事件后，拉高GPIO35，SPM收到EINT后，依次打开释放的资源，如通知PMIC 离开low power mode，打开VCORE和26M，发送命令通知DRAM离开自刷新。

2. 给CPU0上电,执行CPU Resume流程,再跳到suspend前设定的地址,开始执行系统Resume流程。
3. 拉高GPIO128,将soc已经醒来的消息发送给MCU。(可根据需求调整通知MCU的时间点,如等Android Resume完成再拉高GPIO128)。
4. Hypervisor 中vCPUx线程恢复后,跳到Host OS Linux kernel,再执行Native kernel Resume Flow,将唤醒事件通过key event上报。
5. SoS的SleepManager 通过input sub system上报的事件判断后,将决定再次休眠或者继续唤醒 Flow,像MCU 拉高 GPIO35 这种就会继续唤醒Flow.
6. 如果是可以唤醒整个系统的 Key Event , 则上报 keyevent 87 去 resume Weston, 并 hold wakelock, 防止再次休眠。
7. 写节点/sys/guest\_os/android(tbox)/resume, 触发 Guest OS 唤醒。
8. Hypervisor收到后,恢复vCPU0线程。
9. Enable Non-Boot CPUs, 恢复vCPU1~7线程,开始Android kernel Resume Flow
10. IVI 基于 CPMS 架构, 无需判断 keyevent , 默认所有事件都可以触发整个系统 Resume。
11. 在 Host OS 端, 可以通过节点/sys/guest\_os/android (tbox)/pm\_state, 使用 Mailbox 机制查询两个 Guest OS 系统(Android/Tbox)状态,如果 Guest OS 端 kernel Resume 完成则更新状态为 Running。
12. Android 端, Car Power Management Service 收到唤醒事件后, Disable System Suspend , 防止误触发休眠。Tbox 端, 由 SleepManager 主动 hold wakelock。
13. 通知打开显示模块,至此,整个唤醒流程完成,可以开始人机交互。

## RTC 或 Modem 唤醒流程

RTC 或 Modem , 从实际场景以及节省功耗的角度来说, 它们唤醒的流程有所不同。

参考图 1-2 , SPM 收到 RTC 或 Modem 唤醒后, 1-6 的流程没有变化。第 7 步, SOS 的 SleepManager 通过 Key Event 的 Device Name , 判断出是 RTC 或 Modem 触发的唤醒,则根据实际需求做不同的唤醒逻辑,例如仅需要唤醒 Tbox OS。Tbox Resume 后,需要 hold wake\_lock 再处理任务,处理完任务后 release wake\_lock。而 Host OS 端,当触发Tbox Resume 后,会持一个临时的 wake\_lock,通过节点/sys/guest\_os/tbox/pm\_state,使用 Mailbox 机制查询Tbox OS 的状态,如果 Tbox 端已完成任务并重新睡下去, Host OS 端则会释放临时的wake\_lock。最后, SOS 端监测到没有 hold wakelock , 会重新睡下去。



7.13.2.3 kernel suspend/resume流程

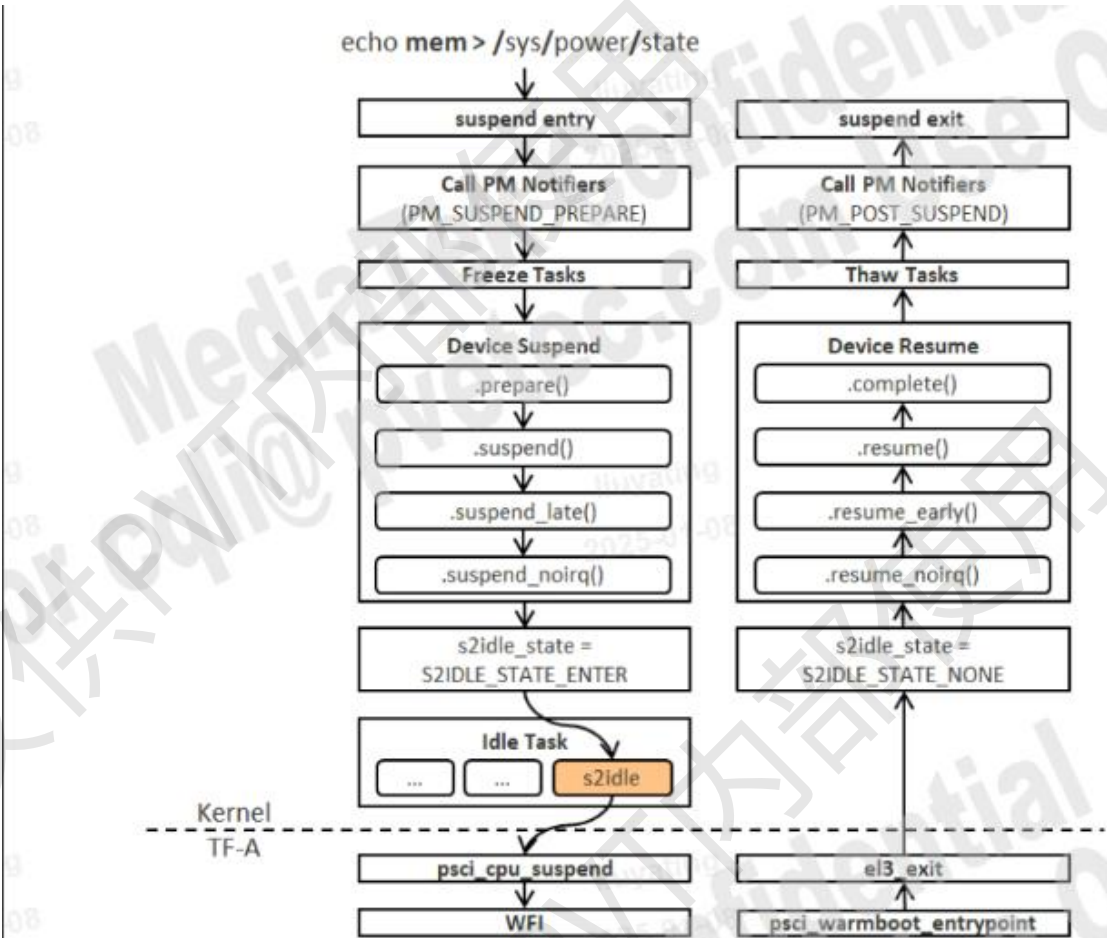


图 1-3. Kernel Suspend/Resume 流程

系统休眠

```
cat /sys/power/wakeup_count
echo mem > /sys/power/state
```

7.13.3 控制guest os suspend/resume方式

- 在 SoS 端检查 IVI (Android) OS 的状态，显示 running 表示 Android 正在运行，显示 suspend 表示 Android 已经挂起。

```
cat /sys/guest_os/android/pm_state
```

- 在 SoS 端触发 IVI (Android) OS 进入休眠状态。

```
echo powerkey > /sys/guest_os/android/pm_state
```

- 在 SoS 端触发 IVI (Android) OS 唤醒。

```
echo 0 > /sys/guest_os/android/resume
```



- 在 SoS 端检查 Tbox OS 的状态，显示 running 表示 Tbox 正在运行，显示 suspend 表示 Tbox 已经挂起。

```
cat /sys/guest_os/tbox/pm_state
```

- 在 SoS 端触发 Tbox OS 进入休眠状态。

```
echo powerkey > /sys/guest_os/tbox/pm_state
```

- 在 SoS 端触发 Tbox OS 唤醒。

```
echo 1 1 > /sys/guest_os/tbox/resume (Tbox Sleepmanager 持锁)
```

```
echo 1 2 > /sys/guest_os/tbox/resume (Tbox Sleepmanager 不持锁)
```

## 7.13.4 调试方法

### 7.13.4.1 休眠查询

如何判定系统休眠成功：

屏幕熄灭可能仅表示系统进入浅睡眠，这并不代表系统已经成功进入休眠状态，判定系统是否休眠成功需要查看 Kernel log，若系统成功进入 suspend，则 Host OS 端不再打印 Kernel log。

如果需要通过 log 判断休眠是否成功，需要先下这几行命令打开更多的 debug log 信息：

```
echo 8 8 8 8 > /proc/sys/kernel/printk
```

```
echo 1 > /sys/module/kernel/parameters/initcall_debug
```

```
echo 1 > /proc/mtprintk
```

kernel log 中，关键字 “PM: Syncing filesystems ...” 表示 kernel 开始执行 suspend 流程

kernel log 中，关键字 “suspend of devices complete after xxx msecs” 表示 device suspend 完成。

kernel log 中，关键字 “noirq suspend of devices complete after xxx msecs” 表示 device noirq suspend 完成。

。

kernel log 中，关键字 “suspend enter” 表示 suspend 流程完成，系统已进入 suspend 状态。

### 7.13.4.2 唤醒源

如何确认唤醒源

在 Kernel log 中搜索关键字 “suspend wake up by”，可以查看唤醒源  
如：

Pwrkey, 唤醒(及其他 EINT 唤醒):

```
[SPM] suspend wake up by R12_EINT_EVENT_B, timer out = 207308
```

如果要看具体是由哪个 EINT 唤醒，需先从日志查看

```
EINT xxx is pending
```

再通过 cat /proc/interrupts 看 xx 对应的是哪个中断，

Modem 相关唤醒:

[SPM] suspend wake up by R12CCIF0\_EVENT\_B, timer out = 1825253

定时器(PCM Timer)唤醒

[SPM] suspend wake up by PCM\_TIMER, timer\_out = 65612

目前MT8676支持的唤醒源列表如下:

Name	Control Bit	Description	Can it be Disabled
R12_PCM_TIMER	0	设定计时器, 超时报唤醒系统	Y
R12_KP_IRQ_B	2	检测到键盘按下/释放动作	Y
R12_APWDT_EVENT_B	3	RGU 在 suspend mode 中唤醒 CPU	Y
R12_APXGPT1_EVENT_B	4	AP GPT 计时器超时事件	Y
R12_CONN2AP_SPM_WAKEUP_B	5	Connectivity IC (Wi-Fi/BT/GPS) 唤醒系统	Y

R12_EINT_EVENT_B	6	EINT 事件唤醒系统	N
R12_CONN_WDT_IRQ_B	7	Connectivity IC watchdog timeout 唤醒系统	Y
R12_CCIF0_EVENT_B	8	MD to AP CCIF 唤醒事件	Y
R12_CCIF1_EVENT_B	9	MD to AP CCIF 唤醒事件	Y
R12_SSPM2SPM_WAKEUP_B	10	SSPM 相关唤醒系统	Y
R12_SCP2SPM_WAKEUP_B	11	SCP sensor 相关唤醒系统	N
R12_ADSP2SPM_WAKEUP_B	12	ADSP 相关唤醒系统	Y
R12_USBX_CDSC_B	14	USB 相关唤醒系统	Y
R12_USBX_POWERDWN_B	15	USB 远程唤醒	Y
R12_SYS_TIMER_EVENT_B	18	System timer 唤醒系统	N
R12_EINT_EVENT_SECURE_B	19	EINT event 唤醒系统	N
R12_SCP_CIRQ_IRQ_B	22	SCP_CIRQ 唤醒事件	Y
R12_MD2AP_PEER_EVENT_B	23	MD 相关唤醒系统	Y
R12_MD1_WDT_B	25	MD1 看门狗超时	Y
R12_REG_CPU_WAKEUP_B	28	内部唤醒源	N
R12_APUSYS_WAKE_HOST_B	29	APUSYS 的唤醒源	Y
R12_PCIE_WAKEUP_EVENT_B	30	PCIe 相关唤醒源	Y
R12_MSDC_WAKEUP_EVENT_B	31	MSDC 相关唤醒源	Y

您可以按照以下方式禁用列表中可以禁用的唤醒源。但是, 请注意, 只有在“can it be disabled”列中标注为可禁用 )的唤醒源才能被禁用:

使用系统提供的调试命令可以快速禁用特定的唤醒源:

① 使用如下命令读取当前系统唤醒源设定:

```
cat /proc/mtk_lpm/power/suspend_ctrl | grep -i wake_src
```

② 在表1-1的"Control Bit"栏位中找到您想要禁用唤醒源所对应的 `control bit` 位, 然后将读出的 `wake_src` 对应bit写0, 最后用下述命令将修改后的 `wake_src`写进系统(不需要禁用的唤醒源的 `control bit` 位不进行修改):

```
echo wake src 0x***** >/proc/mtk_lpm/power/suspend_ctrl
```

③ 如果想要再启用某个唤醒源, 就将对应的 `control bit` 再写1即可。

### 7.13.4.3 分析不能休眠问题

如系统灭屏后无法进入休眠(即没有**suspend entry**打印), 可以用以下命令判断是哪个**wakelock**阻止系统进入休眠:

查看哪些模块持锁:

① `cat /sys/kernel/debug/wakeup_sources`

观察输出结果的第5列`active_since`, 数字不为0且一直在增大的就是阻止系统进入休眠的**wakelock**。

② 亦可使用下面的command查看, 输出的那些就是阻止进休眠的**wakelock**

```
cat /sys/kernel/debug/wakeup_sources | awk -F'\t' '{ if(NR==1) print $1"\t" $7; else if($10) print $1"\t"$10;}'
```

③ 查看kernel log, 搜索关键字

Pending Wakeup Sources:

如果系统已经进入**kernel**休眠流程(**suspend entry**):

① **wakelock**

ex:

```
<6>[426.823002]<1>(0)[1895:system server]PM: Wakeup pending, aborting
```

```
suspend
```

```
<6>[426.823021]<1>(0)[1895:system server]active wakeup source:leds wakelock
```

找申请 **leds wakelock** 的模块即可, 基本上就是这个模块导致。

② **dpm suspend**阶段, 有**device suspend fail**也会导致休眠失败

ex:

```

int __maybe_unused mtk_imgsys_pm_suspend(struct device *dev)
{
 struct mtk_imgsys_dev *imgsys_dev = dev_get_drvdata(dev);
 int ret, num;
 bool idle = true;

 ret = wait_event_timeout
 (imgsys_dev->flushing_waitq, (idle = mtk_imgsys_idle(dev)),
 msecs_to_jiffies(1000 / 30 * DIP_COMPOSING_MAX_NUM * 3));
 num = atomic_read(&imgsys_dev->num_composing);
 if (!ret && !idle) {
 dev_info(dev, "%s: flushing SCP job timeout, num(%d), idle(%d)\n",
 __func__, num, idle);

 return NOTIFY_BAD;
 }
#ifdef NEED_PM
 if (pm_runtime_suspended(dev)) {
 dev_info(dev, "%s: pm_runtime_suspended is true, no action\n",
 __func__);
 return NOTIFY_DONE;
 }
 ret = pm_runtime_put_sync(dev);
 if (ret) {
 dev_info(dev, "%s: pm_runtime_put_sync failed: (%d)\n",
 __func__, ret);
 return NOTIFY_BAD;
 }
#endif
 return NOTIFY_DONE;
}

```

③ 还有很多休眠失败的情形，后续有遇到再进行补充

#### 7.13.4.4 休眠/唤醒各阶段延迟与优化

1. 休眠和唤醒可以大致划分为下面几个阶段，在kernel log中搜索下面的关键字，可以大致得到每个阶段的耗时，针对耗时异常的阶段再进行细化分析

休眠：

suspend entry

Freezing user space processes

Freezing remaining freezable tasks

suspend of devices complete

start suspend of devices complete

late suspend of devices complete

noirq suspend of devices complete

suspend enter

唤醒：

wake up by

noirq resume of devices complete

early resume of devices complete

resume of devices complete

Restarting tasks

suspend exit

2. 可以在各个os中打开下面两个config,

CONFIG\_PM\_DEBUG=y

CONFIG\_PM\_SLEEP\_DEBUG=y

并输入:

echo 0 > /sys/module/printk/parameters/console\_suspend

echo 1 > /sys/module/kernel/parameters/initcall\_debug

echo 1 > /sys/power/pm\_print\_times

echo 8 > /proc/sys/kernel/printk

echo 1 > /sys/power/pm\_debug\_messages

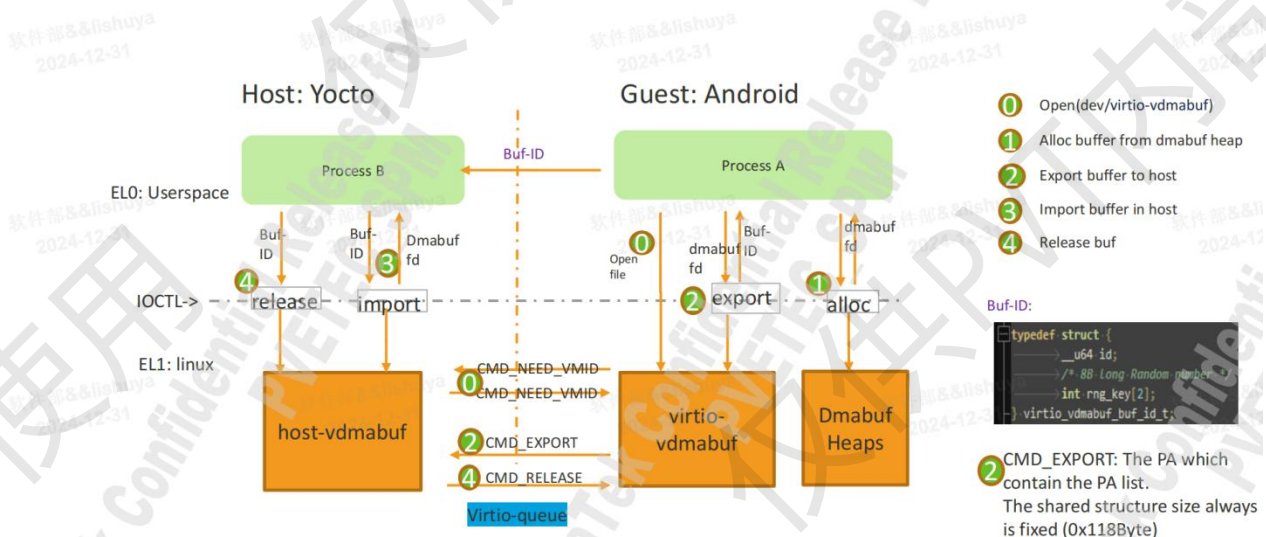
会把各个device, driver的休眠唤醒耗时打印出来。(打开config后, 打印的log会很多, 也可能会增加休眠唤醒的时间, 可以重点看下相对耗时较长的阶段, device)

## 7.14 OS间内存共享

### 7.14.1 Vdmabuf

#### 7.14.1.1 概述

#### Dmabuf sharing from guest OS to host OS





vdmauf功能是用来share Android端申请的buffer到yocto端，目前不具备双向能力，只能share从android端分配的buffer到yocto。vdmauf本身不具备alloc buffer的能力，使用前需要提前从heap里面分配好buffer（可使用7.17 dmabuf 章节中介绍的接口）。

android端根据alloc的dmabuf fd export出Buf-ID，结合OS间通信机制（例如vsock，介绍详见7.7.1.1vsock使用方法），Buf-ID传到yocto端，yocto端根据接收到的Buf-ID import出dmabuf fd后访问共享内存。

PS: vdmauf share buffer的最大size是1GB。

### 7.14.1.2 接口介绍

- 1、static inline int mtk\_vdmauf\_guest\_open(const char \*name): 打开 guest 端的/dev/virtio-vdmauf 节点，获取到 guest 端的 fd
- 2、static inline int mtk\_vdmauf\_guest\_export(const int fd, const int dmabuf\_fd, virtio\_vdmauf\_buf\_id\_t \*buf\_id): 根据 fd 和 dmabuf\_fd，解析出 buf\_id
- 3、static inline int mtk\_vdmauf\_guest\_close(int fd): 关闭 guest 端 fd
- 4、static inline int mtk\_vdmauf\_vhost\_open(const char \*name): 打开 host 端的/dev/vhost-vdambuf 节点，获取到 host 端 fd
- 5、static inline int mtk\_vdmauf\_vhost\_import(const int fd, virtio\_vdmauf\_buf\_id\_t \*buf\_id, int/\* OUT\*/ \*dmabuf\_fd): 根据 host 端 fd 和 从 guest 端接收到 buf\_id，解析出 dmabuf\_fd
- 6、static inline int mtk\_vdmauf\_vhost\_release(const int fd, virtio\_vdmauf\_buf\_id\_t \*buf\_id): 释放 buffer
- 7、static inline int mtk\_vdmauf\_vhost\_close(int fd): 关闭 host 端的 fd

## 7.15 OS内进程间内存共享

### 7.15.1 通过Binder机制

Binder 是 Android 操作系统中一种高效、安全的进程间通信机制。它通过共享内存和内核驱动实现高效的通信，同时通过权限验证和身份认证保证通信的安全性。它允许进程间传递复杂的数据结构，包括文件描述符和共享内存。

MT8676 yocto端也支持Binder机制。

#### 7.15.1.1 通过 IPC Binder 传递文件描述符



### 7.15.1.1.1 接口介绍 (C++)

- 1、writeFileDescriptor: 服务端或客户端使用Parcel的此方法将文件描述符写入Parcel;
- 2、readFileDescriptor: 客户端或服务端使用Parcel的此方法从Parcel中读取文件描述符;

### 7.15.1.1.2 伪代码示例 (C++)

#### 1、服务端

```
#include <binder/Parcel.h>
#include <binder/BinderService.h>
#include <binder/IServiceManager.h>
#include <fcntl.h>
#include <unistd.h>
using namespace android;

class MyService : public BBinder {
public:
 static void instantiate() {
 defaultServiceManager()->addService(String16("MyService"), new MyService());
 }
 virtual status_t onTransact(uint32_t code, const Parcel& data, Parcel* reply, uint32_t flags = 0)
override {
 switch (code) {
 case 0: { // getFileDescriptor
 int fd = open("/path/to/resource", O_RDWR);
 if (fd < 0) {
 return UNKNOWN_ERROR;
 }
 reply->writeFileDescriptor(fd);
 //close(fd);
 return NO_ERROR;
 }
 default:
 return BBinder::onTransact(code, data, reply, flags);
 }
}
};

int main(int argc, char** argv) {
```

```

MyService::instantiate();
ProcessState::self()->startThreadPool();
IPCThreadState::self()->joinThreadPool();
return 0;
}

```

## 2、客户端

```

#include <binder/IServiceManager.h>
#include <binder/Parcel.h>
#include <binder/IBinder.h>
#include <fcntl.h>
#include <unistd.h>
#include <iostream>
using namespace android;
int main(int argc, char** argv) {
 sp<IServiceManager> sm = defaultServiceManager();
 sp<IBinder> binder = sm->getService(String16("MyService"));
 if (binder == nullptr) {
 std::cerr << "Failed to get MyService" << std::endl;
 return -1;
 }
 Parcel data, reply;
 binder->transact(0, data, &reply); // 0 is the code for getFileDescriptor
 int fd = reply.readFileDescriptor();
 if (fd < 0) {
 std::cerr << "Failed to get file descriptor" << std::endl;
 return -1;
 }
 // Use the fd (e.g., read/write data)
 char buffer[128];
 read(fd, buffer, sizeof(buffer));
 std::cout << "Read data: " << buffer << std::endl;
 //使用完之后关闭
 close(fd);
 return 0;
}

```

### 7.15.1.2 通过 IPC Binder 传递 native\_handle\_t 对象

native\_handle\_t是一个结构体，用于封装多个文件描述符和整数数据。

### 7.15.1.2.1 接口介绍 (C++)

- 1、writeNativeHandle：服务端或客户端使用Parcel的此方法将native\_handle\_t对象写入Parcel；
- 2、readNativeHandle：客户端或服务端使用Parcel的此方法从Parcel中读取native\_handle\_t对象；

### 7.15.1.2.2 伪代码示例 (C++)

#### 1、服务端

```
#include <binder/Parcel.h>
#include <binder/BinderService.h>
#include <binder/IServiceManager.h>
#include <hardware/hardware.h>
#include <fcntl.h>
#include <unistd.h>
using namespace android;
class MyService : public BBinder {
public:
 static void instantiate() {
 defaultServiceManager()->addService(String16("MyService"), new MyService());
 }
 virtual status_t onTransact(uint32_t code, const Parcel& data, Parcel* reply, uint32_t flags = 0)
override {
 switch (code) {
 case 0: { // getNativeHandle
 int fd = open("/path/to/resource", O_RDWR);
 if (fd < 0) {
 return UNKNOWN_ERROR;
 }
 native_handle_t* handle = native_handle_create(1, 0);
 handle->data[0] = fd;
 reply->writeNativeHandle(handle);
 native_handle_delete(handle);
 //close(fd);
 return NO_ERROR;
 }
 }
}
```

```

 default:
 return BBinder::onTransact(code, data, reply, flags);
 }
}
};

int main(int argc, char** argv) {
 MyService::instantiate();
 ProcessState::self()->startThreadPool();
 IPCThreadState::self()->joinThreadPool();
 return 0;
}

```

## 2、客户端

```

#include <binder/IServiceManager.h>
#include <binder/Parcel.h>
#include <binder/IBinder.h>
#include <hardware/hardware.h>
#include <fcntl.h>
#include <unistd.h>
#include <iostream>
using namespace android;

int main(int argc, char** argv) {
 sp<IServiceManager> sm = defaultServiceManager();
 sp<IBinder> binder = sm->getService(String16("MyService"));
 if (binder == nullptr) {
 std::cerr << "Failed to get MyService" << std::endl;
 return -1;
 }
 Parcel data, reply;
 binder->transact(0, data, &reply); // 0 is the code for getNativeHandle
 const native_handle_t* handle = reply.readNativeHandle();
 if (handle == nullptr) {
 std::cerr << "Failed to get native handle" << std::endl;
 return -1;
 }
 int fd = handle->data[0];
 // Use the fd (e.g., read/write data)
 char buffer[128];
 read(fd, buffer, sizeof(buffer));
}

```

```

std::cout << "Read data: " << buffer << std::endl;
//使用完毕后关闭
close(fd);
native_handle_close(handle);
native_handle_delete(const_cast<native_handle_t*>(handle));
return 0;
}

```

## 7.16 debug入口办法

### 7.16.1 常用debug方式

1, android remount方式

需要重新5个分区img, 如下:

preloader\_a: bl2\_sboot\_dis.img

preloader\_b: bl2\_sboot\_dis.img

vendor\_boot\_a: vendor\_boot-debug.img

bl2-an: bl2-an\_sboot\_dis.img

boot\_a: boot-debug.img

2, yocto remount

先更新镜像为preloader\_a: bl2\_sboot\_dis.img

preloader\_b: bl2\_sboot\_dis.img

remount指令如下

mount -o remount,rw /

3, yocto抓取ftrace方式

第一步, 启动trace

mount -t debugfs nodev /sys/kernel/debug

echo 40960 > /sys/kernel/debug/tracing/buffer\_size\_kb

echo > /sys/kernel/debug/tracing/trace

echo 1 > /sys/kernel/debug/tracing/events/sched/sched\_switch/enable

echo 1 > /sys/kernel/debug/tracing/events/drm/enable

echo 1 > /sys/kernel/debug/tracing/tracing\_on

第二步, 关闭trace, 并导出trace log

echo 0 > /sys/kernel/debug/tracing/tracing\_on

echo 0 > /sys/kernel/debug/tracing/events/sched/sched\_blocked\_reason/enable

echo 0 > /sys/kernel/debug/tracing/events/sched/sched\_waking/enable

```
echo 0 > /sys/kernel/debug/tracing/events/sched/sched_wakeup/enable
echo 0 > /sys/kernel/debug/tracing/events/sched/sched_switch/enable
echo 0 > /sys/kernel/debug/tracing/events/sched/sched_process_fork/enable
echo 0 > /sys/kernel/debug/tracing/events/v4l2/enable
echo 0 > /sys/kernel/debug/tracing/events/vb2/enable
echo 0 > /sys/kernel/debug/tracing/events/drm/enable
dd if=/sys/kernel/debug/tracing/trace of=/tmp/atrace.html bs=2048
```

4, android 抓取perfetto

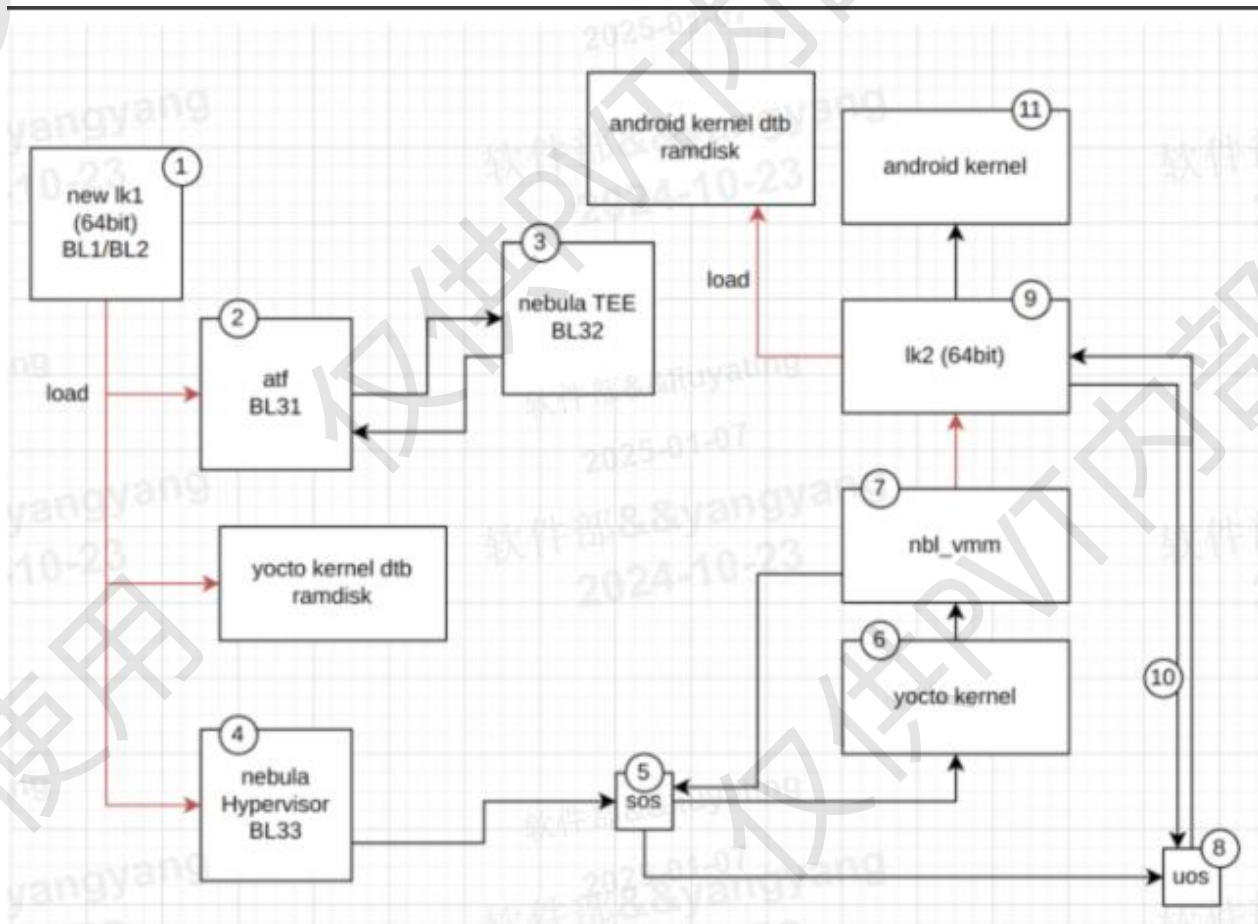
指令如下

```
adb shell perfetto -o /data/misc/perfetto-traces/trace_file.perfetto-trace -t 10s sched freq idle am wm gfx view
binder_driver hal dalvik camera input res memory irq
adb pull /data/misc/perfetto-traces/trace_file.perfetto-trace
```

## 7.16.2 串口log入手确认启动时序

多系统所有阶段的 log 都会输出到同一个串口，很容易混淆，可借助 log 关键词进行标识，方便 debug。

先看下启动流程，后对每个阶段 log 关键词确认。





### 7.16.2.1 lk

lk 的每句 log 是没有关键词进行区分的，只能通过 start 和 end log 进行辨识。

起始 log，多系统的 lk 关键词都可以通过

welcome to lk2

进行标识，顺序 sos lk->android uos lk -> tbox uos lk

LLA 会搜索到三次“welcome to lk2”

```
Search "welcome to lk2" (3 hits in 1 file)
D:\work_tools\sscom\ReceivedToFile-COM15-2025_1_2_10-43-48.DAT (3 hits)
Line 506: welcome to lk2
Line 16509: welcome to lk2
Line 24705: welcome to lk2
```

结束 log，会有所不一样，如下：

1, sos lk 结束 log

Turn off BL31 console

2, android uos lk 结束 log

lk finished --> jump to linux kernel 64Bit

3, tbox uos lk 结束 log

LK run time

### 7.16.2.2 hypevisor 串口 log

可以确定是 hypevisor log 关键词

ARM boot EL2

通过这句 log，再可以通过关键词

nebula\_log\_driver

搜索确定周围 hypevisor uart log

```
[7.957349][T1100068] kworker/u16:2: [name:nebula_log_driver]nebula: (0)[00008.994] 00000.00000> ARM boot EL2
[7.957352][T1100068] kworker/u16:2: [name:nebula_log_driver]nebula: (0)[00008.994] 00000.00000> ARM cpu 0: midr 0x41fd461 'ARM Cortex-a510 rip1' mpidr 0x51000000 aff 0;0;0;0
[7.957356][T1100068] kworker/u16:2: [name:nebula_log_driver]nebula: (0)[00008.994] 00000.00000> ARM Features: fp asimd aes pmull sha1 sha2 crc32 atomics rdm sha3 sm3 sm4 dpb
```

### 7.16.2.3 Android 串口 log

1, Android uos early console

Android UOS early console 也会以 hypevisor log 形式输出，所以 log 还是以“nebula:”为开头

```
[18.221396][T1400068] kworker/u16:4: [name:nebula_log_driver]nebula: (2)[00028.243] 04101.04568> [0.000000][T0] swapper: printk: bootconsole [p11] enabled
[18.223235][T1400068] kworker/u16:2: [name:nebula_log_driver]nebula: (2)[00028.245] 04101.04568> [0.000000][T0] swapper: [name:panic]panic_on_taint: bitmask=0x20 nouseartaint_mode=disabled
```

2, Android uos early console end

```
[20.268552][T1400067] kworker/u16:4: [name:nebula_log_driver]nebula: (6)[00030.294] 04101.05056> [1.197389][T600001] swapper/0: ashmem: [name:ashmem]initialized
[20.270429][T1400067] kworker/u16:4: [name:nebula_log_driver]nebula: (6)[00030.295] 04101.05056> [1.620332][T600001] init: printk: console [hvc0] enabled
```

如上图，当看到“init: printk: console [hvc0] enabled” 就表示 early console 已经结束，准备要切换到 virtio-c

onsole

### 3, Android uos virtio-console

```
[20.520461] nbl_vm_srv[2705]: [INFO:virtio_pci.cc(555)] virtio PCI device ready: 58
[20.520475] nbl_vm_srv[2705]: [1.759591][T400001] init: virtio_apu virtio6: [name:virtio_apu]virtio-apu initialized
[20.520480] nbl_vm_srv[2705]: [1.759591][T400001] init: virtio_apu virtio6: [name:virtio_apu]virtio-apu initialized
```

如上图所示，virtio-console 输出的 log 会以“nbl\_vm\_srv[XXX]”为开头，其中 XXX 是 nbl\_vmm 的 pid 号。

3 系统的话可以通过 nbl\_vmm 的 pid 号区分 不同的 uos。

## 7.16.3 从问题种类选择合适debug工具

### 7.16.2.1 功能问题

从 log 分析入手，会用到以下方式：

#### 1, debuglogger

bootloader, kernel, userspace log 都会存在 mobilelog, modem log 会存在 Mdlogger。

mobilelog 和 Mdlogger log 都存到/data/debuglogger 路径下。

Android 和 yocto debuglogger 路径同样是/data/debuglogger

#### 2, logcat

用于分析 Android userspace 层问题

#### 3, journalctl

用于分析 yocto userspace 层问题

#### 4, dmesg

用于分析 yocto 和 Android kernel 层问题

### 7.16.3.2 稳定性

#### 7.16.3.2.1 KE, HWT, HW reboot, NE, JE, SWT, EE

当系统出现 KE, HWT, HW reboot, NE, JE, SWT, EE, 会在当前 OS /data/aee\_exp 生成对应 db 文件，  
即会用到以下 debug 工具：

#### 1, GAT

DB 解析 工具

#### 2, SP flash tool

当系统一直触发 KE 无法正常进入系统时，lk kedump 回写 expdb 分区后，关机，再通过 SP flash tool 工具  
read back 对应 os expdb 分区。

#### 3, dump expdb 分区解析

python expdb\_parser.pyc expdb(db) (需要 GAT 带的 python 和里面的脚本)

### 7.16.3.2.2 内存泄露

1, Asan

可抓 native 层内存泄露现场 log;

2, 使能 page\_owner

可抓 kernel 内存泄露问题

3, libmemunreachable+mallocdebug

可抓 Android 端 native 内存泄露现场 log;

### 7.16.3.2.3 踩内存

1, Hwasan

适用 native 层

2, kasan

适用 kernel

### 7.16.3.2.4 性能

1, qian\_debug-tool

可抓取多系统整个 cpu 状态。

2, ftrace

主要用于内核空间的性能分析，userspace 性能数据也可以抓到。

可用 <https://ui.perfetto.dev/> 打开 trace log, 进行图形分析。

3, perfetto

抓取单 os cpu 状态数据。

4, top

5, vmstat

## 7.16.4 基础器件debug 方式

针对以下器件，linux 框架或是平台会向 userspace 层暴露控制节点，通过操作节点方式可以查询、控制器件的一种 debug 方式。

### 7.16.3.1 cpu

1, 查询 cpu 可设置频点

cat /sys/devices/system/cpu/cpufreq/policy0/scaling\_available\_frequencies

2, cpu 锁频设置

例如锁频 cpu0 锁频 1.9GHz

echo 1900000 > /sys/devices/system/cpu/cpufreq/policy0/scaling\_min\_freq

echo 1900000 > /sys/devices/system/cpu/cpufreq/policy0/scaling\_max\_freq

### 7.16.3.2 内存

1, 系统内存 size

1) linux 通用方式

cat /proc/meminfo

2) android

dumpsys meminfo

2, Android 端 dmabuf 数据

1) 查询 dmabuf 使用情况

dmabuf\_dump

2) mtk 查询每个 dmabuf node 所持对象

cat /proc/dma\_heap/all\_heaps

### 7.16.3.3 中断

cat /proc/interrupts

### 7.16.3.4 gpio

1, 查询 gpio 状态

cat /proc/mtk\_gpio/soc.pinctrl

### 7.16.3.5 i2c

集成 i2c-tools, 目前 LLA 三个系统都已经集成

1, i2cdetect

1) 列举 i2c bus

i2cdetect -l

2) 列举 i2c-1 bus 上面连接所有设备

i2cdetect -y 1

```
sh-3.2# i2cdetect -y 1
 0 1 2 3 4 5 6 7 8 9 a b c d e f
00: -- -- -- -- -- -- -- -- -- -- UU 0d 0e --
10: -- -- -- -- -- -- -- -- -- -- UU UU -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

发现 I2C 设备的位置显示为 UU 或者表示设备地址的数值，UU 表示该设备在 driver 中被使用。

#### 2, i2cdump

dump I2C 设备大批量 register 的值

```
i2cdump -y -f [i2c-bus] [chip addr]
```

#### 3, i2cget

读取 I2C 设备某个 register 的值

```
i2cget -y -f [i2c-bus] [chip addr] [register]
```

#### 4, i2cset

设置 I2C 设备某个 register 的值

```
i2cset -y -f [i2c-bus] [chip addr] [register] [value]
```

### 7.16.3.6 uart

#### 1, 设置串口波特率

```
stty -F /dev/ttyS2 ispeed 921600 ospeed 921600 cs8
```

#### 2, 查询串口设置

```
stty -F /dev/ttyS2 -a
```

#### 3, 读串口

```
cat /dev/ttyS2
```

#### 4, 向串口发数据

```
echo 123 > /dev/ttyS2
```

发送 16 进制数

```
echo -ne '\x41\x42\x43' > /dev/ttyS2
```

### 7.16.3.7 pmic

首先切换目录，找到 pmic\_access 节点：

```
adb root
adb shell
cd /sys/bus/platform/devices/mt-pmic
```

read 方法:

```
echo addr > pmic_access
addr 格式: SXXXX (S 是 slave address, XXXX 为想要查的寄存器地址)。
ex.
echo 40008 > pmic_access
cat pmic_access
```

write 方法:

```
echo addr val > pmic_access
addr 格式: SXXXX (S 是 slave address, XXXX 为想要查的寄存器地址)。
val 格式: 是 16 进制但是不要写 0x。
ex.
```

```
echo 5024D 1 > pmic_access
```

常见的 slave address:

```
slave 4: MT6363/MT6377
slave 5: MT6373/MT6368/MT6369
slave 6: MT6319/MT6316
slave 7: MT6319/MT6316
slave 8: MT6319/MT6316
slave 3: MT6319
```

slave-id 查询方法, 以 mt6363.dtsi 为例, slave address 即为 4:

```
main_pmic: pmic@4 {
 compatible = "mediatek,mt6363";
 reg = <0x4 SPMIUSID>;
 ...
};
```



## 7.17 dmabuf

DMA-BUF (Direct Memory Access Buffer) 是一种内核机制, 用于在不同的设备驱动程序之间共享缓冲区。它主要用于图形和多媒体应用程序中, 以实现高效的内存共享和数据传输。DMA-BUF 允许多个设备驱动程序在不复制数据的情况下共享同一块物理内存, 从而提高性能和减少延迟。

### 7.17.1 接口介绍

- 1、`BufferAllocator* CreateDmabufHeapBufferAllocator()`: 创建一个新的 `BufferAllocator` 实例
- 2、`void FreeDmabufHeapBufferAllocator(BufferAllocator* buffer_allocator)`: 释放一个 `BufferAllocator` 实例
- 3、`int DmabufHeapAlloc(BufferAllocator* buffer_allocator, const char* heap_name, size_t len, unsigned int heap_flags, size_t legacy_align)`: 从指定的 DMA 缓冲区堆中分配内存
- 4、`int DmabufHeapAllocSystem(BufferAllocator* buffer_allocator, bool cpu_access, size_t len, unsigned int heap_flags, size_t legacy_align)`: 从系统堆中分配内存, 支持 CPU 访问
- 5、`int MapDmabufHeapNameToIonHeap(BufferAllocator* buffer_allocator, const char* heap_name, const char* ion_heap_name, unsigned int ion_heap_flags, unsigned int legacy_ion_heap_mask, unsigned int legacy_ion_heap_flags)`: 将 DMA 缓冲区堆名称映射到 ION 堆名称
- 6、`int DmabufHeapCpuSyncStart(BufferAllocator* buffer_allocator, unsigned int dmabuf_fd, SyncType sync_type, int (*legacy_ion_cpu_sync)(int, int, void *), void *legacy_ion_custom_data)`: 开始 DMA 缓冲区的 CPU 同步
- 7、`int DmabufHeapCpuSyncEnd(BufferAllocator* buffer_allocator, unsigned int dmabuf_fd, SyncType sync_type, int (*legacy_ion_cpu_sync)(int, int, void*), void* legacy_ion_custom_data)`: 结束 DMA 缓冲区的 CPU 同步
- 8、`bool CheckIonSupport()`: 检查系统是否支持 ION

## 八、模块

### 8.1 Display

#### 8.1.1 Display 总览

##### 8.1.1.1 名词解释

Abbreviation	Explanation
DSI	Display Serial Interface 显示串行接口
DPI	Digital Parallel Interface 数字并行接口
SerDes	Serializer/Deserializer 加串器/解串器
LCM	Liquid Crystal Display Module 液晶显示模组
DTS	Device Tree Source设备树
HWC	Hardware Composer 硬件合成器
DP	Display Port 数字式视频接口标准
DSC	Display Stream Compression 视觉无损显示流压缩标准
CRTC	Cathode Ray Tube Controller(阴极射线管控制器)即显示控制器

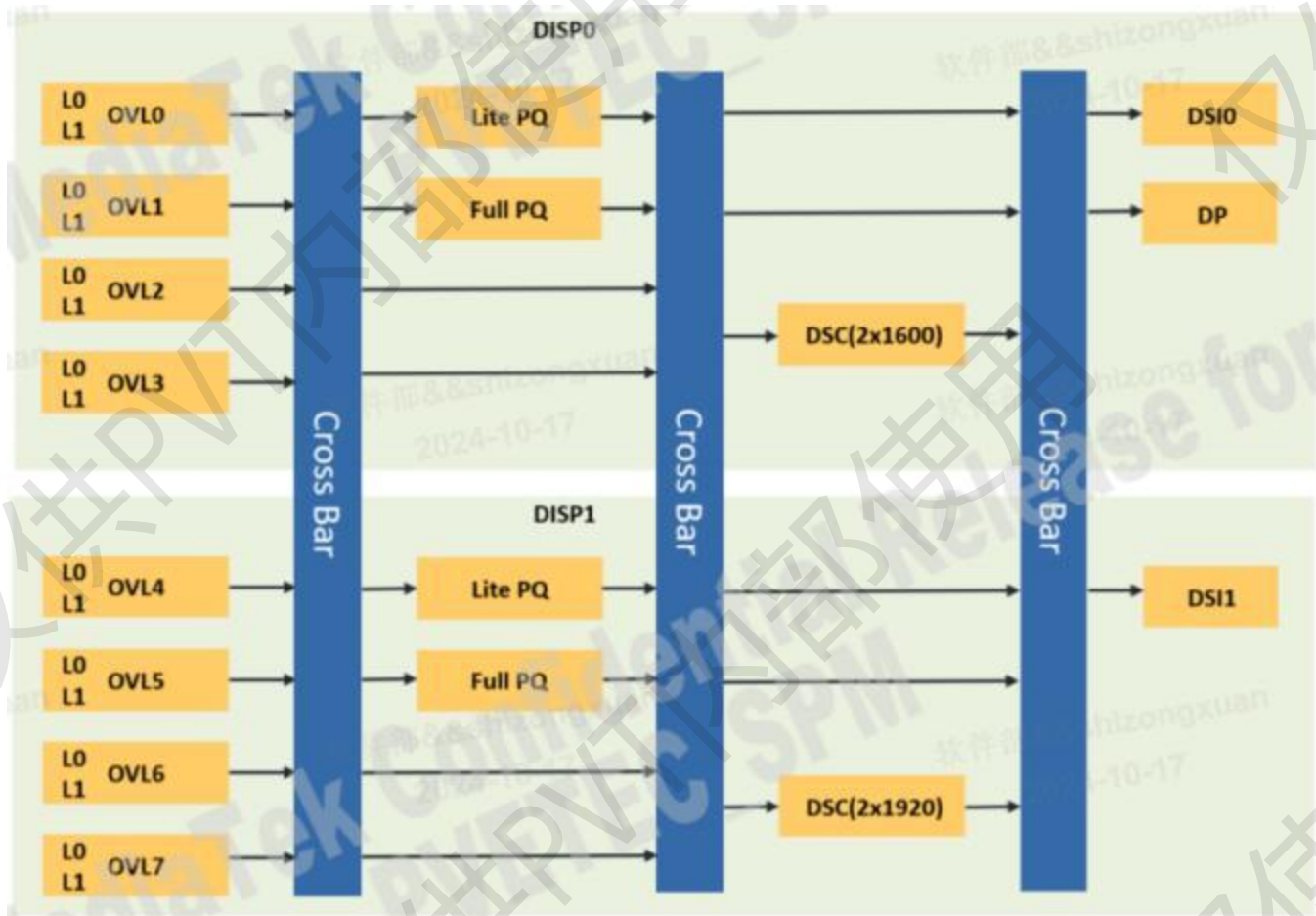
##### 8.1.1.2 Display Capability

Display module	Capbility
DISP pipeline	10bit pipe *3 (MAX 688Mhz at 0.75v)

DSI	DSI0 + DSI1 C/D PHY Combo 4-lane * 2 DPHY: 2.5Gbps/lane
DP	DP1.4, 4-lane 8.1Gbps/lane with USB2.0 or DP1.4, 2-lane 8.1Gbps/lane with USB3.0
Panel number	1 - 6
OVL Layers	OVL0-OVL7, 16 layers
Compression	VESA DSC 1.2 (2 slice *2)

### 8.1.1.3 Architecture

#### 8.1.1.3.1 HW Architecture



带宽限制:

DSI0: 2880\*1800@60fps or 2\*1920\*1080@60fps

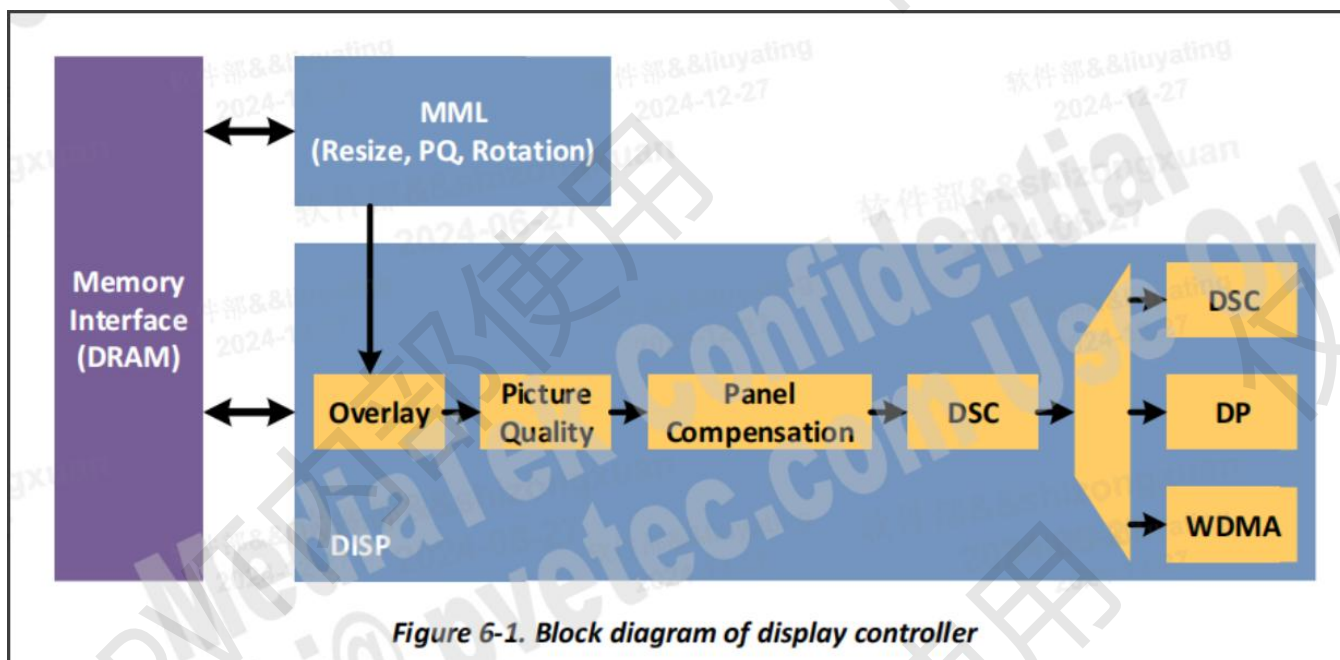
DP: 7680\*1080@60fps or 2\*2560\*1600@60fps

DSI1: 2880\*1800@60fps or 2\*1920\*1080@60fps

Full PQ和Lite PQ区别

Full PQ: tdshp, all3.0, ccorr, gamma, c3d, all2.0, dither

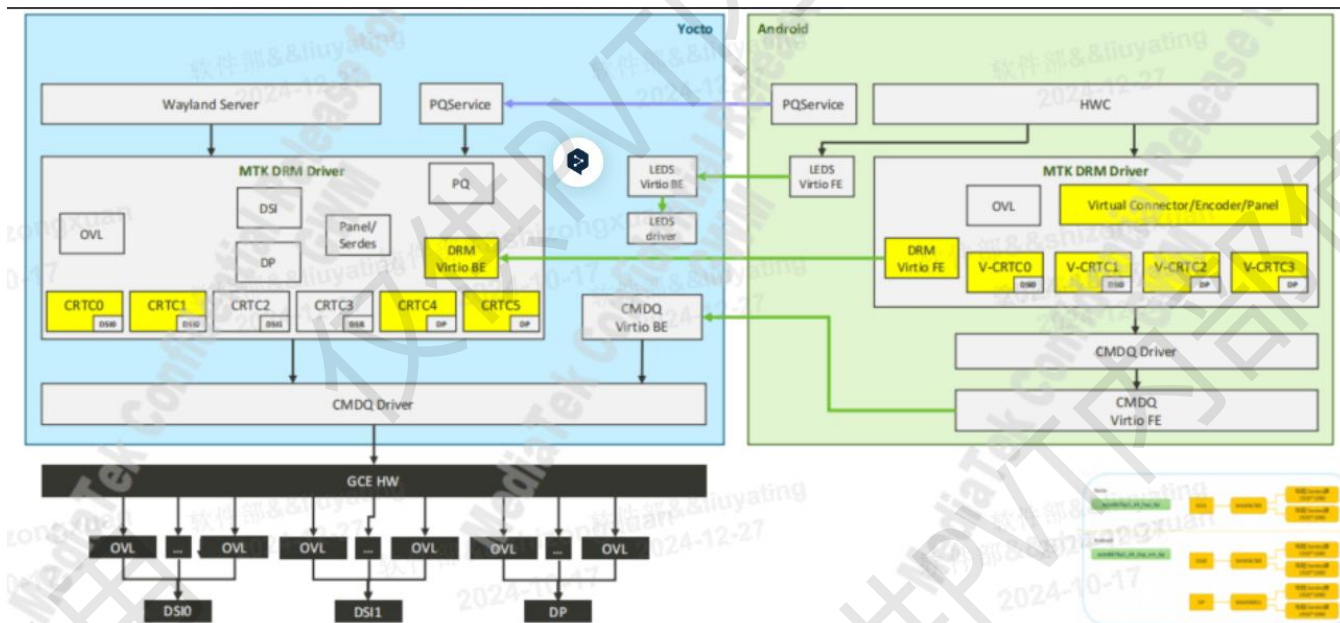
Lite PQ: tdshp, ccorr, gamma



从上图可以看出MML/MDP主要是功能在于处理图层缩放，PQ，Rotation

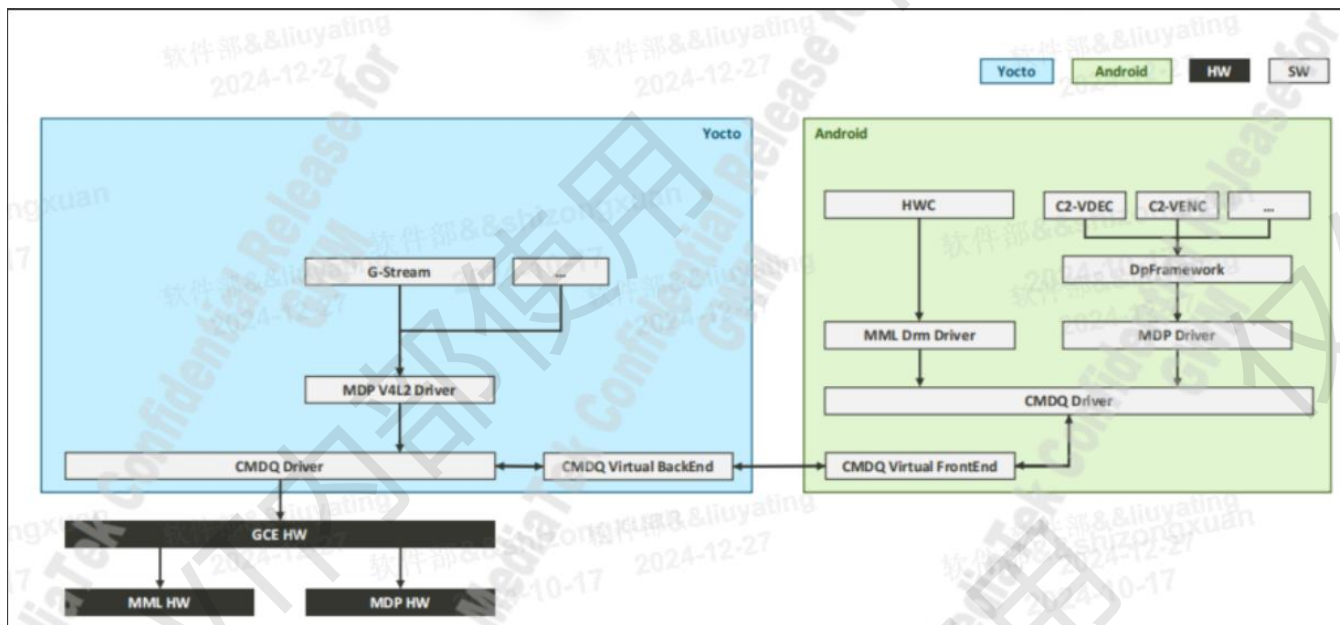
### 8.1.1.3.2 SW Architecture

#### 8.1.1.3.2.1 Hypervisor (Yocto + Android)

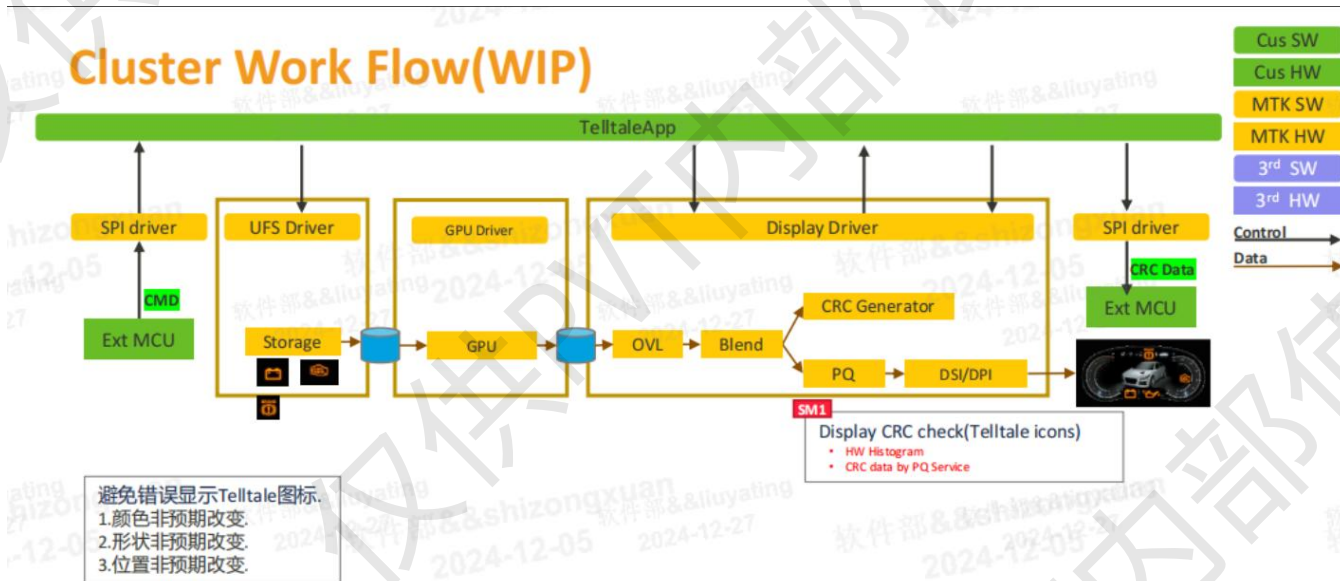


当前采用驱动层虚拟化方案，将 disp\_pq 和 disp interface(DSI/DP) 虚拟化，ovl 仍是直通控制，兼顾性能。完整的 CRTC 驱动（包括 ovl, disp\_pq, dsi/dp）在 Host OS，Guest OS 采用 Virt-Connector/Encoder，通过 virtio 去 Host OS 的 DRM 驱动获取 display mode 以及开关 CRTC。Guest OS 更新图层则采用直通方式，直接驱动 OVL，无需和 Host 端通信。

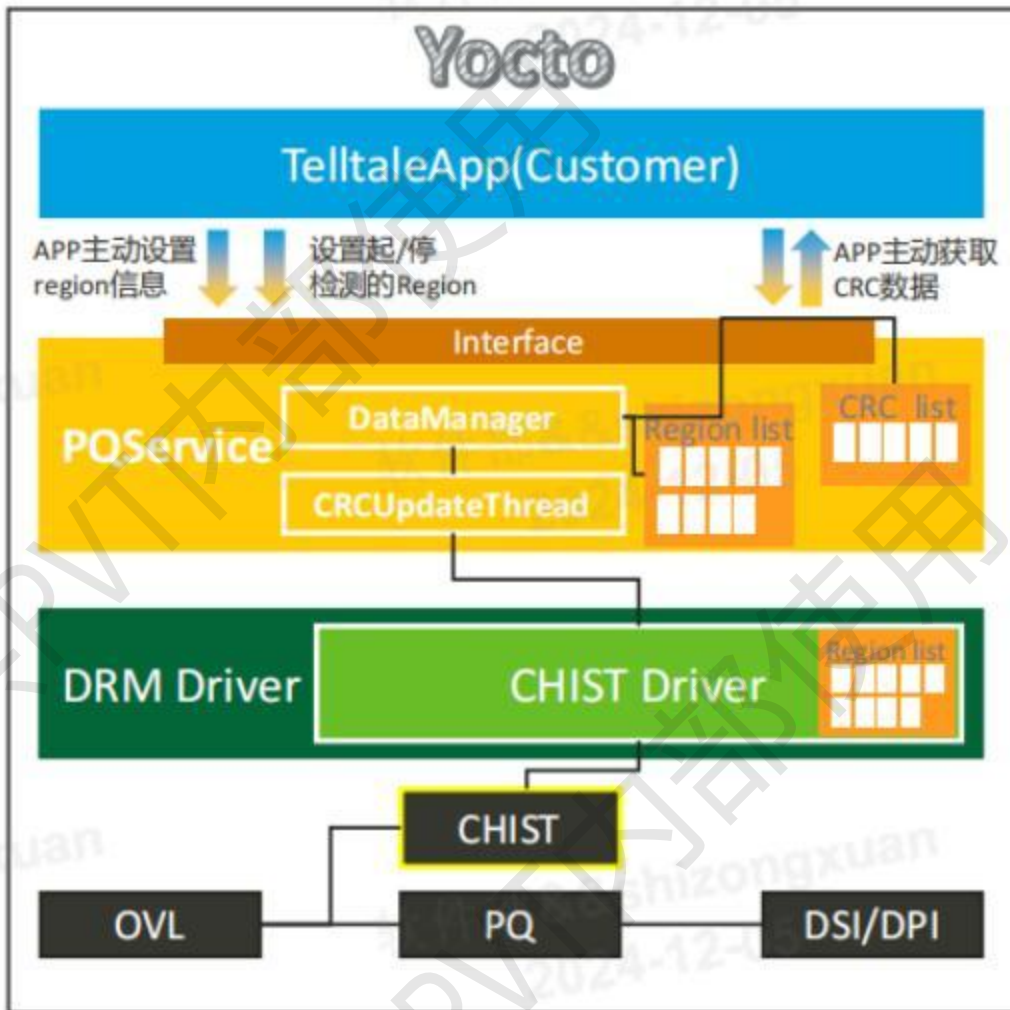
MML/MDP 虚拟化框架如下图所示：



### 8.1.1.3.2.2 Display功能安全CRC







Yocto OS支持一些安全图标的校验功能，包括颜色非预期改变，形状非预期改变，位置非预期改变。实现原理是利用了Display Pipeline上的chist功能模块，chist模块可以计算出指定区域的Y-直方图，PQService会拿到这个Y-直方图，然后利用CRC算法，计算出指定区域的CRC，客户可以拿到这个CRC值和CRC golden值去做比较，判断图标非预期改变。

当前硬件支持两个chist模块，每个chist支持7 channel，也就是同时可以支持14个区域的安全校验。

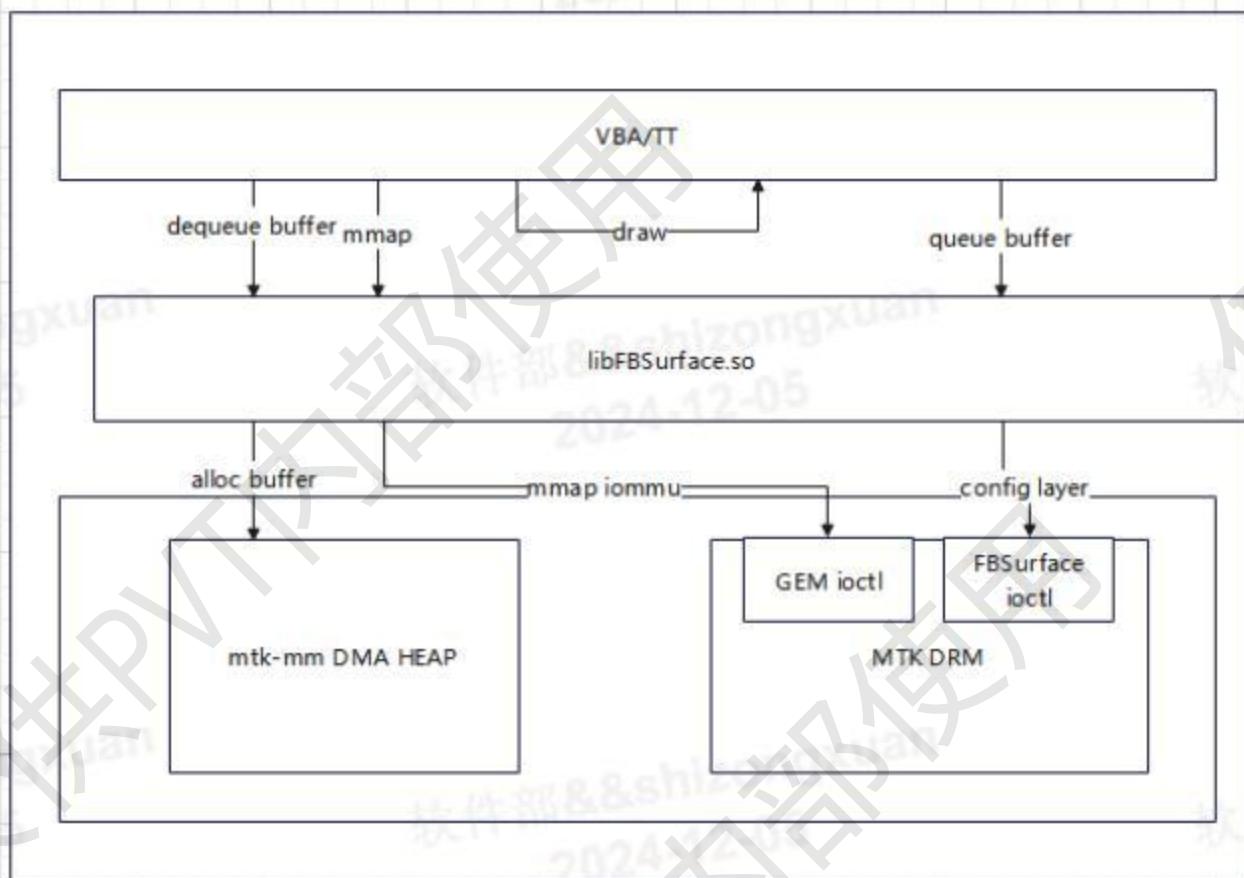
**sample code:** `yocto/src/graphics/pq/litepq/test/PQ_Test.cpp`

### 8.1.1.3.2.3 Yocto Display Architecture

Yocto os里面Display对接了两个框架，如下

#### 8.1.1.3.2.3.1 FBSurface

FBSurface是一个简单的显示框架，一般用于显示animation，仪表图标等。FBSurface会提供一些类似alloc buffer，显示参数设置，送显等API。FBSurface 依赖MTK DRM私有的ioctl接口用于显示数据。



**sample code:** yocto/src/multimedia/libfbsurface/test/libfbsurface\_multi.cpp

yocto/src/multimedia/libfbsurface/test/libfbsurface\_ut.cpp

### 8.1.1.3.2.3.2 weston

开源的显示架构，底层对接MTK DRM。支持显示Cluster仪表，也可以用来显示VBA，仪表图标等

## 8.1.2 Configuration / Customization Guideline

### 8.1.2.1 Host Yocto配置

#### 8.1.2.1.1 Display DTS配置

增加serdes请按照示例添加，比如ti983 / max96789。并根据linux component规则设置好endpoint以及remote-endpoint，能把dsi encoder / dsi phy / dsi serdes / dsi panel 或者 dp encoder / dp phy / dp serdes / dp panel 串起来

8.1.2.1.2 DTS配置开关

dsi/dp encoder以及phy， dsi/dp serdes dts开关在如下路径， 根据自己需求设置disabled或者okay  
yocto/meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/dtsi/cust\_mt8676\_display\_config\_hyp.dtsi

8.1.2.1.3 Display interface归属os配置

yocto/meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/dtsi/cust\_mt8676\_display\_config\_hyp.dtsi

interface	归属
DSI0	增加host-os-use属性，表示interface分给yocto
DSI0_VRIT1	增加host-os-use属性，表示interface分给yocto
DSI1	增加host-os-use属性，表示interface分给yocto
DSI1_VRIT1	增加host-os-use属性，表示interface分给yocto
DP	增加host-os-use属性，表示interface分给yocto
DP_VIRT1	增加host-os-use属性，表示interface分给yocto

8.1.2.1.4 Display path配置

yocto/meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/dtsi/cust\_mt8676\_display\_config\_hyp.dtsi

设备节点	描述
------	----

dsi0	host端connector设备节点
dsi1	根据自己使用场景，打开或者关闭
dp_intf	
virt_dsi0_1	开启superframe之后，第二屏的connector设备节点，必须要打开
virt_dsi1_1	根据自己使用场景，打开或者关闭
virt_dp_1	

### 8.1.2.1.5 Display ovl配置（PVT会根据客户使用场景配置好，客户无需自己配置）

#### 8.1.2.1.5.1 Drm Driver默认配置

mtk\_drm\_drv.c

interface	ovl配置
DSI0	DDP_COMPONENT_OVL0_2L DDP_COMPONENT_OVL1_2L DDP_COMPONENT_OVLSYS_DLO_ASYNC3
DSI0_VRIT1	DDP_COMPONENT_OVL1_2L
DSI1	DDP_COMPONENT_OVL6_2L DDP_COMPONENT_OVL7_2L DDP_COMPONENT_OVLSYS_DLO_ASYNC10

DSI1_VRIT1	DDP_COMPONENT_OVL7_2L
DP	DDP_COMPONENT_OVL2_2L DDP_COMPONENT_OVL3_2L DDP_COMPONENT_OVLSYS_DLO_ASYNC6
DP_VIRT1	DDP_COMPONENT_OVL3_2L

8.1.2.1.5.2 客户客制化配置

yocto/meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/auto8676p1\_64\_hyp.dts, 在如下节点里配置 display interface的ovl信息。这个DTS 里面的OVL配置会覆盖上一节描述的代码默认配置

```
&dispsys_config {
/*
 crtc0-ovl-path = <DDP_COMPONENT_OVL0_2L>,
 <DDP_COMPONENT_OVL1_2L>,
 <DDP_COMPONENT_OVL2_2L>,
 <DDP_COMPONENT_OVL3_2L>,
 <DDP_COMPONENT_OVLSYS_DLO_ASYNC3>;
 crtc-layer-nr = <8>;
*/
 pinctrl-names = "mode_te_te";
 pinctrl-0 = <&mtkfb_pins_lcm_dsi_te>;
 status = "okay";
};
```

8.1.2.2 Guest Android配置

8.1.2.2.1 Display DTS配置

android/kernel/kernel\_device\_modules-  
6.1/arch/arm64/boot/dts/mediatek/cust\_mt8676\_display\_config\_vm.dtsi  
Android DTS里面dsi encoder / phy / serdes, dp encoder / phy / serdes都不需要配置，必要时还需要手动  
disable掉已经打开的。需要配置的是以下虚 拟节点

设备节点	描述
virt_dsi0_0 virt_dsi1_0 virt_dp_0	根据自己使用场景，打开或者关闭
virt_dsi0_1 virt_dsi1_1 virt_dp_1	开启superframe之后，第二屏的connector设备节点，必须要打开 根据自己使用场景，打开或者关闭

8.1.2.2.2 Display ovl配置（PVT会根据客户使用场景配置好，客户无需自己配置）

8.1.2.2.2.1 Drm Driver默认配置

mtk\_drm\_drv.c

interface	ovl配置
-----------	-------



DSI0_VRIT0	DDP_COMPONENT_OVL0_2L DDP_COMPONENT_OVL1_2L DDP_COMPONENT_OVLSYS_DLO_ASYNC3
DSI0_VRIT1	DDP_COMPONENT_OVL1_2L
DSI1_VRIT0	DDP_COMPONENT_OVL6_2L DDP_COMPONENT_OVL7_2L DDP_COMPONENT_OVLSYS_DLO_ASYNC10
DSI1_VRIT1	DDP_COMPONENT_OVL7_2L
DP_VIRT0	DDP_COMPONENT_OVL2_2L DDP_COMPONENT_OVL3_2L DDP_COMPONENT_OVLSYS_DLO_ASYNC6
DP_VIRT1	DDP_COMPONENT_OVL3_2L

#### 8.1.2.2.2.2 客户客制化配置

android/kernel/kernel\_device\_modules-6.1/arch/arm64/boot/dts/mediatek/auto8676p1\_64\_bsp\_vm.dts，在如下节点里配置display interface的 ovl信息。这个DTS里面的OVL配置会覆盖上一节描述的代码默认配置

```

&dispsys_config {
/*
 crtc0-ovl-path = <DDP_COMPONENT_OVL0_2L>,
 <DDP_COMPONENT_OVL1_2L>,
 <DDP_COMPONENT_OVL2_2L>,
 <DDP_COMPONENT_OVL3_2L>,
 <DDP_COMPONENT_OVLSYS_DLO_ASYNC3>;
 crtc-layer-nr = <8>;
*/

 pinctrl-names = "mode_te_te";
 pinctrl-0 = <&mtkfb_pins_lcm_dsi_te>;
 status = "okay";
};

```

### 8.1.2.3 OVL配置QA

1. ovl之间比较，比如ovl0和ovl1，zorder是否固定，软件可以调吗？

--> 软件不可调，OVL之间没有zorder的概念，图层的zorder依赖dts里面配置的顺序，比如crtc0顺序配置了OVL4和OVL5，那就固定了ovl4图层在OVL5下面。对于android和yocto应用，weston和android hwc可以拿到某个crtc的图层资源数组，如果客户想显示的软件图层zorder比较低，就选择数组index低的图层

2. ovl的2个layer之间比较，zorder是否固定，软件可以调吗？

-->zorder也是固定的，ovl-layer1比ovl-layer0的zorder高

3. android和yocto各使用一层layer的场景，比如android使用ovl0的其中一层layer，yocto使用ovl0的其中一层layer。针对这种场景图层怎么配置，软件里怎么做区分的？

-->这种就需要借助修改中的crtcx\_layer\_table，可以规划当前ovl 的两个layer，哪一个给android 使用，按一个给yocto 使用

4. async和ovl绑定规则是什么样的？DDP\_COMPONENT\_OVLSYS\_DLO\_ASYNCx和DDP\_COMPONENT\_OVLx\_2L的关系

--> 这里的async 是display path 上串path的一个连接模块，OVL组合之后会选某个async连接后面path模块，这是硬件path细节，当前没有可release给客户的文档

5. 输入到ovl硬件图层的分辨率大小有限制吗？

--> 有，ovl layer 最大support的图层宽度为1920，如果图层宽度超过1920，就需要两个ovl layer，比如ovl0 layer0和ovl0 layer1

6. 修改CRTC的顺序

(1) &dispsys\_config {} 顺序

(2) mtk\_drm\_drv.c crtc\_create 顺序

(3) arch/arm64/boot/dts/mediatek/mt6897.dts dsi/dp 相关节点的定义先后顺序

(4) 开superframe 的屏需要保持先后顺序

7. crtc的顺序可以是这样的吗? android: dp0 dp1 dsi0 dsi1 yocto: dsi0 dsi1 dp0 dp1

--> 不行, android 端是虚拟的屏, 需要和yocto 端的物理屏对应, yocto 端的物理顺序和android 端的虚拟顺序要keep 一致

## 8.1.3 Debug Tips

### 8.1.3.1 显示PATH信息

#### DRM节点

(1) 简单查看当前显示path是怎么样的, 显示所有的connector以及crtc信息, 以及每个connector的timing信息

yocto和android 路径一致: ls /sys/class/drm/card0/

#### weston debug

(1) 查看weston display信息

source data/wayland\_env\_file

weston-debug -a && ctrl + C

查看输出前面output信息以及有哪些图层显示

### 8.1.3.2 display pattern (Yocto os)

#### Interface Encoder pattern

##### DSI0

打开: adb shell "echo gce\_wr:1400d178,c61,ffffff > /sys/kernel/debug/mtkfb"

关闭: adb shell "echo gce\_wr:1400d178,0,ffffff > /sys/kernel/debug/mtkfb"

##### DSI1

打开: adb shell "echo gce\_wr:1420d178,c61,ffffff > /sys/kernel/debug/mtkfb"

关闭: adb shell "echo gce\_wr:1420d178,0,ffffff > /sys/kernel/debug/mtkfb"

##### DP

打开: adb shell "echo gce\_wr:1400bf00,41,ffffff > /sys/kernel/debug/mtkfb"

关闭: adb shell "echo gce\_wr:1400bf00,0,ffffff > /sys/kernel/debug/mtkfb"

#### blending pattern

None

### 8.1.3.3 android和yocto 图层测试

#### modetest刷图

```
/usr/bin/modetest -M mediatek -D 0 -s 32@69:1920x1080@RG24 -P 46@69:1920x1080@RG24
/usr/bin/modetest -M mediatek -D 0 -s 38@104:1920x1080@RG24 -P 100@104:1920x1080@RG24
/usr/bin/modetest -M mediatek -D 0 -s 40@110:1920x720@RG24 -P 106@110:1920x720@RG24
```

#### am start android应用

可以dumpsys display 查看当前android os支持哪些display id

```
am start --display 0 -a android.settings.INPUT_METHOD_SETTINGS
```

```
am start --display 1 -a android.settings.INPUT_METHOD_SETTINGS
```

```
am start --display 2 -n com.android.deskclock/.DeskClock
```

```
am start --display 3 -a android.intent.action.VIEW -d http://www.baidu.com
```

#### gst-launch-1.0 weston刷图

1.找个match 屏幕大小的PNG图片, push 该PNG 图片到yocto 端 /data/

2.播放cmd: yocto 端 adb root -> adb shell 后 下cmd

```
source /data/wayland_env_file
```

a.DSI0 显示

```
gst-launch-1.0 filesrc location=/data/RGB.png ! pngparse ! pngdec ! imagefreeze ! videoconvert ! video/x-raw,format=BGRA ! waylandsink render-rectangle='<0,0,1920,1080>'
```

b.DSI1 显示

```
gst-launch-1.0 filesrc location=/data/RGB.png ! pngparse ! pngdec ! imagefreeze ! videoconvert ! video/x-raw,format=BGRA ! waylandsink render-rectangle='<1920,0,1920,1080>'
```

c.DP 显示

```
gst-launch-1.0 filesrc location=/data/RGB.png ! pngparse ! pngdec ! imagefreeze ! videoconvert ! video/x-raw,format=BGRA ! waylandsink render-rectangle='<3840,0,1920,1080>'
```

### 8.1.3.4 Display pipeline dump (Yocto os / Android os)

```
adb shell "echo diagnose > /sys/kernel/debug/mtkfb && cat /sys/kernel/debug/mtkfb" > diagnose0810 .log
```

### 8.1.3.5 Android强制GPU合成 (Android os)

开: adb shell service call SurfaceFlinger 1008 i32 1

关: adb shell service call SurfaceFlinger 1008 i32 0

### 8.1.3.6 Display模块clk动态调整 (Yocto os)

拉到最大 (分析闪屏花屏问题的时候使用)

```
adb shell "echo 0 0 > /sys/module/mtk_mmdvfs_debug/parameters/force_step"
```

### 8.1.3.7 PQ关闭

```
adb shell "echo helper:MTK_DRM_OPT_USE_PQ,0 > /sys/kernel/debug/mtkfb"
```

执行suspend / resume

### 8.1.3.8 Buffer DUMP

SurfaceFlinger BufferQueue dump (Android os) (有bug, Hypervisor系统暂时不支持)

hwc buffer dump (Android os)

MML/MDP buffer dump (Android os)

1.请在Android端敲echo 1 > sys/module/mtk\_mml/parameters/mtk\_mml\_msg和echo 1 >

sys/module/mtk\_mml/parameters/mml\_log\_rec, 打开mml debug log。(复制到现象时应该会有大量带mml关键字log输出)

2.在复制到问题时分别dump input 和 output

// dump input 1 frame

```
adb shell "echo 1 > /sys/module/mtk_mml/parameters/mml_frame_dump"
```

```
adb pull /sys/kernel/debug/mml/mml-frame-dump-in frame_dump_in.bin
```

// dump output 1 frame

```
adb shell "echo 2 > /sys/module/mtk_mml/parameters/mml_frame_dump"
```

```
adb pull /sys/kernel/debug/mml/mml-frame-dump-out frame_dump_out.bin
```

查看out.bin的时候, 从kernel log搜索[mml]out关键字, 里面有描述out.bin的参数信息

FBSurface dump(input && output)

FBSurface使用场景是bootanimation, fast display (rvc, avm) 等

```
adb shell setprop vendor.fbsurface.dump.enable 1
```

dump出的图路径在: /sdcard/fb\_dump/

**PS:** FBSurface当前dump数据是在刷图流程里, 如果应用只刷了一帧, 一定要在应用起来之前执行adb shell setprop vendor.fbsurface.dump.enable 1

### 8.1.3.9 Display log

hwc log (Android os)

```
adb shell setprop persist.vendor.debug.hwc.log V && adb shell setprop vendor.debug.hwc.skip_log 0
```

## android display driver log

log通过cat /proc/kmsg或者debuggloger log目录查看

常用display mobile log

实时开关:

```
adb shell "echo mobile:on > /d/mtkfb"
```

```
adb shell "echo detail:on > /d/mtkfb"
```

开机log:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_mobile_log = 1
```

```
bool g_detail_log = 1
```

## android fence log

log通过cat /proc/kmsg或者debuggloger log目录查看

实时开关:

```
adb shell "echo fence:on > /d/mtkfb"
```

开机log:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_fence_log = 1
```

## irq log

log通过cat /d/mtkfb > /data/mtkfb.txt 查看IRQ相关log

实时开关:

```
adb shell "echo irq:on > /d/mtkfb"
```

开机log:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_irq_log = 1
```

## yocto display driver log

常用display mobile log

实时开关:

```
adb shell "echo mobile:on > /sys/kernel/debug/mtkfb"
```

```
adb shell "echo detail:on > /sys/kernel/debug/mtkfb"
```

开机log:

```
drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_mobile_log = 1
```

```
bool g_detail_log = 1
```

## yocto fence log

实时开关:

```
adb shell "echo fence:on > /sys/kernel/debug/mtkfb"
```

开机log:



drivers/gpu/drm/mediatek/mediatek\_v2/mtk\_debug.c

bool g\_fence\_log = 1

### yocto irq log

实时开关:

adb shell "echo irq:on > /sys/kernel/debug/mtkfb"

开机log:

drivers/gpu/drm/mediatek/mediatek\_v2/mtk\_debug.c

bool g\_irq\_log = 1

## 8.1.4 Troubleshooting

### 8.1.4.1 黑屏

常规debug顺序以及方法

- 1.查看背光是否开启
  - 2.如果有bridge ic, 查看bridge ic状态是否正常
  - 3.bridge pattern是否正常, soc dp/dsi pattern是否正常
  - 4.android os: vsyor / screencap截图是否正常, yocto os: weston 截图是否正常
- 如果都ok, 要通过详细分析log来定位问题

### 8.1.4.2 花屏/闪屏

常规debug顺序以及方法

- 1.bridge ic pattern, dsi / dp pattern是否正常
  - 2.vsyor / android screencap / weston screencap查看原始数据是否正常
  - 3.force GPU 是否正常
  - 4.查看log中是否有DISP\_OVL / RDMA underflow / abnormal 异常日志
  - 5.通过driver日志, 查看显示参数是否配置错误
  - 6.提升display clk以及dram, 查看是否有改善
- 如果还有问题, 想通过详细分析log来定位问题

### 8.1.4.3 卡顿/冻屏

常规debug顺序以及方法

- 1.查看android main log里面是否有fence timeout
- 2.查看driver log是否有underflow/abnormal 异常log

3. 开启fence log，看对应fence是否release

4. 抓取systrace看耗时位置

[config.pbtX](#)附件

```
adb root
```

```
adb push config.pbtX /data/ //将config push到开发板
```

```
adb shell
```

```
cat /data/config.pbtX | perfetto --txt -c - -o /data/misc/perfetto-traces/trace_XXX //开发板执行此命令,开始复现问题,复现后ctrl +x 停止抓取trace
```

```
adb pull /data/misc/perfetto-traces/trace_XXX //将trace文件拉取到本地
```

## 8.2 Camera

### 8.2.1 camera虚拟化概念介绍

MT8676 camera虚拟化是camera跑在yocto端，android端访问camera通过vsocket方式交互。android端向yocto端要数据，数据的处理都在yocto端。

#### 8.2.1.1 基本架构介绍

(1) 宿主机Yocto：直接访问相机硬件

(2) 虚拟机Android：通过 VSOCKET 与宿主机通信

(3) 中间层RPC（Remote Procedure Call，远程过程调用）：处理相机操作并通过 VSOCKET 传输数据。是一种重要的分布式计算技术，允许程序调用另一个地址空间（通常是在网络上的另一台计算机）的子程序，就像调用本地程序一样。

#### 8.2.1.2 大致实现步骤

(1) Yocto端：

创建一个RpcService，直接与相机硬件交互，使用 VSOCKET 监听来自 Android 的连接

(2) Android端：

创建一个RpcSession，使用 VSOCKET 连接到Yocto服务并获取yocto端的camerahalservice服务

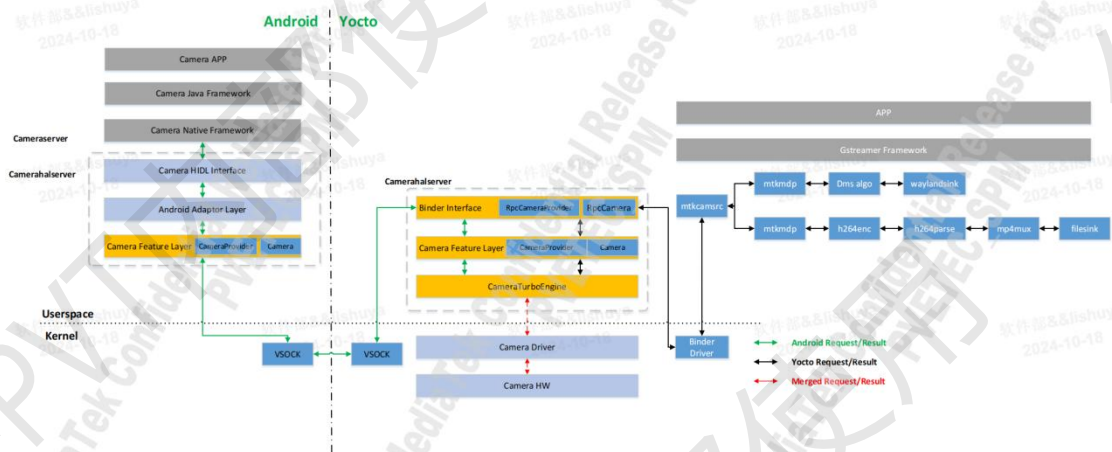
#### 8.2.1.3 数据流

Android -> VSOCKET -> Yocto服务 -> 相机硬件

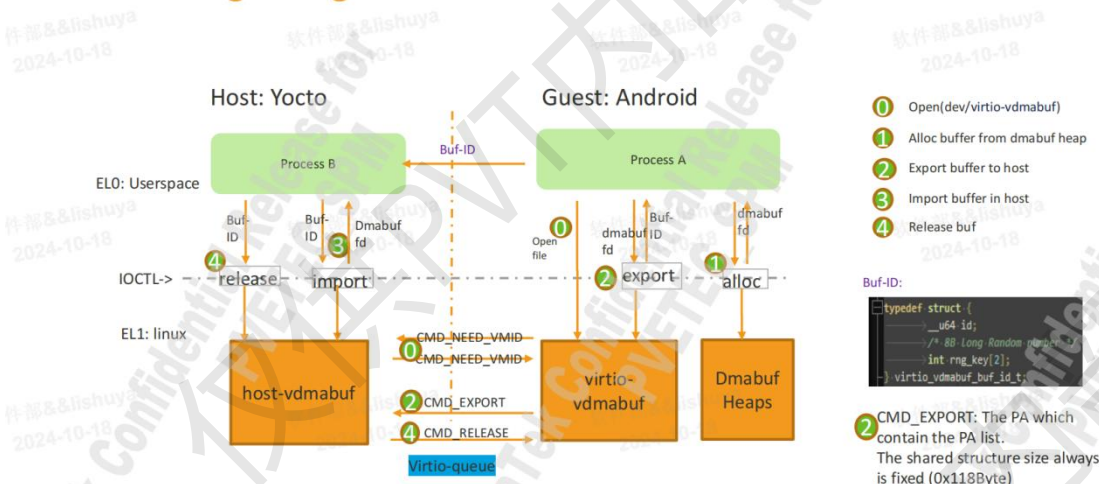
相机硬件 -> Yocto服务 -> VSOCKET -> Android

## 8.2.2 camera虚拟化框架介绍

### Camera virtualization architecture



### Dmabuf sharing from guest OS to host OS



(此图介绍详见7.14.1章节)

## 8.2.3 camera主要code位置

### 8.2.3.1 Yocto端

Camera 虚拟化是在 src\multimedia\mtkcam-mt8676\camera\_turbo\_rpc\service\service.cpp 中 setupRpcService 接口通过 RpcServer 创建 vsock server, 将 camerahaserver 服务设置为 root object.

### 8.2.3.2 Android端

Camera 虚拟化的主要实现：vendor\mediatek\proprietary\hardware\mtkcam-turbo-android\mtkcam-turbo\entry\hypervisor 中的 CameraProvider::initialize 接口通过 RpcSession 创建 vsock client，获取 yocto 端的 root object 即 camerahalservice 服务，从而访问 yocto 端的 camera。

### 8.2.4 配置/客制化指南

#### 8.2.4.1 如何添加一个新的sensor



MT8676\_Yocto\_  
Camera\_Driver\_L

附件

如上文档是单yocto版本添加一个新的sensor的方法。

Hyper版本camera driver也是在yocto端，添加一个新sensor的方法与上面文档只有一点不同：文档中1.3.1.1和1.3.1.2提到的config文件不同。

单yocto版本config files：

src\kernel\linux\v6.1\_mt8676\co\_device\_module\arch\arm64\configs\auto8676p1\_64\_defconfig

hyper版本config files：meta\meta-mediatek-mt8676-hyp\recipes-

kernel\linux\files\auto8676p1\_64\_hyp\_defconfig

#### 8.2.4.2 如何配置某个camera在android端支持多开

android端camera受framework限制，如果想要camera支持多开，需要在yocto端按照需求对某个或某些真实的cameraID虚拟出虚拟cameraID，这样让android的framework层可以多拿到虚拟的cameraID（实际对应底层开的还是同一个sensor）。

##### 8.2.4.2.1 修改文件列表

yocto\src\multimedia\mtkcam-mt8676\camera\_turbo\custom\include\Customization.h

##### 8.2.4.2.2 如何修改

修改示例：

```
// for customer to set which camera is needed to support multi-open for android client
```

```
// yocto client multi-open use shift bit in custLayer already
const std::unordered_map<std::string, int32_t> CUSTOMER_MULTI_OPEN_DEVICES_MAP =
{
 //virtualId is real cameraId + 32, 64, 96, ...
 //Due to the t/p limitation of pqdip HW, random operations in the upper layer may result in frame loss
 {"SENSOR_DRVNAME_MAX96714_MIPI_YUV", 4},
}
```

sensor name名字需要与kd\_imgsensor.h中定义的sensor name一致。

上述示例表示配置SENSOR\_DRVNAME\_MAX96714\_MIPI\_YUV支持四开，即其真实的cameraID在yocto端会虚拟出三个虚拟的cameraID，虚拟ID分别是：真实cameraID的值+32，真实cameraID+64，真实cameraID+96。

### 8.2.4.2.3 注意事项

- 1、由于PQDIP的t/p有上限，8676的大约是960MPixels，由于有硬件限制，不需要多开需求的sensor不要对其进行配置。如果同时打开太多cameraID或乱操作，可能会导致掉帧等问题。
- 2、如果只是yocto端想要多开，则不用做上述修改。yocto端可以使用同一个cameraID，然后根据不同的UserId区分，实现多开（实际对应底层开的还是同一个sensor）。

### 8.2.4.3 yocto端上层如何取流（一）

基于 camerahal 封装一层接口——适配层，不同的操作系统可以通过调用适配层接口进行 camera 取流。

#### 8.2.4.3.1 code位置

yocto\src\multimedia\mtkcam-mt8676\ipc\_adapter\ipcClient

#### 8.2.4.3.2 生成物

libipcclient.so

### 8.2.4.3.3 yocto端上层对接ipcclient取流参考示例

#### 8.2.4.3.3.1 code位置

yocto\src\multimedia\mtkcam-mt8676\ipc\_adapter\test

#### 8.2.4.3.3.2 生成物

bin/ipcclienttest

### 8.2.4.3.3 测试方法

执行命令如下 `ipcclienttest <stream> <dumpAll> <dumpFrameNum> <seconds> <cameraid>`

**测试示例 1:** 打开 camera 0 的一路流，取流 1s，之后关闭 camera 0。此场景执行命令 `ipcclienttest 0 0 -1 1`

0 即可，终端正常输出 `test done: FPS = xxx`，表示测试正常。

**测试示例2:** 测试cameravid 0是否能正常取流且dump camera data查看数据是否正常

(1) 创建/data/camera\_dump路径;

(2) dump某一指定帧的数据，例如dump帧号为1的数据：执行`ipcclienttest 0 0 1 5 0`;

(3) dump所有帧数据：执行`ipcclienttest 0 1 -1 5 0`;

数据放置路径：/data/camera\_dump

### 8.2.4.4 yocto端上层如何取流（二）

Yocto应用基本都是调用源生的gststream，MTK有提供mtkcamsrc插件。

#### 8.2.4.4.1 code位置

src\multimedia\gst-mtkcamsrc

#### 8.2.4.4.2 yocto端app对接gststream mtkcamsrc取流参考示例

##### 8.2.4.4.2.1 code位置

src\multimedia\gst-mtkcamsrc\tests

##### 8.2.4.4.2.2 测试方法

**测试示例:** 执行如下命令，可在屏幕上显示出camid 32的数据

`gst-launch-1.0 mtkcamsrc camid=32 ! video/x-`

`raw,format=YUY2,width=1920,height=1080,framerate=30/1 ! v4l2convert disable-passthrough=true output-io-mode=dmabuf-import capture-io-mode=dmabuf ! video/x-`

`raw,format=BGRA, width=640,height=360 ! waylandsink sync=false render-rectangle="<0,400,-1,-1>"`

## 8.2.5 Debug手段

### 8.2.5.1 log放开方法

#### 8.2.5.1.1 Yocto端log放开方法

1. 如何设置camerahalservice的输出log等级

```
adb shell mtkcam_setprop persist.mtk.camera.log_level 4
adb shell mtkcam_setprop vendor.debug.camera.log 4
adb shell mtkcam_setprop vendor.debug.camera.rpclog 1
adb shell mtkcam_setprop vendor.debug.camera.ipcclientlog 1
adb shell systemctl restart camerahalservice
```

注意：重启平台后失效

2. 获取native log

```
adb shell journalctl > journalctl.txt
```

3. 获取kernel log

```
adb shell dmesg > dmesg.txt
adb shell cat /proc/kmsg > kmsg.txt
```

4. Yocto debuglogger抓取

```
adb pull /data/debuglogger/ ./
```

#### 8.2.5.1.2 Android端log放开方法

1. 如何将kernel log打印在串口上

```
adb shell "echo 1 > \proc\sys\kernel\printk" (重启失效)
```

2. 打开libcameraservice中的ALOV和ALOGVV

frameworks/av/services/camera/libcameraservice/Android.bp 作如下修改：

```
cflags: [
 "-Wall",
 "-Wextra",
 "-Werror",
 "-Wno-ignored-qualifiers",
 "-DLOG_NDEBUG=0", //增加这行
 "-DLOG_NNDEBUG=0",
],
```

3. android camera log放开命令



(1) 放开camerahal log

```
adb root
```

```
adb shell setprop persist.vendor.mtk.camera.log_level 4
```

```
adb reboot或者adb shell "pidof camerahalservice | xargs kill"
```

(2) 放开黄标Camera apk的log

```
adb shell setprop vendor.debug.mtkcam.loglevel 4
```

(3) 比较完整的放开camera各层级的log可以执行下面的命令

```
adb root
```

```
adb shell setprop persist.vendor.mtk.camera.log_level 3
```

```
adb shell setenforce 0
```

```
adb shell logcat -G 100M
```

```
adb shell logcat -c
```

```
adb shell setprop vendor.debug.camera.uilog.details 4294963200
```

```
adb shell setprop vendor.debug.camera.uilog.level 5
```

```
adb shell setprop vendor.debug.mtkcam.loglevel 4
```

```
adb shell setprop persist.vendor.mtk.camera.log_level 1
```

```
adb shell setprop persist.vendor.debug.camera.log 1
```

```
adb shell setprop debug.camera.log 4
```

```
adb shell setprop vendor.debug.camera.log 4
```

```
adb shell setprop vendor.debug.camera.metadata 4
```

```
adb shell setprop persist.log.tag 1
```

```
adb shell setprop persist.mtk.camera.log_level 4
```

```
adb shell setprop vendor.debug.isp 4
```

```
adb shell -> echo 8 > /proc/sys/kernel/printk (重启会失效)
```

```
adb shell -> echo -n 'file *camera_isp* +tp' > /sys/kernel/debug/dynamic_debug/control (重启会失效)
```

```
adb shell "pidof camerahalservice | xargs kill"或者adb reboot (有些重启会失效，所以建议只用第一个)
```

4. android debuglogger抓取

```
adb pull /data/debuglogger/ ./
```

## 8.2.5.2 dump camera data

yocto端dump p1,p2图片方法:

1. 创建/data/vendor/camera\_dump路径

2. 打开camera前执行如下命令:

```
mtkcam_setprop vendor.debug.camera.coredevice.wpe.dump 2
```

3. 停止dump:

```
mtkcam_setprop vendor.debug.camera.coredevice.wpe.dump 0
```

4. camera data放置路径:

/data/vendor/camera\_dump

5. 查看yuv数据工具：7yuv附件



7yuv安装包.tar

### 8.2.5.3 dump meta

android端dump meta的方法：

```
adb shell dumpsys media.camera > dump_meta.txt
```

说明：meta中包含所有android用户可见的camera devices。

### 8.2.5.4 search camera sensor

在yocto端执行：sentest\_v4l2命令，即可看到目前search到的sensor信息，示例如下所示：

```
sh-3.2# sentest_v4l2
[main]sizeof long : 4
debug---[init_sensor_list] ioctl fail, line=300
[show_Sensors]sensorNum 8
[show_Sensors]name:SENSOR_DRVNAME_MAX96714_MIPI_YUV type:1 max_scen: 4
[show_Sensors]index:0, SensorDevIdx:1
...
```

### 8.2.5.5 测试验证yocto端camera是否正常的用例

调用 HIDL: yocto\src\multimedia\mtkcam-mt8676\camera\_turbo\_rpc\include\

#### 8.2.5.5.1 code位置

yocto\src\multimedia\mtkcam-mt8676\camera\_turbo\_rpc\rptest\

#### 8.2.5.5.2 测试方法

执行命令如下

```
rpctest <stream> <dumpAll> <dumpFrameNum> <seconds> <cameraid>
```

**测试示例1：**打开camera 0的一路流，取流5s后，关闭camera 0。此场景执行命令rpctest 0 0 -1 5 0即可，终端正常输出test done: FPS = xxx，表示测试正常。

**测试示例2：**测试cameraid 0是否能正常取流且dump camera data查看数据是否正常

(1) 创建/data/camera\_dump路径；

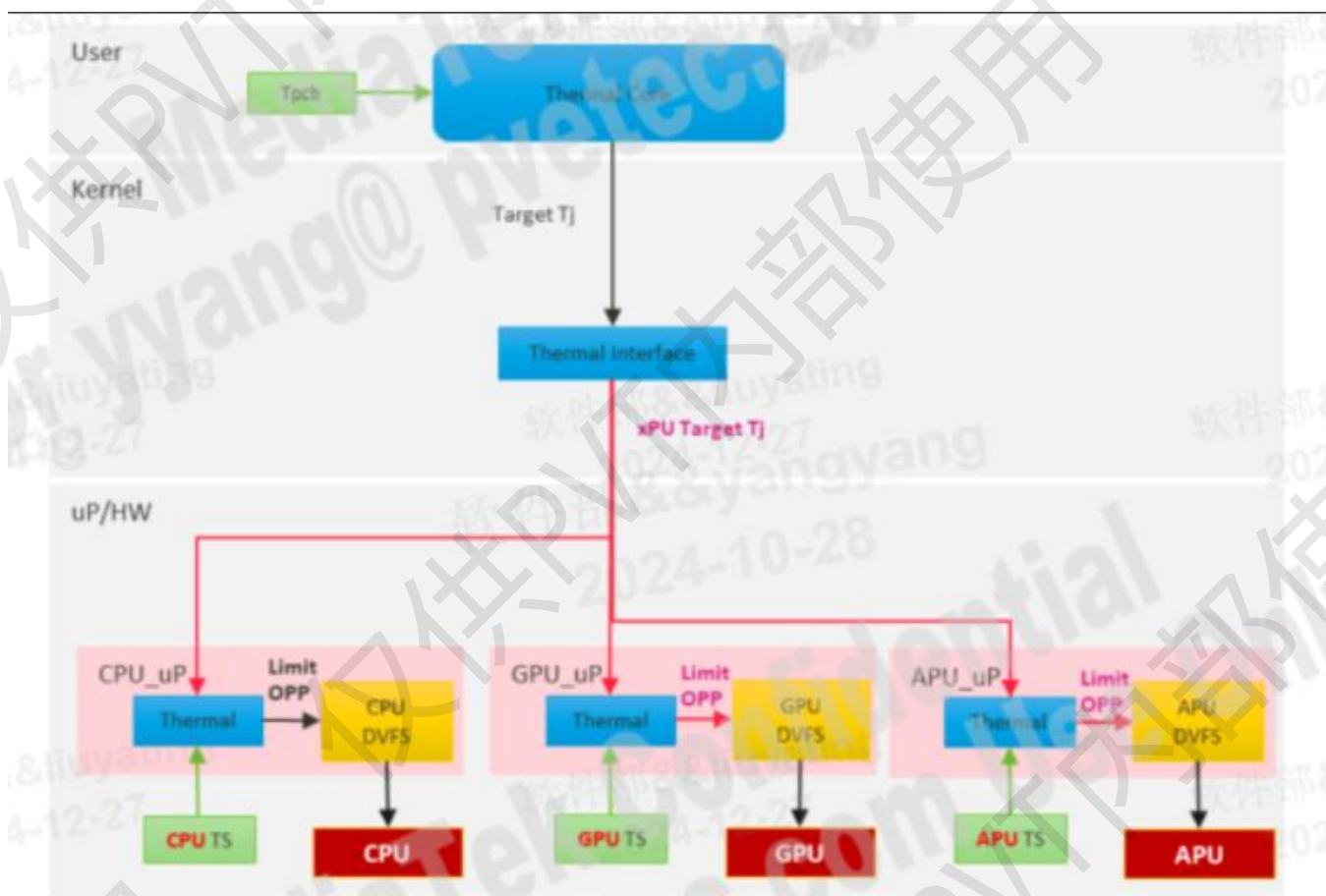
(2) dump某一指定帧的数据，例如dump帧号为1的数据：执行rpctest 0 0 1 5 0；

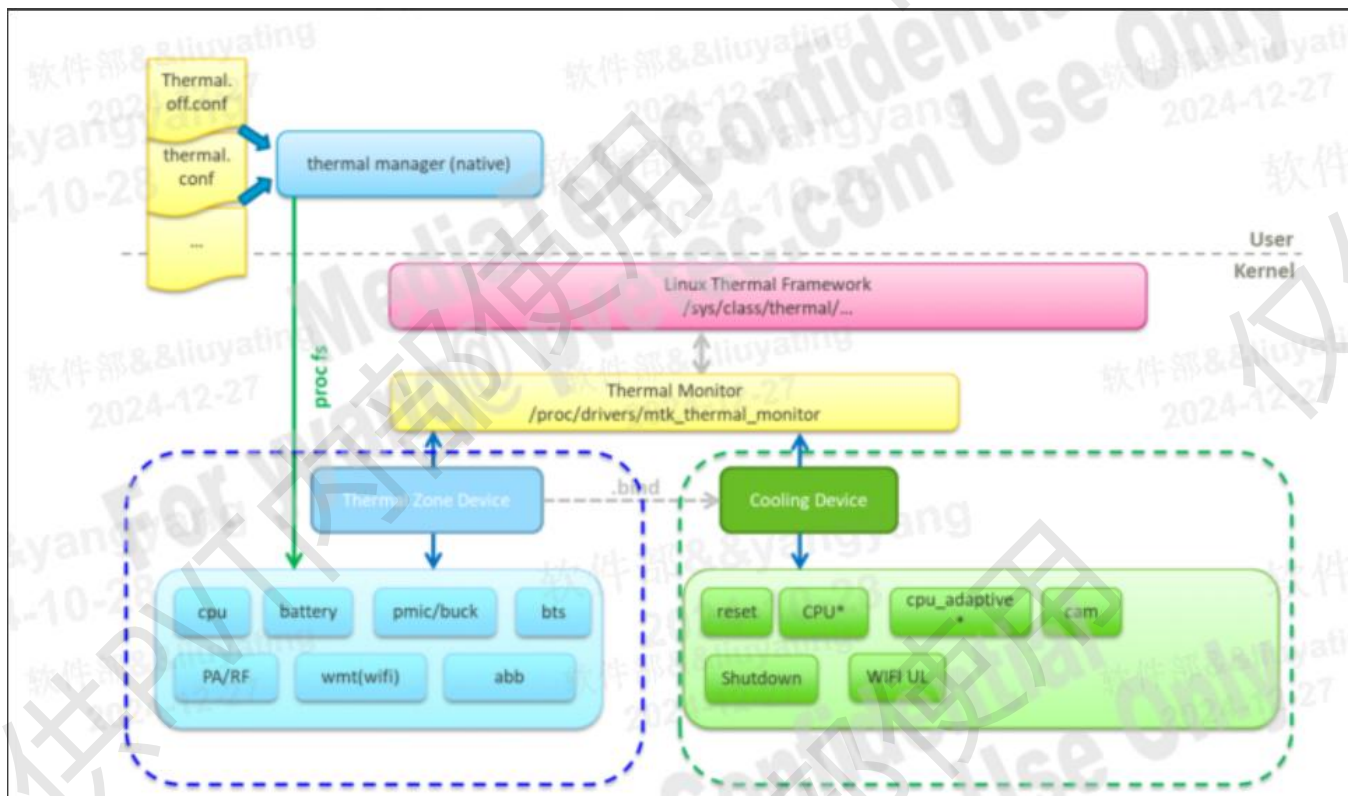
(3) dump所有帧数据：执行rpctest 0 1 -1 5 0；

数据放置路径：/data/camera\_dump

## 8.3 thermal

### 8.3.1 Architecture

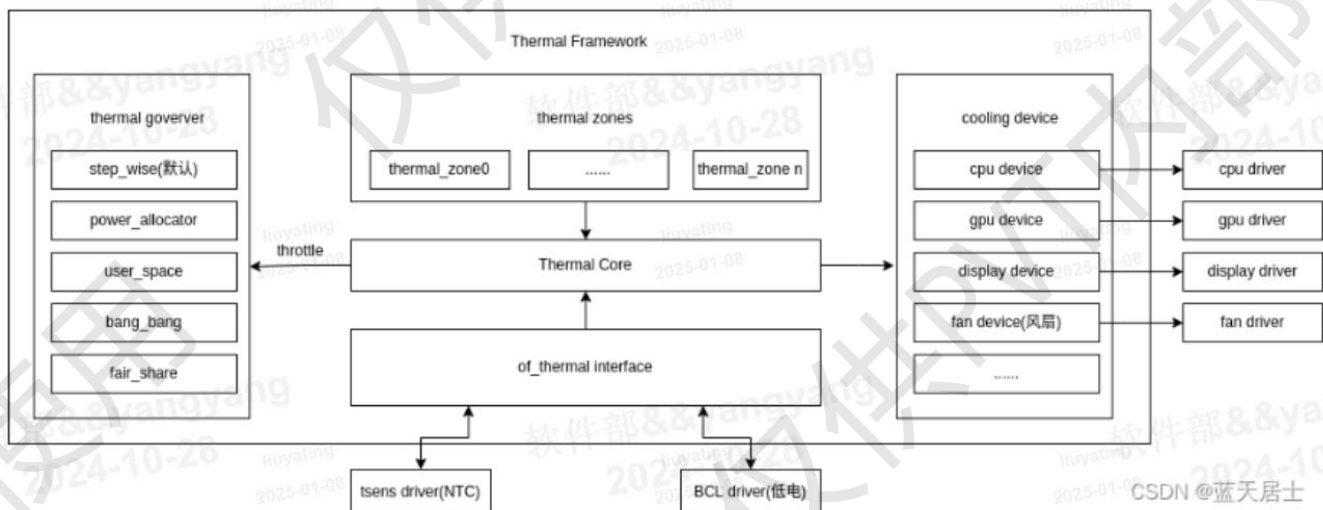




### 8.3.2 Thermal sensor

### 8.3.3 Linux thermal framework

1. Linux thermal的整体框架如下图所示



**Thermal Core:** Thermal Core 作为 Thermal 框架的中枢，提供了一系列注册 Governor、注册 Thermal 类、基于Device Tree注册Thermal Zone 的一系列 接口；

**Thermal Driver/Thermal Sensor:** 为整个 Thermal 提供读取温度、设置 trip 温度等功能的接口；

Thermal Governor: Thermal Governor 决定了当温度达到某个阈值时，系统应该采取什么措施，常见的 Governor 有：Power Allocator、Step Wise、User Space、FairShare等；

Thermal Cooling: Thermal Cooling作为温控系统的输出，包含了用于降温的实际硬件或软件方法。 主要通过降频降压实现降功耗，这些设备可以是 CPU、GPU等；

可以获取温度的设备抽象为thermal\_zone\_device， dts配置的thermal zone如下图所示：

```

xref: /mt8676_pvt_demo/yocto/src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/mt6897.dts
Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download Search current directory
thermal_zones: thermal-zones
2732 soc_max {
2733 polling-delay = <0>; /* milliseconds */
2734 polling-delay-passive = <0>; /* milliseconds */
2735 thermal-sensors = <&lvt5 0>;
2736 trips {
2737 soc_max_crit: so-max-crit {
2738 temperature = <119000>;
2739 hysteresis = <2000>;
2740 type = "critical";
2741 };
2742 };
2743 };
2744
2745 cpu-medium-core4-1 {
2746 polling-delay = <0>; /* milliseconds */
2747 polling-delay-passive = <0>; /* milliseconds */
2748 thermal-sensors = <&lvt5 1>;
2749 trips {

```

Thermal Cooling目前dts没有配置，是通过conf文件来实现的

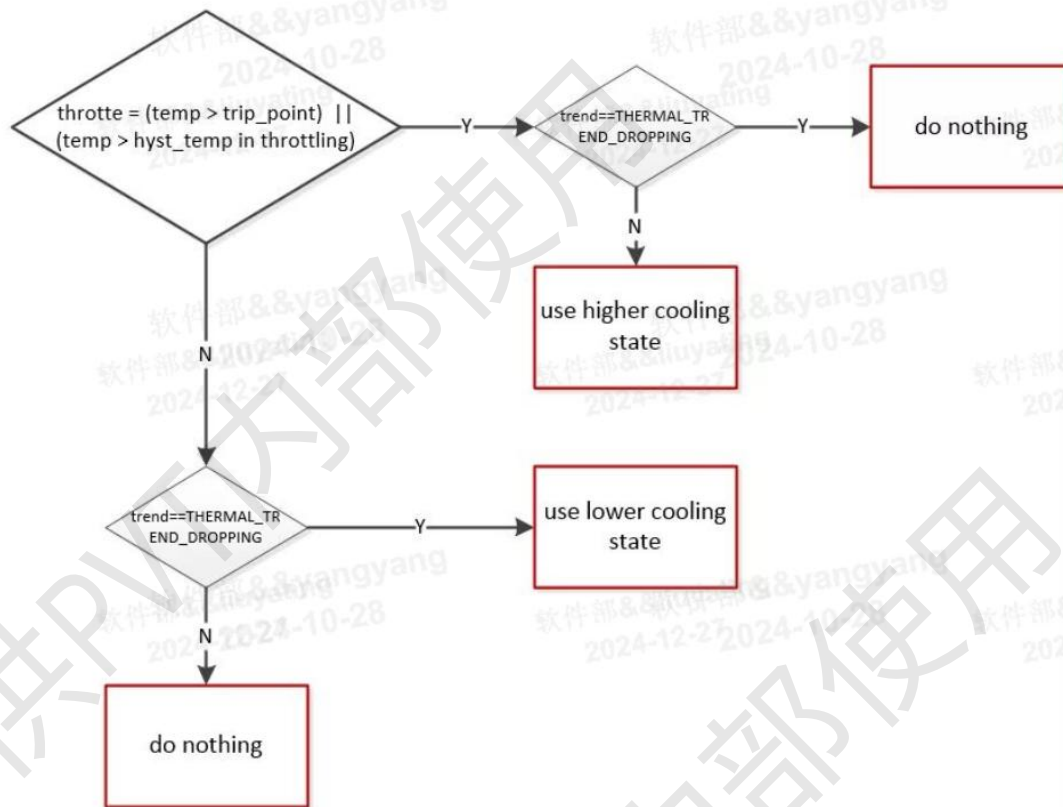
cooling device 维护一个 cooling 等级，即 state，一般 state 越高即系统的冷却需求越高。

cooling device 根据不同等级的冷却需求进行冷却行为。cooling device 只根据 state 进行冷却操作，是实施者，而 state 的计算由 thermal governor 完成。

1).step\_wise governor

step\_wise 算法在计算 target cooling state 的过程中，除了需要知道是否 throttle，还添加了一个 trend 作为参考条。

trend 表示温升趋势，Linux Thermal Framework 定义了五种 trend type，见 enum thermal\_trend，即稳定(THERMAL\_TREND\_STABLE)，上升（THERMAL\_TREND\_RAISING），下降（THERMAL\_TREND\_DROPPING），最高温线（THERMAL\_TREND\_RAISE\_FULL），最低温线（THERMAL\_TREND\_DROP\_FULL）。



step\_wise governor 对于 cooling\_state 选择的策略：

- 当 throttle 发生且温升趋势为上升，使用更高一级的 cooling state;
- 当 throttle 发生且温升趋势为下降，不改变 cooling state;
- 当 throttle 解除且温升趋势为下降，不改变 cooling state;
- 当 throttle 解除且温升趋势为上升，使用更低一级的 cooling state;

## 2).gov\_power\_allocator

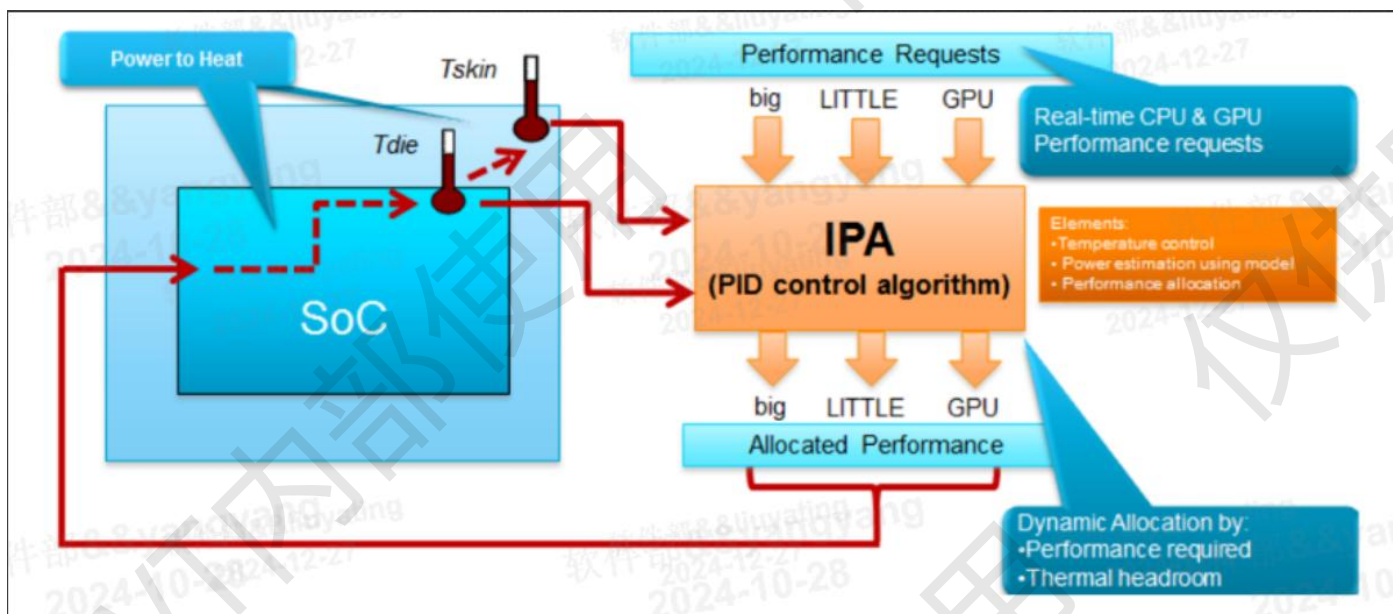
原理：

对于IPA来讲，主要的输入为温度和各个Core的性能需求。温度可以来自SOC，也可以来自板温，IPA把采集温度与目标控制温度的温差作为控制输入。

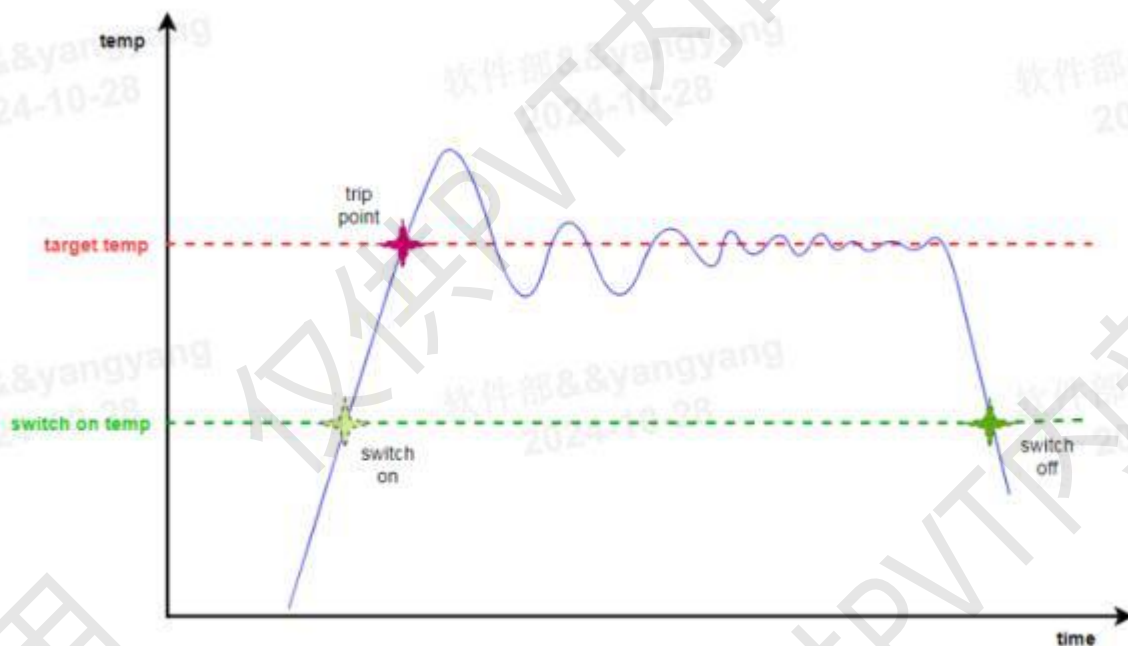
各个IP Core的性能需求是通过当前的负载来计算的，然后利用OPP(Operating Performance Point)及EM(Energy Model)转换为Power Request作为IPA的输入。

而IPA的输出是PowerGranted，然后再通过OPP及EM转换为性能指标（比如最大运行频率）。如此的循环往复最终实现对硬件的温度控制。





优点：IPA实现了多个IP core之间Power的动态分配，实现资源的优化配置。以小核、大核、GPU为控制对象来讲，即使温度已经达到目标温度，但是根据各个小核、大核、GPU的负载情况，最需要性能的处理器仍然有可能跑到最高频率，这种基于负载的power动态分配策略对性能比较友好



switch\_on\_temp是IPA策略启动的触发温度  
control\_temp是IPA策略的目标温度阈值，即是降频的温度

### 8.3.4 配置文件

#### 1. 配置文件介绍



目前thermal的温控管理是放在yocto侧的，

配置策略在source code的路径是src/app/spm-base/thermal-conf/mt8676, 支持加密格式

Thermal policy	Permanent?	Encrypted?	Description
thermal.conf	Yes	Yes	Default thermal policy
disable_thermal.conf	Yes	Yes	Disable thermal throttling and thermal protection
disable_thermal_temp.conf	No	Yes	Same as the above, except it needs to re-apply after device rebooted
disable_throttling.conf	No	Yes	Disable thermal throttling
disable_skin_control.conf	No	Yes	Disable MTK skin control close loop (always keep Target T <sub>j</sub> to 95°C)
Thermal_policy_XX.conf (XX = 00~19 except 00, 02, 08)	On demand	Yes	Can add your own policy setting and switch via power HAL
thermal_policy_08.conf	No	Yes	Thermal policy for benchmark

Thermal 2.0 配置文件可采用文本编辑方式直接修改，不再需要Thermal config Tool。具体格式如下：

```
[policy_type]
If permanent= Yes, this policy is still valid after reboot
permanent= Yes
[LTF-disable-throttling]
If HW_protection= disabled, it will turn off
SW shutdown protections and LVTTS HW reboot protection
HW_protection= enabled
[mtk-skin-control]
sensor= ap_ntc #设定此策略绑定的sensor为ap_ntc(SOC附近NTC)
max_target_tj= 95000 #设定soc_max最大目标温度为95度
min_target_tj= 68000 #设定soc_max最小目标温度为68度
trip_tpcb= 53000 #设定Tpcb1, 与soc目标温度下降的斜率有关系
target_tpcb= 56000 #设定目标板温(Tpcb2)为56度, 即通过限制CPU/GPU频率核数, 将板温控制在56度
cluster0_min_throttle_freq= 1725000 #设定CPU小核可调节的频率下限为1.725G。如不需要设定频率下限, 此行可删除。
cluster1_min_throttle_freq= 1624000 #设定CPU中核可调节的频率下限为1.624G。如不需要设定频率下限, 此行可删除。
cluster2_min_throttle_freq= 1745000 #设定CPU大核可调节的频率下限为1.745G。如不需要设定频率下限, 此行可删除。
gpu_min_throttle_freq= 715000 #设定GPU可调节的频率下限为715M。如不需要设定频率下限, 此行可删除。
[mtk-cpu-table-mapping] #如不需要使用, 可以删除
```

mode= enabled           #该策略使能设定。如不使用该策略，则将“enabled”修改为“disabled”或删除此配置单元

sensor= ap\_ntc           #设定此策略绑定的sensor为ap\_ntc(SOC附近NTC)

cluster= 0           #设定此策略控制的是CPU小核

threshold= [39000, 42000, 44000, 46000, 48000, 50000, 52000]   #设定板温(ap\_ntc)门限档位，这些档位与freq中设定的频率档位一一对应

hysteresis= [2000, 2000, 2000, 2000, 2000, 2000, 2000]       #设定各档位退出门限，2000代表2度，即板温降低2度则退出该档CPU频率限制

freq= [2000000, 1800000, 1800000, 1800000, 1800000, 1725000, 1725000]   #设定CPU的频率限制档位，与threshold中设定的板温门限档位一一对应

#举例：当板温达到39度时，CPU频率限制为2G，当板温降低2度至37度时，解除该档位限制

[mtk-cpu-table-mapping]           #如不需要使用，可以删除

mode= enabled           #该策略使能设定。如不使用该策略，则将“enabled”修改为“disabled”或删除此配置单元

sensor= ap\_ntc           #设定此策略绑定的sensor为ap\_ntc(SOC附近NTC)

cluster= 1           #设定此策略控制的是CPU中核

threshold= [39000, 42000, 44000, 46000, 48000, 50000, 52000]   #设定板温(ap\_ntc)门限档位，这些档位与freq中设定的频率档位一一对应

hysteresis= [2000, 2000, 2000, 2000, 2000, 2000, 2000]       #设定各档位退出门限，2000代表2度，即板温降低2度则退出该档CPU频率限制

freq= [2600000, 2354000, 2200000, 1985000, 1855000, 1624000, 1624000]   #设定CPU的频率限制档位，与threshold中设定的板温门限档位一一对应

#举例：当板温达到39度时，CPU频率限制为2.6G，当板温降低2度至37度时，解除该档位限制

[mtk-cpu-table-mapping]           #如不需要使用，可以删除

mode= enabled           #该策略使能设定。如不使用该策略，则将“enabled”修改为“disabled”或删除此配置单元

sensor= ap\_ntc           #设定此策略绑定的sensor为ap\_ntc(SOC附近NTC)

cluster= 2           #设定此策略控制的是CPU大核

threshold= [39000, 42000, 44000, 46000, 48000, 50000, 52000]   #设定板温(ap\_ntc)门限档位，这些档位与freq中设定的频率档位一一对应

hysteresis= [2000, 2000, 2000, 2000, 2000, 2000, 2000]       #设定各档位退出门限，2000代表2度，即板温降低2度则退出该档CPU频率限制

freq= [2600000, 2141000, 1820000, 1745000, 1745000, 1745000, 1745000]   #设定CPU的频率限制档位，与threshold中设定的板温门限档位一一对应

#举例：当板温达到39度时，CPU频率限制为2.6G，当板温降低2度至37度时，解除该档位限制

[mtk-cpu-isolate] #如不需要使用，可以删除

mode= enabled #该策略使能设定。如不使用该策略，则将“enabled”修改为“disabled”或删除此配置单元

sensor= ap\_ntc #设定此策略绑定的sensor为ap\_ntc(SOC附近NTC)

cpu= [cpu4, cpu5, cpu6, cpu7] #关闭cpu策略。cpu4代表关闭cpu4，其他依此类推

threshold= [52000, 52000, 48000, 48000] #设定板温(ap\_ntc)门限档位，与cpu4~cpu7分别对应，即当板温大于48度，关闭cpu6/cpu7,当板温大于52度，关闭cpu4/cpu5

hysteresis= [2000, 2000, 2000, 2000] #设定各档位退出门限，2000代表2度，即板温降低2度则退出对应的关闭cpu策略

[mtk-gpu-table-mapping] #如不需要使用，可以删除

mode= enabled #该策略使能设定。如不使用该策略，则将“enabled”修改为“disabled”或删除此配置单元

sensor= ap\_ntc #设定此策略绑定的sensor为ap\_ntc(SOC附近NTC)

threshold= [39000, 42000, 44000] #设定板温(ap\_ntc)门限档位，与freq中GPU频率档位分别对应

hysteresis= [2000, 2000, 2000] #设定各档位退出门限，2000代表2度，即板温降低2度则退出该档GPU频率限制

freq= [854000, 815000, 715000] #设定GPU的频率限制档位，与threshold中设定的板温门限档位一一对应

[charger-cooler]

mode= enabled #该策略使能设定。如不使用该策略，则将“enabled”修改为“disabled”或删除此配置单元

sensor= ap\_ntc #设定此策略绑定的sensor为ap\_ntc(SOC附近NTC)

threshold= [43000, 46000, 49000] #设定板温(ap\_ntc)门限档位，与current中充电电流(进电池)档位分别对应

hysteresis= [2000, 2000, 2000] #设定各档位退出门限，2000代表2度，即板温降低2度则退出该档充电电流限制

current= [1800, 1000, 0] #设定充电电流限制档位，与threshold中设定的板温门限档位一一对应

[thermalhal-threshold]

type= SKIN

sensor= ap\_ntc

level= [severe, critical, emergency, shutdown]

threshold= [50000, 56000, 58000, 80000]

#Google定义的thermal HAL，绑定板温（无法通过thermal config修改）。

#定义了severe, critical, emergency, shutdown四个事件，shutdown事件会Trigger Android shutdown,

emergency事件Trigger MTK camera app exit。

[wifi-cooler]

mode= enabled

sensor= ap\_ntc #设定此策略绑定的sensor为ap\_ntc(SOC附近NTC)

threshold= [55000, 60000, 65000] #设定不同的板温对应不同的wifi chip温度门限

hysteresis= [2000, 2000, 2000] #设定各档位退出门限，2000代表2度。

target\_temp= [90000, 85000, 80000] #设定不同的wifi chip温度门限。

#根据不同的板温设定wifi chip的目标温度，通过wifi throttle来守住wifi chip的温度，按2T2R→1T2R→1T1R→

Duty 50%过程加严控制，2T2R代表2个TX天线和2个接收天线。

[backlight-cooler]

mode= enabled

sensor= ap\_ntc #设定此策略绑定的sensor为ap\_ntc(SOC附近NTC)

threshold= [43000, 46000, 49000, 53000] #设定板温(ap\_ntc)门限档位，与reduce-brightness中不同的背光档位分别对应

hysteresis= [2000, 2000, 2000, 2000] #设定各档位退出门限，2000代表2度，即ap\_ntc温度降低2度则退出该档背光控制

reduce-brightness= [10, 50, 80, 95] #限制背光亮度为Max亮度的百分比。95代表95%

[flashlight-cooler]

mode= enabled

sensor= ap\_ntc #设定此策略绑定的sensor为ap\_ntc(SOC附近NTC)

threshold= [43000, 46000, 49000, 53000] #设定板温(ap\_ntc)门限档位，与limit-state中不同的电流档位分别对应

hysteresis= [2000, 2000, 2000, 2000] #设定各档位退出门限，2000代表2度，即ap\_ntc温度降低2度则退出该档电流控制

limit-state= [1, 2, 3, 5] #限制电流档位

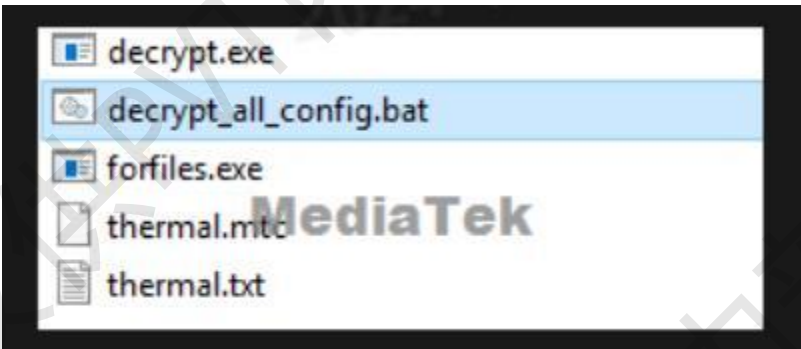
Limit-state	Flashlight limit current (mA)
0	No limit
1	200
2	150
3	100

4	50
5	25

2.配置文件加解密

解密：

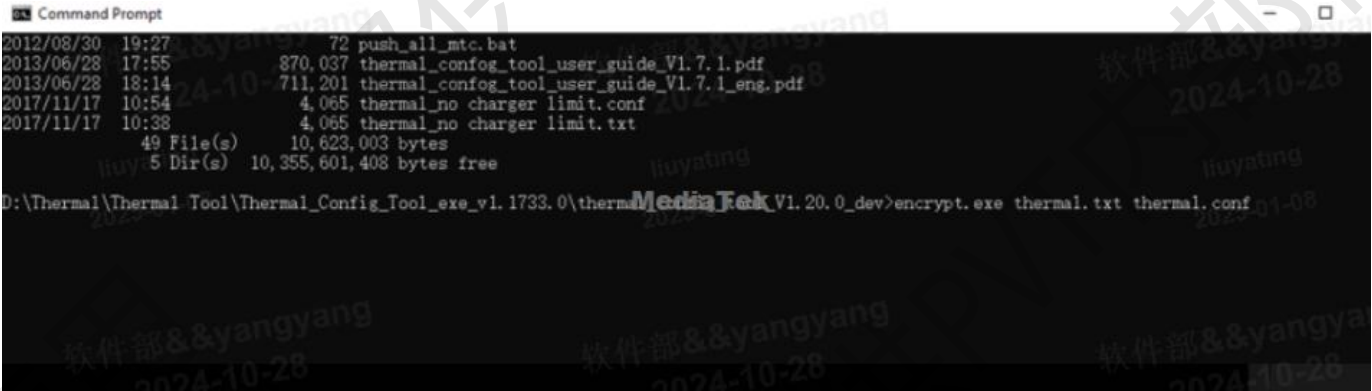
- 1) 解压Thermal config tool后，将要解密的thermal配置文件复制到如下目录。  
\\Thermal\_Config\_Tool\_exe\_v1.1942.1\\thermal\_config\_tool\_V1.20.0\_dev\\decrypt
- 2) 将thermal配置文件重命名为.mtc文件。
- 3) 运行decrypt\_all\_config.bat，可在该目录下生成.txt文件，打开可查看thermal策略并做修改。



修改完成后，可直接使用.txt文件让修改策略生效。

加密：

- 1) 文本编辑修改好.txt文件并将文件放在如下路径下：  
\\Thermal\_Config\_Tool\_exe\_v1.1942.1\\thermal\_config\_tool\_V1.20.0\_dev
- 2) PC CMD界面下运行“encrypt.exe thermal.txt thermal.conf”生成thermal而配置文件，如下图所示



3.thermal\_core

thermal\_core是一个本地Linux应用程序，解析上面conf文件热管理策略，CPU/GPU/APU会各自根据thermal\_core设置下来的目录温度和检测到的温度做热管理调节。



## 8.3.5 thermal debug

### 1. 配置文件修改生效

具备动态切换 thermal policy 的功能:

- Yocto:

```
adb shell "thermal-int apply [policy_name]"
```

比如:

```
adb shell "thermal-int apply disable_throttling.conf"
```

```
adb shell cat /data/thermal/.current_tp
```

可以查看当前使用的 policy (Yocto)。

### 2. 读温度的接口

yocto侧读各个sensor温度接口如下:

```
/mt8676_pvt_demo/yocto/src/kernel/linux/v6.1_mt8676/co_device_module/drivers/thermal/mediatek/
HAD vtskin_temp.c 41 if (IS_ERR_OR_NULL(tzd) || !tzd->ops->get_temp) { in vtskin_get_temp()
47 ret = tzd->ops->get_temp(tzd, &tz_temp); in vtskin_get_temp()
49 dev_err(skin_data->dev, "%s get_temp fail %d\n", sensor_name, ret); in vtskin_get_temp()
73 .get_temp = vtskin_get_temp,
HAD board_temp.c 238 .get_temp = board_ntc_get_temp,
HAD soc_temp.c 467 .get_temp = soc_thermal_read_temp,
HAD pmic_temp.c 128 .get_temp = pmic_get_temp,
HAD soc_temp_lvts.c 972 .get_temp = soc_temp_lvts_read_temp,
```

### 3. thermal log 的打开

thermal\_core log 目前默认打开, log level 为 0。

```
enum {
 LEVEL_DEBUG,
 LEVEL_INFO,
 LEVEL_ERROR,
};
```

可以通过以下 command 调整 log level 的等级:

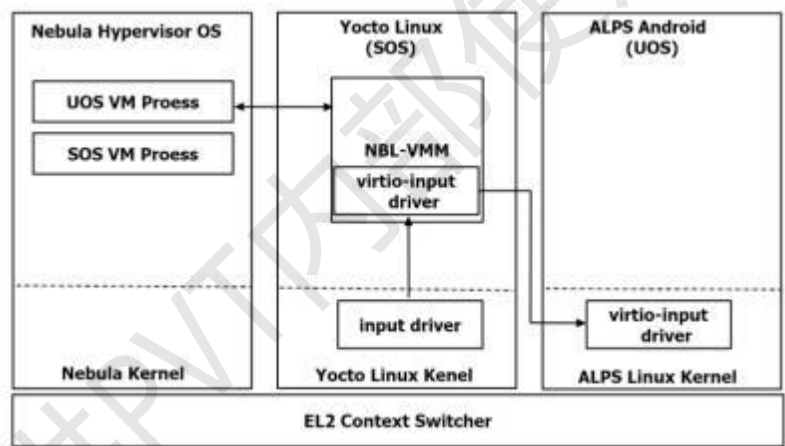
```
adb shell " thermal-int debug_log 1"
```

抓取 thermal core/daemon 的 log:

```
adb shell journalctl -fu thermal-daemon.service
```

## 8.4 virtio\_input

### 8.4.1 虚拟化框架介绍



1、MTK的TP驱动，为了兼容多个不同的TP驱动IC，它会去遍历每一个编译并添加进 `tpd_driver_list` 中的IC驱动，直到其中一个初始化成功。具体的TP驱动文件，一般由IC厂提供，然后移植到各平台的代码中，主要实现了对IC的上电和初始化，注册输入设备，获取并上报数据等。

2、`virtio_input`是input虚拟化设备的接口定义，定义实际设备和虚拟设备event上传协议。完成实际设备到虚拟设备事件上报，和虚拟设备到实际设备的回调。

3、整体流程是，yocto端virtio读取event设备，并将input事件通过virtio\_input发送到android端（写入虚拟event设备）。然后android通过虚拟化的

event设备上传到上层应用。整个开发流程yocto跟单系统开发TP是一样的，需要添加新的TP IC。而virtio\_input

一般是定义好，一般不需要更改。1、MTK的TP驱动，为了兼容多个不同的TP驱动IC，它会去遍历每一个编译并添加进 `tpd_driver_list` 中的IC驱动，直到其中一个初始化成功。具体的TP驱动文件，一般由IC厂提供，然后移植到各平台的代码中，主要实现了对IC的上电和初始化，注册输入设备，获取并上报数据等。

2、`virtio_input`是input虚拟化设备的接口定义，定义实际设备和虚拟设备event上传协议。完成实际设备到虚拟设备事件上报，和虚拟设备到实际设备的回调。

3、整体流程是，yocto端virtio读取event设备，并将input事件通过virtio\_input发送到android端（写入虚拟event设备）。然后android通过虚拟化的event设备上传到上层应用。整个开发流程yocto跟单系统开发TP是一样的，需要添加新的TP IC。而virtio\_input一般是定义好，一般不需要更改。



8.4.2 Input数据传输



### 8.4.3 input快速开发

#### 1、TP驱动的移植

(1)、首先，将触摸IC厂提供的驱动文件XXX放入路径：

yocto/src/kernel/linux/v6.1\_mt8676/co\_device\_module/drivers/input/touchscreen/gt9xx

(2)、将TP驱动加入编译规则，以 gt9xx为例

obj-\$(CONFIG\_TOUCHSCREEN\_GT9XX) += gt9xx/

上述添加方法使用宏控制的目的是为了更方便选择编译触摸IC的驱动，需在Kconfig和项目的xxxdeconfig文件加

入宏 CONFIG\_TOUCHSCREEN\_GT9XX的配置

(3)、DTS设备树配置

yocto/src/kernel/linux/v6.1\_mt8676/co\_device\_module/arch/arm64/boot/dts/mediatek/cust\_mt8676\_display\_interface.dtsi 配置TP设备信息

```
104 &i2c0 {
105
106 gt9xx@31{
107 compatible = "goodix,gt9xx";
108 reg = <0x31>;
109 status = "okay";
110 touch-id = <2>;
111 touch-identity = "gt9xx_dsi1_1";
112
113 interrupt-parent = <&pio>;
114 interrupts = <10 0x2800>;
115 irq-gpios = <&pio 10 0x2800>;
116 reset-gpios = <&pio 108 0x0>;
117 irq-flags = <2>;
118
```

(4)、DWS文件配置

修改DWS文件的目的是为了配置系统的中断号、设备树I2C地址和对应的名称等。使用DrvGen.exe工具,配置yocto/src/devtools/dct/dws/mt6897目录下的dws (如: auto8676p1\_64\_hyp.dws)文件。

(5)、将ko文件编译进系统

yocto/src/kernel/linux/v6.1\_mt8676/co\_device\_module/kernel/kleaf/mgk\_64\_k61.bzl

"drivers/input/touchscreen/gt9xx/gt9xx\_touch.ko",

yocto/meta/meta-mediatek-mt8676-hyp/recipes-

kernel/linux/ko\_order\_table/auto8676p1\_64\_hyp/ko\_order\_table.csv

gt9xx\_touch.ko,../kernel\_device\_modules-

6.1/drivers/input/touchscreen/gt9xx/gt9xx\_touch.ko,vendor,Y,N,user/userdebug/eng

## 2、Hypervisor 配置

启动nbl\_vmm时会执行vendor/etc/hyper\_android/query\_input\_event.sh脚本

会识别给定的input设备（DEVICE\_ARRAY=("himax-touchscreen" "mtk-tpd" "goodix\_ts" "goodix\_ts"））并将  
其写到vendor/etc/hyper\_android/uos\_alps\_pv8676.lua中input栏。

```
-- input
uos_config:setInput("/dev/input/event0")
uos_config:setInput("/dev/input/event3")
uos_config:setInput("/dev/input/event4")
uos_config:setInput("/dev/input/event4")
-- *****
```

上面设备分别对应给到android的

```
devName[0] = "mtk-tpd";
devName[1] = "mtk-tpd1";
devName[2] = "mtk-tpd2";
devName[3] = "mtk-tpd3";
```

Android shell下getevent -i可以查看input信息

```
add device 12: /dev/input/event3
bus: 0006
vendor 0000
product 0000
version 0000
name: "mtk-tpd3"
location: "virtio6/input0"
id: "serial-number"
version: 1.0.1
```

最后android端需要绑定display\_id和input设备，touch才能到对应屏幕上

mt8676/device/mediateksample/auto8676p1\_64\_bsp\_vm/input-port-associations.xml最后android端需要绑定  
display\_id到对应屏幕上 mt8676/device/mediateksample/auto8676p1\_64\_bsp\_vm/input-port-  
associations.xml

```
<ports>
<port display="0" input="virtio3/input0" />
<port display="1" input="virtio4/input0" />
<port display="2" input="virtio5/input0" />
<port display="3" input="virtio6/input0" />
```

## 8.4.4 input debug

检查IC引脚电压，此时是否有中断电平输出。查看串口log或使用adb命令：`getevent` 检查系统是否有接收到上报的数据。再则可以检测driver 是否初始化 成功 如i2c通路是否有问题。

在android端可以打开触摸显示点，直观地检测touch问题：

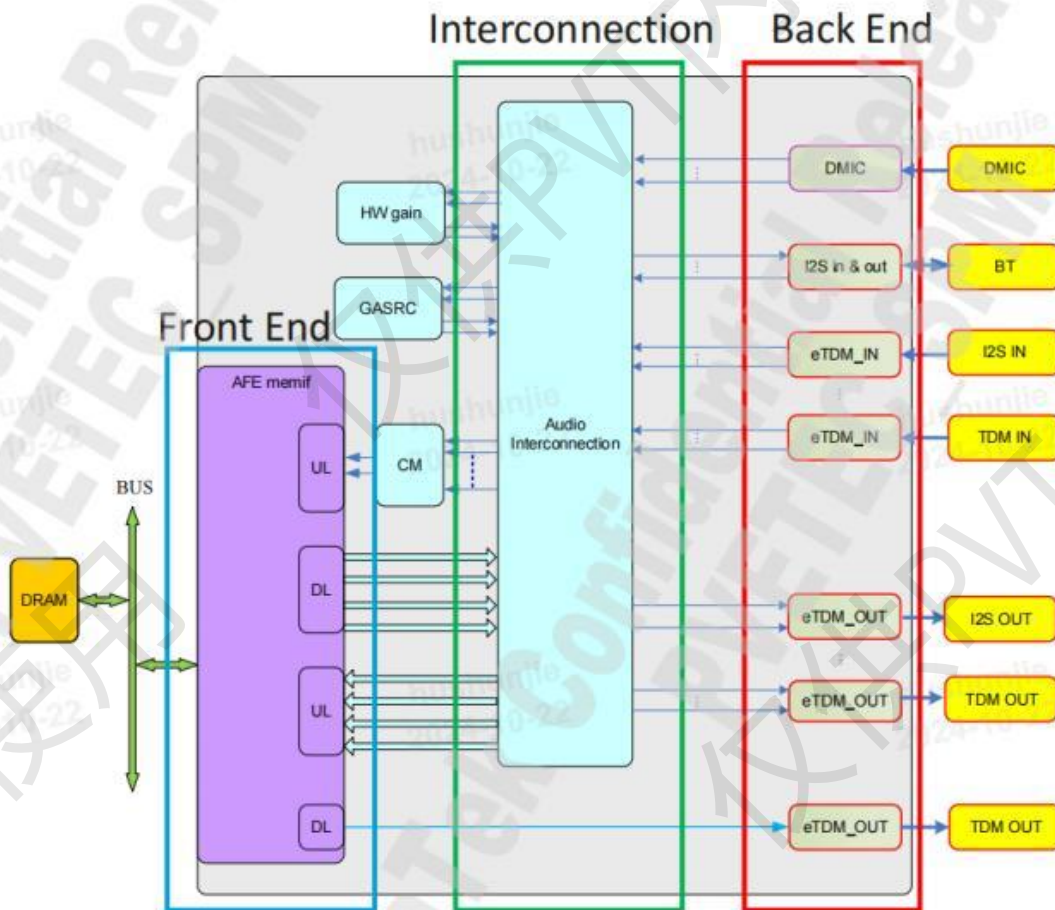
`settings put system pointer_location 1`

- 1、各模块虚拟化框架
- 2、各模块快速开发手册
- 3、各模块快速debug方法

## 8.5 Audio

### 8.5.1 Audio AFE HW

1.MTK AFE概述：



- AFE: Audio Front End HW 表示整个音频前端硬件



- AFE MEMIF(FE): PCM DMA,memory read/write 指Audio DMA数据传输通道，audio数据流通过MEM IF在内存和Audio IF之间传递数据

- Audio interconnection: connection fabric for audio sub module 代表MEM IF与Audio IF之间的连接矩阵

—— ex: DL MEMIF → Intercon → I2S Out

- DAI (BE): Digital Audio I/F, eTDM/I2S/DMIC，代表系统的数字音频接口，比如I2S，TDM

—— Enhanced TDM: can be configured to I2S or TDM

. MISC

—— CM: channel merge，主要用于合并回显数据和Mic输入

—— GASRC: General audio sampling rate convertor，用于采样率转换

—— HW gain: digital gain process，用于增益调节

—— Bus: Soc Top fabric bus，互联总线，音频数据的通路

## 2.MTK Audio AFE hw:

描述如下图：

1.主要包括24ch的下行，表示MTK Audio AFE的播放通路，其中包括4组2ch I2S out，1组8ch TDM out，1组8ch Internal DP-TX out和

2.上行通路18ch的上行，表示MTK Audio AFE的录音通道，其中包含5组2ch I2S out，1组8ch TDM in

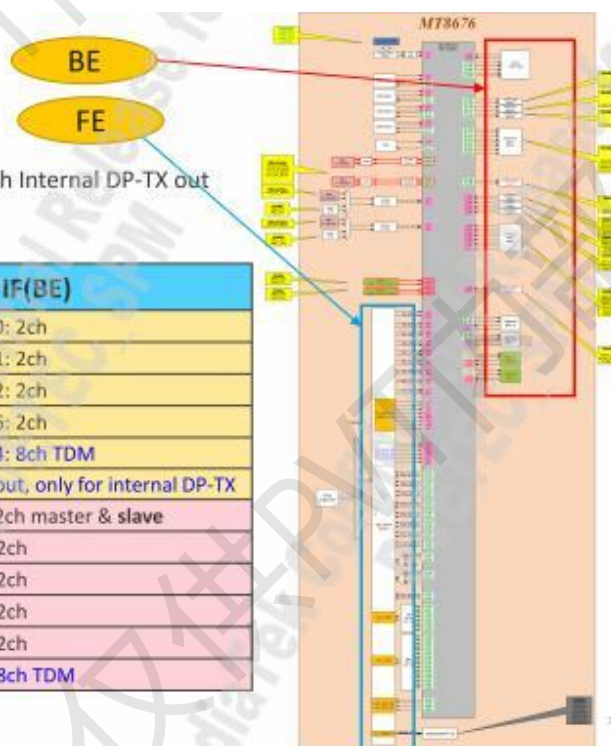
### MT8676 AFE HW capability

- 下行24ch**: 4组2ch I2S out，1组8ch TDM out，1组8ch Internal DP-TX out
- 上行18ch**: 5组2ch I2S in，1组8ch TDM in

In/Out	Max Valid Format	Mem IF(FE)	Audio IF(BE)
Max 24ch Output	2ch/192k/32b	DL_2ch_X	I2SOUT0: 2ch
	2ch/192k/32b	DL_2ch_X	I2SOUT1: 2ch
	2ch/192k/32b	DL_2ch_X	I2SOUT2: 2ch
	2ch/192k/32b	DL_2ch_X	I2SOUT6: 2ch
	8ch/48k/32b	DL_8CH_0 (8ch)	I2SOUT4: 8ch TDM
	8ch/96k/32b	DL_TDMOUT (8ch)	No PIN out, only for internal DP-TX
Max 18ch Input	2ch/192k/32b	VUL_2CH_X	FMI2S: 2ch master & slave
	2ch/192k/32b	VUL_2CH_X	I2SIN0: 2ch
	2ch/192k/32b	VUL_2CH_X	I2SIN1: 2ch
	2ch/192k/32b	VUL_2CH_X	I2SIN2: 2ch
	2ch/192k/32b	VUL_2CH_X	I2SIN6: 2ch
	8ch/48k/32b	VUL_CM0 (8ch)	I2SIN4: 8ch TDM

MEDIATEK

MediaTek Proprietary and Confidential. © 2024 MediaTek Inc. All rights reserved.



## 3.Audio AFE软件设计:

mt6897-afe-common.h中定义了整个audio子系统支持的audio FE(mem if)和BE(I2S、TDM)硬件ID

```
enum {
MT6897_MEMIF_DL0,
MT6897_MEMIF_DL1,
MT6897_MEMIF_NUM,
MT6897_DAI_ADDA = MT6897_MEMIF_NUM,
MT6897_DAI_ADDA_CH34,

- - -
MT6897_DAI_NUM,

- -
MT6897_GPIO_EXT_HP_AMP,

- - - -
};
```

mt6897-mt6368.c中mt6897\_mt6368\_dai\_links定义了audio\_afe中CPU端到Codec端的链接组件

```
#define SND_SOC_DAILINK_DEF(name, def...) \
 static struct snd_soc_dai_link_component name[] = { def }

#define SND_SOC_DAILINK_DEFS(name, cpu, codec, platform...) \
 SND_SOC_DAILINK_DEF(name##_cpus, cpu); \
 SND_SOC_DAILINK_DEF(name##_codecs, codec); \
 SND_SOC_DAILINK_DEF(name##_platforms, platform)

#define DAILINK_COMP_ARRAY(param...) param
#define COMP_EMPTY() {}
#define COMP_CPU(_dai) { .dai_name = _dai, }
#define COMP_CODEC(_name, _dai) { .name = _name, .dai_name = _dai, }
#define COMP_PLATFORM(_name) { .name = _name }
#define COMP_AUX(_name) { .name = _name }
#define COMP_CODEC_CONF(_name) { .name = _name }
#define COMP_DUMMY() { .name = "snd-soc-dummy", .dai_name = "snd-soc-dummy-dai", }
```

```
SND_SOC_DAILINK_DEFS(playback0,
DAILINK_COMP_ARRAY(COMP_CPU("DL0")),
DAILINK_COMP_ARRAY(COMP_DUMMY()),
DAILINK_COMP_ARRAY(COMP_EMPTY()));
```

```

#define SND_SOC_DAILINK_REG1(name) SND_SOC_DAILINK_REG3(name##_cpus,
name##_codecs, name##_platforms)
#define SND_SOC_DAILINK_REG2(cpu, codec) SND_SOC_DAILINK_REG3(cpu, codec,
null_dailink_component)
#define SND_SOC_DAILINK_REG3(cpu, codec, platform) \
.cpus = cpu, \
.num_cpus = ARRAY_SIZE(cpu), \
.codecs = codec, \
.num_codecs = ARRAY_SIZE(codec), \
.platforms = platform, \
.num_platforms = ARRAY_SIZE(platform)
#define SND_SOC_DAILINK_REGx(_1, _2, _3, func, ...) func
#define SND_SOC_DAILINK_REG(...) \
SND_SOC_DAILINK_REGx(__VA_ARGS__, \
SND_SOC_DAILINK_REG3, \
SND_SOC_DAILINK_REG2, \
SND_SOC_DAILINK_REG1)(__VA_ARGS__)

```

```

static struct snd_soc_dai_link mt6897_mt6368_dai_links[] = {
/* Front End DAI links */
{
.name = "Playback_1",
.stream_name = "Playback_1",
.trigger = {SND_SOC_DPCM_TRIGGER_PRE,
SND_SOC_DPCM_TRIGGER_PRE},
--
.dynamic = 1,
.dpcm_playback = 1,
SND_SOC_DAILINK_REG(playback0),
},

```



```
static struct snd_soc_dai_driver mt6897_memif_dai_driver[] = {
/* FE DAIs: memory interfaces to CPU */
{
.name = "DL0",
.id = MT6897_MEMIF_DL0,
.playback = {
.stream_name = "DL0",
.channels_min = 1,
.channels_max = 2,
.rates = MTK_PCM_RATES,
.formats = MTK_PCM_FORMATS,
},
.ops = &mt6897_memif_dai_ops,
},

```

mt6897-afe-pcm.c 为每个memif定义了专用的

```
static const struct mtk_base_memif_data memif_data[MT6897_MEMIF_NUM] = {
[MT6897_MEMIF_DL0] = {
.name = "DL0",
.id = MT6897_MEMIF_DL0,
.reg_ofs_base = AFE_DL0_BASE,
.reg_ofs_cur = AFE_DL0_CUR,
.reg_ofs_end = AFE_DL0_END,
.reg_ofs_base_msb = AFE_DL0_BASE_MSB,
.reg_ofs_cur_msb = AFE_DL0_CUR_MSB,
.reg_ofs_end_msb = AFE_DL0_END_MSB,
.fs_reg = AFE_DL0_CON0,
.fs_shift = DL0_SEL_FS_SFT,
.fs_maskbit = DL0_SEL_FS_MASK,
.mono_reg = AFE_DL0_CON0,
.mono_shift = DL0_MONO_SFT,
.enable_reg = AFE_DL0_CON0,
.enable_shift = DL0_ON_SFT,
.hd_reg = AFE_DL0_CON0,
.hd_mask = DL0_HD_MODE_MASK,
.hd_shift = DL0_HD_MODE_SFT,
.hd_align_reg = AFE_DL0_CON0,
.hd_align_mshift = DL0_HALIGN_SFT,
.agent_disable_reg = -1,

```

```

.agent_disable_shift = -1,
.msb_reg = -1,
.msb_shift = -1,
.pbuf_reg = AFE_DL0_CON0,
.pbuf_mask = DL0_PBUF_SIZE_MASK,
.pbuf_shift = DL0_PBUF_SIZE_SFT,
.minlen_reg = AFE_DL0_CON0,
.minlen_mask = DL0_MINLEN_MASK,
.minlen_shift = DL0_MINLEN_SFT,
},

```

```

static const int memif_irq_usage[MT6897_MEMIF_NUM] = {
/* TODO: verify each memif & irq */
[MT6897_MEMIF_DL0] = MT6897_IRQ_0,

```

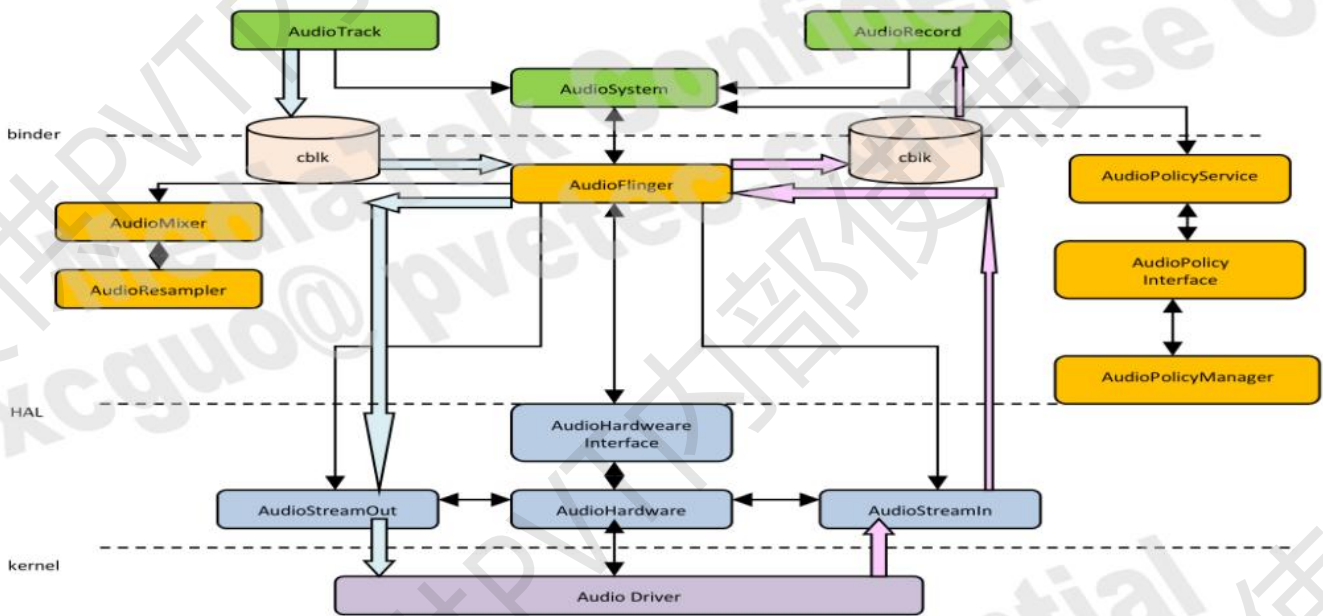
```

unsigned int virtio_id_memif_index_mapping[MT6897_MEMIF_NUM] = {
MT6897 MEMIF DL0,
_ _
MT6897 MEMIF DL1,
_ _
MT6897 MEMIF DL2,
_ _
MT6897 MEMIF DL3,
_ _
MT6897 MEMIF DL4,
_ _
MT6897 MEMIF DL5,
_ _
MT6897 MEMIF DL6,

```

## 8.5.2 Audio HAL

Audio HAL软件架构图



**AudioALSASStreamManager**: 管理下面的AudioALSASStreamIn和AudioALSASStreamOut;

**AudioALSASStreamOut**: 管理着AudioALSAPlaybackXXXX;

**AudioALSASStreamIn**: 管理着AudioALSACaptureXXXX;

**AudioALSAPlaybackXXXX**与**AudioALSACaptureXXXX**: 这两个类里面的主要函数是open(), read()和write(), 主要是负责对PCM buf的读写到Linux 的ALSA里面;

**AudioALSASpeechXXXX**: 是Aduio的一个算法处理;

**AudioALSAHardwareResourceManager**: 这个类主要用于打开和关闭硬件设备, 如MIC, 喇叭等;

**AudioALSAVolumeController**: 主要用于Audio系统的音量控制, 音量补偿, 音频参数也在此得到应用, (手机平台)

HAL 的顶层控制接口是 **AudioALSAHardware**、**AudioALSASStreamOut** 和 **AudioALSASStreamIn**。这些类都使用**AudioALSASStreamManager** 来控制音频模式、打开/关闭, 输入/输出流。因此

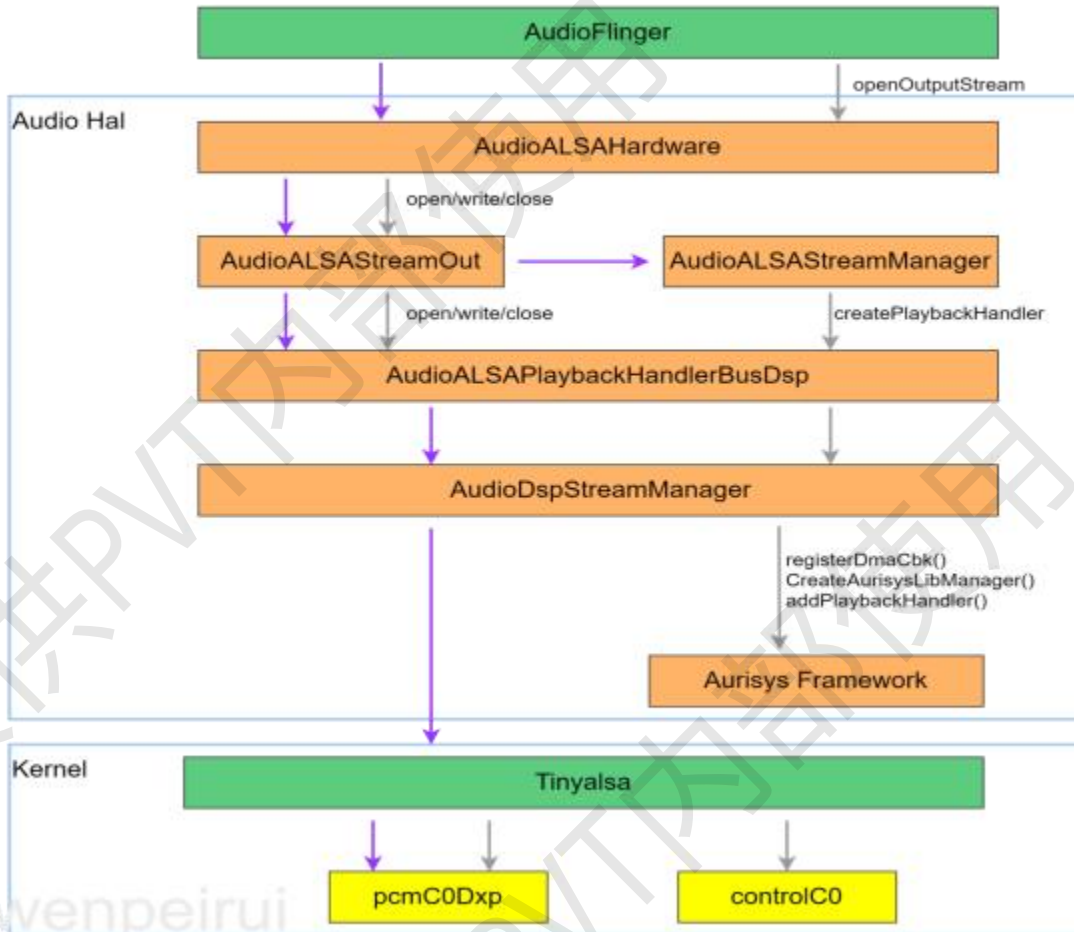
**AudioALSASStreamManager** 将拥有所有音频环境信息, 如模式 路由 音量等信息

Audio HAL数据流图









framework调用到了 AudioALSASStreamOut->open()函数后，由于我们定义了 MTK\_AUDIO\_AUTO\_SUPPORT宏，所以new了AudioALSAPlaybackHandlerBusDsp类，传入了 stream\_attribute\_source参数，new成功后调用AudioALSAPlaybackHandlerBusDsp->open()函数，对音频流进行处理，并开始与ADSP通

播放结束后，framework会调用 AudioALSASStreamOut->close()函数，释放 AudioALSAPlaybackHandlerBusDsp类，以上是adsp handler的生命周期。



```
AudioALSASStreamOut.cpp
mPlaybackHandler = mStreamManager-
>createPlaybackHandler(&mStreamAttributeSource);
```

与policy的配置xml对应

```
AudioALSASStreamManager.cpp
if (AudioDspConfigManager::getInstance()-
>isBusDspStream(stream_attribute_source->mBusNum)) {
 if (stream_attribute_source->mAudioOutputFlags ==
AUDIO_OUTPUT_FLAG_DIRECT)
 pPlaybackHandler = new
AudioALSAPlaybackHandlerBusDirectDsp(stream_attribute_source);
 else
 pPlaybackHandler = new
AudioALSAPlaybackHandlerBusDsp(stream_attribute_source);
 break;
}
```

根据flag区分多声道和非多声道使用的handler

```
AudioALSAPlaybackHandlerBusDsp.cpp
dspPcmIndex = AudioALSADeviceParser::getInstance()-
>GetPcmIndexByString(mBusStreamConfig->mPcmStreamName);
dspCardIndex = AudioALSADeviceParser::getInstance()-
>GetCardIndexByString(mBusStreamConfig->mPcmStreamName);
pcmC0D116p 设备节点, 来自配置文件auto_audio_config.xml,
DSP_Playback_Playback0
```

```
mAudioDspConfigManager->setDevicePathOnOff(mStreamAttributeSource->mBusNum,
true);
```

```
AudioDspConfigManager.cpp
AudioALSAHardwareResourceManager::getInstance()-
>enableTurnOnSequence(busStreamConfig->mDevicePaths[i]);
```

来自配置文件auto\_audio\_config.xml, 有的通路配置了device\_path, 再到audio\_device.xml查找

```
mDspStreamManager->addPlaybackHandler(this);
AudioDspStreamManager.cpp
openPlaybackTask(playbackHandler);
mAudioDspConfigManager-
>setDevicePathOnOff(BUS_ID_DSP_TASK(TASK_SCENE_AUDPLAYBACK), true);
```

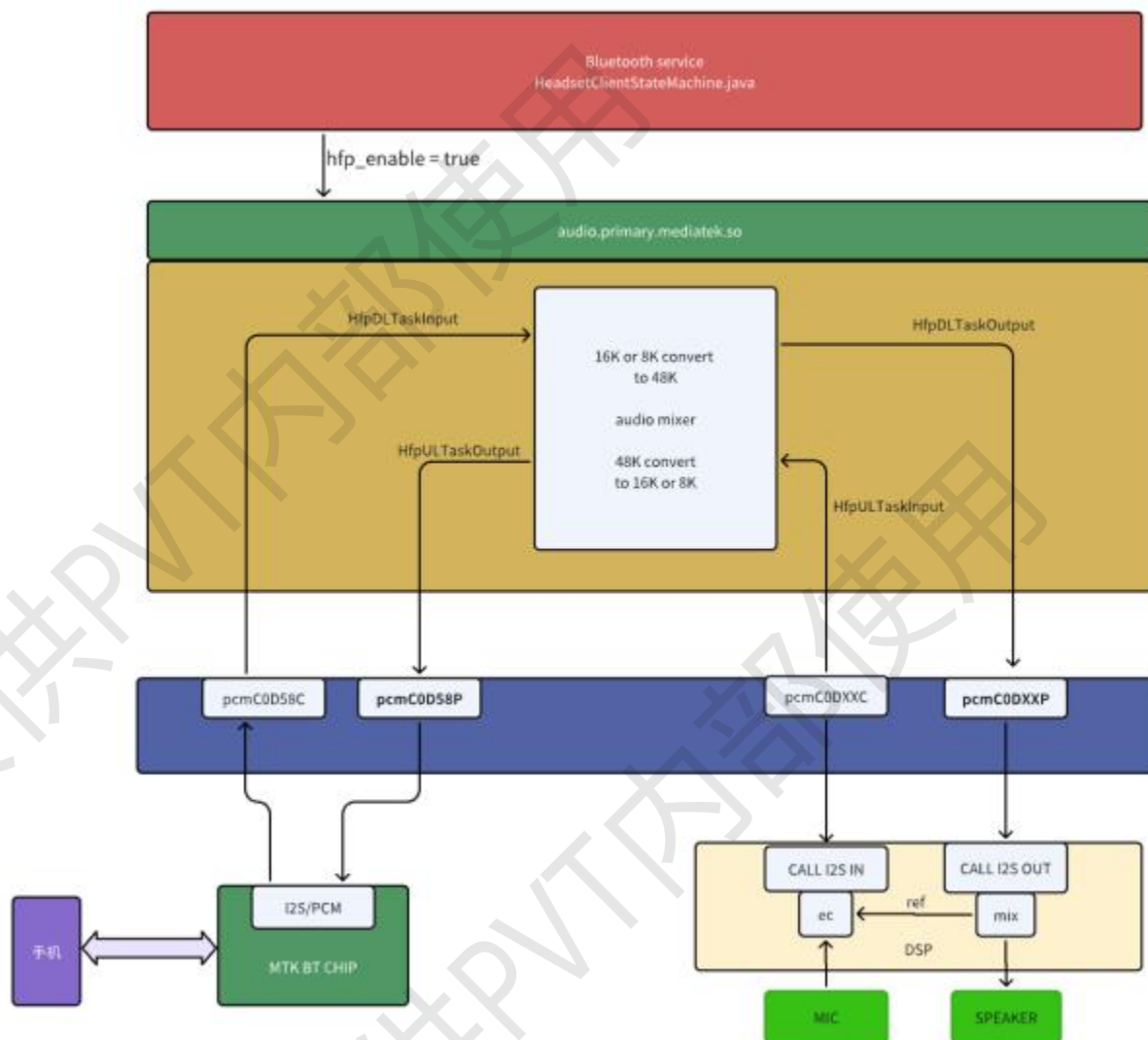
转换来自AudioDspConfigManager.cpp

```
const static std::map<std::string, uint8_t> pcmStreamToSceneMap = {

 {"DSP_Playback_Playback", TASK_SCENE_AUDPLAYBACK},

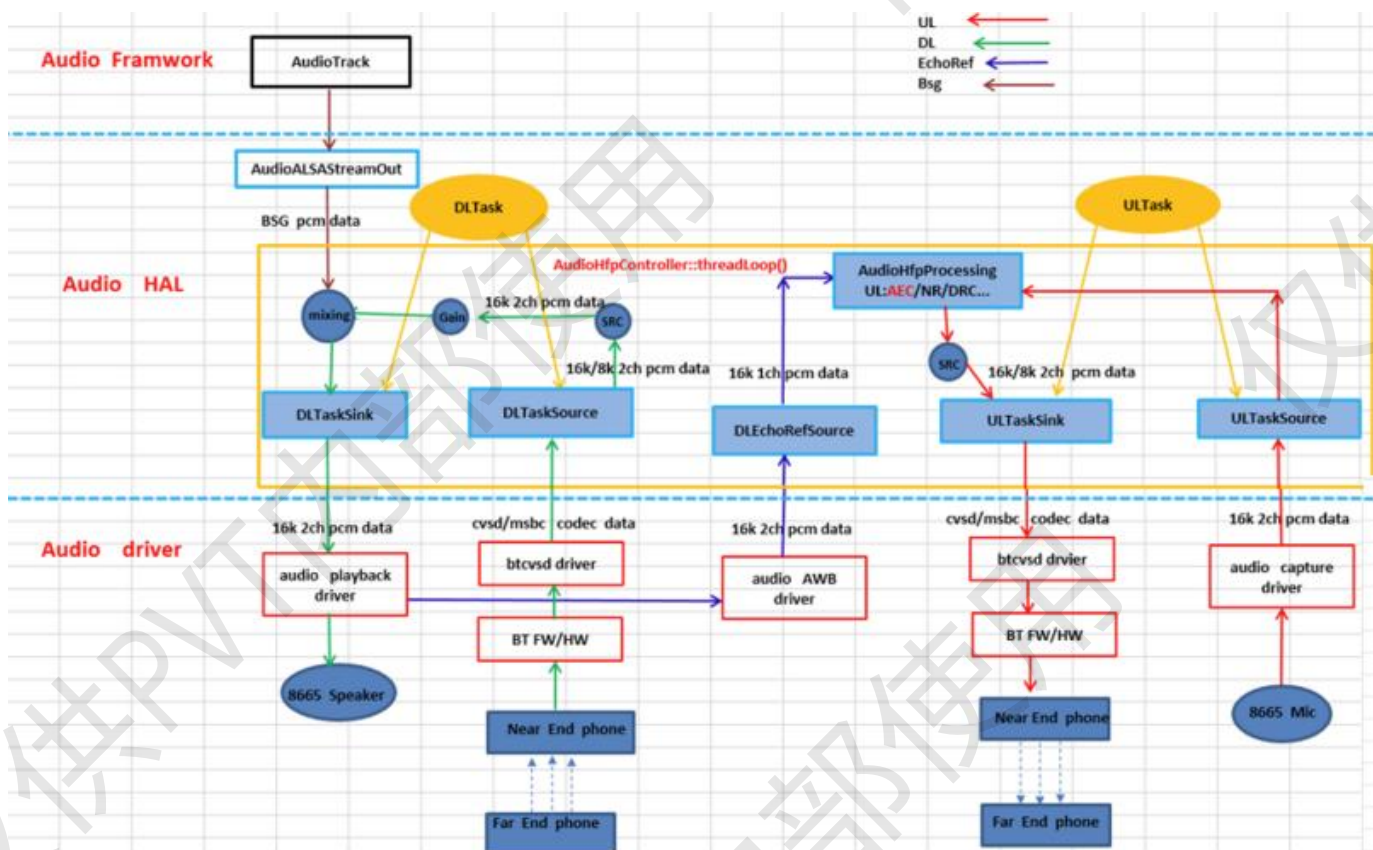
};
```

蓝牙hfp功能方案设计框图



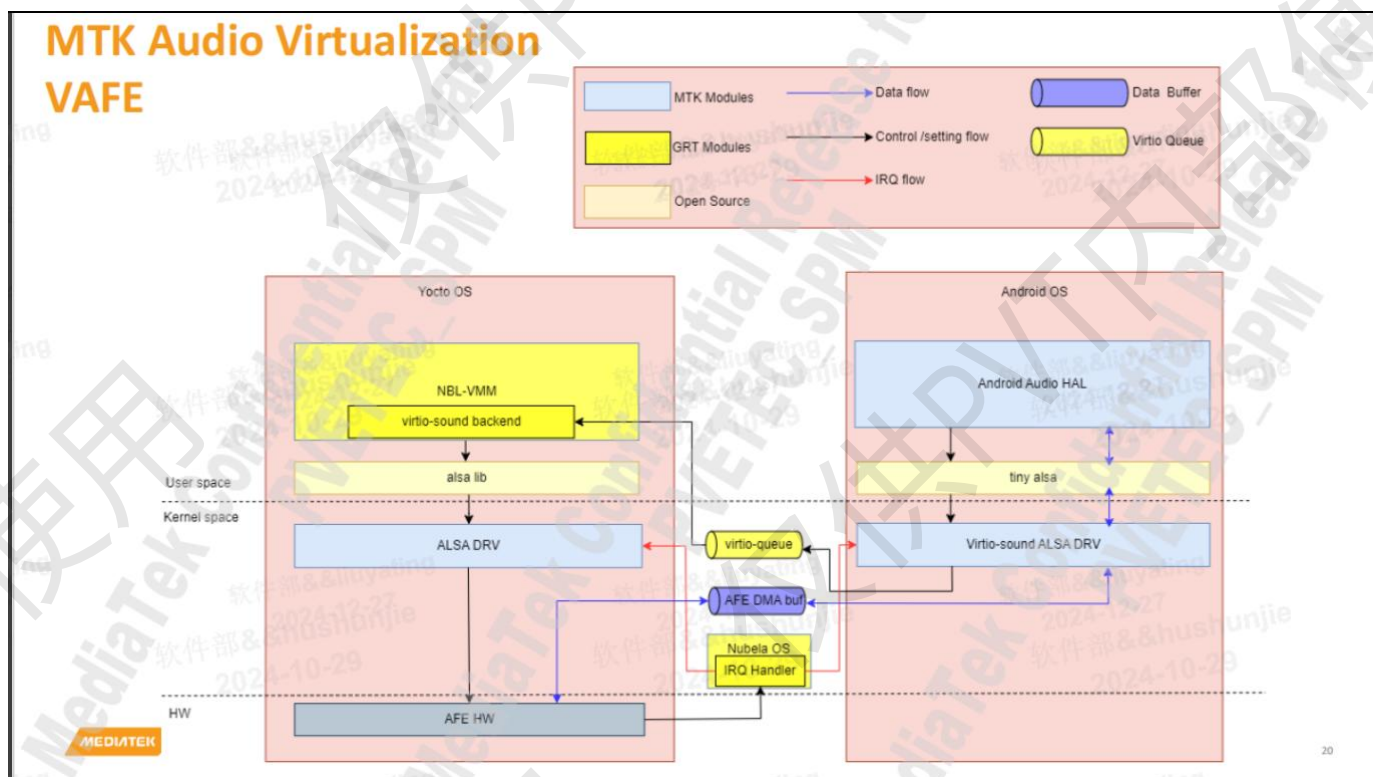
公版的hfp功能实现，以hfp\_enable为控制接口，该接口通过setparameter设置到audiohal层，而后通过audiohfpcontro接口控制hfp通道的打开关闭等操作，并控制数据流操作，该操作中包含主要上行下行数据流读取和写入线程，实现通话功能

蓝牙hfp数据流各节点的数据流



### 8.5.3 Audio Virtualization

Audio VAFE虚拟化框架图:





Audio VAFE BE: nbl vmm中会创建VirtioSound作为AFE的虚拟化后端, 在nbl\_vm的main.c中有:

```
:
: #ifdef WITH_VIRTIO_SOUND
: if (config.is vsound enabled()) {
: status = machina::attach_sound_device(instance.get(), config);
: CHECK(status == ZX_OK);
: LOG(ERROR) << "sound status " << status;
: }
: #endif
```

主要操作如下

1. 解析vm\_audio\_cfg.pb.txt

ParseAudioCfg(absl::GetFlag(FLAGS\_audio\_cfg), vmid);

2. 解析vm\_audio\_shared\_irq.pb.txt

ParseAudioSharedIrqCfg(absl::GetFlag(FLAGS\_audio\_shared\_irq\_cfg));

3. 创建虚拟化队列, 打开声卡设备

```
1:
2: zx_status_t VirtioSound::Init(async_t* async, uint32_t vmid) {
3: VirtioQueue* control = reinterpret_cast<VirtioQueue*>(control_queue());
4: VirtioQueue* rx = reinterpret_cast<VirtioQueue*>(rx_queue());
5: VirtioQueue* tx = reinterpret_cast<VirtioQueue*>(tx_queue());
6: vmid = vmid;
7: if (async != nullptr) {
8: async_ = async;
9: } else {
10: sound_loop_ = std::make_unique<async::Loop>();
11: sound_loop_>StartThread("virtio-sound");
12: async_ = sound_loop_>async();
13: }
14: zx_status_t status = control->PollAsync(async_, &control_queue_wait_, &VirtioSound::CtrlQueueHandler, this);
15: if (status != ZX_OK) {
16: return status;
17: }
18: status = rx->PollAsync(async_, &rx_queue_wait_, &VirtioSound::PcmQueueHandler, this);
19: if (status != ZX_OK) {
20: return status;
21: }
22: status = tx->PollAsync(async_, &tx_queue_wait_, &VirtioSound::PcmQueueHandler, this);
23: if (status != ZX_OK) {
24: return status;
25: }
26: alsamixer_.Open(0);
27: VirtioSoundBase::Init(vmid);
28: PrepareSndConfig();
29: auto pci = reinterpret_cast<VirtioPci*>(pci_device());
30: //
31: pci->RegisterDriverReadyHandler([this] {
32: PrepareSharedIrq(VirtioSoundBase::GetSharedIrqRecord(), VirtioSoundBase::GetCustomVirtSndPcmRecord());
33: });
```

创建虚拟化队列与FE通信

打开驱动声卡设备

虚拟化队列中主要交互的是pcm的基本操作, 还有dapm控件的操作:

```

enum {
/* jack control request types */
VIRTIO_SND_R JACK_INFO = 1,
VIRTIO_SND_R JACK_REMAP,
/* PCM control request types */
VIRTIO_SND_R PCM_INFO = 0x0100,
VIRTIO_SND_R PCM SET PARAMS,
- - - - -
VIRTIO_SND_R PCM PREPARE,
- - - - -
VIRTIO_SND_R PCM RELEASE,
- - - - -
VIRTIO_SND_R PCM START,
- - - - -
VIRTIO_SND_R_PCM_STOP,
VIRTIO_SND_R_PCM_FREE,
/* channel map control request types */
VIRTIO_SND_R CHMAP_INFO = 0x0200,
/* control element request types */
VIRTIO_SND_R CTL_INFO = 0x0300,
VIRTIO_SND_R CTL ENUM ITEMS,
- - - - -
VIRTIO_SND_R CTL READ,
- - - - -
VIRTIO_SND_R CTL WRITE,
- - - - -
VIRTIO_SND_R CTL TLV READ,
- - - - -
VIRTIO_SND_R CTL TLV WRITE,
- - - - -
VIRTIO_SND_R CTL TLV COMMAND,
- - - - -

/* jack event types */
VIRTIO_SND_EVT_JACK_CONNECTED = 0x1000,
VIRTIO_SND_EVT_JACK_DISCONNECTED,
/* PCM event types */
VIRTIO_SND_EVT_PCM_PERIOD_ELAPSED = 0x1100,
VIRTIO_SND_EVT_PCM_XRUN,

```

```

/* control element event types */
VIRTIO_SND_EVT_CTL_NOTIFY = 0x1200,
/* common status codes */
VIRTIO_SND_S_OK = 0x8000,
VIRTIO_SND_S_BAD_MSG,
VIRTIO_SND_S_NOT_SUPP,

- - - -
VIRTIO_SND_S_IO_ERR

- - - -
};

```

amixer用于打开/dev/snd的声卡设备，操作sound驱动中所有的控件

4. 创建虚拟audio\_streams\_， std::vector<std::shared\_ptr<VirtioAudioStream>> audio\_streams\_， 用于接收fe端对pcm的所有操作并打开真实的pcm设备

备

```

void VirtioSoundBase::PrepareAudioStreams() {
 uint8_t direction;
 std::lock_guard<std::mutex> lock(stream_mutex);

 #if 0
 for (int i = 0; i < arraysize(virtio_pcm_info_init); i++) {
 direction = s_snd_data.pcm_info[i].direction;
 std::shared_ptr<VirtioAudioStream> stream = std::make_shared<VirtioAudioStream>(
 this, direction, direction == VIRTIO_SND_D_OUTPUT ? tx_queue() : rx_queue());
 stream->set_stream_id(i);
 audio_streams_.push_back(stream);
 }
 #endif
 for (int i = 0; i < pcmdev_valid_cnt; i++) {
 direction = s_snd_data.pcm_info[i].direction;
 std::shared_ptr<VirtioAudioStream> stream = std::make_shared<VirtioAudioStream>(
 this, direction, direction == VIRTIO_SND_D_OUTPUT ? tx_queue() : rx_queue());
 stream->set_stream_id(i);
 audio_streams_.push_back(stream);
 }
}

```

5. 查询alsa控件数量，创建控件

```

:
:
: for (i = 0; i < alsamixer_.CtrlsCount(); i++) {
: virtio_snd_ctl_info_t ctlInfo;
: // struct mixer_ctl* ctl = alsamixer_.GetCtl(i);
: MixerCtl* ctl = alsamixer_.GetCtl(i);
: if (ctl) {
: ctlInfo.index = i;
:
: // CtlUtils::convert_virtio_snd_ctl_info(ctlInfo, *(ctl->info));
: CtlUtils::convert_virtio_snd_ctl_info(ctlInfo, *(ctl->info));
: s_snd_data.ctl_info.push_back(ctlInfo);
: }
: }
:
:

```



Audio VAFE BE: android的android\kernel\kernel\_device\_modules-6.1\sound\virtio中实现了audio的AFE作为虚拟化的前端，主要是虚拟了出audio设备，用于接收hal层audio相关的操作，主要围绕创建virtio\_snd来实现，大致步骤如下：

### 1.创建虚拟化队列virtqueue，队列与nbl\_vmm后端对应

```
{
 struct virtio_snd *snd;
 struct device *dev = &vdev->dev;
 unsigned int i;
 int rc;
 int ret;

 snd = devm_kzalloc(&vdev->dev, sizeof(*snd), GFP_KERNEL);
 if (!snd)
 return -ENOMEM;

 snd->vdev = vdev;
 INIT_LIST_HEAD(&snd->ctl_msgs);
 INIT_LIST_HEAD(&snd->pcm_list);

 vdev->priv = snd;

 for (i = 0; i < VIRTIO_SND_VQ_MAX; ++i)
 spin_lock_init(&snd->queues[i].lock);

 rc = virtsnd_find_vqs(snd);
 if (rc)
 goto on_exit;

 virtio_device_ready(vdev);

#ifdef CONFIG_SND_VIRTIO_MTK_PASSTHROUGH
 /* This func was used by original virtio_snd, but cannot be used in kernel-6.1 */
 arch_setup_dma_ops(dev, -1, -1, NULL, 0);
#else
 dev->coherent_dma_mask = DMA_BIT_MASK(64);
 dev->dma_mask = &dev->coherent_dma_mask;
 ret = dma_set_mask_and_coherent(dev, DMA_BIT_MASK(64));
 if (ret) {
 pr_info("%s() call dma_set_mask_and_coherent return ret = %d\n", __func__, ret);
 return ret;
 }
#endif

 rc = virtsnd_build_devs(snd);
 if (rc)
 goto on_exit;
}
```

创建虚拟化队列，用于与后端通信

创建虚拟化设备

### 2.创建虚拟化声卡和虚拟化audio设备

```

00210: static int virtsnd_build_devs(struct virtio_snd *snd)
00211: {
00212: struct virtio_device *vdev = snd->vdev;
00213: struct device *dev = &vdev->dev;
00214: int rc;
00215:
00216: rc = snd_card_new(dev, SNDRV_DEFAULT_IDX1, SNDRV_DEFAULT_STR1, ← 创建虚拟声卡
00217: THIS_MODULE, 0, &snd->card);
00218: if (rc < 0)
00219: return rc;
00220:
00221: snd->card->private_data = snd;
00222:
00223: strncpy(snd->card->driver, VIRTIO_SND_CARD_DRIVER,
00224: sizeof(snd->card->driver));
00225: strncpy(snd->card->shortname, VIRTIO_SND_CARD_NAME,
00226: sizeof(snd->card->shortname));
00227: if (dev->parent->bus)
00228: snprintf(snd->card->longname, sizeof(snd->card->longname), ← 设置声卡名称
00229: VIRTIO_SND_CARD_NAME " at %s/%s/%s",
00230: dev->parent->bus->name, dev_name(dev->parent),
00231: dev_name(dev));
00232: else
00233: snprintf(snd->card->longname, sizeof(snd->card->longname),
00234: VIRTIO_SND_CARD_NAME " at %s/%s",
00235: dev_name(dev->parent), dev_name(dev));
00236:
00237: rc = virtsnd_jack_parse_cfg(snd); ← 查询jack设备
00238: if (rc)
00239: return rc;
00240:
00241: rc = virtsnd_pcm_parse_cfg(snd); ← 查询声卡设备
00242: if (rc)
00243: return rc;
00244:
00245: rc = virtsnd_chmap_parse_cfg(snd);
00246: if (rc)
00247: return rc;
00248:
00249: if (virtio_has_feature(vdev, VIRTIO_SND_F_CTL5)) { ← 查询kctl控件
00250: rc = virtsnd_kctl_parse_cfg(snd);
00251: if (rc)
00252: return rc;
00253: }

```

查询操作主要是调用virtsnd\_ctl\_query\_info把构建virtio\_snd\_msg结构体发到后端，根据查询到的信息创建对应的设备：

```

00248:
00249:
00250: if (virtio_has_feature(vdev, VIRTIO_SND_F_CTL5)) {
00251: rc = virtsnd_kctl_parse_cfg(snd);
00252: if (rc)
00253: return rc;
00254: }
00255:
00256: if (snd->njacks) { ← 创建jack设备
00257: rc = virtsnd_jack_build_devs(snd);
00258: if (rc)
00259: return rc;
00260: }
00261:
00262: if (snd->nsubstreams) { ← 创建pcm设备
00263: rc = virtsnd_pcm_build_devs(snd);
00264: if (rc)
00265: return rc;
00266: }
00267:
00268: if (snd->nchmaps) {
00269: rc = virtsnd_chmap_build_devs(snd);
00270: if (rc)
00271: return rc;
00272: }
00273:
00274: if (snd->nkctls) { ← 创建控件设备
00275: rc = virtsnd_kctl_build_devs(snd);
00276: if (rc)
00277: return rc;
00278: }
00279:
00280: #if IS_ENABLED(CONFIG_MTK_VIRT_ADSP)
00281: rc = vadsp_build_devs(snd->card);
00282: if (rc)
00283: return rc;
00284: #endif
00285:
00286: rc = virtsnd_btcvsd_build_devs(snd);
00287: if (rc)
00288: dev_info(dev, "build BTCVSD device error: %d\n", rc);

```

VADSP虚拟化：

Audio VADSP框架图：

MTK的ADSP是基于两个Xtensa DSP来实现，用于对音频信号进行处理，实现常见的3A算法（AGC、ANC、AEC);包括常见的音效处理、消回声，降噪等,8676有两个800Mhz的HIFI3核心，8678有两个800Mhz的HIFI5核心

- Cadence HiFi：Cadence Tensilica HiFi DSP 架构，用于音频解码
- Tensilica：嵌入式处理器公司，已被芯片 EDA 巨头 Cadence 收购
- Xtensa：DSP 处理器核心，也是该系列的型号名称，也是该处理器中指令集架构的名称
- HiFi：特指 32 位音频/语音处理数字信号处理 (DSP) 核，基于 Xtensa® 架构

MT8676 ADSP vs. MT8678 ADSP		
Check Items	MT8676 ADSP	MT8678 ADSP
DSP Core	HiFi3 x 2	HiFi5 x 2
Clock Rate	800MHz	800MHz
SRAM Size	I-Cache 64KB *2 D-Cache 128KB *2	I-Cache 64KB *2 D-Cache 128KB *2 768KB L2TCM
GMACs	32x32: 2 16x16: 4	32x32: 8 16x16: 16
Coremark	3.87	5.57
Instruction parallelism	3 (VLIW)	5 (VLIW)
Vector operations(SIMD)	64-bit	128-bit

在开机后，lk中会为ADSP保留一个专门的地址空间，用来存放一个基于FreeRTOS的镜像，也就是adsp.img，adsp镜像主要就是用来负责使用HIFI Core来对 audio数据实现3A算法处理。

audio adsp代码位置：yocto\src\kernel\linux\v6.1\_mt8676\co\_device\_module\drivers\misc\mediatek\adsp

audio ipi代码位置：yocto\src\kernel\linux\v6.1\_mt8676\co\_device\_module\drivers\misc\mediatek\audio\_ipi

ipi实现：



```

00636:
00637: static int __init audio_ipi_init(void)
00638: {
00639: uint32_t dsp_id = 0;
00640:
00641: struct audio_task_info_t *task_info = NULL;
00642: uint8_t task_id = 0;
00643:
00644: int ret = 0;
00645:
00646: ipi_queue_init();
00647:
00648: audio_task_manager_init();
00649: audio_messenger_ipi_init();
00650:
00651: init_audio_ipi_dma();
00652:
00653: #if IS_ENABLED(CONFIG_MTK_AUDIODSP_SUPPORT)
00654: adsp_register_notify(&audio_ctrl_notifier);
00655: #endif
00656:
00657: #if IS_ENABLED(CONFIG_MTK_TINYSYS_SCP_SUPPORT)
00658: if (is_audio_scp_support())
00659: scp_A_register_notify(&audio_ctrl_notifier_scp);
00660: #endif
00661:
00662: for (task_id = 0; task_id < TASK_SCENE_SIZE; task_id++) {
00663: task_info = &g_audio_task_info[task_id];
00664:
00665: dsp_id = audio_get_dsp_id(task_id);
00666:
00667: task_info->dsp_id = dsp_id;
00668: task_info->is_dsp_support = is_audio_dsp_support(dsp_id);
00669: task_info->is_adsp = is_audio_use_adsp(dsp_id);
00670: task_info->is_scp = is_audio_use_scp(dsp_id);
00671: task_info->task_ctrl = get_audio_controller_task(dsp_id);
00672: }
00673:
00674: ret = misc_register(&audio_ipi_device);
00675: if (unlikely(ret != 0)) {
00676: pr_notice("misc register failed");
00677: return ret;
00678: }
00679:
00680: return ret;
00681: }

```

vhost-adsp实现:

vhost-adsp.c中有实现vhost-adsp驱动程序:

```

01742:
01743: static const struct file_operations vhost_adsp_fops = {
01744: .owner = THIS_MODULE,
01745: .open = vhost_adsp_open,
01746: .release = vhost_adsp_release,
01747: .llseek = noop_llseek,
01748: .unlocked_ioctl = vhost_adsp_ioctl,
01749: };
01750:
01751: static struct miscdevice vhost_adsp_misc = {
01752: MISC_DYNAMIC_MINOR,
01753: "vhost-adsp",
01754: &vhost_adsp_fops,
01755: };
01756:
01757: static int vhost_adsp_init(void)
01758: {
01759: pr_info("[VADSP] %s() L%d.", __func__, __LINE__);
01760: init_debugfs();
01761: register_virt_ipi_cb(vadsp_send_ipi);
01762: register_virt_adsp_reg_feature_cb(vadsp_reg_feature);
01763: register_virt_adsp_query_status_cb(vadsp_query_status);
01764: return misc_register(&vhost_adsp_misc);
01765: }

```

vhost-adsp文件操作接口:

```

142:
143: static const struct file_operations vhost_adsp_fops = {
144: .owner = THIS_MODULE,
145: .open = vhost_adsp_open,
146: .release = vhost_adsp_release,
147: .llseek = noop_llseek,
148: .unlocked_ioctl = vhost_adsp_ioctl,
149: };
150:

```

vhost\_adsp\_open接口:

```

1166:
1167: static int vhost_adsp_open(struct inode *inode, struct file *file)
1168: {
1169: struct vhost_adsp *vadsp;
1170: struct vhost_virtqueue **vqs;
1171: int ret;
1172:
1173: pr_info("[VADSP]s() L%d begin!!", __func__, __LINE__);
1174:
1175: vadsp = kzalloc(sizeof(*vadsp), GFP_KERNEL);
1176: if (!vadsp)
1177: {
1178: pr_err("[VADSP]s() L%d NOMEM!!", __func__, __LINE__);
1179: ret = -ENOMEM;
1180: goto out;
1181: }
1182:
1183: vqs = kmalloc(VHOST_ADSP_VQ_MAX * sizeof(*vqs), GFP_KERNEL);
1184: if (!vqs)
1185: {
1186: kvfree(vadsp);
1187: pr_err("[VADSP]s() L%d NOMEM!!", __func__, __LINE__);
1188: return -ENOMEM;
1189: }
1190:
1191: ret = ida_simple_get(&vhost_adsp_index_ida, 0, 0, GFP_KERNEL);
1192: if (ret < 0) {
1193: pr_err("[VADSP]s() L%d ida get fail ret:%d!!", __func__, __LINE__, ret);
1194: goto out_dev;
1195: }
1196: vadsp->index = ret;
1197:
1198: vadsp->evt_vq.handle_kick = vhost_adsp_evt_handle_guest_kick;
1199: vadsp->req_vq.handle_kick = vhost_adsp_handle_guest_kick;
1200: vqs[VHOST_ADSP_VQ_EVT] = &vadsp->evt_vq;
1201: vqs[VHOST_ADSP_VQ_REQ] = &vadsp->req_vq;
1202: vhost_work_init(&vadsp->work, vhost_adsp_handle_host_kick);
1203: vhost_dev_init(&vadsp->dev, vqs, VHOST_ADSP_VQ_MAX,
1204: UIO_MAXIOV, VHOST_ADSP_PKT_WEIGHT,
1205: VHOST_ADSP_WEIGHT, true, NULL);
1206:

```



**Adsp.cc**

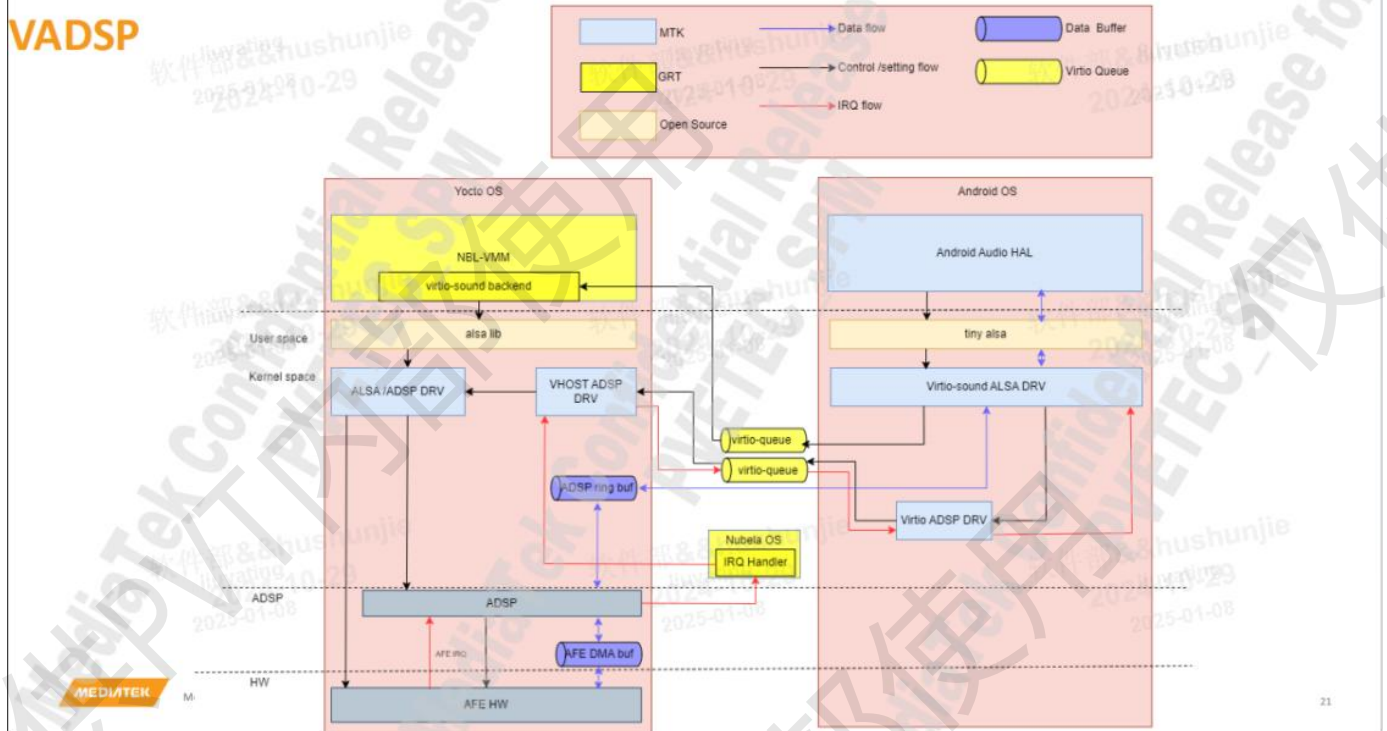
```

01303: static long vhost_adsp_ioctl(struct file *f, unsigned int ioctl,
01304: unsigned long arg)
01305: {
01306: struct vhost_adsp *vadsp = f->private_data;
01307: void __user *argp = (void __user *)arg;
01308: u64 __user *featurep = argp;
01309: u64 features;
01310: int ret;
01311: struct adsp_audio_reserve_mem rsv_mem;
01312: struct adsp_audio_reserve_mem *memp;
01313:
01314: pr_info("[VADSP]%() L%d ioctl:%d\n", __func__, __LINE__, ioctl);
01315: switch (ioctl)
01316: {
01317: case VHOST_GET_FEATURES:
01318: features = VHOST_ADSP_FEATURES;
01319: if (copy_to_user(featurep, &features, sizeof(features)))
01320: return -EFAULT;
01321: return 0;
01322: case VHOST_SET_FEATURES:
01323: if (copy_from_user(&features, featurep, sizeof(features)))
01324: return -EFAULT;
01325: if (features & ~VHOST_ADSP_FEATURES)
01326: return -EQPMOTSUPP;
01327: return vhost_adsp_set_features(vadsp, features);
01328: case VHOST_RESET_OWNER:
01329: return vhost_adsp_reset_owner(vadsp);
01330: case VHOST_ADSP_GET_RSV_DRAM:
01331: get_adsp_audio_reserve_dram(&rsv_mem.phy_addr,
01332: &rsv_mem.size);
01333: pr_info("[VADSP]%() L%d adsp rsv mem:0x%llx, size:0x%llx!!\n", __func__,
01334: __LINE__, rsv_mem.phy_addr, rsv_mem.size);
01335: if (copy_to_user(argp, &rsv_mem, sizeof(rsv_mem)))
01336: return -EFAULT;
01337: return 0;
01338: case VHOST_ADSP_SET_RSV_DRAM:
01339: //struct adsp_evt_cb_data data;
01340: memp = argp;
01341: if (copy_from_user(&rsv_mem, memp, sizeof(struct adsp_audio_reserve_mem)))
01342: return -EFAULT;
01343: pr_info("[VADSP]%() L%d set adsp rsv mem:0x%llx, size:0x%llx!!\n", __func__,
01344: __LINE__, rsv_mem.phy_addr, rsv_mem.size);
01345: adsp_audio_rsv_mem.phy_addr = rsv_mem.phy_addr;
01346: adsp_audio_rsv_mem.size = rsv_mem.size;
01347: adsp_rsv_mem_sent = 0;
01348: return 0;
01349: default:
01350: mutex_lock(&vadsp->dev.mutex);

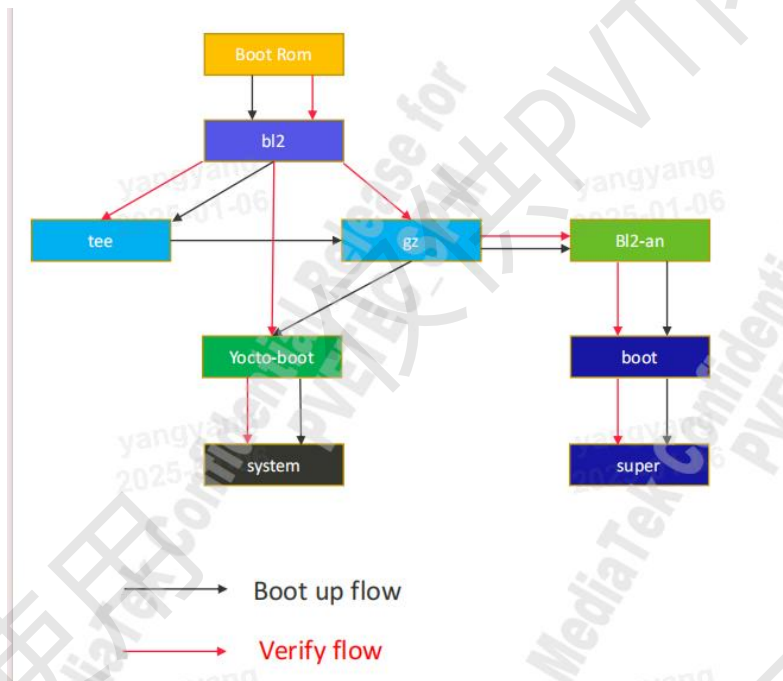
```



## MTK Audio Virtualization VADSP



## 8.6 secure boot



### 8.6.1 sign flow

1. Android branch 使用case5 对mtk format image进行签名。

Image list: tee.img, dtbo.img。

Sign script: vendor/mediatek/proprietary/scripts/sign-image\_v2/hsm.py

2. Android branch 使用case6 对google image 进行签名,

image list: vbmeta, vbmeta\_system, vbmeta\_vendor, boot, vendor\_boot, init\_boot, super

3. Yocto branch 使用 case5 对android prebuilt image 进行签名

image list: spmfw.img sspm.img dpm.img gz.img pi\_img.img mcupm.img audio\_dsp.img modem.img

consys\_gnss.img consys\_bt.img connsys\_wifi.img

sign script: meta/meta-mediatek-mt8676/scripts/sign-image\_v2/hsm.py

4. Yocto branch 使用 case2 对 yocto format image 进行签名

image list: yocto-boot.img apusys.img gpueb.img scp.img vcp.img logo.img

sign script: meta/meta-mediatek/recipes-bsp/secure-boot/hsm-sign-env/hsmstool/hsm\_sign\_tool.py

5. Yocto branch 使用case3 对yocto system image 进行签名

image list: system.ext4

sign script: meta/meta-mediatek/recipes-bsp/secure-boot/hsm-sign-env/hsmstool/hsm\_sign\_tool.py

6. Yocto branch 使用 case1 对yocto bootloader 进行签名

image list: bl2.img

sign script: meta/meta-mediatek/recipes-bsp/lk/files/pbp/hsm.py

## 8.6.2 verify flow

1. Brom 通过efuse keyhash校验bl2.img

2. Bl2 通过 android root public key校验case5签名的android image

3. Bl2 通过 yocto image public key 校验 case2签名的yocto image

4. Yocto-boot 通过 yocto image public key校验case3签名的system.ext4

5. Bl2-an 通过 android root public key校验case5签名的android image

6. Bl2-an 通过avb public key校验 avb相关image

7. boot 通过avb public key校验super image

## 8.6.3 key配置

### 8.6.3.1 Bootloader keys

适用image: bl2.img

使用私钥:

<yocto branch>/meta/meta-mediatek/conf/machine/keys/sbc\_key.pem

<yocto branch>/meta/meta-mediatek/conf/machine/keys/verified\_key.pem

使用公钥:

<yocto branch>/meta/meta-mediatek/conf/machine/keys/sbc\_key.pem 自动转换

公钥位置: bl2.img 尾部

### 8.6.3.2 yocto format的key

适用image: logo.img, vcp.img, apusys.img, yocto-boot.img

使用私钥:

<yocto branch>/meta/meta-mediatek/conf/machine/keys/verified\_key.pem

使用公钥:

<yocto branch>/meta/meta-mediatek/conf/machine/keys/verified\_key.pem 自动转换

公钥位置: bl2.img 内部

### 8.6.3.3 yocto rootfs keys

适用image: system.ext4

使用私钥:

<yocto branch>/meta/meta-mediatek/conf/machine/keys/verified\_key.pem

使用公钥:

<yocto branch>/meta/meta-mediatek/conf/machine/keys/verified\_key.pem 自动转换

公钥位置: yocto-boot.img

### 8.6.3.4 hypervisor keys

待谦川提供补充

### 8.6.3.5 mtk android format image keys

适用image: audio\_dsp.img, consys\_gnss.img, dpm.img, dtbo.img, gpueb.img, gz.img, mcupm.img, modem.img, mvpu\_algo.img, pi\_img.img, scp.img, spmfw.img, sspm.img, tee.img

使用私钥:

<android branch>/ vendor/mediatek/proprietary/custom/{Platform}/security/cert\_config

使用公钥:

<yocto code>/sbc/bsp/lk2/target/{Project\_Name}/include/oemkey.h

<yocto code>/sbc/bsp/lk2/target/{Project\_Name}/include/oemkey.h

公钥位置:

bl2.img 和bl2-an.img 内部

### 8.6.3.6 avb keys

适用image: vbmeta.img, vbmeta\_system.img, vbmeta\_vendor.img, super.img, boot.img, vendor\_boot.img, init\_boot.img

使用私钥:

<android branch>/device/mediatek/system/common/key/rsa2048/oem\_prvk.pem

<android branch>/device/mediatek/vendor/common/key/rsa2048/oem\_prvk.pem

使用公钥:

<yocto branch>/src/bsp/lk2/target/auto8678p1\_64\_ufs/include/avbkey.h

公钥位置:

vbmeta.img, vbmeta\_system.img, vbmeta\_vendor.img 内部

## 8.6.4 常用的配置

### 8.6.4.1 新增镜像添加校验flow

#### 1. 编译签名

新增镜像建议使用FIT镜像格式签名，签名步骤如下，

1). 实现基类，可以参考下图bbclass文件，主要实现do\_assemble\_fitimage，这是一个常见的任务，用于生成FIT (Flattened Image Tree) 镜像，并签名

```
src: /mt8676_pvt_lla/yocto/meta/meta-mediatek/classes/linyys-apusys-fitimage.bbclass (revision bdc646d4)
Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download | Search ☐ current directory

102 apusys_line=kernel = \apusys-1\;
103 sign_images_line=sign-images = \kernel\;
104 if ["${FIT_PSS_PADDING_SUPPORT}" = "yes"]; then
105 padding_line=padding = \pss\;
106 else
107 padding_line=""
108 fi
109 cat << EOF >> ${WORKDIR}/fit-image.its
110 default = "conf-1";
111 conf-1 {
112 description = "${conf_desc}";
113 ${apusys_line}
114 signature-1 {
115 algo = "${conf_algo}";
116 ${padding_line}
117 key-name-hint="${conf_key_name}";
118 ${sign_images_line}
119 }
120 }
121 EOF
122
123
124 do_assemble_fitimage() {
125 rm -f ${WORKDIR}/fit-image.its
126 fitimage_emit_fit_header
127
128 #
129 # Step 1: Prepare a apusys image section.
130 #
131 fitimage_emit_section_maint imagetart
132 fitimage_emit_section_apusys ${APUSTS_OUT}/${APUSTS_BINARY} ${APUSTS_COMPRESS}
133 fitimage_emit_section_maint sectend
134
135 #
136 # Step 2: Prepare a configurations section
137 #
138 fitimage_emit_section_maint confstart
139 fitimage_emit_section_config
140 fitimage_emit_section_maint confend
141 }
```

2). 新增镜像的bb文件定义这个基类（即上一步的bbclass），然后就会调用do\_assemble\_fitimage这个函数可以参考下图的bb文件

按照上面步骤，就可以对新镜像签名了

在lk阶段加载镜像时候，添加校验流程，`.verify`这个钩子函数需要指定`fit conf verify sig`，

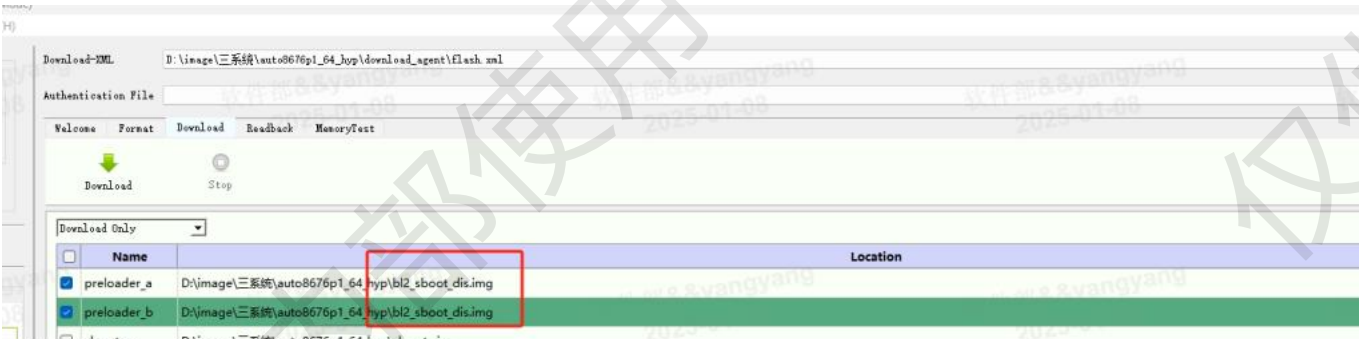
```

xref: /mt8676_pvt_11a/yocto/src/bsp/lk2/platform/mediatek/common/image_loaders/fit_loaders/fit_logo_loader.c
Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download
37 logo_info = (struct imageinfo *)calloc(1, sizeof(struct imageinfo));
38 if (!logo_info) {
39 LTRACEF("malloc %lu bytes for logo_info failed.\n",
40 sizeof(struct imageinfo));
41 return -1;
42 }
43
44 logo_info->image_type = IMAGE_TYPE_LOGO;
45 logo_info->load_va = load_va;
46 logo_info->size = size;
47 logo_info->entry_pa = entry_pa;
48 ret = imageinfo_set(logo_info);
49 if (ret) {
50 LTRACEF("imageinfo_set logo failed.\n");
51 free(logo_info);
52 return -1;
53 }
54
55 dprintf(0, "fit_load_logo 22222\n");
56 return 0;
57 }
58
59 __WEAK int logo_postload(void)
60 {
61 return 0;
62 }
63
64 #ifndef LOGO_PART_BASE_NAME
65 #define LOGO_PART_BASE_NAME "logo"
66 #endif
67
68 LOADER_START(logo)
69 .part_name = LOGO_PART_BASE_NAME,
70 .recovery_part_name = NULL,
71 .skip_check = NULL,
72 .preload = NULL,
73 .read = &fit_get_image,
74 .verify = &fit_conf_verify_sig,
75 .load = &fit_load_logo,
76 .postload = logo_postload,
77 LOADER_END
78

```

### 8.6.4.2 disable secure boot

1. 替换preloader镜像，替换成bl2\_sboot\_dis.img



2. 替换Android lk镜像，替换成bl2-an\_sboot\_dis.img

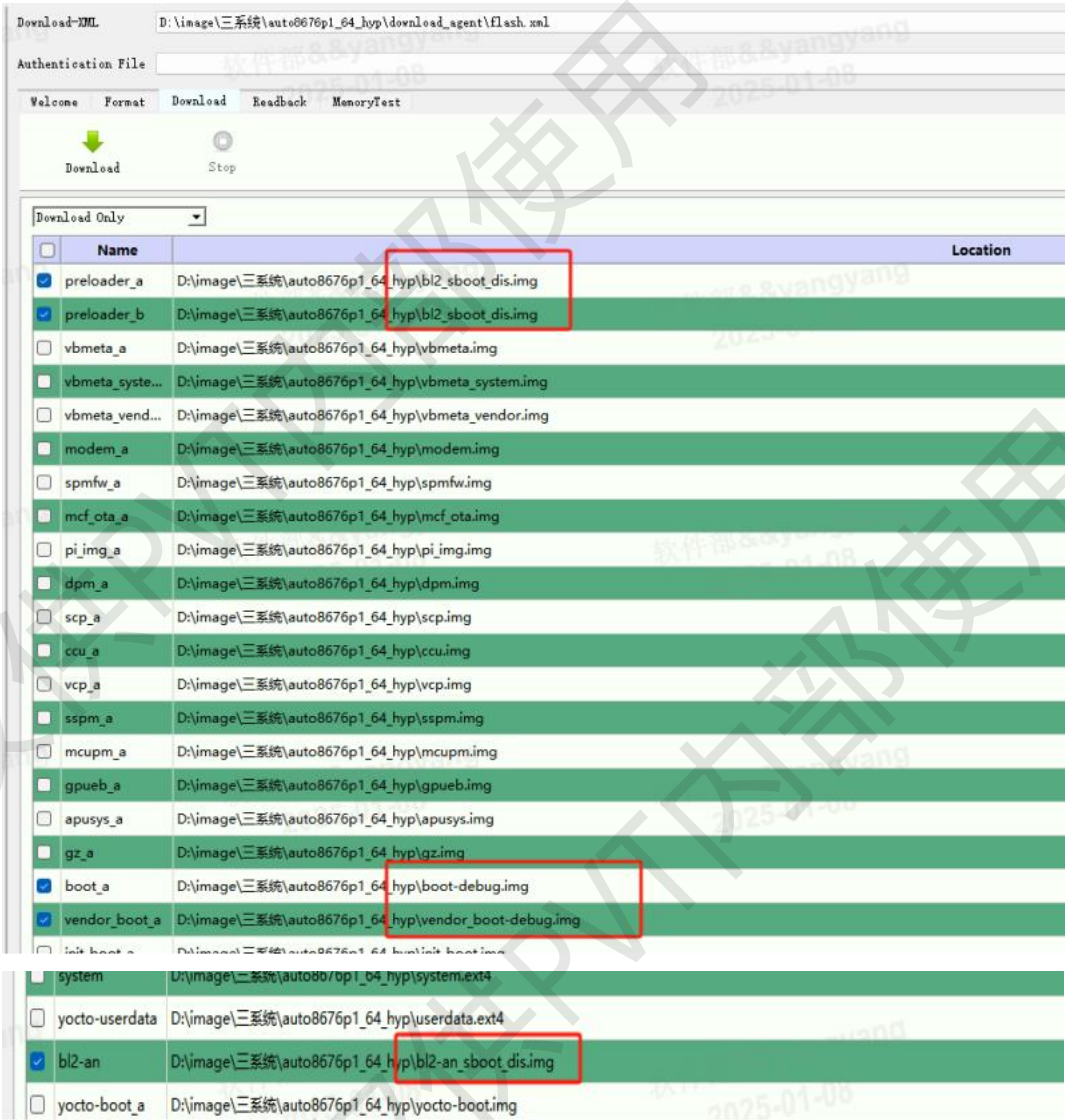
### 8.6.4.3 disable avb

android/device/mediatek/common/device.mk  
BOARD\_AVB\_ENABLE:= false

### 8.6.4.4 remount system分区



需要替换bl2\_sboot\_dis.img, bl2-an\_sboot\_dis.img, boot-debug.img, vendor\_boot-debug.img, bl2-an\_sboot\_dis.img这几个镜像



## 8.7 optee

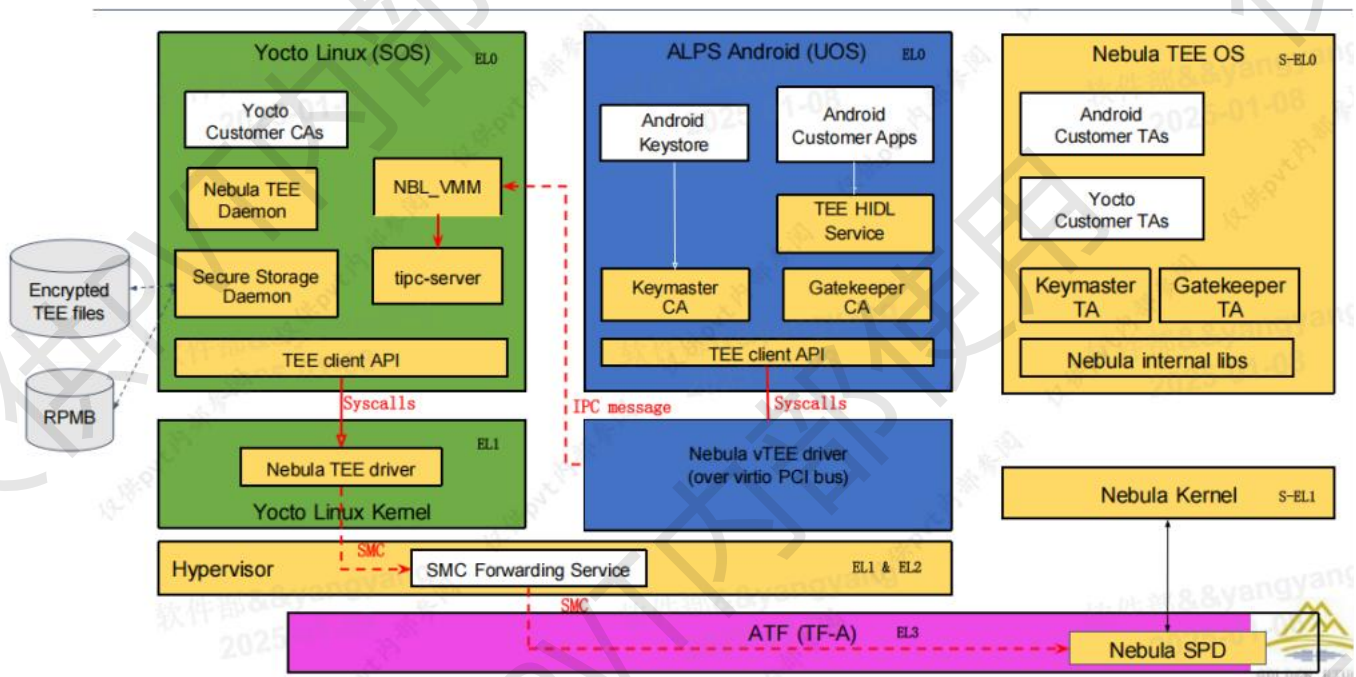
### 8.7.1 optee 简介

OP-TEE 是一个开源的 Secure os。下图是 OP-TEE 官方提供的架构图。

OP-TEE 内核是运行在S.EL1 和 S.EL0 中的可信应用程序。可信应用程序通过 TEE 内部 AP1与 OP-TEE 内核通信。TEE内部 API是由 GlobaiPlatfom 组织开发的标准 API。GlobalPlatform 致力于开发标准 API, 这些 API不仅支持 OP-TEE,还支持许多不同的 TEE。采用 GP 标准意味着在其他支持 GP 接口的 TEE 应用可以方便地移植到 OP-TEE 上, 反之亦然, 为 OP-TEE 设计的 TEE 应用也可以方便地移植到其他支持 GP 接口的 TEE 上。

在非安全状态下，内核空间中有一个 OP-TEE 驱动程序。它负责处理与 OP-TEE 内核的低级别通信。在非安全用户空间(EL0)中，有一个用户空间库 (libopenteec.so)实现了 GlobalPlatform API。TEE Client API是应用程序用来访问 TA(可信应用程序)的接口。

OP-TEE还包括一个称为 tee-supplciant 的组件。tee-supplciant 处理 OP-TEE 需要返回到 REE world 处理的事务，比如从文件系统中加载 UTA，再比如 TEE 安全存储功能需要把加密后的数据保存在 Flash 或者 RPMB 中，但 TEE 本身无法读写 fash 或 RPMB，需要切回 REE，通过 tee-supplciant 完成读写。



## 8.7.2 optee应用开发

在Yocto域和Android域都支持OP-TEE，且Yocto域和Android域所需的API都是一样的。这意味着 yocto 域和Android域的CA/TA代码可以完全一样。

如果您的程序需要运行在Yocto域，需要将CA和TA的执行文件打包进Yocto域的文件系统。Android域同理，若需要运行于Android域，也需要将执行文件打包进Android域的文件系统。

### 8.7.2.1 CA文件

CA 部分的开发主要是包括如何初始化上下文，如何打开/关闭会话，以及如何使用不同的参数类型进出 TA。

```

static TEEC_Session g_stTeeSession = {0};
static TEEC_Context g_stTeeContext = {0};
static bool g_bIsTeeInitialized = false;

TEEC_Result initTzContext()
{
 const TEEC_UUID stTeeUuid = {0xa60589fd, 0x0158, 0x40eb, {0xa7, 0x74, 0xa3, 0x98,
0xb1, 0xd6, 0x98, 0x32}};
 TEEC_Result u32TeeResult = TEEC_ERROR_GENERIC;
 uint32_t u32RetOrig = 0;

 if (true == g_bIsTeeInitialized)
 {
 u32TeeResult = TEEC_SUCCESS;
 goto exit;
 }

 u32TeeResult = TEEC_InitializeContext(NULL, &g_stTeeContext);
 if (TEEC_SUCCESS != u32TeeResult)
 {
 DLOGE("TEEC_InitializeContext fail, %X", u32TeeResult);
 goto exit;
 }

 u32TeeResult = TEEC_OpenSession(&g_stTeeContext, &g_stTeeSession,
&stTeeUuid, TEEC_LOGIN_PUBLIC, NULL, NULL, &u32RetOrig);
 if (TEEC_SUCCESS != u32TeeResult)
 {
 TEEC_FinalizeContext(&g_stTeeContext);
 DLOGE("TEEC_OpenSession fail, %X, orig: %X", u32TeeResult, u32RetOrig);
 goto exit;
 }

 g_bIsTeeInitialized = true;
exit:
 return u32TeeResult;
}

TEEC_Result uninitTzContext()
{
 TEEC_Result u32TeeResult = TEEC_ERROR_GENERIC;

 if (false == g_bIsTeeInitialized)
 {
 u32TeeResult = TEEC_ERROR_GENERIC;
 DLOGE("Session not exist!");
 goto exit;
 }

 TEEC_CloseSession(&g_stTeeSession);
 TEEC_FinalizeContext(&g_stTeeContext);
}

```

### 8.7.2.2 TA文件

TA 部分实现了基于不同的命令标识来执行 CA 请求的各种操作，TA 的开发者要定义自己的 UUID，在 CA 调用 TEEC\_OpenSession 时传入。同时该 demo 也在 user\_ta\_header\_defines.h 中将 TA 定义为单实例，即 TA 的二进制在 OP-TEE 内存中只会存一份，不会多次加载。TA\_STACK\_SIZE 和 TA\_DATA\_SIZE 则需要根据各 TA 的实现自行调整大小。

```
#define TA_UUID { 0xa60589fd, 0x0158, 0x40eb, {0xa7, 0x74, 0xa3, 0x98, 0xb1, 0xd6, 0x98, 0x32} }

TEE_Result TA_CreateEntryPoint(void)

#define TA_FLAGS (TA_FLAG_SINGLE_INSTANCE | \
```

NOTES

MediaTek Proprietary and Confidential. © 2022 MediaTek Inc. All rights reserved. Unauthorized reproduction or disclosure of this document, in whole or in part, is strictly prohibited.

21

[more Feedback](#)

Confidential

MT8676 Hypervisor

User!

```
TA_FLAG_MULTI_SESSION \
)

#define TA_STACK_SIZE (2 * 1024)
#define TA_DATA_SIZE (32 * 1024)
```

当 TA 收到 CA 发送的命令时，即可根据命令标识来进行参数检查，并进行相应的功能实现。

```
TEE_Result TA_InvokeCommandEntryPoint(void *pSessionContext, uint32_t nCommandID, uint32_t
nParamTypes, TEE_Param pParams[4])
{
(void)pSessionContext;
DLOGE("[TA]CommandID : %d \n", nCommandID);
switch (nCommandID)
{
case E_DEMO_CMD_NO_PARA_TEST:
(void)pParams;
return demo_ta_no_para_test(nParamTypes);
case E_DEMO_CMD_VALUE_INPUT_TEST:
return demo_ta_value_input_test(nParamTypes, pParams);
case E_DEMO_CMD_VALUE_OUTPUT_TEST:
return demo_ta_value_output_test(nParamTypes, pParams);
case E_DEMO_CMD_VALUE_INOUT_TEST:
return demo_ta_value_inout_test(nParamTypes, pParams);
case E_DEMO_CMD_MEMREF_TEMP_INPUT_TEST:
return demo_ta_memref_temp_input_test(nParamTypes, pParams);
case E_DEMO_CMD_MEMREF_TEMP_OUTPUT_TEST:
return demo_ta_memref_temp_output_test(nParamTypes, pParams);
case E_DEMO_CMD_MEMREF_TEMP_INOUT_TEST:
return demo_ta_memref_temp_inout_test(nParamTypes, pParams);
default:
DLOGE("Unexpected command id: %d", nCommandID);
return TEE_ERROR_BAD_PARAMETERS;
}
}
```

## 8.8 OTA

### 8.8.1 OTA 简介

#### 1、Hypervisor OTA Update 架构



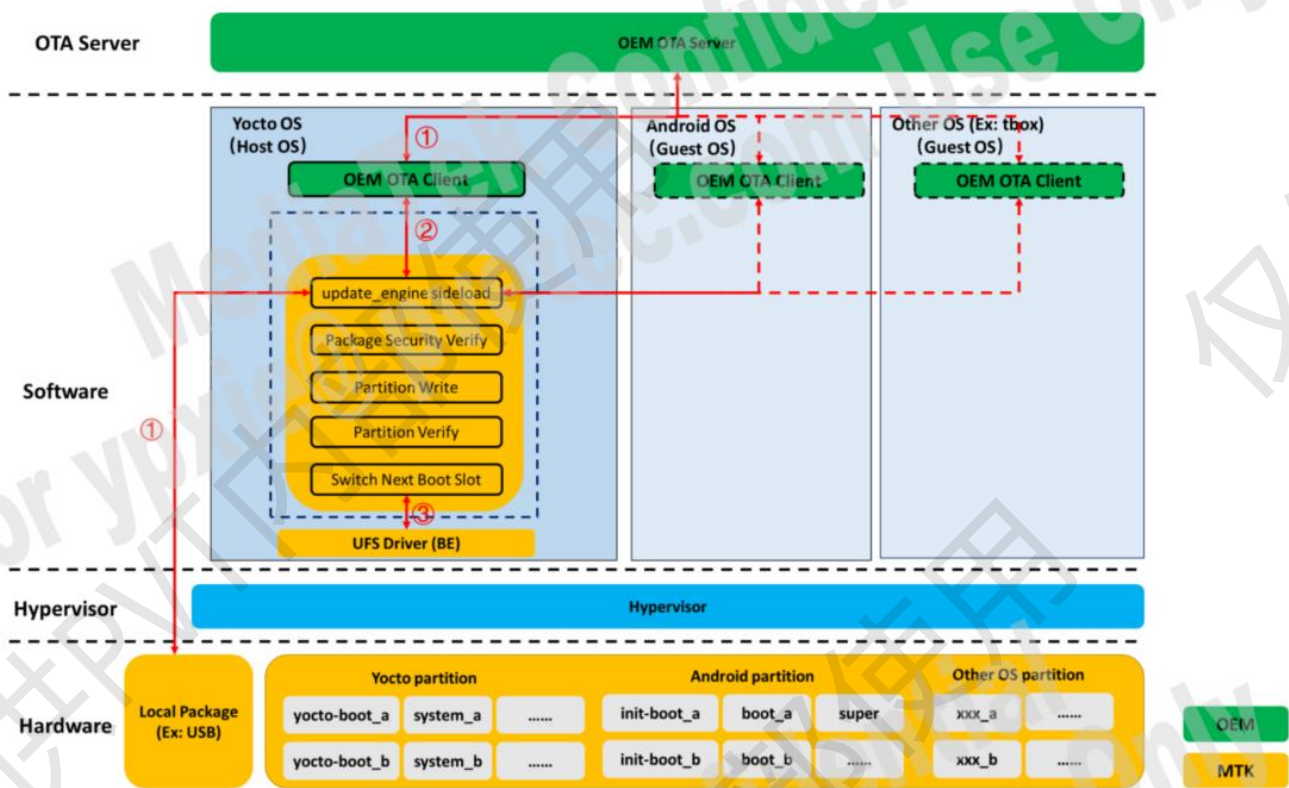


图3-1. Hypervisor (L+L+A) OTA Update 架构

图3-1描述了Hypervisor (L+L+A) 整体升级的整体架构，从图中能获取到如下信息：

1 Hypervisor (L+L+A) 升级支持两种触发方式：本地升级和网络升级，联发科技只支持本地升级，网络升级需要客户自行实现，本地升级是指升级包存放在本地环境内，利用升级脚本去触发升级；网络升级是指升级包存放在远程服务器内，升级时需要OTAAPP通过无线网络远程下载和安装更新包

2 Hypervisor OTA升级全部在Yocto OS (host OS)上完成，全程不需要Android OS (Guest OS)参与

3.以本地升级为例，Hypervisor OTA升级的大致流程如下

- 1)将升级包存放到本地PC环境内
- 2)执行OTA update script调用update\_engine\_sidedload触发升级
- 3)对升级包进行完整性与准确性校验
- 4)将升级包内的升级数据分别更新到对应分区内
- 5)对已经更新到分区内的数据进行校验，确保数据的准确性
- 6)更新系统启动控制参数，即Bootctl信息，确保系统下一次启动加载新的slot分区数据
- 7)切换boot region,确保系统的下一次启动时加载新的preloader分区数据
- 8)重启设备

2、Hypervisor build otapackage架构

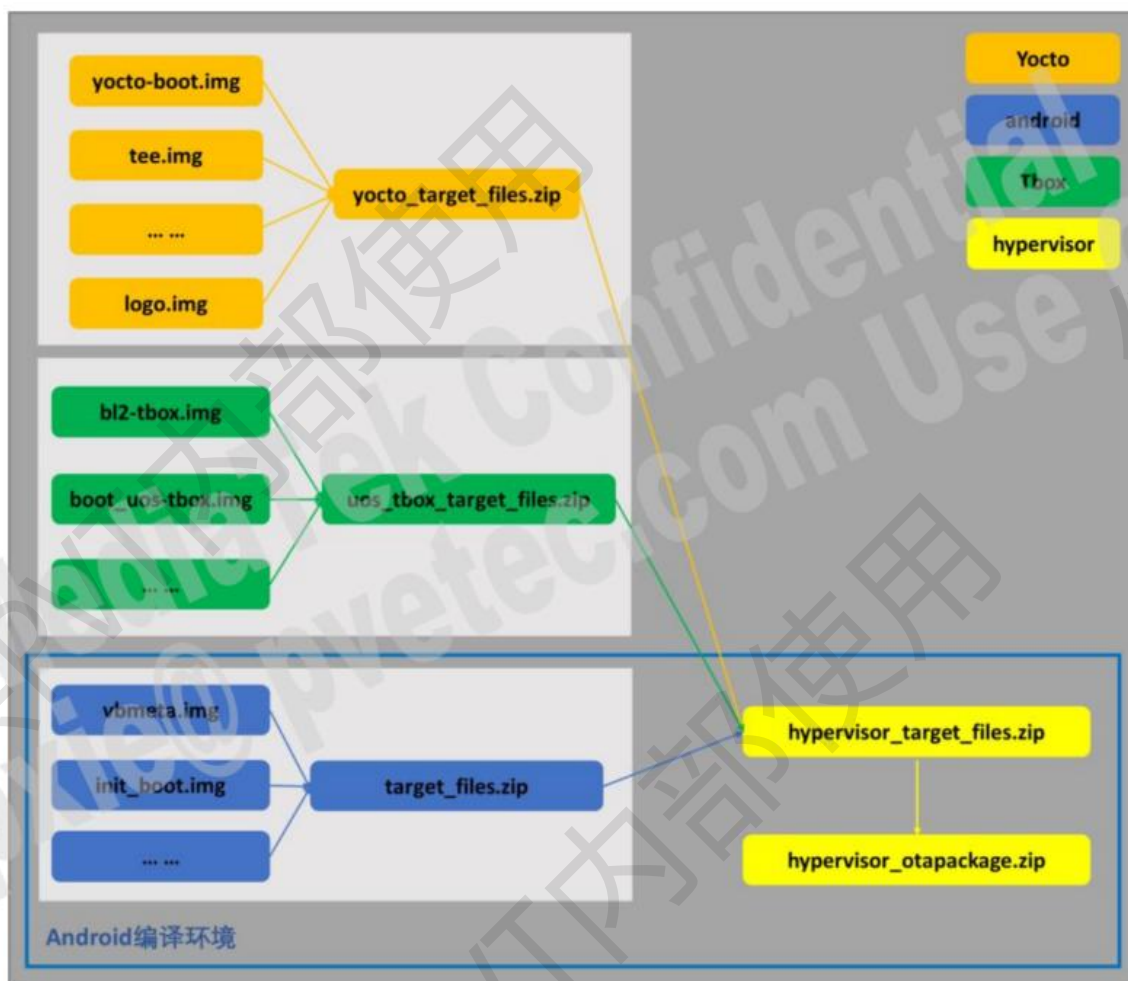


图3-2. Hypervisor (L+L+A) build otpackage 架构

图3-2描述了Hypervisor (L+L+A) OTA编译升级包的架构，从图中能获取到如下信息：

- 1.Hypervisor (L+L+A) 整体升级只需要编译一个OTA升级包
- 2.Hypervisor (L+L+A) 编译升级包必须在Hypervisor Android的编译环境下进行
- 3.Hypervisor编译升级包的大致流程如下：
  - 编译Hypervisor yocto\_target\_files.zip
  - 编译Hypervisor uos\_tbox\_target\_files.zip
  - 编译Hypervisor android target\_files.zip
  - 将yocto\_targetfiles.zip、uos\_tbox\_targetfiles.zip和 targetfiles.zip合并成最终的hypervisor\_target\_files.zip
  - 利用hypervisor\_target\_files.zip在Android编译环境下编译生成全量升级包和差分升级包

## 8.8.2 如何编译Hypervisor(L+L+A) OTA升级包

- 1、编译hypervisor\_target\_files.zip
  - a.Build yocto\_target\_files.zip



Yocto的target\_files.zip由Hypervisor\_make\_yocto\_targetfiles.py编译产生，脚本位于meta/meta-mediatek-mt8676-hyp/recipes-devtools/ota-tools/files。

Hypervisor Yocto编译完成后，在Yocto的编译环境下执行如下指令：

```
python3 Hypervisor_make_yocto_targetfiles.py <参数1><参数2>
```

批注：

- Hypervisor\_make\_yocto\_targetfiles.py:编译yocto targetfiles.zip的脚本
- <参数1>:Yocto targetfiles.zip的输出路径
- <参数2>:Project scatter.txt文件

在Yocto OS编译阶段，系统会自动执行该脚本进行打包，最终的yocto\_target\_files.zip位于：  
build-sos/tmp/deploy/images/<project\_name>/yocto\_target\_files.zip

#### b.Build uos\_tbox\_target\_files.zip

T-box OS (Guest os)的target\_files.zip由Hypervisor\_make\_uos\_tbox\_targetfiles.py编译产生，脚本位于meta/meta-mediatek-mt8676-hyp/recipes-devtools/uos-tbox-ota-tools/files。

Hypervisor Yocto编译完成后，在Yocto的编译环境下执行如下指令：

```
python3 Hypervisor_make_uos_tbox_targetfiles.py <参数1><参数2>
```

批注：

- Hypervisor\_make\_uos\_tbox\_targetfiles.py:编译Tbox targetfiles.zip的脚本
- <参数1>:Tbox targetfiles.zip的输出路径
- <参数2>:Project scatter.txt文件

在T-box OS编译阶段，系统会自动执行该脚本进行打包，最终的uos\_tbox\_target\_files.zip位于：  
build-tbox/tmp/deploy/images/<project\_name>/uos\_tbox\_target\_files.zip

#### c.Build Android target\_files.zip

Android的target\_files.zip默认在build阶段完成，无需手动编译，其位于out/target/product/<projectname>/merged/目录下，名称为target\_files.zip。  
示例：out/target/product/auto8676p1\_64\_bsp\_vm/merged/target\_files.zip

#### d.Build Hypervisor targetfiles.zip

Hypervisor\_target\_files.zip需要使用Hypervisor\_3OS\_merge\_targetfiles.py脚本对yocto\_target\_files.zip、uos\_tbox\_target\_files.zip和targetfiles.zip 进行整合产生，脚本位于device/mediateksample/<project name>。  
在获取到yocto\_target\_files.zip、uos\_tbox\_target\_files.zip和targetfiles.zip后，在Android的编译环境下执行如下指令：

```
python3 Hypervisor_3OS_merge_targetfiles.py <参数1> <参数2> <参数3>
```

批注：

·Hypervisor\_3OS\_mrege\_targetfiles.py:编译hypervisor targetfiles.zip的脚本

·<参数1>:yocto\_target\_files.zip的路径

·<参数1>:uos\_tbox\_target\_files.zip的路径

·<参数2>:targetfiles.zip的路径

#### 输出文件:

在当前目录下会产生名称为hypervisor\_target\_files.zip的文件, 这就是Hypervisor的targetfiles.zip包

#### 示例:

```
python3 device/mediateksample/auto8676p1_64_bsp_vm/Hypervisor_3OS_merge_targetfiles.py
build-sos/tmp/deploy/images/auto8676p1_64_hyp/yocto_target_files.zip
build-tbox/tmp/deploy/images/auto8676p1_64_uos_tbox/uos_tbox_target_files.zip
out/target/product/auto8676p1_64_bsp_vm/merged/target_files.zip
```

### 2、编译全包

在参考上面编译hypervisor\_target\_files.zip小节编译得到hypervisor\_target\_files.zip后, 就可以在Hypervisor Android的编译环境下编译Hypervisor的全量升级包。

#### 编译指令:

```
./out_sys/host/linuxe-x86/bin/ota_from_target_files -v --block -p out_sys/host/linux-x86 -k
build/make/target/product/security/testkey --skip_postinstall hypervisor_target_files.zip otapackage_full.zip
```

#### 批注:

·ota\_from\_target\_files:升级包编译脚本

·build/make/target/product/security\_testkey:对升级包进行签名的key路径, 联发科技默认使用Google提供的testkey, 贵司可指定其它的key路径, 使用-k参数指定, 如何更换签名key, 请参考: 1.6如何修改升级包的签名

·hypervisor\_target\_files.zip:Hypervisor的targetfiles.zip

·otapachage\_full.zip:最终产生的Hypervisor全量升级包, 包名可随意指定

· 请务必确保在Hypervisor Android的build环境下编译升级包, 并确保编译全包前有执行source & lunch, 否则将导致全包编译失败

### 3、编译差分包

#### 前提条件:

编译差分包需要预先准备两份hypervisor\_target\_files.zip。一份是基底版本(source版本)的

hypervisor\_target\_files.zip, 此处命名为source\_hypervisor\_target\_files.zip; 一份是目标版本(target版本)的

hypervisor\_target\_files.zip, 此处命名为target\_hypervisor\_target\_files.zip。

批注: 请务必确保平台上烧录的基底版本load和基底版本的target\_files.zip是同一次编译得到的, 否则将导致OTA差分升级失败。

#### 编译指令:

```
./out_sys/host/linux-x86/bin/ota_from_target_files -v --block -p out_sys/host/linux-x86-
kbuild/make/target/product/security/testkey --skip_postinstall -i source_hypervisor_target_files.zip
target_hypervisor_target_files.zip otapackage_delta.zip
```

批注：

- ota\_from\_target\_files: 升级包编译脚本
- build/make/target/product/security/testkey: 对升级包进行签名的key路径，联发科技默认使用Google提供的testkey, 贵司可指定其它的key路径，使用-k参数指定。
- source\_hypervisor\_target\_files.zip: source版本的hypervisor\_target\_files.zip
- target\_hypervisor\_target\_files.zip: target版本的hypervisor\_target\_files.zip
- otapackage\_delta.zip: 最终产生的Hypervisor差分升级包，包名可随意指定
- 请务必确保在Hypervisor Android的build环境下编译升级包，并确保编译全包前有执行source & lunch, 否则将导致差分包编译失败

### 8.8.3 如何进行Hypervisor OTA升级

#### 1、全包升级(normal mode)

测试环境

- (1) The PC has Python3 environment
- (2) Hash base load in DUT
- (3) Get OTA package

测试步骤

- (1) Device connection yocto adb environment
- (2) Execute the upgrade script:

```
python3 hypervisor_update.py --file otapackage_delta.zip > update.txt 2&1
```

-hypervisor\_update.py: Update script

-otapackage\_delta.zip: OTA package, unlimited package name

- > update.txt 2>&1: Store the upgrade log in update.txt in the current directory

预期结果

- (1) Update should be completed, and update.txt prints the following  
log: [INFO:update\_attempter\_android.cc(600)] Update successfully applied, waiting to reboot.
- (2) The first restart after the upgrade should successfully enter the HomeScreen
- (3) Check whether setting-version has been updated

Note: If the test upgrade fails, please provide the update log and uart log in CR

#### 2、差分升级(normal mode)

测试环境

- (1) The PC has Python3 environment

(2) flash base load in DUT,must be consistent with the base version provided to the OTA owner

(3) Get OTA package from OTA owner

#### 测试步骤

(1) Device connection yocto adb environment

(2) Execute the upgrade script:

```
python3 hyperisor_update.py --file otapackage_delta.zip > update.tx t2>&1
```

-hypervisor\_update.py Update script

-otapackage\_delta.zip:OTA package,unlmitd package name

- >update.txt 2>81:Store the upgrade log in update.txt in the current directory

#### 预期结果

(1) Update should be completed,and update.txt prints the following

log:[INFO:update\_attemptr\_android.cc(600)]Update successfully applied,walting to reboot.

(2) The first restart after the upgrade should successfuly enter the HomeScreen

(3) Check whether setting-version has been updated

Note:If the test upgrade fais,please provide the update log and uart log in CR